

Sommario

Capitolo 1 - Concorrenza	6
1.1 La metafora del pancake	6
1.2 Partial Order e POSET	8
1.2.1 Diagramma di Hasse	9
1.3 Processo e thread	10
1.3.1 Catena	10
1.4 Esecuzione del DAG	11
1.4.1 Percorso critico e sincronizzazione	11
1.4.2 Algoritmo di Kahn	12
1.5 Benefici della parallelizzazione	15
Capitolo 2 - Sistemi a scambio di messaggi	18
2.1 Rete di interconnessione	18
2.2 Middleware	21
2.2.1 Semantica di Send/Receive	21
Capitolo 3 - Erlang	24
3.1 Programmazione funzionale	24
3.1.1 Uso della ricorsione	27
3.1.2 Programmazione Funzionale e Concorrenza nell'accesso ai dati	28
3.2 Le basi di Erlang	29
3.2.1 Scheduler di Erlang	30
3.2.2 La shell di Erlang	31
3.2.3 Applicazioni Erlang	31
3.2.3 Struttura di un programma Erlang	32
3.2.4 Tipi di dato	34
3.3 Moduli Erlang	39
3.4 Funzioni	40
3.4.1 Definizione di una funzione	41
3.4.2 Chiamata di una funzione	42
3.5 Controllo del flusso	43
3.5.1 Espressioni Guardia	43
3.5.2 Espressione Case	43
3.5.3 Espressione If	44
3.5.4 Loop in Erlang	44
3.6 High order functions in Erlang	45
3.7 Programmi con Erlang	47
3.7.1 Funzione spawn()	47
3.7.2 Funzione bang(!)	48
3.7.3 Segnali in Erlang	49
3.7.4 Funzione receive	50
3.7.5 Ping Pong	51
3.8 Client-server in Erlang	52
3.8.1 Processo Server	52

3.8.2 Processo Client.....	55
3.8.3 Pubblicazione del PID di un processo.....	57
3.9 Programmi distribuiti in Erlang.....	57
3.9.1 Connessione tra due nodi Erlang.....	59
3.9.2 Comunicazione remota.....	60
3.9.3 Funzione di callback.....	62
3.9.4 Data Storing: ETS & DETS.....	64
3.10 Errori in Erlang.....	65
3.10.1 Link tra processi.....	66
Capitolo 4 - Java.....	68
4.1 JVM - Java Virtual Machine.....	68
4.2 Java Generics.....	69
4.2.1 Type Erasure.....	71
4.3 Java Collection Framework - JCF.....	73
4.3.1 Gerarchie di interfacce.....	73
4.3.4 Implementation classes.....	75
4.3.5 Immutable collections.....	76
4.3.6 Interfacce e variabili funzionali.....	77
4.3.7 Espressioni lambda.....	78
4.4 Java streams.....	79
4.4.1 Stream pipeline.....	80
4.4.2 Valutazione “pigra” degli stream.....	82
4.4.3 Parallelismo nella pipeline.....	82
4.5 Java annotations.....	83
Capitolo 5 - Java Threads & Synchronization.....	85
5.1 Language-level Threading.....	85
5.1.1 La classe Thread.....	85
5.2 JVM Runtime Data Area.....	86
5.2.2 Method Area.....	87
5.2.3 Heap.....	87
5.3 Gestione dei thread in Java.....	89
5.3.1 Terminazione di un thread.....	89
5.4 Thread Pooling ed Executor Framework.....	92
5.4.1 Thread pooling.....	92
5.4.2 L'interfaccia Executor.....	93
5.4.3 Il Task e la classe Future.....	95
5.4.4 Off-the-shelf ExecutorServices.....	97
5.5 Java synchronizers.....	100
5.5.1 Latch.....	100
5.5.2 Barriera.....	100
5.5.3 Fasori.....	100
5.5.4 Semafori.....	101
5.5.5 Exchanger.....	101
5.6 Thread-safe Data Structures.....	102

5.6.1 Decorator Pattern.....	102
5.6.2 Collezione sincronizzata.....	103
5.6.3 ConcurrentHashMap.....	104
5.6.4 Iteratori per collezioni concorrenti.....	104
5.6.5 Blocking Queues.....	106
5.7 Memory Consistency Rules.....	107
5.7.1 Lazy Singleton Initialization.....	107
5.7.2 Double-Checked Locking e la keyword volatile.....	108
5.7.3 Modelli di consistenza della memoria.....	110
5.8 JMM - Java Memory Model.....	112
5.8.1 Obiettivi di JMM.....	112
5.8.2 Regole della JVM.....	114
Capitolo 6 - Logical Time.....	116
6.1 Process history.....	116
6.2 Space-time Diagrams.....	117
6.3 Nozione di tempo.....	119
6.3.1 Relazioni di precedenza.....	119
6.3.2 Relazioni di precedenza su variabili condivise.....	121
6.3.3 Eventi e stati locali.....	122
6.3.5 Stato globale.....	123
6.4 Clock per tempo logico.....	126
6.4.1 Lamport Timestamp.....	126
6.4.2 Totally Ordered Lamport Timestamp.....	128
6.4.3 Strong Clock Consistency - Timestamp Vettoriali.....	128
6.4.4 Cuts & Vector Timestamps.....	130
Capitolo 7 - Supporto per la com. in sist. distr.....	132
7.1 Protocolli di richiesta/risposta.....	132
7.2 Failure Model.....	134
7.2.1 Strategie per gestire le difficoltà.....	134
7.2.2 Riesecuzione vs caching.....	135
7.3 Esempio di prot. Richiesta-Risposta - HTTP.....	136
7.4 RPC - Remote Procedure Call.....	138
7.4.1 IDL - Interface Definition Language.....	138
7.4.2 Chiamate remote.....	138
7.4.3 Implementazione di una RPC.....	139
7.5 gRPC - Google RPC.....	142
7.5.1 ProtoBuf.....	142
7.5.2 Localizzare il Server gRPC.....	144
7.6 RMI - Remote Method Invocation.....	145
7.6.1 Riferimenti remoti.....	145
7.6.2 Implementazione di una RMI.....	147
7.6.3 Esempio di funzionamento.....	148
1. Il client invoca un metodo sul proxy (client stub).....	148
7.6.4 Servants.....	148

7.6.5 RMI e Multithreading.....	149
7.7 Class Loading in Java.....	150
7.7.1 Distributed Class Loader.....	150
7.8 Garbage Collection.....	151
7.8.1 Approccio mark and sweep.....	151
7.8.2 Performance della garbage collection.....	153
7.8.3 DGC - Distributed Garbage Collection.....	154
Capitolo 8 - Servizi Web.....	158
8.1 Modalità per sviluppare servizi web.....	158
8.1.1 Big web services.....	158
8.1.2 RESTful web services.....	159
8.1.3 JSON.....	161
8.2 SOA - Service Oriented Architecture.....	162
8.3 CGI - Common Gateway Interface.....	165
8.4 Tomcat Web Server.....	167
8.4.1 Architettura di Tomcat.....	168
8.4.2 Host and Context.....	168
8.4.3 Class Loader.....	171
8.5 Servlet.....	174
1. Creazione degli Oggetti Richiesta e Risposta.....	176
2. Accesso ai Parametri di Richiesta.....	176
3. Creazione del Corpo della Risposta.....	176
4. Impostazione del Tipo di Contenuto e Intestazioni.....	176
5. Configurazione dei Servlet.....	177
8.6 Tracciamento delle sessioni.....	178
8.6.1 Tecniche di tracciamento delle sessioni.....	179
8.6.2 Gestione degli oggetti sessione.....	180
8.7 Servlet e richieste HTTP.....	181
8.7.1 Filtri Servlet.....	181
8.8 Collaborazione tra Servlet.....	183
8.8.1 Condivisione di informazioni tra Servlet.....	184
8.8.2 Organizzare le applicazioni con i Servlet.....	185
8.9 JSP - Jakarta Server Pages.....	186
8.9.1 Funzionamento di JSP.....	188
8.9.2 Struttura di un template HTML.....	189
8.9.3 Separazione delle Responsabilità nei JSP.....	190
8.10 Sviluppo di WebApp: Struttura dei Progetti.....	191
8.10.1 JSP e Architettura MVC.....	192
Capitolo 9 - Applicazioni enterprise.....	193
9.1 Application Servers.....	193
9.2 JEE Server.....	194
9.3 Spring Framework.....	195
9.3.1 Architettura di Spring.....	196
9.3.2 Spring MVC.....	199

9.4 JMS - Jakarta Message System.....	202
9.4.1 Funzionamento di JMS.....	202
9.4.2 JMS programming objects.....	203
9.5 EJB - Enterprise JavaBeans.....	206
9.5.1 Session Beans.....	206
9.5.2 Message Driven Beans.....	208
9.5.3 Ciclo di vita di uno Stateless Session EJB.....	209
9.5.4 Ciclo di vita di uno Stateful Session EJB.....	210
9.6 Ricerca di risorse remote in Applicazioni Enterprise.....	212
9.6.1 JNDI - Java Naming and Directory Interface.....	212
Caratteristiche principali:.....	213
Capitolo 10 - Algoritmi distribuiti fondamentali.....	215
10.1 Wave algorithms.....	216
10.2 Snapshot di una computazione distribuita.....	219
10.2.1 Chandy-Lamport Algorithm.....	221
10.2.2 Algoritmo Lai-Yang.....	226
10.3 Algoritmi di broadcast e convergecast.....	229
10.3.1 Spanning Tree.....	229
10.4 Algoritmi di mutua esclusione.....	234
10.4.1 Algoritmo Ricart-Agrawala.....	235
10.4.2 Token Ring Mutex.....	241
10.4.3 Mutex Centralizzato.....	242
10.5 Classificazione dei Fallimenti.....	243
10.5.1 Modelli sincroni.....	243
10.6 Algoritmi per l'elezione di un leader.....	244
10.6.1 Algoritmo di Chang & Roberts (C&R).....	245
10.6.2 Bully Algorithm.....	249
10.7 Multicast.....	253
10.7.1 Le basi della comunicazione di gruppo.....	253
10.7.2 BMulticast - Basic Multicast.....	255
10.7.3 Rmulticast - Reliable Multicast.....	255
10.7.4 Altre semantiche di Multicast.....	256
Capitolo 11 - Sincronizzazione dei clock fisici.....	258
11.1 Sincronizzazione esterna, interna e ripristino.....	260
11.1.1 Sincr. esterna - Algoritmo dei Cristiani 	261
11.1.2 Sincr interna - Algoritmo di Berkeley.....	262
11.2 NTP - Network Time Protocol.....	263
11.2.1 Modalità di Sincronizzazione in NTP.....	264
Laboratorio 1 - Federate learning.....	267
L1.1 Approccio algoritmico.....	269
L1.2 FedLang.....	269
Laboratorio 2 - GRPC.....	272
Laboratorio 3 - Apache Maven.....	273
L3.1 Il file pom.xml.....	273

L3.2 Maven repository.....	274
Le domande tipiche sono evidenziate in questo colore.	

Capitolo 1 - Concorrenza

I sistemi distribuiti sono un caso specifico di sistemi concorrenti, tutto dipende dal fatto che il programma risieda in una singola macchina oppure sia diffuso su diverse macchine.

Ci riferiamo a due tipi di modelli, il modello a **memoria condivisa** e il **modello a scambio di messaggi**. Noi vedremo il secondo modello, ogni cosa che può essere espressa in un modello, può essere espressa anche nell'altro.

Rappresentazione formale di una esecuzione concorrente

Per lavorare al meglio ci serve una rappresentazione formale di un'esecuzione concorrente:

- **Azioni (A)**, l'insieme di azioni in una computazione concorrente è indicata come (*A - insieme di azioni...*).
- **Vincoli di precedenza** che indichiamo come (*PC - insieme di vincoli...*).

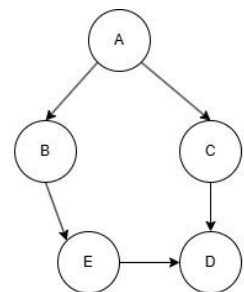
Definizione di concorrenza

Siano due azioni $a_i, a_j \in A$.

Queste due azioni sono dette **concorrenti** ($a_i || a_j$) **se e solo se** non ci sono relazioni del tipo $(a_i \leq_p a_j) \vee (a_j \leq_p a_i)$.

Attenzione: la relazione di concorrenza **non è transitiva**.

Sia il grafo nella figura a destra: C ed E sono concorrenti, così come B e C, però B ed E non sono concorrenti perché B deve precedere E ($B \leq E$).



1.1 La metafora del pancake

Quando vogliamo cucinare un pancake dobbiamo effettuare certe azioni in sequenza, questo può essere immaginato come una computazione composta da varie azioni collegate tramite diversi vincoli di precedenza.

- Prendi una ciotola grande.
- Inserisci la farina all'interno della ciotola.
- Inserisci il sale.
- Inserisci lo zucchero.
- "*Make the well*".
- Sciogli il burro.
- Metti il latte nella ciotola.
- Metti le uova nella ciotola.
- Metti il burro fuso nella ciotola.
- Mescola tutto.
- Metti l'olio nella pentola per friggere.
- Scalda la pentola per friggere.
- ...

Come si può vedere ogni azione è descritta e possiede dei vincoli di precedenza; per fare ciò usiamo una notazione grafica:

Relazione matematica

L'azione B viene dopo l'azione A; quindi lo rappresentiamo in questa notazione:

$$A \rightarrow B$$

Una **relazione matematica** è un elemento appartenente al prodotto cartesiano tra due insiemi. In questo caso uno di questi elementi è la relazione $A \rightarrow B$.

L'ordine (*a volte*) conta, quindi:

$$A \rightarrow B \neq B \rightarrow A$$

Se però cambiamo l'ordine delle azioni B, C, D, E il risultato non cambia, basta che vengano eseguiti dopo A ma prima di F .

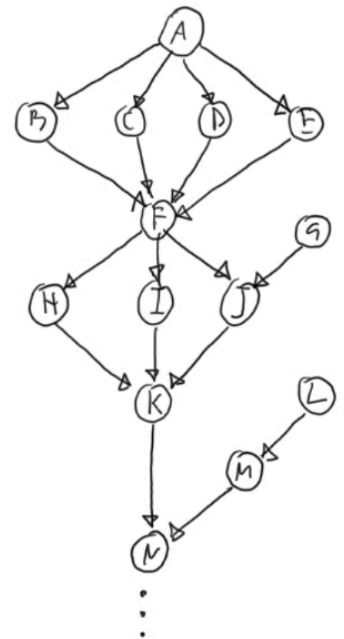
Quindi si hanno le seguenti relazioni:

- $A \rightarrow B$
- $A \rightarrow C$
- $A \rightarrow D$
- $A \rightarrow E$
- $B \rightarrow F$
- $C \rightarrow F$
- $D \rightarrow F$
- $E \rightarrow F$

F non può essere completata fino a che tutte le azioni precedenti non sono state completate.

Nel grafo in figura sopra non c'è nessuna freccia diretta da A a K , però ci sono vari percorsi indiretti che partendo da A arrivano a K .

Le azioni tra F e K (H, I, J) sono **concorrenti** perché possono essere svolte in parallelo.



DAG - Directed Acyclic Graph

Alla fine otteniamo un “*grafo diretto, aciclico*”, ovvero un **DAG**.

- **Diretto**: ogni arco ha una direzione, cioè collega un nodo di partenza a un nodo di arrivo.
- **Aciclico**: non esistono cicli, ovvero non puoi partire da un nodo, seguire una sequenza di archi e ritornare allo stesso nodo.

A cosa servono i DAG?

I DAG vengono utilizzati per rappresentare **dipendenze tra compiti** o **operazioni**. Ogni nodo rappresenta un'unità di lavoro (ad esempio, un'operazione o un task), e ogni arco rappresenta una dipendenza tra due compiti (cioè, un task deve essere completato prima che un altro inizi).

1.2 Partial Order e POSET

Vincolo di precedenza

Un vincolo di precedenza indica che una determinata attività **non può iniziare finché un'altra attività non è stata completata**.

$$A \rightarrow B$$

L'azione B viene dopo l'azione A.

PC - Precedence Constraints

Definiscono l'ordine in cui alcune attività o compiti devono essere eseguiti in un sistema.

Questo insieme di regole è indicato con PC .

$$PC = \{ A \rightarrow B, B \rightarrow C \}$$

Un modo per rappresentare il PC per 3 azioni (A, B e C) è tramite una matrice binaria:

	A	B	C
A	0	1	0
B	0	0	1
C	0	0	0

Partial Order

Un *Partial Order* descrive una relazione di ordine tra elementi di un insieme, ma in cui non tutti gli elementi sono necessariamente confrontabili.

Partendo dal PC è possibile definire un **partial order** tra gli elementi in A , l'ordine parziale viene indicato con $\leq_p = PC^+$

Questa relazione è detta **transitive closure**.

Applicando la transitività possiamo aumentare il numero di membri in PC , se utilizziamo la transitive closure possiamo capire quali azioni vengono prima di una certa azione.

$$a \leq b \rightarrow \text{"L'azione } a \text{ viene prima dell'azione } b\text{"}$$

Non-strict partial order ($\leq$) ha le seguenti proprietà:

- **Riflessività:** $a \leq a$
- **Antisimmetria:** Se $(a \leq b \wedge b \leq a) \Rightarrow a = b$.
 - Questo significa che se $(a \leq b \wedge b \neq a) \Rightarrow b \not\leq a$
- **Transitività:** Se $a \leq b \wedge b \leq c \Rightarrow a \leq c$

Strict partial order ($<$) ha le seguenti proprietà:

- **Irriflessività:** $\neg a < a$

Le altre proprietà sono le stesse del caso precedente.

POSET - Partial Order SET

A e $\leq_p = PC^+$ compongono un modello matematico formale per una esecuzione concorrente.

Un **POSET** (*insieme parzialmente ordinato*) può essere utilizzato per descrivere l'ordine parziale tra eventi in un sistema. In un ordine parziale, alcuni eventi possono essere ordinati temporalmente, mentre altri no, poiché avvengono in parallelo o non esiste una relazione di precedenza deterministica tra di essi.

Qual è una rappresentazione grafica migliore di un POSET? Una soluzione consiste nell'utilizzare un diagramma di Hasse.

Total order

Total order è composto da un partial order più la proprietà della **totalità** per la quale:

$$\forall a_i, a_j \in A \Rightarrow (a_i \leq_T a_j) \vee (a_j \leq_T a_i)$$

1.2.1 Diagramma di Hasse

Un diagramma di Hasse è una rappresentazione grafica di un POSET.

Viene utilizzato per visualizzare la relazione di ordine parziale tra gli elementi di un insieme.

Nel contesto dell'esecuzione concorrente, il diagramma di Hasse può essere usato per rappresentare le dipendenze tra eventi concorrenti.

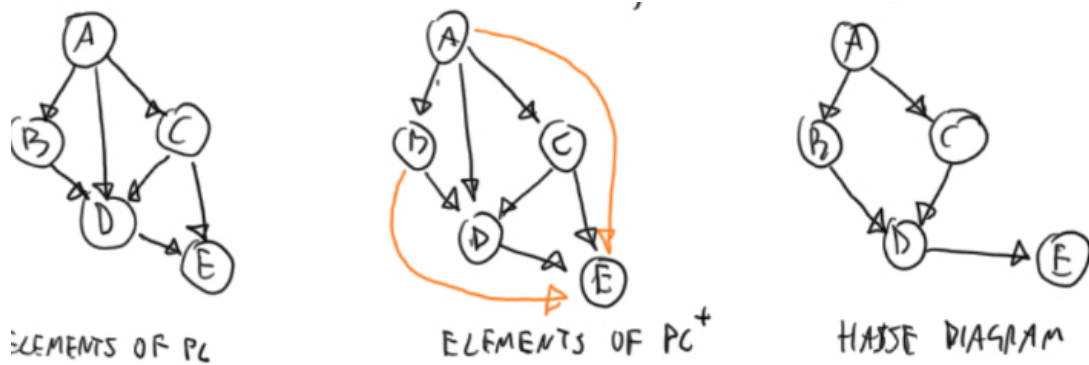
Riduzione transitiva

R^+ è l'insieme più piccolo tale che la transitive closure di R è la stessa di PC^+ ($R^+ = PC^+$). Ovvero risultano equivalenti.

Nel diagramma di Hasse si **omettono gli archi ridondanti**. Non si rappresenta direttamente la transitività: se $a \leq b$ e $b \leq c$, allora non si disegna l'arco diretto tra a e c . Questo perché l'arco $a \rightarrow b \rightarrow c$ contiene più informazioni che lasciando solo $a \rightarrow c$.

Per i DAG la riduzione transitiva è univoca, se ad esempio abbiamo PC con A, B, C, D, E, con le seguenti relazioni: $A \rightarrow B$, $A \rightarrow C$, $A \rightarrow D$, $B \rightarrow D$, $C \rightarrow D$, $D \rightarrow E$, $C \rightarrow E$, mentre abbiamo come elementi di PC^+ le seguenti relazioni: $A \rightarrow B$, $A \rightarrow C$, $A \rightarrow D$, $B \rightarrow D$, $C \rightarrow D$, $D \rightarrow E$, $C \rightarrow E$, $A \rightarrow E$, $B \rightarrow E$.

Il diagramma di Hasse sarà il seguente: $A \rightarrow B$, $A \rightarrow C$, $B \rightarrow D$, $C \rightarrow D$, $D \rightarrow E$, le altre relazioni ($A \rightarrow D$, $C \rightarrow E$) vengono rimosse perché A può raggiungere D, mentre C può raggiungere E attraverso altri percorsi non rimovibili (*non ridondanti*).



Nella figura sopra vediamo la differenza tra PC , PC^+ e R^+ .

1.3 Processo e thread

Il grafo in figura sotto è il modello di un'esecuzione concorrente.

Da notare che è anche un diagramma di Hasse perché ogni nodo può raggiungere un altro nodo tramite percorsi che non possono essere rimossi (*perché un nodo può raggiungere un altro nodo solo tramite quel percorso*).

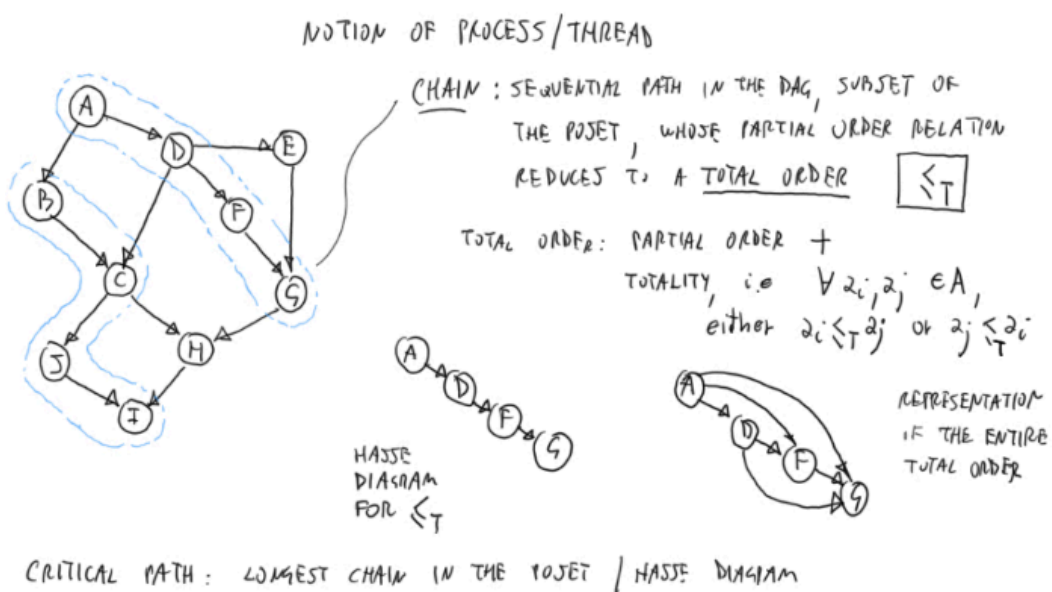
1.3.1 Catena

Una **catena** è un sottoinsieme di elementi del POSET in cui **ogni coppia di elementi è comparabile** rispetto alla relazione d'ordine.

In altre parole, dato un sottoinsieme $C \subseteq P$ esso è una catena se $\forall x, y \in C, x \leq y \vee y \leq x$.

Ovvero la relazione di ordine parziale di C si riduce ad un **total order** ($\leq_T$).

Prendiamo come esempio il percorso $A \rightarrow D \rightarrow F \rightarrow G$.



Esso rappresenta la sequenza di azioni che devono essere eseguite per arrivare a G partendo da A .

L'esecuzione di processo/thread seguirà sempre una catena.

Da notare che anche l'azione E è necessaria per arrivare a G : ma non sappiamo quale worker la eseguirà, ma fino a che E non sarà completata, il nostro worker non potrà eseguire G .

Quindi ci sarà per forza un altro processo/thread che invece eseguirà l'azione E .

Se osserviamo invece la sequenza $B \rightarrow C \rightarrow J \rightarrow I$, questa come la sequenza precedente è anch'essa una **catena**.

Quindi il diagramma di Hasse della prima catena è rappresentato dalle seguenti relazioni: $A \rightarrow D$, $D \rightarrow F$, $F \rightarrow G$, mentre la rappresentazione del total order è la seguente: $A \rightarrow D$, $A \rightarrow F$, $A \rightarrow G$, $D \rightarrow F$, $D \rightarrow G$, $F \rightarrow G$ (include gli archi da una sorgente ad una destinazione raggiungibile da quella sorgente non solo in modo diretto).

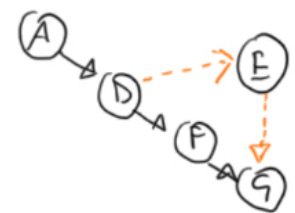
1.4 Esecuzione del DAG

Usando le nozioni di catena e DAG possiamo creare un modello matematico per rappresentare un processo o un thread.

L'esecuzione dell'intero DAG può essere portata avanti usando possibilmente molteplici processi.

1.4.1 Percorso critico e sincronizzazione

Come possiamo utilizzare questo modello per trovare alcune proprietà di questa computazione? La catena più lunga possibile nel DAG ci dà un'idea su come ogni azione è associata al tempo di esecuzione più corto che necessita la nostra computazione. La catena più lunga in R^+ è detta **percorso critico**.



Consideriamo la catena $A \rightarrow D \rightarrow F \rightarrow G$ e anche la catena $D \rightarrow E \rightarrow G$.

- Se vogliamo eseguire G dobbiamo attendere che E completi.
- Se vogliamo eseguire E dobbiamo attendere che D completi, se vogliamo eseguire G dobbiamo attendere che E completi ed E è esterna alla catena.

Se consideriamo le frecce che escono dalla catena o che entrano nella catena, queste indicano azioni che devono essere eseguite per il completamento del processo, dobbiamo tenere conto quindi anche della sincronizzazione.

La sincronizzazione è il meccanismo che forza vincoli di precedenza ad elementi che non appartengono alla catena.

Dentro una catena la sincronizzazione non è un problema, per la sua definizione le attività all'interno della catena sono sequenziali.

Posso pensare di mettere E in mezzo a D e F o tra F e G nel caso in cui abbiamo un solo worker. Senza eseguire E non posso eseguire G.

Nel caso in cui avessimo **un solo** worker, c'è la possibilità di inserire E all'interno della catena (*rendiamo tutto sequenziale*).

Per fare questa cosa dobbiamo posizionare tutte le attività in un total order, continuando però a soddisfare l'ordine parziale.

Questa operazione è detta topological sorting (*oppure linear extension*) e viene risolta tramite l'algoritmo di Kahn.

Dati a_i, a_j se $a_i \leq_p a_j$ allora $a_j \leq_T a_i$.

1.4.2 Algoritmo di Kahn

Abbiamo un modello con un singolo worker e tutte le attività devono essere riorganizzate in un percorso sequenziale.

In questo modo lo worker riesce ad eseguire tutte le attività soddisfacendo l'ordine corretto (*questo processo è detto topological sorting*).

Il problema quindi è “*riordinare un grafo in modo da ottenere una singola catena che soddisfi comunque i vincoli di precedenza del grafo originale*”.

Dividiamo i nodi in tre sottoinsiemi:

1. **SN - Start Nodes**: L'insieme di tutti i nodi senza archi entranti.
2. **S - Solution List**: Inizialmente vuota, verrà riempita dall'algoritmo un nodo alla volta.
3. **M - Middle**: Tutti gli altri nodi.

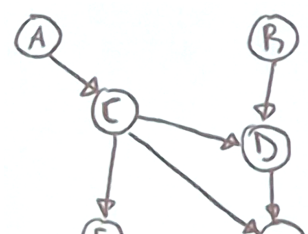
Step dopo step, sposto i nodi tra i diversi set popolando S. Alla fine avrò tutti i nodi nel sottoinsieme S lasciando SN e M vuoti.

```
While SN ≠ ∅
    Pick - Sia n ∈ SN chiamato "Current Node"
    Append - Appendiamo n alla lista S
    ∀ m ∈ M raggiungibile da n tramite un arco r:
        Remove - Si rimuove r dal grafo
        IF - m non ha altri archi entranti:
            Append - m viene tolto da M e messo in SN
    IF - M ≠ ∅:
        Non era un DAG → Errore
    ELSE:
        return S
```

Esempio

All'inizio:

- $SN = \{A, B\}$



- $S = \{\}$
- $M = \{C, D, E, F, G\}$

Le attività all'interno di SN sono eseguibili in maniera concorrente.

1. Prendiamo un nodo di SN (A).
2. Ora dobbiamo considerare tutti gli archi uscenti da A : in questo caso uno che punta a C .

I tre vettori a questo punto saranno:

- $SN = \{B, C\}$
- $S = \{A\}$
- $M = \{D, E, F, G\}$

3. Adesso selezioniamo un altro nodo di SN (C).
4. Ora dobbiamo considerare tutti gli archi uscenti da C , in questo caso tre che puntano a D, E, F .

I tre vettori a questo punto saranno:

- $SN = \{B, E\}$
- $S = \{A, C\}$
- $M = \{D, F, G\}$

Continuiamo:

5. Rimuovo l'arco che parte da C verso D
6. D ha ancora l'arco entrante proveniente da B .
7. Rimuovo l'arco che parte da C verso F
8. F ha ancora l'arco entrante proveniente da D .
9. Rimuovo l'arco che parte da C verso E
10. E non ha altri archi entranti, può andare in SN .
11. Adesso selezioniamo un altro nodo di SN (B).
12. Ora dobbiamo considerare tutti gli archi uscenti da B , in questo caso uno che punta a D .
13. D non ha altri archi entranti, può andare in SN .

I tre vettori a questo punto saranno:

- $SN = \{E, D\}$
- $S = \{A, C, B\}$
- $M = \{F, G\}$

Continuiamo:

14. Adesso selezioniamo un altro nodo di $SN(D)$.

15. Ora dobbiamo considerare tutti gli archi uscenti da D , in questo caso uno che punta a F .

16. F non ha altri archi entranti, può andare in SN .

I tre vettori a questo punto saranno:

- $SN = \{E, F\}$
- $S = \{A, C, B, D\}$
- $M = \{G\}$

Continuiamo:

17. Adesso selezioniamo un altro nodo di $SN(F)$.

18. Ora dobbiamo considerare tutti gli archi uscenti da F , in questo caso uno che punta a G .

I tre vettori a questo punto saranno:

- $SN = \{E\}$
- $S = \{A, C, B, D, F\}$
- $M = \{G\}$

Continuiamo:

19. Rimuovo l'arco da G , ha ancora l'arco entrante da E .

20. Adesso selezioniamo l'unico nodo rimasto in $SN(E)$.

21. Ora dobbiamo considerare tutti gli archi uscenti da E , in questo caso uno che punta a G .

I tre vettori a questo punto saranno:

- $SN = \{G\}$
- $S = \{A, C, B, D, F, E\}$
- $M = \{\}$

Continuiamo:

22. Adesso selezioniamo l'ultimo nodo G

I tre vettori a questo punto saranno:

- $SN = \{\}$
- $S = \{A, C, B, D, F, E, G\}$
- $M = \{\}$

Infine:

23. I subset SN e M sono vuoti, l'algoritmo termina correttamente (*non ci sono archi rimasti*), la soluzione S è un **total order**.

$A \rightarrow C \rightarrow B \rightarrow D \rightarrow F \rightarrow E \rightarrow G$

Notiamo che ci sono delle frecce speciali (**arancioni**) che non sono presenti nel partial order originale, quindi quando otteniamo il total order dal partial order abbiamo forzato alcuni vincoli spuri, questo perché abbiamo un singolo worker.

Se abbiamo un solo worker è necessario molto tempo per completare l'intera computazione, nel caso in cui si abbiano più worker otteniamo benefici nel tempo di esecuzione.

Riorganizziamo le attività in un modo che possano essere eseguite in parallelo esattamente allo stesso tempo.

1.5 Benefici della parallelizzazione e legge di Amdahl

Indichiamo con P il **programma originale** nella forma base (ovvero *sequenziale*), mentre indichiamo con P' , la **versione parallelizzata** del programma (*si spera migliorata*).

Proviamo a confrontare P e P' utilizzando lo **stesso input**, indichiamo con $T(P)$ il tempo di esecuzione di P e con $T(P')$ il tempo di esecuzione di P' .

Quello che vogliamo raggiungere è: $T(P') \leq T(P)$ ovvero che la versione parallelizzata risulti più performante.

Indichiamo con il termine di **speed-up** il rapporto tra $T(P)$ e $T(P')$ che indichiamo con

$$\sigma = \frac{T(P)}{T(P')}$$

$$T(P) = s + p$$

- s il tempo per svolgere la parte puramente sequenziale
- p il tempo per eseguire la parte parallelizzabile.

Se abbiamo n unità computazionali, p può essere ridotto in $\frac{p}{n}$ poiché parallelizziamo la parte parallelizzabile dividendola in n unità di esecuzione.

Indichiamo invece la **sequentia fraction**:

$$\alpha = \frac{s}{s + p}$$

Più α è grande e meno è importante la parte parallelizzabile e quindi meno vantaggi ottengo dalla parallelizzazione.

Questo avviene nei casi ideali, nella realtà questo difficilmente viene raggiunto perché molto spesso le unità computazionali richiedono sincronizzazione e scambio di informazioni, è un lavoro addizionale che comporta un certo overhead computazionale.

Quindi lo speed-up può essere espresso come:

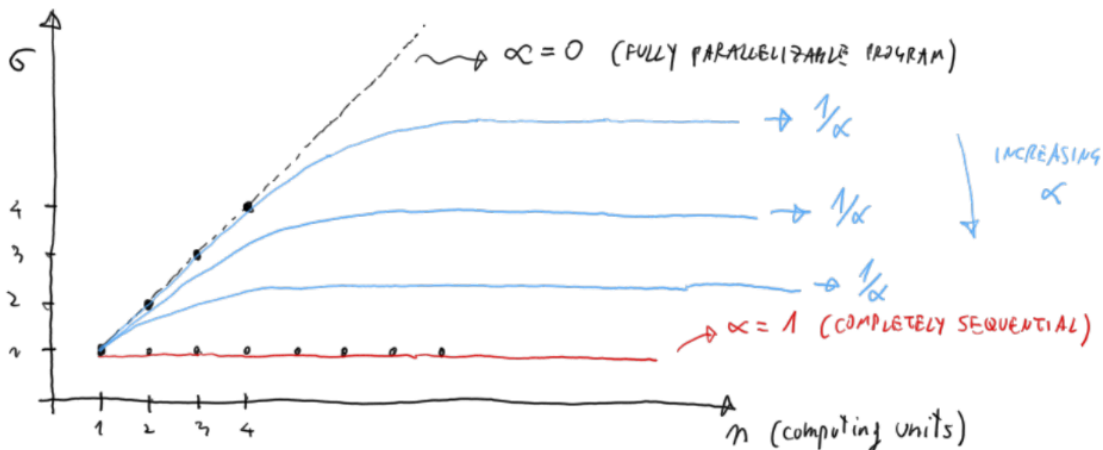
$$\sigma(n, \alpha) = \frac{1}{\alpha + \frac{1-\alpha}{n}}$$

Concorrenza, legge di Amdahl, grafico al variare di n . Quali sono i casi in cui all'aumentare di n la curva diminuisce?

Se facciamo il limite per $n \rightarrow \infty$ della funzione $\sigma(n, \alpha)$ otteniamo quello che si chiama **legge di Amdahl** ed il risultato è $\frac{1}{\alpha} = \frac{s+p}{s}$.

Nel grafico vediamo come cambiano le cose all'aumentare di α : la versione parallelizzata con due unità computazionali impiega metà del tempo della versione originale. Otteniamo una linea retta che tende ad $\alpha = 0$ (tende ad un programma completamente parallelizzabile).

Mentre nel caso di $\alpha = 1$ non abbiamo nessun vantaggio, è il caso completamente sequenziale.



Le **linee azzurre** nel grafico valgono $\frac{1}{\alpha}$, più α cresce, più la linea si avvicina alla soluzione completamente sequenziale.

Tipicamente abbiamo un trend inferiore rispetto alla linea retta teorica, quindi abbiamo una linea che cresce però non così velocemente come nel caso ideale. Aumentando il numero di unità computazionali, ad un certo punto le prestazioni decrescono perché aumenta il contributo della coordinazione tra le varie unità (*overhead*).

I punti possono anche essere sopra la linea retta, però ad un certo punto, anche in questo caso le prestazioni decadono, questo caso è detto **comportamento superlineare**.

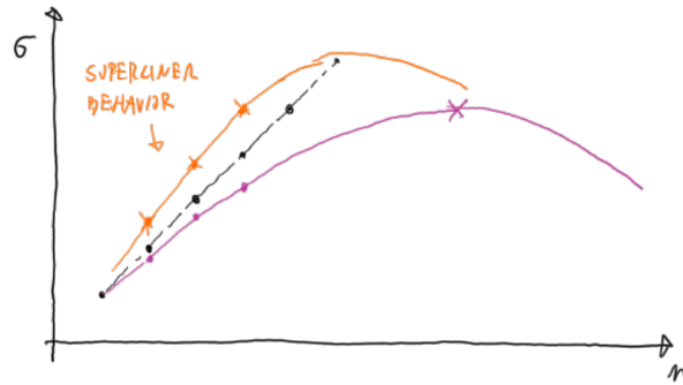
Questo avviene perché i nostri sistemi sono più complessi di quello che ci aspettiamo, la presenza della memoria cache può determinare strani effetti.

Quando definiamo un'applicazione dobbiamo definire il working set, se questo entra nella cache, ogni volta che il programma necessita un valore appartenente al working set, ci

accederà immediatamente, se questo non avviene perché il working set è troppo grande, si ha una cache miss che richiede molto più tempo.

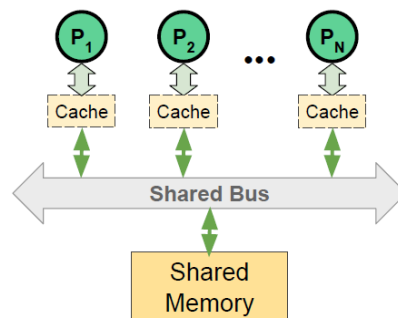
Ad esempio se dovessimo considerare la cache e altri dispositivi hardware ormai installati in tutti i sistemi le cose si complicano.

Quindi il grafico al crescere di n è più una cosa di questo tipo.

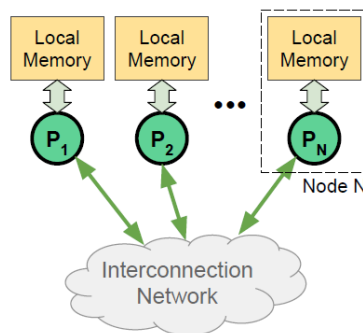


Capitolo 2 - Sistemi a scambio di messaggi

Non ci sono domande raccolte su questo capitolo.



Nei sistemi a memoria condivisa, la memoria è acceduta tramite un bus condiviso.



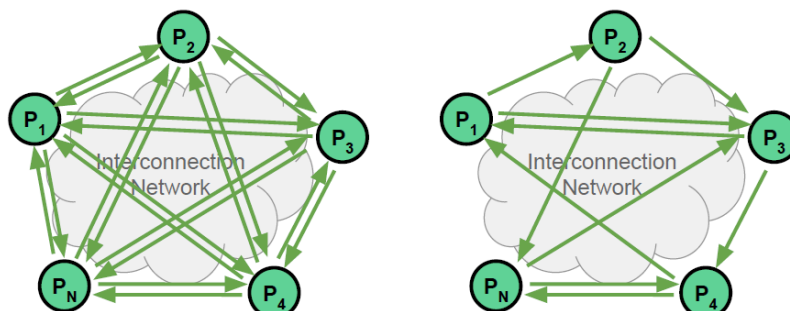
Mentre nel caso di un sistema basato su scambio di messaggi ogni processo lavora con una propria memoria locale e comunica con gli altri attraverso una rete di interconnessione.

2.1 Rete di interconnessione

I canali di interconnessione possono essere:

- **Fully connected (sinistra)**: ogni processo è connesso con tutti gli altri processi.
- **Partially connected (destra)**: non c'è ogni possibile percorso tra ogni processo, ma solo alcuni.

Il canale è un mezzo per comunicare e può essere **unidirezionale** oppure **bidirezionale**, inoltre un canale potrebbe essere FIFO o non-FIFO relativamente ai messaggi che vengono inviati.



Il networking è il veicolo con cui questi sistemi distribuiti si scambiano messaggi, quindi in questo corso non verranno esplorati gli argomenti di networking.

Le informazioni vengono suddivise in diverse parti, ciascuna delle quali viene inviata ad un'unità come messaggio, il messaggio è composto dal **payload** più **informazioni aggiuntive**. Il payload è il contenuto del messaggio, il resto è il necessario per inviare il messaggio.

Si può lavorare con diversi tipi di modelli in modo da caratterizzare diversi aspetti dei sistemi target a diversi livelli di astrazione.

I modelli possono essere:

- **Fisici:** Rappresentazione dell'hardware sottostante di un sistema distribuito, astruendo i dettagli tecnologici dei dispositivi di computazione e di networking.
- **Architetturali:** Si concentrano sulla struttura in termini di componenti e loro relazioni (ad esempio client-server, peer-to-peer) - utilizzo di middleware.
- **Fondamentali:** Tengono in conto le proprietà fondamentali delle caratteristiche di base del sistema, nonché dei suoi potenziali guasti.

Modelli fondamentali

Contengono solo gli elementi essenziali per comprendere e ragionare su alcuni aspetti del comportamento di un sistema.

Lo scopo di questi modelli è:

- Rendere esplicite tutte le ipotesi rilevanti sul sistema modellato.
- Fare generalizzazioni su cosa è possibile o impossibile, sotto tali ipotesi.
- Generalizzazioni: algoritmi di uso generale o proprietà desiderabili e garantite.

Le garanzie dipendono dall'analisi logica e, possibilmente, dalla dimostrazione matematica. Gli aspetti comportamentali di base sono: interazione, guasti, sicurezza.

Un messaggio viene inviato dalla memoria locale di un processo alla memoria locale di un altro processo.

Assieme ai messaggi vengono inviate anche informazioni aggiuntive:

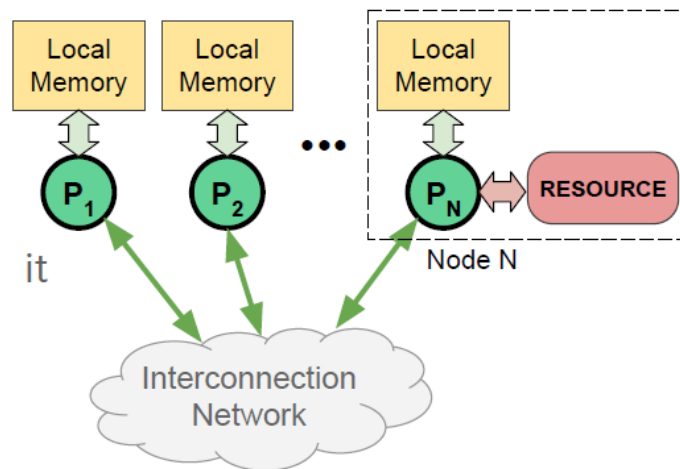
- Ulteriore semantica, come tipi di messaggio o tag allegati.
- Metadati (*che tipo di cose nel payload? Come interpretarlo?*)
- Chi è il mittente? (*può essere importante perché chi riceve il messaggio, senza l'identificatore del mittente, non può inviargli la risposta*).

Bisogna trovare un modo per esprimere un indirizzo da associare ad un mittente/destinatario.

Gestione delle risorse

In un sistema di passaggio di messaggi, una risorsa può essere associata a un processo in grado di interagire con essa. Se la risorsa è destinata a essere condivisa tra i processi, il processo associato ad essa deve comportarsi come il suo **manager**.

Qualsiasi accesso alla risorsa deve essere mediato dalla comunicazione con il relativo **manager**.



Process addressing

Un processo in un'applicazione distribuita deve essere identificato in modo univoco ai fini dell'applicazione.

Non è un problema relativo al networking, non dobbiamo identificare una macchina in un certo modo, possiamo identificare un processo come vogliamo.

Inoltre, deve anche essere trovato, in modo che i messaggi possano essere inviati.

L'uso di un'infrastruttura di comunicazione (*sia concreta che astratta*) richiede un sistema di indirizzamento.

Spesso, ID specifici sono sufficienti come indirizzi logici, ma devono essere in qualche modo mappati sugli indirizzi reali utilizzati dalla rete di comunicazione sottostante.

Overlay networks

Astrazione, è un modo logico per gestire come vari componenti devono essere connessi.

Consiste nella definizione a livello applicativo di una rete di comunicazione sopra una reale. Il risultato è detto **Overlay network**.

Esempi di overlay network:

- Un sistema client-server
- Applicazioni Peer-to-peer, come Skype

Il sistema di indirizzamento può essere sia **implicito** che **esplicito**. Implicito significa che a volte un servizio può essere fornito come destinazione, senza indicare il processo effettivo che eseguirà il job.

Messaggi diversi possono essere inviati a un singolo processo per molte ragioni diverse.

Per facilitarne la gestione, più endpoint di comunicazione possono essere associati a un processo/nodo.

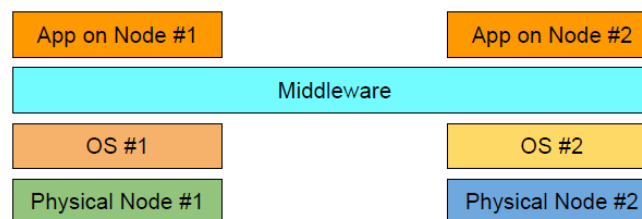
Un endpoint di comunicazione può essere specificato dall'indirizzo del processo/nodo, più un altro identificatore noto come porta.

Una **porta** indica un access point per i messaggi di un certo tipo, quando dobbiamo inviare un messaggio bisogna specificare non solo l'indirizzo, ma anche la porta, quindi le porte specificano la semantica di messaggio rendendone più semplice la gestione.

Quando inviamo dei messaggi ci sono vari modi di comunicare, una comunicazione può essere **broadcast**, **multicast** oppure se i processi vengono suddivisi in gruppi, questi cambiano dinamicamente e bisogna tenerne conto.

2.2 Middleware

L'implementazione di costrutti di comunicazione in genere nasconde i dettagli di rete: questo può essere fatto usando un software specializzato detto **Middleware**, che fornisce i servizi di comunicazione richiesti.



Ogni macchina ha il suo OS, sopra i OS troviamo il middleware che implementa la comunicazione e la sincronizzazione tra processi che troviamo in un sistema eterogeneo (*architetture diverse e sistemi operativi diversi*).

Ci sono due modi per organizzare il middleware e quindi fornire servizi di comunicazione, tramite **librerie** oppure tramite **supporto runtime**, in ogni nodo abbiamo qualcosa in esecuzione che interrompe il processo se deve intervenire per certe funzionalità.

L'utilizzo di componenti middleware (*standard*) è in genere preferibile alla scrittura di codice personalizzato, perché è solitamente la soluzione più conveniente per supportare l'integrazione di parti software, in particolare quelle distribuite.

2.2.1 Semantica di Send/Receive

Abbiamo due tipi di modelli, **sincrono** e **asincrono**, corrispondono a come le operazioni in processi differenti devono essere eseguite le une rispetto alle altre.

Nel caso sincrono i processi avanzano in modo simultaneo, l'esecuzione procede in round sincroni. Modello semplice ma non è realistico.

Invece nel caso asincrono non serve fare assunzioni riguardo il tempo di esecuzione dei nodi e ognuno può andare secondo la propria velocità, i ritardi sono illimitati.

In questo caso la principale caratteristica è la **mancanza di un clock globale** che ci permette di tenere traccia della nostra applicazione.

La send e la receive possono avere quindi due comportamenti possibili:

- **Bloccante:** L'esecuzione si blocca fino a che l'interlocutore non ha inviato/ricevuto il messaggio.
- **Non bloccante:** l'operazione viene attivata e il processo continua ad essere eseguito, richiede il buffering dei messaggi sia per inviarli che per riceverli.

In entrambi i casi si richiede l'uso di code di ricezione e di invio (*chiamate anche "mailboxes"*), ovvero dei buffer dove tenere i messaggi in uscita e in entrata.

Primitiva Send Bloccante

Il messaggio è inviato a destinazione, il processo mittente può riprendere l'esecuzione solo dopo che ha ricevuto la risposta di avvenuta ricezione del destinatario.

Questa primitiva contiene un handshake, in cui si crea una connessione.

Primitiva Send Non Bloccante

Il messaggio è inviato alla destinazione e il processo mittente può subito andare avanti con l'esecuzione delle istruzioni successive.

Primitiva Receive Bloccante

Il destinatario si mette in ascolto e aspetta l'arrivo di un messaggio. Se ci sono messaggi nel buffer, rimuoverà uno di essi dal buffer. Altrimenti si blocca in attesa che un messaggio sia disponibile su un endpoint.

Primitiva Receive Non Bloccante

Il processo controlla se un messaggio in entrata è disponibile e possibilmente lo legge. Restituisce indicazioni sul successo/fallimento.

Rischio di deadlock

Bisogna stare attenti nell'utilizzo di una specifica semantica nelle coppie blocking/non-blocking perché potrebbe portare a deadlock.

Il tempo di attesa per un messaggio non è infinito, dipende dal valore del timeout, se il tempo viene superato, allora si ha una condizione di errore.

Uno dei costrutti per l'operazione receive è il *guarded command*, ovvero specifichiamo il guard e possibilmente eseguiamo un blocco di istruzioni se il guard è soddisfatto.

Esempio Bounded Buffer Problem

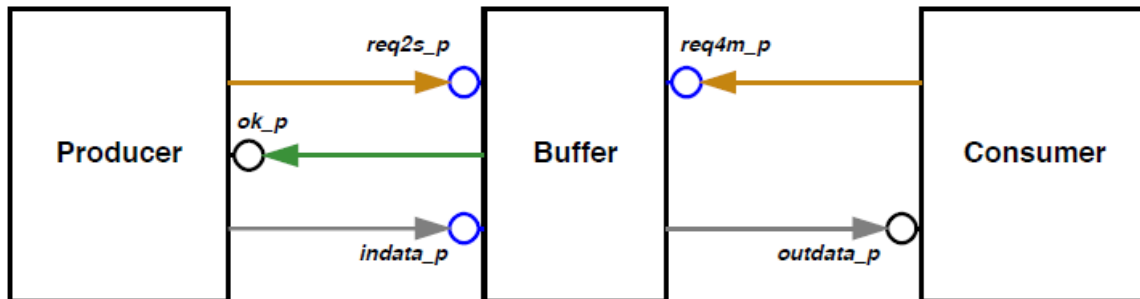
Abbiamo un produttore ed un consumatore che scambiano dei dati, il produttore incapsula i messaggi e li inoltra ad un terzo processo, detto buffer, il buffer ha una quantità di spazio limitata.

Quando il buffer è pieno, il produttore non può inserire nuovi dati, quando è vuoto, il consumatore non può prelevarne di nuovi.

Il produttore invia il messaggio `req2s_p` e si blocca finché il buffer non risponde con `ok_p`, quando il produttore riceve questo messaggio, può inviare i dati al buffer (`indata_p`).

Mentre il consumatore, se vuole leggere dei dati dal buffer, invia il messaggio `req4m_p`, il consumatore rimane in attesa finché il buffer non gli invia i dati (`outdata_p`).

La struttura dati reale in cui i dati sono memorizzati esiste già, non c'è bisogno di implementare all'interno del buffer.



Capitolo 3 - Erlang

Si tratta di un linguaggio a programmazione funzionale.

In che senso funzionale?

Un paradigma di programmazione è un modo di programmare: influenza la struttura, le prestazioni di un programma e la nostra capacità di ragionare su di esso e sui problemi che risolve.

Ad esempio programmare in Basic e Java è completamente differente, quindi un paradigma di programmazione è un modo di programmare, consideriamo due paradigmi: **imperativo** e **funzionale**.

Nel primo caso abbiamo una sequenza di comandi che guidano il flusso di controllo, mentre nel secondo non si ha nessuna mutazione di stato, la computazione riguarda come valutare espressioni/funzioni.

Nella programmazione imperativa, la computazione è intesa come una sequenza di comandi che operano sulle informazioni di stato. L'effetto dell'esecuzione di un comando (*istruzione*) è una modifica allo stato del programma.

L'adozione di determinate regole di organizzazione del programma ha portato a paradigmi più specifici: "Strutturato", "Procedurale", "Orientato agli oggetti", ecc.

Consideriamo un semplice esempio di programmazione imperativa (Python) come questo:

```
if x > 0 :  
    result = 15/3  
else :  
    result = 2*3  
result = result - 1
```

3.1 Programmazione funzionale

Una delle caratteristiche tipiche della programmazione funzionale è la **mancanza di stato** durante il calcolo.

Mancanza di uno stato

Si riferisce al fatto che le operazioni non dipendono da uno stato mutabile o condiviso.

Questo concetto è fondamentale nella programmazione funzionale, dove le funzioni sono **pure** e lavorano esclusivamente sui loro input, senza modificare variabili o strutture di dati esterne.

Espressioni e sottoespressioni

Un calcolo è una sequenza di espressioni risultanti dalla valutazione (e *successiva sostituzione*) di sottoespressioni.

Lo stesso esempio di prima in programmazione funzionale è il seguente:

```
(15/3 if x > 0 else 2*3) - 1
=> (5 if 7 > 0 else 6) - 1
=> 5 - 1
=> 4
```

Appare strano dal punto di vista di chi è abituato alla programmazione imperativa, infatti in questo caso la programmazione è basata su **valutazione + sostituzione**.

Per eseguire questo programma dobbiamo andare avanti step dopo step valutando ogni sottoespressione e sostituendo ogni sottoespressione con il suo valore corrispondente.

Sostituendo ogni sottoespressione con il suo valore, otteniamo un'espressione semplificata ad ogni step, alla fine otteniamo il risultato.

Molti linguaggi adottano questo paradigma, alcuni sono molto vicini a questo paradigma, altri prendono solo alcuni aspetti. Quelli più vicini alla programmazione funzionale sono Haskell e Scala, mentre altri linguaggi tipo JavaScript, Java, Python e C# utilizzano alcuni principi.

Molti concetti popolari nella programmazione funzionale **non sono presenti** nella programmazione imperativa:

- Integrità referenziale.
- Uso massiccio della ricorsione.
- **Referential Transparency** e Funzioni pure.
- **Lack of State** - Mancanza di uno stato.
- Lazy e Eager Evaluation.
- Funzioni di prima classe e di ordine superiore.
- Ruolo primario della ricorsione.

Referential Transparency

I calcoli nella programmazione funzionale sono visti come valutazione di espressioni.

Un'espressione con questa proprietà può essere sostituita con il suo valore senza comportare effetti collaterali nella computazione del risultato.

Una funzione è detta **referenzialmente trasparente** se rimpiazzarla col suo valore finale non provoca effetti collaterali nel comportamento del programma.

Una funzione è detta **pura** se le sue chiamate sono tutte referenzialmente trasparenti, quindi non hanno effetti collaterali.

Ovvero nell'esecuzione della funzione non vengono toccate altre cose esterne, quindi eseguirla 1 volta o 100 volte non cambia nulla.

Funzione pura - Memoization

Se una funzione è pura può essere soggetta a **memoization**.

Ovvero se abbiamo una funzione $f()$ con input che vale 10, se eseguiamo la funzione, vediamo che impiega 10 minuti.

Se successivamente riproviamo ad eseguire $f(10)$ otterremo lo stesso risultato nello stesso tempo, invece di eseguire la stessa funzione sugli stessi dati, tutte le volte che dobbiamo eseguirla nel programma, salviamo il risultato in un file e utilizziamo quel risultato la prossima volta che dovremmo eseguire quella funzione (risparmiamo il tempo di esecuzione).

Lack of state

Una variabile nel linguaggio funzionale è immutabile: una volta che una variabile assume un valore questo non cambierà mai.

Possiamo creare nuove variabili, ma non modificare quelle già esistenti.

Infatti serve un **garbage collector** che si occupi di gestire oggetti non utilizzati per liberare la memoria.

Eager evaluation e lazy evaluation

Nei calcoli, la valutazione delle espressioni (*principalmente per gli argomenti delle funzioni*) può essere eseguita, in diversi linguaggi/situazioni, secondo diverse strategie:

- Una funzione con **Eager evaluation** è valutata subito appena la si incontra ed è solitamente utilizzata nel passaggio per valore e per riferimento degli argomenti nelle funzioni.
- Una funzione con **Lazy evaluation** è valutata dopo, solo quando i valori che produce sono effettivamente richiesti.

Questo tipo di valutazione è utile perchè effettuare immediatamente delle computazioni potrebbe rallentare l'esecuzione, quindi questa tecnica consente di migliorare le prestazioni in certe situazioni.

First-class e Higher-order Functions

Le funzioni sono trattate come ogni altro valore, quindi, possono essere passate/restituite ad/da altre funzioni.

Inoltre è possibile avere una funzione che viene restituita come risultato da un'altra funzione (*le funzioni sono dette "first-class citizens"*).

Poiché una funzione restituita può dipendere da parametri effettivi della funzione che l'ha generata, tali valori devono essere mantenuti in un ambiente speciale detto *closure*.

Le **funzioni di ordine superiore** sono quelle che possono accettare funzioni come argomenti e possono restituire funzioni come risultato.

3.1.1 Uso della ricorsione

La ricorsione è a volte l'unico modo per portare avanti certi tipi di computazioni. Ciò che può essere fatto con l'iterazione può essere fatto anche con la ricorsione e viceversa.

Non potendo usare delle variabili a cui è possibile cambiare valore, l'unico modo per ripetere gli step richiesti è utilizzare la ricorsione.

Nell'esempio sotto è il caso di una tail recursion, ovvero la ricorsione è l'ultima operazione.

```
def it_fact(n):  
    res = 1 #base case  
    while n>1:  
        res *= n  
        n -= 1  
    return res
```



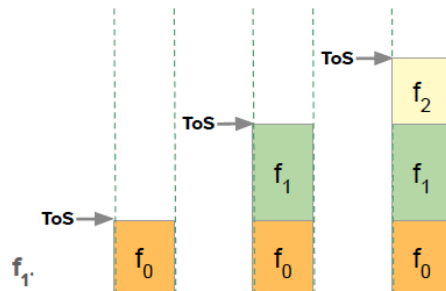
```
def rc_fact(n, res=1):  
    #res acts as an "accumulator"  
    if n <= 1:  
        return res #base case  
    return rc_fact(n-1, res*n)
```

Tail call optimization

Il problema della ricorsione è che consumiamo molta memoria, c'è un modo per utilizzare la ricorsione senza utilizzare troppa memoria, ovvero quello di richiamare la funzione come ultima operazione (tail recursion).

La tail recursion può essere gestita in modo efficiente adottando il **tail call optimization** per evitare una crescita eccessiva dello stack delle chiamate.

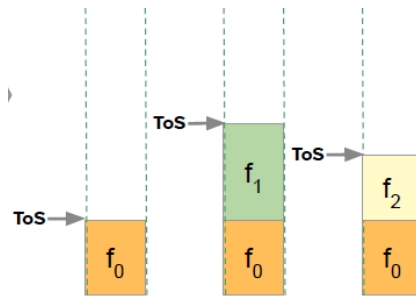
Supponiamo di dover chiamare tre funzioni $f_0 \rightarrow f_1 \rightarrow f_2$. La situazione che si genererà in memoria sarà la seguente:



Per ottimizzare ri-organizziamo le chiamate delle funzioni:

- f_1 sia chiamata all'interno di f_0 .
- f_2 sia l'ultima operazione di f_1 .

In questo caso possiamo sostituire il frame di f_1 con il frame di f_2 , quindi come vediamo in figura sotto abbiamo un risparmio di memoria.



Ogni volta che l'ultima operazione eseguita all'interno di una chiamata di funzione è un'altra chiamata di funzione, possiamo sostituire il frame del chiamante con il frame del chiamato, senza bisogno di mantenere entrambi!

In Python questo trick non viene mai utilizzato perchè se ad un certo punto vogliamo sospendere l'esecuzione e vedere lo stato dello stack, se utilizzassimo questa tecnica, lo stato dello stack relativo ad ogni passaggio andrebbe perso.

Lavorando con strutture dati che sono ricorsive loro stesse è più facile utilizzare la ricorsione, se vogliamo utilizzare la ricorsione, dobbiamo simulare la struttura ricorsiva adottata per memorizzare ogni singolo dato.

Se utilizziamo funzioni ricorsive non abbiamo effetti collaterali, quindi non dobbiamo utilizzare gli array perchè mantengono le informazioni sullo stato.

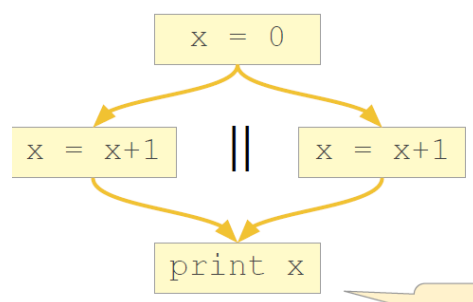
I dati sono immutabili, però trick di basso livello consentono il riutilizzo dei dati.

L'immutabilità è presente anche in Java, le stringhe sono immutabili.

Le liste sono strutture dati ricorsive, hanno elementi composti da dati e un puntatore al prossimo elemento, può essere gestita ricorsivamente in questo modo, il primo elemento è la testa, mentre gli altri elementi costituiscono una lista.

3.1.2 Programmazione Funzionale e Concorrenza nell'accesso ai dati

Il problema principale proviene dalle *data races*.



In figura la variabile x , il cui valore iniziale è 0, è acceduta da due processi contemporaneamente che effettuano la stessa operazione (*incrementano* x).

In base al tempo relativo di esecuzione delle due operazioni concorrenti, il risultato cambierà.

Se queste operazioni avvengono esattamente allo stesso tempo, il processo di sinistra legge x che vale 0 e lo incrementa, allo stesso tempo il processo a destra legge x che vale ancora 0 perchè ha letto il valore allo stesso tempo dell'altro processo, quindi incrementa 0 ed il risultato è 1.

Mentre se il processo a destra è un pò più lento a leggere il valore, leggerà 1 e incrementando, il valore ottenuto è 2. Il valore ottenuto non è deterministico, dipende dal tempo relativo.

Il problema è correlato alla mutabilità dei dati.

Come risolvere questo problema?

Questo problema viene risolto alla radice rendendo i dati immutabili (come appunto nella programmazione funzionale), quando abbiamo stati mutabili dobbiamo utilizzare la sincronizzazione che limita l'utilizzo delle risorse.

Nei linguaggi imperativi, i calcoli simultanei devono essere specificati esplicitamente. Nella programmazione funzionale pura: nessuno stato, nessun effetto collaterale.

Nessuno stato mutabile condiviso → Nessun problema nell'accesso simultaneo → Calcoli per forza thread-safe (nessuna critical race)

“Il problema è che tutte le applicazioni concorrenti comportano mutazioni controllate e deliberate dello stato condiviso”.

3.2 Le basi di Erlang

Il linguaggio Erlang è costruito secondo la visione della programmazione funzionale.

Adotta questa visione di concorrenza, con i "processi" che scambiano messaggi e reagiscono alle ricezioni di messaggi.

Scriviamo il programma con una certa struttura e in base a come mappiamo i processi in core locali o CPU distribuite, avremo un programma locale o un programma distribuito, non possiamo avere entrambi contemporaneamente.

Utilizzato per applicazioni simultanee con parallelismo fine-grained come:

- Server di rete, sistemi distribuiti.
- Meccanismi middleware:
 - Database distribuiti.
 - Server di code di messaggi.
- Applicazioni soft in tempo reale; strumenti di monitoraggio, controllo e test.

La necessità di gestire applicazioni su larga scala ha portato a elixir.

ERTS - Erlang Run-Time System

Erlang è eseguito da un interprete **ERTS**, come accade in Python e Java, utilizza un intermediario (quindi è necessaria la compilazione).

ERTS ha delle strutture di **garbage collection** per gestire la memoria dinamica.

BEAM - Bogdan/Björn's Erlang Abstract Machine

Il bytecode (*prende questo nome perchè le istruzioni sono codificate in un byte*) viene eseguito in una virtual machine detta BEAM.

Il BEAM è la macchina virtuale su cui viene eseguito il codice Erlang. È il cuore del runtime di Erlang e si occupa di eseguire il codice compilato in un formato intermedio chiamato bytecode BEAM, garantendo efficienza, gestione della concorrenza, tolleranza ai guasti e altre caratteristiche chiave del linguaggio.

- Il codice Erlang viene compilato in **bytecode BEAM**.
- Il bytecode è caricato nella macchina virtuale BEAM.

I moduli sono compilati e caricati da BEAM, un BEAM è eseguito da un singolo processo del sistema operativo (*gli aspetti di concorrenza vengono gestiti internamente*).

Il BEAM si occupa anche della schedulazione dei processi Erlang concorrenti.

Un processo Erlang non ha niente a che fare con un processo/thread a livello del OS, ovvero un processo Erlang non per forza corrisponde ad un processo del OS.

3.2.1 Scheduler di Erlang

Infatti Erlang utilizza i propri scheduler (*uno per core della CPU*) per distribuire l'esecuzione dei processi Erlang sui thread di "*supporto*", ovvero i veri thread del sistema operativo (*uno per core*).

La pianificazione dei processi BEAM si basa su una coda di processi separata per scheduler.

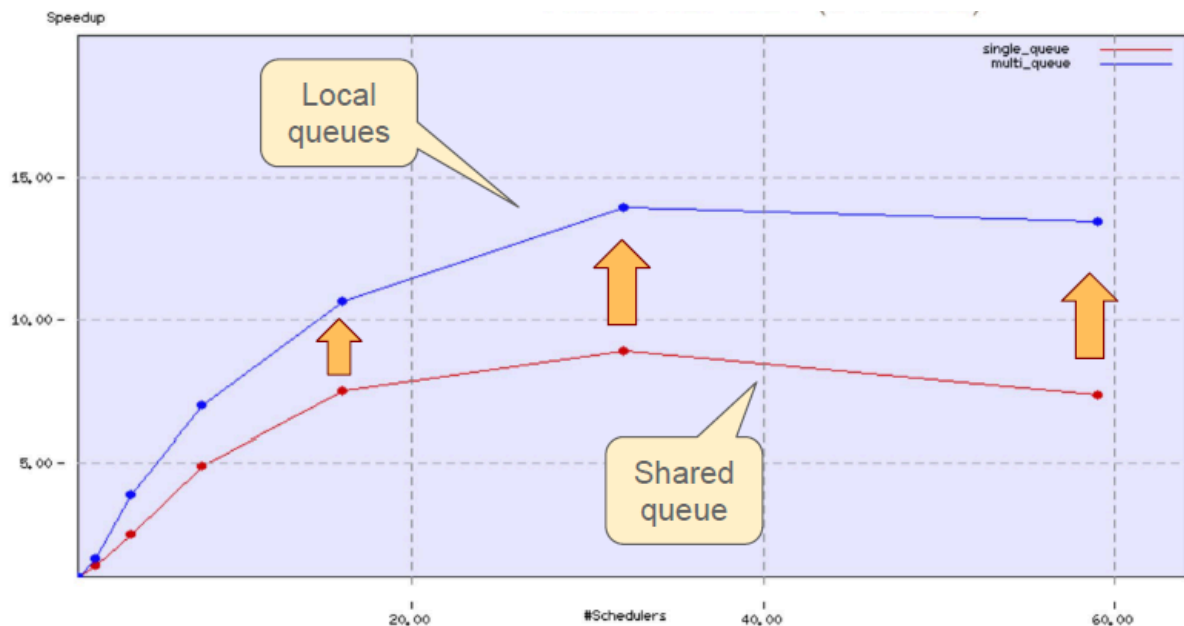
Scheduler differenti gestiscono processi Erlang differenti, in base al modo in cui questi scheduler sono organizzati, le prestazioni sono migliori.

Per migliorare il load balancing è possibile utilizzare lo **work stealing**.

Ad un certo punto una CPU finisce il lavoro e non ha nulla da fare, allora ruba il lavoro dalle altre CPU in modo da bilanciare il carico.

Local queue

Local queues significa che abbiamo una coda per ogni CPU, piuttosto che una singola coda in comune, local queues utilizza lo work stealing.



3.2.2 La shell di Erlang

Gli utenti possono interagire con il sistema tramite la shell Erlang (*erl*). La compilazione ed il caricamento vengono eseguiti dalla shell.

```
erl
```

```
1> c(hello).
```

```
2> hello:start().
```

I moduli possono essere compilati e sostituiti anche mentre l'applicazione è in esecuzione!

Shell ("*repl*"): le espressioni vengono valutate, ma le funzioni non possono essere definite.

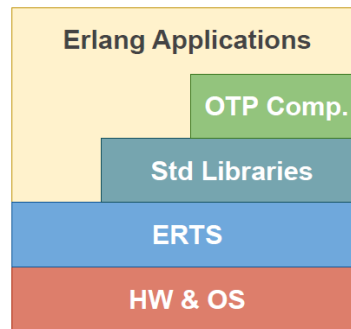
In Erlang, la shell può essere utilizzata anche per effettuare alcune operazioni come dei calcoli. Le funzioni devono essere scritte in moduli, e devono essere esportate da un modulo per poter essere utilizzate.

Le funzioni esportate dai moduli sono accessibili dopo il caricamento del modulo.

3.2.3 Applicazioni Erlang

Le applicazioni Erlang sono indipendenti dall'hardware e dal sistema operativo sottostante, quindi le porzioni distribuite possono interagire naturalmente.

I processi possono essere generati su nodi diversi e la comunicazione tra processi è trasparente per quanto riguarda il posizionamento sui nodi.



ERTS - Erlang Run-Time System

Include tutti i componenti necessari per eseguire codice Erlang.

ERTS è la base su cui si appoggia la macchina virtuale **BEAM** e fornisce le funzionalità essenziali per la gestione dei processi, la comunicazione, l'I/O, la gestione della memoria e altro ancora.

ERTS rende un programma Erlang **largamente indipendente dall'hardware sottostante**, grazie al fatto che il codice Erlang viene eseguito all'interno della macchina virtuale **BEAM**, che è parte integrante di ERTS.

OTP - Open Telecom Platform

Set di strumenti/componenti/librerie/pattern per potenziare lo sviluppo delle app Erlang.

OTP funziona da middleware per le applicazioni Erlang.

3.2.3 Struttura di un programma Erlang

Un programma Erlang è costituito da espressioni valutate una dopo l'altra.

Termine

L'espressione più semplice in Erlang è un **termine**, ovvero un pezzo di dati di qualsiasi tipo, che può essere scritto dal suo letterale.

La valutazione di un termine, restituisce il termine stesso.

Le espressioni possono essere costituite da sottoespressioni combinate da operatori: Tutte le sottoespressioni vengono valutate prima che un'espressione stessa venga valutata (Eager-evaluation), a meno che non sia esplicitamente dichiarato diversamente.

Variabile

Una **variabile** è un'espressione. Se una variabile è vincolata a un valore, la valutazione della variabile è il valore stesso.

Un'espressione nella shell, come detto prima, deve terminare con il punto più uno spazio vuoto. Più espressioni successive devono essere separate da “,”.

Ci sono due tipi di letterali numerici, **integer** e **float**, per esprimere gli interi in ogni base la notazione è `base#value`.

Mentre per esprimere un carattere in codice ASCII/Unicode la notazione è `$char`.

Se scriviamo:

```
2> 1+2, 2+3.
```

La prima somma viene valutata, però viene stampata solo l'ultima espressione.

In Erlang le variabili non variano come in ogni linguaggio funzionale, le variabili iniziano con una lettera maiuscola oppure con un underscore.

Erlang è tipizzato dinamicamente, non c'è nessuna indicazione di tipo statico per le variabili, può essere associata ad un valore solo una volta.

Una variabile può essere vincolata a un valore solo una volta (*alias "assegnazione singola"*), quindi in realtà non varia.

Le variabili sono vincolate ai valori tramite pattern matching.

Un pattern è strutturato come un termine, ma include variabili illimitate.

La variabile anonima (*solo* `_`) può essere utilizzata quando una variabile è richiesta ma il suo valore può essere ignorato.

Pattern matching

Tentativo di rendere identici un pattern (*lato destro*) e un termine (*lato sinistro*), eventualmente legando le variabili non vincolate nel pattern.

Caso banale: uso dell'operatore di corrispondenza `=`.

```
2> A = 2.
```

Questa espressione si legge "A matches 2"; non è un assegnamento.

Se scriviamo `A` otteniamo 2.

In figura vediamo come nella terza riga A e B matchano 0 e 1, però A, nella riga 2 era stata assegnata a 2, ed essendo immutabile, viene generato un errore.

Mentre nella riga 4 non abbiamo problemi perché A viene matchata con 2.

Altri casi di occorrenza di corrispondenza di pattern:

- Valutazione di una chiamata di funzione.
- Espressioni `case- receive- try-`

Se la corrispondenza fallisce, si verifica un errore di runtime.

```
1> 1+2.  
3  
2> 1+2, 2+3.  
5  
3> 7 rem 3.  
1  
4> 2.3*1.1.  
2.53  
5> 16#A5F2.  
42482  
6> $A.  
65
```

```
1> A.  
** 1: variable 'A' is  
unbound **  
2> A = 2.  
2  
3> {A, B} = {0, 1}.  
** exception error: no match  
of right hand side value  
{0,1}  
4> {A, B} = {2, 1}.  
{2,1}  
5> B.  
1
```

3.2.4 Tipi di dato

In Erlang ci sono le `Fun` che corrispondono alle funzioni lambda in Python. Ogni cosa è immutabile, però le liste hanno un modo per essere gestite cosicché è possibile avere un numero variabile di termini, utilizzando la ricorsione.

Tipi di dati:

- Numero (int, float)
- Atomo
- Booleani non presenti: ci sono gli atomi `true` e `false`
- Bit string
- Riferimento
- Fun (lambda)
- Port Identifier
- Pid (per identificare un processo)

Tipi di dati composti:

- Tupla - con un numero fisso di termini.
- Record - zucchero sintattico per le tuple (con campi nominati), non disponibile nella shell.
- Lista - con un numero variabile di termini.
- Stringa - in realtà, una lista di interi.

Atomi

Rappresentano valori costanti, in modo che il valore di un atomo sia univoco all'interno del programma, corrispondono ai tipi enumerati in C/Java.

La valutazione di un atomo, restituisce l'atomo stesso.

Gli atomi iniziano con **lettere minuscole**, seguite da caratteri alfanumerici più `_` e `@`.

Altrimenti, possono essere delimitati da virgolette singole `' '`, con qualsiasi carattere all'interno.

Nella riga 2 in figura sotto vediamo che il valore a sinistra inizia con una lettera maiuscola, quindi è una variabile, mentre quello a destra è un atom perchè inizia con la lettera minuscola. Non è possibile assegnare alla variabile un valore differente (*tiger*), abbiamo un errore in questo caso.

```

1> lion.
lion
2> Animal = lion.
lion
3> Animal = tiger.
** exception error: no match of
right hand side value tiger
4> {zoo, Animal1, Animal2} = {zoo,
'zebra joe', 'monkey bob'}.
{zoo,'zebra joe','monkey bob'}
5> Animal1.
'zebra joe'
6> Animal2.
'monkey bob'

```

Nella riga 4, abbiamo un atomo che viene matchato con se stesso e due variabili matchate con due atom (sono atomi perché delimitati dai singoli apici).

Tuple

Le tuple vengono delimitate dalle `{ }`.

Una tupla è un tipo di dato composto con un numero fisso di termini (*elementi*) e vengono utilizzate per raggruppare elementi; sono simili alle strutture in C, senza campi denominati.

I valori possono essere estratti dalle tuple utilizzando il pattern matching. Variante "denominata" delle tuple: record.

```

1> {alan,turing}.
{alan,turing}
2> Logician = {alonso, church,
{birthyear, 1903}}.
{alonso,church,{birthyear,1903}}
3> {Name,church,{birthyear,1903}}
= Logician.
{alonso,church,{birthyear,1903}}
4> Name.
alonso
5> {_, Lastname, _} = Logician.
{alonso,church,{birthyear,1903}}
6> Lastname.
church

```

Nell'esempio, alla riga 1 abbiamo una tupla composta da due atomi, mentre nella riga 2 abbiamo una tupla di 3 atomi e un numero. Nella riga 3 abbiamo una tupla composta da una variabile che matchiamo con una variabile (Logician), un atomo e un'altra tupla. Nella riga 5 vogliamo solo che la variabile Lastname prenda il valore di Logician.

Liste

Una lista è un tipo di dati composto per memorizzare un numero arbitrario di termini (*elementi*).

Sono delimitate da parentesi quadre, le liste sono divise in **head** e **tail**, in cui head è la testa e tail è il resto della lista.

Elenco vuoto: []

Nota: [E1, ..., EN] è quindi equivalente a [E1 | [... | [EN | []]]]

I valori possono essere estratti dagli elenchi utilizzando il pattern matching, anche nella forma [H | T].

In riga 1 dell'esempio abbiamo una lista che ha un atom, un numero e una tupla composta da due numeri, da notare che i valori in una lista possono essere eterogenei.

Nell'esempio 3 invece, X prenderà l'atomo (bob), mentre Y prenderà il resto della lista.

Nell'esempio 6, X prende l'atomo (era già delimitato da quel valore), Z prende il numero, mentre W prende la tupla.

```
1> L1 = [bob, 42, {4,2}].  
[bob,42,{4,2}]  
2> L2 = [3 | L1].  
[3,bob,42,{4,2}]  
3> [X|Y] = L1.  
[bob,42,{4,2}]  
4> X.  
bob  
5> Y.  
[42,{4,2}]  
6> [X,Z,W] = L1.  
[bob,42,{4,2}]  
7> W.  
{4,2}  
8> length(L1).  
3
```

List comprehension

Costrutto che consente di creare liste in modo conciso, filtrando ed elaborando elementi da una o più sorgenti (*liste o insiemi*).

È una tecnica molto potente per manipolare collezioni di dati, applicando funzioni a ogni elemento della lista e includendo o escludendo elementi in base a condizioni.

```
[Espressione || Pattern <- Lista, Condizione]
```

- **Espressione:** è il valore che sarà inserito nella nuova lista.
- **Pattern <- Lista:** è l'iterazione su ogni elemento della lista originale.
- **Condizione:** (*opzionale*) è una condizione che filtra gli elementi della lista. Solo gli elementi per cui la condizione è vera verranno inclusi nella nuova lista.

Creare una lista con i quadrati dei numeri da 1 a 5.

```
[X * X || X <- [1, 2, 3, 4, 5]].
```

% Risultato: [1, 4, 9, 16, 25]

Filtrare solo i numeri pari e calcolare i quadrati.

```
[X * X || X <- [1, 2, 3, 4, 5], X rem 2 == 0].
```

% Risultato: [4, 16]

Built-in Functions (BIFs)

Alcune operazioni non possono essere sviluppate utilizzando i costrutti Erlang di base, o almeno non in modo molto efficiente.

Per questo motivo, sono state rese disponibili diverse funzioni "integrate" (*all'interno di BEAM*); alcune sono "auto-importate".

Sono in genere utilizzate per l'accesso al sistema, la conversione dei dati, la gestione efficiente dei dati composti, I/O, ecc.

Ad esempio `date()` restituisce la data, nella riga 4 vediamo una built-in function `element()` che consente di ottenere un elemento di una tupla. Nella riga 7 la funzione `io:format()` consente di stampare a video.

```
1> date().
{2022,10,10}
2> time().
{10,11,42}
3> Tup = {lion,rhino,ostrich}.
{lion,rhino,ostrich}
4> element(2,Tup).
rhino
5> Lis = tuple_to_list(Tup).
[lion,rhino,ostrich]
6> length(Lis).
3
7> io:format("hello~n").
hello
ok
```

Stringhe

Una stringa è rappresentata come una lista di interi. Ogni intero corrisponde al codice UNICODE del carattere in questione.

Anche se c'è una sintassi speciale per esse, come possiamo vedere sotto (*vengono utilizzati i delimitatori " "*):

```

1> S1 = "Hello".
"Hello"
2> S2 = S1 ++ " World!". %str-list concat
"Hello World!"
3> S2 ++ [10].    %10: newline
"Hello World!\n"
4> S2 ++ [-3].
[72,101,108,108,111,32,87,111,114,108,100,
33,-3]
5> S3 = "\x{2200} e \x{2208} A \x{2a01}
B".
[8704,32,101,32,8712,32,65,32,10753,32,66]
6> io:format("~ts~n",[S3]).
∀ e ∈ A ⊕ B
ok

```

Nella prima linea `S1` matcha la stringa "Hello", il risultato della valutazione di questa espressione sarà "Hello".

Nella riga 2, `S2` è composta dalla concatenazione di `S1` più "World!" (*l'operatore di concatenazione è ++*), quindi la sua valutazione restituirà "Hello World!".

Nella terza linea ad `S2` viene concatenato l'intero 10 che rappresenta il carattere *new line* (`\n`).

Nella quarta linea, concatenare `S2` con un array composto da un solo numero che non ha una codifica di un carattere corrispondente, ovvero `-3`. A questo punto, `S2` viene vista come una lista di interi e `-3` viene regolarmente concatenato in fondo alla lista.

Utilizzando il formato mostrato nel comando sotto:

```
io:format("~ts~n" , [S3])
```

(*~ts~n è il placeholder*)

Possiamo visualizzare la stringa in un determinato template, nell'esempio, viene mostrato il simbolo corrispondente al codice assegnato ad `S3`.

3.3 Moduli Erlang

Un modulo in Erlang è l'unità base del codice ed è contenuto in un singolo file con estensione `.erl`.

Metadati

Ogni file di questo tipo include metadati che descrivono il modulo e contengono le definizioni delle funzioni.

Uno dei metadati fondamentali è il nome del modulo, che deve corrispondere al nome del file in cui è contenuto. Un file denominato `esempio.erl` deve definire un modulo chiamato `esempio`.

Applicazioni modulari

Un'applicazione può essere composta da molti moduli, e in alcuni casi un modulo potrebbe dover richiamare funzioni definite in un altro modulo. È frequente che alcuni moduli contengano funzioni che fungono da punti di ingresso per l'applicazione.

Funzioni pubbliche e private

All'interno di un modulo è possibile definire delle funzioni. Le funzioni pubbliche possono essere richiamate da altri moduli, mentre quelle private sono accessibili esclusivamente all'interno del modulo stesso.

Per rendere una funzione pubblica, è necessario dichiararla esplicitamente nell'attributo `-export`, che specifica l'elenco delle funzioni visibili all'esterno del modulo.

Compilazione di un modulo

Prima di essere eseguiti, i moduli devono essere compilati.

Questo processo traduce il codice sorgente `.erl` in un formato eseguibile da parte della macchina virtuale di Erlang, tipicamente in file `.beam`.

Per compilarlo ci sono vari modi (*comandi*). Ad esempio nell'immagine sopra, nella seconda riga con `export`.

Una funzione è definita dal suo nome e dal numero di parametri (*arity*), nel comando `-export`, il numero che segue il carattere `"/` è il numero di parametri.

Oppure possiamo anche farlo dal terminale:

- Il **compilatore** di Erlang è `erlc`.
- L'**interprete** di Erlang è `erl`.

```
erlc [flag] myfirstmod.erl % Da terminale.
```

```
compile:file("myfirstmod.erl") % Da dentro il modulo.
```

Un modulo può essere compilato come in riga 1, oppure come in riga 2, però il metodo utilizzato in riga 3 è quello più comune (`c(myfirstmod)`).

```
-module(myfirstmod).  
%% API  
-export([sayhello/0,addinc/2]).  
  
add(X,Y) ->  
    X+Y.  
  
%% not exported!  
addinc(X) ->  
    X+1.  
addinc(X,Y) ->  
    add(addinc(X), Y).  
  
sayhello() ->  
    io:format("Hello World!~n").
```

Il metodo utilizzato per compilare un modulo Erlang:

```
c(myfirstmod).
```

Usare un modulo

```
1> ls().  
myfirstmod.erl  
ok  
2> compile:file("myfirstmod.erl").  
{ok,myfirstmod}  
3> c(myfirstmod).  
{ok,myfirstmod}  
4> ls().  
myfirstmod.beam    myfirstmod.erl  
ok
```

Le funzioni in un modulo possono essere utilizzate esternamente con la sintassi:
`mymodule:myfunction(...)`

Importazione di funzioni specifiche: `-import(mymodule, [myfunct/1 ...]).`

3.4 Funzioni

All'interno di un modulo è possibile definire delle funzioni (*pubbliche o private*).

Le funzioni pubbliche possono essere richiamate da altri moduli, mentre quelle private sono accessibili esclusivamente all'interno del modulo stesso.

Una funzione pubblica deve essere dichiarata esplicitamente nell'attributo `-export`, che specifica l'elenco delle funzioni visibili all'esterno del modulo.

3.4.1 Definizione di una funzione

In Erlang, una funzione è definita come una sequenza di clausole, separate da punti e virgola (;) e terminate da un punto (.).

Ogni clausola specifica un comportamento in base ai pattern dei suoi argomenti e, opzionalmente, può includere una guardia introdotta dalla keyword `when`.

Il nome di una funzione è un atomo. Gli argomenti di una funzione: pattern.

All'interno di un modulo, una funzione è identificata dalla coppia nome/arity.

```
-module(factorial).  
-export([fact/1]).  
% Caso base: il fattoriale di 0 è 1  
fact(0) -> 1;  
% Caso ricorsivo: N! = N * (N-1)!  
fact(N) when N > 0 -> N * fact(N - 1).
```

```
c(factorial).
```

```
factorial:fact(5). % Output: 120
```

Nome di una funzione

Il nome di una funzione (`fact`) è rappresentato come un atomo.

Argomenti di una funzione

Gli argomenti di una funzione sono espressi mediante **pattern matching**, che consente di destrutturare i dati in ingresso.

Arity di una funzione

La **arity** di una funzione in Erlang (*e in realtà anche in altri linguaggi funzionali*) è il numero di argomenti che la funzione accetta.

```
-export([fact/1]).
```

In questo caso “1” è la arity della funzione `fact`.

Una funzione è univocamente identificata dalla coppia **nome/arity**, dove il nome è il suo identificatore e l'arity è il numero di argomenti che accetta.

Clausola

Il corpo di una clausola è una sequenza di espressioni, separate da (,).

Una **clausola** corrisponde a ciascun ramo della definizione della funzione, identificato da un pattern (*gli argomenti tra parentesi*) e dal relativo corpo.

```
fact(0) -> 1;
```

```
fact(N) when N > 0 -> N * fact(N - 1).
```

Queste qua sopra sono due clausole della funzione `fact`.

Ogni clausola è valutata in base ai pattern forniti; nella seconda clausola il pattern è:

```
when N > 0
```

Quando il pattern corrisponde, il relativo corpo della clausola viene eseguito e il risultato viene restituito come risultato della funzione.

3.4.2 Chiamata di una funzione

In questo esempio definiamo una funzione che calcola l'area di differenti forme geometriche.

```
-module(shapes) .
-export([area/1]) .

area({square,Side}) ->
    Side*Side;
area({circle,Radius}) ->
    Radius*Radius*3.1415;
area({triangle,B,H}) ->
    B*H/2.

...
```

Un “*trucchetto*” che possiamo utilizzare è quello di utilizzare un parametro come una tupla per indicare il tipo di forma geometrica che stiamo trattando. Lo indicheremo con la lettera minuscola in quanto si tratta di un atomo.

```
{square , Side}
```

Nell'esempio sopra la tupla è composta da un atomo che rappresenta il nome della figura (`square`) più il lato (`Side`).

```
1> c(shapes). %compile module
{ok,shapes}
2> shapes:area({square,3.2}).
10.240000000000002
3> shapes:area({circle,3.2}).
32.168960000000006
4> shapes:area({triangle,3.2, 2}).
3.2
5> shapes:area({rectangle,3.2, 2}).
** exception error: no function clause matching
shapes:area({rectangle,3.2,2}) (...)
```

Eseguiamo il comando alla riga 2 specificando il valore del lato del quadrato.

Nella riga 5 abbiamo un errore perché non abbiamo una clausola della funzione che matcha la funzione relativa al rettangolo (*abbiamo specificato due valori invece che uno*).

3.5 Controllo del flusso

3.5.1 Espressioni Guardia

Una **guardia** in Erlang è un'espressione booleana utilizzata per controllare il flusso di esecuzione, aumentando la potenza del **pattern matching**.

Formalmente, una guardia è una sequenza di **espressioni** progettate per essere sicure e prive di effetti collaterali: pertanto, non è consentito utilizzare funzioni definite dall'utente all'interno delle guardie.

Nell'esempio specifichiamo una funzione per ottenere il segno di un certo numero.

```
sign(X) when is_number(X), X>0 ->
1;
sign(X) when is_number(X), X==0 ->
0;
sign(X) when is_number(X) ->
-1;
sign(_) -> argerror.
```

1. **Controllo del tipo di dato:** in Erlang le variabili non sono tipizzate quindi x può essere qualsiasi cosa, quindi per controllare se una variabile è un numero utilizzeremo `is_number(x)`.

In questo caso la guardia è la sequenza di espressioni "when is_number".

2. **Controllo del segno:** Se $x > 0$, la funzione restituisce 1.
3. **Controllo se 0:** Se $x = 0$, il risultato sarà 0.
4. **Controllo del segno:** Se $x < 0$, la funzione restituisce -1.

In tutti gli altri casi (se il valore di x non è un numero) restituisce un errore (è un atomo).

Viene utilizzata la variabile senza nome "_", per valutare il segno in questo caso.

3.5.2 Espressione Case

A volte, invece di affidarsi al pattern matching su molte clausole, può essere conveniente usare espressioni `case`.

L'espressione `Expr` viene valutata in `T` e i pattern `P1 ... Pn` vengono confrontati in sequenza con `T`.

```
case Expr of
  P1 [when C1] -> E1;
  :
  Pn [when Cn] -> En
end
```


Non appena si verifica una corrispondenza e la relativa guardia (*facoltativa*) è vera, viene valutata la corrispondente `Ei` come valore di ritorno del `case`.

Se però non c'è un pattern `catch-all` alla fine, viene restituito un errore in caso nessuno dei precedenti pattern sia rispettato (*corrisponde al default nello switch-case*).

3.5.3 Espressione If

Guardie `Guard1 ... Guardn` vengono controllate in sequenza.

Non appena una guardia "*riesce*", il valore di ritorno dello `if` è il valore di ritorno della sequenza di espressioni `Ei`.

Le clausole di un `if` sono solitamente chiamate "rami".

```
if
  Guard1 -> E1;
  :
  Guardn -> En
end
```

Quando una guardia riesce non si esce dallo `if` ma si continua a valutare anche le successive.

Nelle espressioni intermedie possiamo creare dei valori che vengono poi utilizzati dall'ultima espressione.

Nessuna guardia riesce viene lanciato errore di runtime `if_clause`. Per evitarla spesso viene inserita una guardia finale sempre `true`.

3.5.4 Loop in Erlang

Spesso dobbiamo ripetere dei calcoli nel programma, l'unico modo per farlo nei linguaggi funzionali, teoricamente, è utilizzando la ricorsione. Infatti questa cosa è particolarmente desiderata in Erlang grazie all'ottimizzazione della tail recursion.

Tuttavia a volte risolvere problemi con la ricorsione risulta molto complicato.

Quindi può risultare comodo implementare un ciclo iterativo utilizzando la ricorsione.

```
-module(helloworld) .
-export([forhw/1,start/0]) .

forhw(0) -> done;
forhw(N) ->
  io:fwrite("Hello~n"),
  forhw(N-1) .

start() ->
  forhw(5) .
```

L'espressione `forhw` serve per specificare quante volte (`N`) dobbiamo ripetere una certa azione. È una funzione ricorsiva, scrive in output e richiama se stessa per `N-1` volte. Nel caso `forhw(0)` non vengono fatte più iterazioni.

Loop in una lista

Nella prima clausola mettiamo il caso base, ovvero quando la lista è vuota, in questo caso viene restituito `Index`.

Nella seconda clausola dividiamo la lista in testa (`H`) e coda (`T`). Ed effettuiamo la chiamata ricorsiva sulla coda incrementando il valore di `Index`.

```

-module(showlist).
-export([scanl/1,start/0]).

scanl(L) -> scanl(L,0).

scanl([], Index) -> Index;
scanl([H|T], Index) ->
    io:fwrite("~w: ~w~n", [Index,H]),
    scanl(T, Index+1).

start() ->
    X = [10,2,7,4],
    scanl(X).

```

3.6 High order functions in Erlang

Le **funzioni di alto livello** in Erlang consentono di esprimere operazioni generiche con un livello di astrazione di controllo, delegando dettagli specifici a una funzione passata come parametro.

Questa funzione (*chiamata fun*) rappresenta l'operazione da eseguire, mentre il flusso di esecuzione è determinato dalle regole specificate.

Un esempio tipico è una funzione ricorsiva che applica un'operazione a un intervallo di valori o a una struttura dati, adattandosi dinamicamente al comportamento richiesto tramite il parametro **fun**.

Funzione come parametro di ingresso

Nell'esempio sotto abbiamo una funzione con parametro di input composto da una tupla composta a sua volta da un atomo ed un valore (*corrispondente alla conversione da Celsius a Fahrenheit e viceversa*).

```

1> TempConv = fun({cel,C}) -> {far, 32 + C*9/5};
1>               ({far,F}) -> {cel, (F-32)*5/9}
1>               end.
#Fun<erl_eval.6.128620087>
2> TempConv({cel,22}).
{far,71.6}
3> TempConv({far,0}).
{cel,-17.77777777777778}

```

Come si può vedere, la funzione permette anche di avere più clausole.

```

myfilter(_ , [] ) -> [];
myfilter(Pred,[H|T]) -> case Pred(H) of
    true -> [H|myfilter(Pred,T)];
    false -> myfilter(Pred,T)
end.

```

Vediamo come è implementato il filtro con la funzione "myfilter/2".

Un **predicato** `Pred` è una funzione il cui dominio è l'insieme dei valori booleani (**true** o **false**). I predicati sono utili per operazioni di verifica, ad esempio per controllare se un elemento soddisfa una certa condizione o proprietà.

Applichiamo il predicato alla testa `H`.

- Se vero, la testa viene mantenuta.
- Se falso, allora viene richiamato il filtro sulla coda, scartando la testa.

```
is_even(X) -> case (X rem 2) of
  0 -> true;
  _ -> false
end.
```

"is_even/1" controlla se `x` è dispari:

- Se il resto è 1 restituisce `true`.
- Altrimenti restituisce `false`.

```
test() -> myfilter(fun is_even/1,
[1,2,3,4,5]).
```

Quindi la funzione "is_even/1" viene passata come parametro di `my_filter` per implementare il predicato `Pred`.

Per specificare che il parametro è una funzione utilizziamo `fun is_even`.

Funzione come valore di ritorno

Nell'esempio abbiamo due funzioni come parametri, `F()` e `G()`, come risultato viene restituita un'altra funzione che è la composizione delle due (`F(G(X))`).

Functions can be returned by functions as well.

In the example, `fcomp` returns the composition of two functions (with arity 1) passed as input.

```
-module(mymod).
...
fcomp(F,G) ->
  fun(X) -> F(G(X)) end.
...
```

shell

```
1> Myf1 = mymod:fcomp(fun(X)->X*X end,
fun(X)->X+1 end).
#Fun<mymod.1.70048917>
2> Myf2 = mymod:fcomp(fun(X)->X+1 end,
fun(X)->X*X end).
#Fun<mymod.1.70048917>
3> io:format("~p\n", [Myf1(5)]).
36
ok
4> io:format("~p\n", [Myf2(5)]).
26
ok
```

Function values
as params

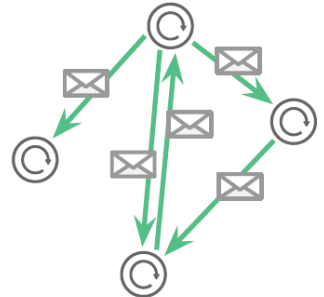
3.7 Programmi con Erlang

Erlang è un linguaggio concorrente che permette di sviluppare una soluzione indipendentemente dove si trova l'entità che deve eseguire quell'attività.

Actor model

In questo modello di concorrenza, le unità computazionali fondamentali sono gli attori.

- Ogni computazione è effettuata da uno o più attori.
- Un attore non condivide nulla con gli altri attori.
- Un attore viene sempre creato da un altro attore.
- Ogni attore ha un identificatore univoco che lo identifica rispetto agli altri attori.
- Due attori comunicano tramite scambio di messaggi asincrono.
 - Il mittente deve conoscere l'identificatore del destinatario.
 - Un attore viene creato da un altro attore.
- Ogni attore reagisce a ciò che riceve effettuando alcune attività.
 - Possiamo pensare ai messaggi come al “carburante” del nostro sistema, senza i messaggi gli attori non reagirebbero e non farebbero nulla.



Comportamento di un attore

Il comportamento indica che cosa un attore fa nell'elaborazione di un messaggio ricevuto, un attore può cambiare comportamento come conseguenza della ricezione di un messaggio (*comportamento reattivo*).

Dopo aver ricevuto un messaggio, un attore può (*anche contemporaneamente*):

- Inviare un numero finito di messaggi ad altri attori.
- Creare un numero finito di nuovi attori.
- Designare il comportamento da utilizzare per il messaggio successivo che riceve.

In pratica, dobbiamo specificare meglio come deve essere strutturata l'operazione di ricezione complessiva.

In Erlang **Processi** ↔ **Attori** e ci sono solo 3 funzioni fondamentali per la concorrenza:

- `spawn()`
- `!` (*bang*)
- `receive()`

3.7.1 Funzione `spawn()`

Per creare un processo in Erlang dobbiamo utilizzare la funzione `spawn()`.

Questa funzione ha come argomenti:

- Argomenti che identificano la funzione di riferimento.
- Valori effettivi degli argomenti per la funzione di riferimento.

`spawn()` **restituisce il PID** del processo appena creato, questo valore verrà utilizzato come identificatore (*ricorda che i processi sono gli attori*).

Questa attività corrisponde all'esecuzione della funzione con i relativi parametri.

```
-module(test).  
-export([body/2]).  
body(X,Y) ->  
    io:format("~w+~w=~w~n", [X,Y,X+Y]).
```

In figura sopra vediamo un modulo che contiene una funzione `body` e definiamo una funzione `F` (è una *fun* perchè nella shell non è possibile definire funzioni, ma è possibile utilizzare le *fun*) da associare ai nuovi processi.

```
1> F=fun(X,Y)->io:format("~w+~w=~w~n",  
    [X,Y,X+Y]) end.  
#Fun<erl_eval.12.128620087>  
  
2> [spawn(fun() -> F(A,A+1) end) || A<-[3,4]].  
3+4=7  
4+5=9  
[<0.134.0>, <0.135.0>]
```

Alla riga 2> tramite una “*list comprehension*” viene creata una lista dove ogni elemento corrisponde al PID di un processo spawnato (0.134.0 e 0.135.0). I due processi appena creati vengono eseguiti concorrentemente.

Alla riga 3> invece creiamo un terzo processo, ma con `body` differente.

```
3> spawn(test,body,[1,2]).  
1+2=3  
<0.137.0>
```

Questa espressione viene valutata e restituisce 0.137.0.

Il metodo in riga 3> è più flessibile perché se nulla cambia dalla creazione, è possibile accedere al valore tramite il file system.

Quando termina un processo in Erlang?

Quando la chiamata della funzione associata terminerà.

3.7.2 Funzione `bang(!)`

In Erlang, l'operatore `!` viene utilizzato per inviare messaggi in modo **non bloccante**. Un messaggio è composto da un destinatario (*specificato tramite il **PID***) e un contenuto (*che può essere qualsiasi termine Erlang*).

Per inviare un messaggio `Msg` al processo con PID `Pid`, si utilizza la sintassi:

```
Pid ! Msg.
```

```
Pid ! {hello, world}.
```

Questa è un'espressione Erlang che valuta il messaggio come termine **Msg**, lo invia al processo identificato da **Pid** e restituisce il termine **Msg**.

Invio associativo

Esiste anche una specie di invio associativo.

```
Pid2 ! Pid1 ! Pid0 ! Msg.
```

1. L'atomo **Msg** viene inviato a **Pid0**.
2. Il valore restituito dall'invio è inviato a **Pid1**.
3. Infine, il risultato dell'intera espressione è inviato a **Pid2**.

Invio riflessivo

Utilizzando: `self() ! emptymessage`

Il processo invia un messaggio vuoto a se stesso.

3.7.3 Segnali in Erlang

I **segnali** in Erlang sono la base della comunicazione e del controllo tra processi. Sono utilizzati per inviare informazioni tra processi o notificare eventi come terminazioni o cambiamenti di stato.

L'invio di messaggi tra processi (!) è una tipologia di segnale presente in Erlang.

Questi segnali sono fondamentali per la gestione della comunicazione e della supervisione tra processi in Erlang.

Altri segnali comuni in Erlang sono:

Exit

Utilizzato per terminare un processo o segnalare che un processo è stato terminato.

```
exit(Pid, reason).
```

Link/Unlink

Stabilisce o rimuove un collegamento tra due processi.

```
link(Pid).
```

```
unlink(Pid).
```

Monitor/Demonitor

Un processo può "monitorare" un altro processo, ricevendo un segnale quando il processo monitorato termina, con la possibilità di gestire l'evento.

```
Ref = monitor(process, Pid).
```

```
demonitor(Ref).
```

3.7.4 Funzione `receive`

I segnali sono gestiti all'interno dei processi tramite il costrutto `receive`, che consente a un processo di aspettare e rispondere a determinati segnali (o *messaggi*). Un processo può anche utilizzare delle guardie per filtrare i segnali che vuole ricevere.

Una `receive` in Erlang è **bloccante**.

```
receive
  {message, Content} ->
    io:format("Received message: ~p~n", [Content]);
  stop ->
    io:format("Process is stopping~n");
  _ ->
    io:format("Unknown message~n")
end.
```

Come si vede sopra questo costrutto applica il *pattern matching* per selezionare il messaggio che deve essere ricevuto.

In base al primo pattern che corrisponde viene eseguita la clausola corrispondente.

Ovviamente è sempre presente una condizione di default.

Mailbox del processo

Ogni processo in Erlang ha una **casella di posta (mailbox)**, una struttura dati che memorizza i messaggi in ingresso seguendo la politica **FIFO**.

I messaggi inviati al processo sono accodati in ordine di arrivo.

La casella di posta può essere ispezionata in questo modo:

```
process_info(Pid, messages).
```

Funzionamento della `receive`

Per estrarre i messaggi nella mailbox, si utilizza l'espressione `receive`. Questo meccanismo sfrutta il **pattern matching** per esaminare i messaggi in attesa nella coda:

1. **Match dei messaggi:**
 - Se un messaggio nella mailbox corrisponde a uno dei pattern definiti nel blocco `receive`, quel messaggio viene rimosso dalla mailbox e il codice associato al pattern (clausola) viene eseguito.
 - Tra i messaggi che corrispondono, viene selezionato il **più vecchio** (estrazione FIFO).
2. **Blocco del processo:**
 - Se nessun messaggio nella mailbox soddisfa almeno uno dei pattern specificati, il processo si blocca in attesa di un messaggio che corrisponda.

Più nello specifico:

1. Il sistema tramite l'istruzione `receive` inizializza un timer solo se è presente la clausola *after* nella `receive`.
2. Si considera il primo messaggio nella mailbox e si scorre la mailbox in cerca di un match.
3. Non appena avviene un match, il messaggio viene rimosso dalla mailbox e le espressioni relative vengono valutate.

Se non c'è nessun match per il primo messaggio nella mailbox, potrebbe essere considerato successivamente, quindi viene rimosso dalla mailbox ed inserito in una struttura dati detta "*save queue*".

Quindi, viene provato il secondo messaggio nella mailbox. Questa procedura viene ripetuta finché:

- Viene trovato un messaggio corrispondente.
- Altrimenti, tutti i messaggi sono stati esaminati.

Se nessuno dei messaggi matcha, il processo è sospeso finché un nuovo messaggio viene inserito nella mailbox. Quando arriva un nuovo messaggio, i messaggi nella *save queue* non vengono nuovamente abbinati; viene abbinato solo il nuovo messaggio.

4. Non appena un match avviene, tutti i messaggi precedentemente salvati nella *save queue* vengono inseriti nella mailbox, mantenendo l'ordine di arrivo. Il timer (se *presente*) viene cancellato.
5. Se il timer scade mentre siamo in attesa di un messaggio, allora valuta le espressioni relative; sposta tutti i messaggi salvati nella mailbox, mantenendo l'ordine di arrivo.

3.7.5 Ping Pong

Si hanno due processi, uno invia un messaggio "*ping*" all'altro, il quale risponde con un messaggio "*pong*". Da notare come i due body siano nello stesso modulo.

```
-module(pp).
-export([start/0, alice/2, bob/0]).
```

Il processo "*ping*" è Alice, mentre il processo "*pong*" è Bob e vengono creati in contemporanea.

1. Bob va in `receive` in attesa di "*ping*".
2. Alice invia "*ping*", ovvero una tupla composta da un `atomo` (il messaggio *ping*) e `self` (indica il proprio PID).

Perché Alice inserisce il proprio PID nel messaggio?

Bob, all'arrivo del messaggio non sa chi è Alice, quindi quest'ultima inserisce il proprio PID in modo che il Bob sappia a chi inviare la risposta.

3. Alice va in `receive` in attesa di "*pong*".

Se riceve qualsiasi altro messaggio, non fa nulla, attende.

4. Bob invia “pong” il quale ha la stessa struttura di “ping”.

5. Alice riceve “pong”, quindi risponde con “ping”.

...

6. Allo scadere del counter N, Alice invia “Alice finished” e termina.

7. Bob riceve “Alice finished”, allora anche Bob termina.

Da notare che l’ultima operazione del processo è una chiamata ricorsiva.

```
alice(0, Other_PID) ->
  Other_PID ! finished, %terminates bob
  io:format("Alice finished~n");
alice(N, Other_PID) ->
  Other_PID ! {ping_msg, self()},
  receive
    pong_msg ->
      io:format("Alice received pong~n")
  end,
  alice(N - 1, Other_PID). %last call opt.
```

```
bob() ->
  receive
    finished ->
      io:format(" Bob finished~n");
      {ping_msg, Other_PID} -> %pttrn with Pid
      io:format(" Bob received ping~n"),
      Other_PID ! pong_msg,
      bob() %last call optimization!
  end.
```

Al termine dell’esecuzione della funzione `start()`, otteniamo il PID di Alice, perchè in Bob l’ultima operazione è lo spawn del processo Alice.

```
start() ->
  Bob_PID = spawn(?MODULE, bob, []),
  spawn(?MODULE, alice, [2, Bob_PID]).
```

MACRO: current module

3.8 Client-server in Erlang

Vediamo le basi dello sviluppo di applicazioni client-server in Erlang.

3.8.1 Processo Server

Un processo server è un processo che continuamente (`loop()`) attende una richiesta, calcola e risponde al mittente la risposta.

```
-module(server).
-export([start/0, loop/0]).

start() -> spawn(?MODULE, loop, []).

loop() ->
  receive
    {From, {plus, X, Y}} ->
      From ! {self(), X+Y},
      loop();
    {From, {minus, X, Y}} ->
      From ! {self(), X-Y},
      loop()
  end.
```

Msg body with {Sender, opID, params}

Reply back, with sender ID as well

Macro ?MODULE

Vediamo che nella funzione `start()`, anziché inserire il nome del modulo (`server`) viene inserito `?MODULE`.

Questa cosa viene fatta per aumentare la flessibilità: se volessimo cambiare nome al modulo continua a funzionare.

Corpo del server

Il corpo del server viene eseguito in `loop()` dove attendiamo continuamente una richiesta (`receive`).

La `receive` utilizza il pattern matching. In questo caso il nostro server offre due servizi (*plus* e *minus*), quindi servono due clausole per differenziare i due servizi.

```
{ From , {plus , X , Y } }
```

- Una variabile (*inizia con lettera maiuscola*) `From` è l'identità del processo client, in questo punto il client inserirà `self()` ovvero il suo PID.
- L'altro attributo è una tupla composta da:
 - Un atomo (*inizia con lettera minuscola*) che specifica il tipo di servizio voluto.
 - `X` e `Y`, ovvero i numeri da sommare o sottrarre.

Successivamente il server invia un messaggio al client contenente la sua identità e il valore calcolato.

```
From ! { self() , X+Y }
```

Dopo questo ritorna all'inizio (*funzione ricorsiva*), ciò avviene tramite la ricorsione richiamando `loop()`.

Se il server non mettesse il suo pid (`self()`) ai messaggi di risposta, il client non riuscirebbe a capire che ha ricevuto risposta dal processo server a cui ha inviato la richiesta.

Aggiornamento del codice server

Se desiderassimo aggiornare il codice del nostro server, normalmente dovremmo fermarlo, ricompilarlo e rieseguirlo. Questo causerebbe un **down time**, ovvero un periodo di inattività in cui il servizio non sarebbe disponibile, una situazione da evitare sempre.

In Erlang, questo problema può essere risolto utilizzando una tecnica che permette di aggiornare il codice del server senza interrompere il servizio.

La soluzione consiste nell'utilizzare una **chiamata remota** al termine di ogni operazione, invece di una chiamata locale.

In particolare, al posto di scrivere `loop()` si usa `server:loop()`.

```
-module(server).  
-export([start/0, loop/0]).  
  
start() ->
```

```

loop().
loop() ->
  receive
    {update, Msg} ->
      io:format("Received: ~p~n", [Msg]),
      server:loop(); % Chiamata remota per il prossimo ciclo
    stop ->
      io:format("Stopping server.~n");
  _ ->
      io:format("Unknown message.~n"),
      server:loop() % Chiamata remota
end.

```

Durante l'esecuzione del server, se viene caricata una nuova versione del modulo (*grazie al sistema di hot code swapping di Erlang*), le successive chiamate a `server:loop()` utilizzeranno automaticamente il nuovo codice.

Per un breve periodo di tempo, le due versioni del codice possono coesistere. I processi in esecuzione continueranno a usare la vecchia versione fino a che non effettueranno una nuova chiamata a `server:loop()`, momento in cui passeranno alla nuova versione.

Pillola avvelenata - Ricezione di messaggi errati

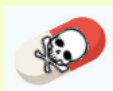
Dobbiamo occuparci dei messaggi in arrivo con un formato inaspettato che non devono essere inseriti nella mailbox perché sprecherebbero memoria.

Per gestire i pattern errati inseriamo una clausola con un pattern rispettato da qualunque messaggio (*figura sotto*), detta `_Unexpected`.

```

loop() ->
  receive
    {From, {plus, X, Y}} ->
      From ! {self(), X+Y},
      loop();
    ...
  stop ->
    ok;
  _Unexpected ->
    loop()
end.

```



Ovviamente deve essere l'ultima clausola perché deve essere eseguita solo se le clausole precedenti non matchano. In questo caso, il server estrae il messaggio senza processarlo e prosegue con la prossima iterazione (*praticamente esegue una continue*).

Per forzare il server a terminare, gli inviamo un messaggio di `stop`.

Mantenere lo stato del processo server

A volte abbiamo il bisogno di mantenere lo stato di un processo server.

Ma noi usiamo solo chiamate ricorsive, perciò basta definire un parametro che contiene lo stato del server e che viene passato ad ogni chiamata ricorsiva.

```

-module(stserver).
-export([start/0, loop/1]).

start() -> spawn(?MODULE, loop, [0]).

loop(N) ->
  io:format("Loop nr.~p ~n", [N]),
  receive
    {From, {plus, X, Y} } ->
      From ! {self(), {X+Y, N} },
      loop(N+1);
    ... <other receive clauses>
  end.

```

3.8.2 Processo Client

Il client invia un messaggio e poi attende una risposta dal server.

```

start(ServID, OpID, X, Y) ->
  spawn(?MODULE, body, [ServID,OpID,X,Y]).

body(ServID, OpID, X, Y) ->
  ServID ! {self(), {OpID, X, Y}},
  receive
    {ServID, Result} ->
      io:format("Result: ~p~n", [Result])
  end.

```

Only replies from the server are received!

Per conservare il PID del server, viene utilizzata la variabile `ServID` (il valore assegnato ad una variabile non cambia nella programmazione funzionale).

Mentre la variabile `[Result]` non è assegnata inizialmente, verrà assegnata non appena riceve il messaggio dal server.

Per terminare un processo in esecuzione viene utilizzata la sintassi alla linea 3>.

```

1> S = server:start().
<0.77.0>
2> C = client:start(S, plus, 2, 2).
<0.79.0>
Result: 4
3> exit(S, kill).
true

```

Requested service: sum

Command to kill a running process

Ottimizzazione del codice del client

Molto spesso il client deve fare sempre le stesse operazioni.

In modo da snellire il codice utilizziamo una funzione **rpc** (*remote procedure call*), nella quale specifichiamo il server, l'operazione e gli operandi, incapsuliamo quindi il body del client.

```

-module(client).
-export([start/4, body/4]).

start(ServID, OpID, X, Y) ->
    spawn(?MODULE, body, [ServID, OpID, X, Y]).

rpc(ServID, Msg) ->
    ServID ! {self(), Msg},
    receive
        {ServID, Result} -> Result
    end.

body(ServID, OpID, X, Y) ->
    Result = rpc(ServID, {OpID, X, Y}),
    io:format("Result: ~p~n", [Result]).

```

Spesso, i dettagli della comunicazione possono essere astratti all'interno delle funzioni.

Da notare che la **rpc** è privata (non c'è nella *export*).

In **rpc** viene svolto tutto il codice relativo alla comunicazione (dove vengono fatte sempre le stesse cose) con il server.

Timeout

Un'operazione bloccante (*receive*) può fermare in modo indefinito la progressione del programma client.

Questo accade ad esempio in caso di caduta del server a cui abbiamo già inviato una richiesta e che quindi siamo in attesa di una risposta.

```

receive
    P1 when C1 -> E1;
    :
    Pn when Cn -> En
after Delay -> ExprDel
end

```

Per risolvere questo problema viene implementata una versione temporizzata della *receive*.

Nel caso in cui non venga ricevuto alcun messaggio entro un dato timeout (*in millisecondi*), viene invece valutata un'espressione dedicata. Specifichiamo delle guardie e al termine specifichiamo un certo delay, in questo caso abbiamo il timeout dell'operazione di *receive*.

Un timeout di 0 millisecondi è utile per **svuotare la casella di posta**.

```

flush_messages() ->
    receive
        _Any ->
            flush_messages()
    after 0 ->
        true
    end.

```

Ricorsivamente, cerca di trovare una corrispondenza (e "*consumare*") qualsiasi messaggio; se non è presente alcun messaggio, il timeout 0 termina le chiamate ricorsive.

Il valore infinito può essere utilizzato anche per un timeout; ha senso quando il valore del ritardo deve essere calcolato/deciso, e una delle possibili opzioni è non avere alcun timeout.

3.8.3 Pubblicazione del PID di un processo

Un PID può essere pubblicato (*in una repository offerta dal sistema*) in modo che ogni processo possa ricavare il PID dell'interlocutore desiderato.

Questa tabella associa un nome al relativo PID.

Le operazioni di registrazione avvengono tramite quattro BIF:

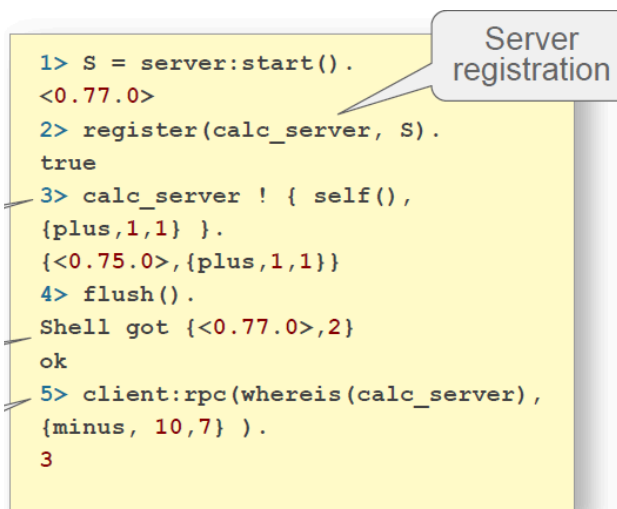
- `register(AnAtom , Pid)`: Registra il `Pid` con il nome `AnAtom`.
- `unregister(AnAtom)`: Rimuovi tutte le registrazioni associate ad `AnAtom`.
- `whereis(AnAtom) -> Pid | undefined -` : Questa è l'operazione di look-up, cerca il `Pid` del processo `AnAtom`.
- `registered() -> [AnAtom::atom()]`: Mentre quest'ultima restituisce la lista di tutti i processi registrati.

La registrazione ha senso solo per processi con funzionalità speciali che devono essere offerte ad altri processi, se utilizziamo 1000 processi differenti nella nostra applicazione, non ha senso utilizzare 1000 entrate nel registry, ha senso solo per un piccolo sottoinsieme di essi.

Le operazioni di register hanno senso all'interno di funzioni in cui vengono effettuate tutte le operazioni di inizializzazione.

Nell'esempio sotto:

- "send" può usare il nome del server (*un atomo*) come destinazione.
- Messaggio di risposta (*come scaricato dalla casella di posta*).
- In rpc, il PID è richiesto per la ricezione selettiva.

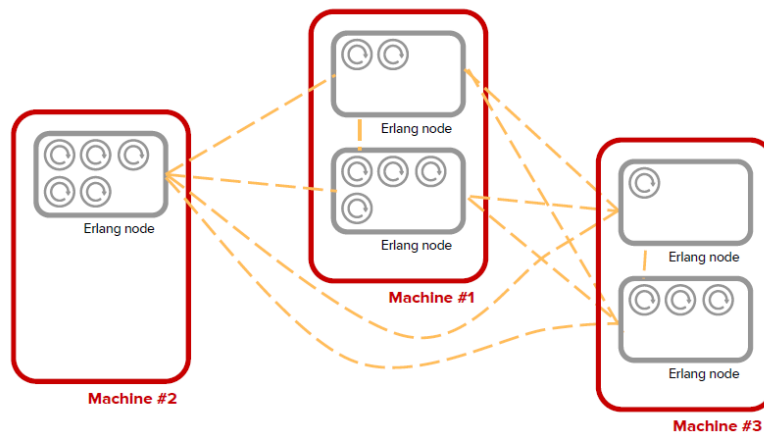


```
1> S = server:start().
<0.77.0>
2> register(calc_server, S).
true
3> calc_server ! { self(),
  {plus,1,1} }.
{<0.75.0>,{plus,1,1}}
4> flush().
Shell got {<0.77.0>,2}
ok
5> client:rpc(whereis(calc_server),
  {minus, 10,7} ).
3
```

3.9 Programmi distribuiti in Erlang

Finora ci siamo concentrati sui processi Erlang all'interno di un singolo nodo.

Tuttavia, in Erlang, ogni nodo può essere connesso agli altri nodi, il che significa che è in grado di interagire con essi. La **programmazione distribuita** in Erlang si basa su questa capacità, consentendo di generare processi su nodi e macchine remoti.



Esistono due principali approcci per sviluppare applicazioni distribuite:

Distribuzione su nodi

I programmi vengono eseguiti su diversi **nodi Erlang**, ciascuno dei quali è un sistema Erlang separato e completo, con il proprio set di processi.

È possibile generare un processo su un nodo remoto utilizzando (*la quale restituisce il PID del processo remoto*):

```
spawn(Node, Fun).
```

I processi che risiedono su nodi differenti possono comunicare tra loro utilizzando il consueto **passaggio di messaggi** come avviene tra processi locali.

Distribuzione basata su socket

Questo approccio offre maggiore sicurezza rispetto alla distribuzione su nodi, poiché consente un controllo più preciso sulle interazioni tra i nodi e su ciò che un client remoto può fare.

È utile per scenari in cui il sistema deve interagire con entità esterne o in cui i nodi non possono fidarsi completamente l'uno dell'altro.

Nome di un processo

I nodi hanno un nome univoco, necessario per essere individuati e contattati da altri nodi.

Ogni processo ha un nome, il quale può essere esteso indicando anche il nome del nodo, il formato è `Name@Host`.

I nomi possono essere di due tipi:

- **Nomi lunghi:** con nomi di dominio completamente qualificati.
- **Nomi brevi:** host senza “.”.

In un singolo cluster può essere utilizzato un solo tipo.

Con il comando sotto vediamo come assegnare un nome ad un processo.

```
erl -sname tom@localhost
```

EPMD (*Erlang Port Mapper Daemon*, sulla porta 4369) viene eseguito in ogni macchina del cluster Erlang, fungendo da name server, abilitando i contatti tra i nodi.

3.9.1 Connessione tra due nodi Erlang

L'interazione tra due nodi è possibile solo dopo che è stata impostata una connessione.

Le connessioni tra nodi Erlang sono transitive: Se connetto un nodo C al nodo A, che a sua volta è già connesso con il nodo B, allora C è connesso anche con B.

```
erl -sname ale@localhost
(ale@localhost)1> net_kernel:connect_node(tom@localhost).
true
(ale@localhost)1> nodes().
[tom@localhost]
```

Un modo alternativo per testare una connessione è il seguente:

```
net_adm:ping(Node).
```

Nota bene! All'inizio conviene scrivere e testare il programma in un singolo sistema (*un solo nodo in una sola macchina*), successivamente testare il programma in due nodi nella stessa macchina. Alla fine bisogna testare il programma in due nodi su due macchine distinte.

I nodi di cluster diversi sullo stesso hardware devono essere tenuti separati.

Cookie

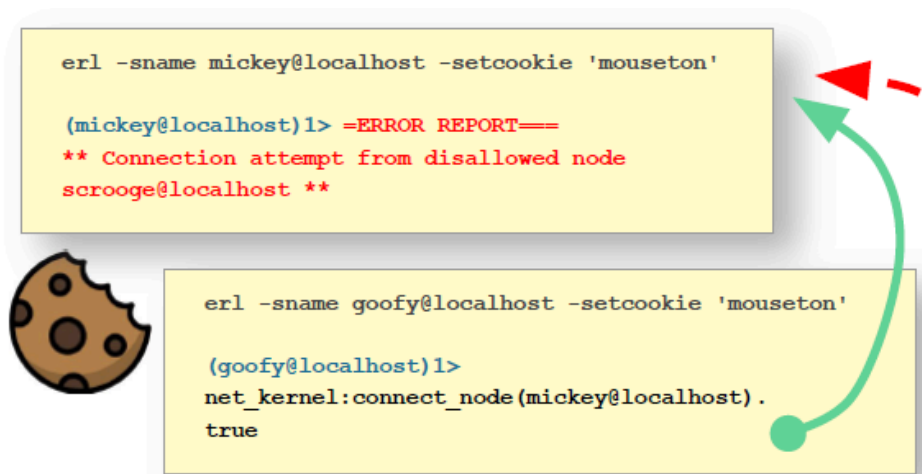
Il **cookie** è un meccanismo di sicurezza fondamentale utilizzato per autenticare la comunicazione tra due nodi Erlang.

Il cookie agisce come una **chiave segreta**; entrambi i nodi devono avere lo stesso cookie configurato per poter comunicare tra loro. Se un nodo tenta di connettersi a un altro nodo con un cookie diverso, la connessione verrà rifiutata.

Questo aiuta a prevenire accessi non autorizzati e garantisce che solo i nodi "*autorizzati*" possano interagire tra loro.

Potrebbe accadere che la stessa macchina ospiti nodi differenti appartenenti a cluster differenti, facciamo in modo che ogni nodo che vuole interagire (*quindi ogni nodo dello stesso cluster*) abbia lo stesso cookie.

Nell'immagine sotto, il cookie è "mouseton".



3.9.2 Comunicazione remota

Spawn di processi da remoto

I processi possono essere spawnati nel nodo remoto.

L'output restituito è il **PID remoto** dove il primo numero non è 0 (nel caso di un PID locale è 0).

<8502.88.0>

I processi possono essere generati su nodi, remoti tramite `spawn/2` e `spawn/4`: si comportano come i classici `spawn/1` e `3`, ma il nome del nodo deve essere aggiunto come primo argomento, questo è utile per:

- Generare server remoti.
- Eseguire comandi remoti.



Comunicazione remota

Per consentire a due nodi remoti di comunicare possiamo utilizzare le `send` e `receive`.

Come si vede in figura sotto, l'unica che cambia rispetto al caso locale è l'operazione di invio dei messaggi.

Send: Expr1 must evaluate to a pid (remote...),
a registered name,
or a tuple {RegisteredProcName, NodeName}.

Expr1 ! Expr2

Al termine dell'esecuzione di Goofie viene restituito il PID remoto per Mickey (*cambia solo il primo numero*).

```
(mickey@localhost)1> net_adm:ping(goofy@localhost).  
Pong  
(mickey@localhost)2> {goofyshell,goofy@localhost} !  
{hello, self()}.  
{hello,<0.82.0>}  
  
(mickey@localhost)3> flush().  
Shell got hello_back  
ok
```



```
(goofy@localhost)1> register(goofyshell, self()).  
true  
  
(goofy@localhost)2> receive {hello, Friend} ->  
Friend ! hello_back end.  
hello_back  
  
(goofy@localhost)3> Friend.  
<8135.82.0>
```



Moduli forniti per supportare programmi Erlang distribuiti:

- `net_kernel`: per connettere/disconnettere nodi, per commutare un nodo in approccio distribuito/non distribuito, per controllare l'heartbeat ecc.
- `global`: un registro di processo globale; → usa `global:send(Name, Msg)`; gestisce i conflitti di nome.
- `rpc`: funzioni per eseguire comandi su nodi remoti →
`rpc:call(Node,Mod,Funct,Args)`: restituisce localmente qualsiasi cosa sia restituita dalla funzione eseguita in remoto; `rpc:call/5 w/ timeout`;

OTP - Open Telecom Platform

Per applicazioni complesse, è consigliabile utilizzare **OTP**, un insieme di librerie e strumenti che fornisce un framework per costruire applicazioni **concorrenti**, **distribuite** e **fault-tolerant**.

Grazie alla sua robustezza e affidabilità, OTP è ampiamente utilizzato in vari contesti, come sistemi bancari, gaming, IoT e altri settori con requisiti critici.

Un concetto fondamentale nello sviluppo di applicazioni OTP è il **behavior** (comportamento).

Un behavior in OTP rappresenta un modello comportamentale comune, ovvero un framework generico che può essere personalizzato tramite un **modulo di callback**. Il comportamento si occupa di gestire gli aspetti non funzionali del sistema (ad esempio, la gestione degli errori o la concorrenza), mentre il codice specifico dell'applicazione viene implementato attraverso le funzioni di callback.

3.9.3 Funzione di callback

Una funzione passata da un chiamante a una funzione chiamata, in modo che tale funzione possa essere chiamata successivamente dal componente che stiamo sfruttando adesso.

Un esempio di funzione di callback è la funzione `onclick` in JavaScript, viene chiamata solo in caso dell'evento click del mouse.

```
-module(server_0).
-export([start/2, rpc/2]).

start(Name, Mod) ->
  register(Name, spawn(fun() -> loop(Name, Mod, Mod:init()) end)).

rpc(Name, Req) ->
  Name ! {self(), Req},
  receive {Name, Response} -> Response
  end.

loop(Name, Mod, State) ->
  receive
    {From, Req} ->
      {Response, NewState} = Mod:handle(Req, State),
      From ! {Name, Response},
      loop(Name, Mod, NewState)
  end.
```

Module with required request-handling function

Keep: Name, Mod, and (evolving) state

In questo esempio il modulo `Mod` che passiamo nella funzione `loop` è un modulo personalizzato. Dobbiamo definire il modulo personalizzato con le funzioni `init()` e `handle(Req, State)`.

Importante: la funzione di callback effettiva deve essere identificata tramite argomenti effettivi e talvolta anche in base alle convenzioni di denominazione.

- **Callback sincroni:** Eseguiti prima che la funzione chiamata restituisca il risultato.
- **Callback differiti:** Eseguiti dopo; utilizzati per la gestione degli eventi, GUI, ecc.

Le funzionalità specifiche devono essere definite separatamente nel modulo `Mod`, che deve contenere `init()` e `handle(Req, State)`.

Parametrizzare un server con una funzione di callback

Dobbiamo inizializzare `init()` e `handle()` come in figura.

Qui specifichiamo un server che riceve un numero per richiesta e restituisce la media di tutti i valori ricevuti finora.

```
-module(avg_server).
-export([init/0, handle/2, get_avg/1]).
-import(server_0, [rpc/2]).

% client routines
get_avg(X) -> rpc(calc_avg, X).

% callback routines
init() -> {0, 0}. % total: 0; # of reqs: 0

handle(X, {Total, Times}) ->
  NewTotal = Total + X,
  NewTimes = Times + 1,
  {NewTotal/NewTimes, {NewTotal, NewTimes}}.
```

Server name to be registered

Initial state

State info: Total, and # of calls

Impostazione del server: chiamare `start/2` dal comportamento, fornendo:

- nome standard per il server.
- riferimento al modulo di callback.

Il server viene utilizzato tramite routine client, definite nel modulo di callback.

```
1> server_0:start(calc_avg, avg_server).
true

2> avg_server:get_avg(4).
4

3> avg_server:get_avg(6).
5

4> avg_server:get_avg(11).
7
```

Name to be registered for the server

Callback module

Requests by clients

In OTP è il `gen_server` (*comportamento*) che adotta l'approccio precedentemente descritto.

Per creare un server, `gen_server` deve essere associato a un modulo callback con API standard, per gestire ogni possibile parte della funzionalità richiesta (*inizializzazione, gestione di rpcs, gestione di "cast", terminazione, ecc.*).

Il modulo callback deve specificare `-behaviour(gen_server)`.

gen_server module	callback module
gen_server:start, start_link, start_monitor	mod:init/1
gen_server:stop	mod:terminate/2
gen_server:call, send_request, multi_call	mod:handle_call/3
gen_server:cast, abcast	mod:handle_cast/3

3.9.4 Data Storing: ETS & DETS

ETS e DETS sono moduli per memorizzare termini di erlang generici. Entrambi forniscono una look-up table, ovvero una collezione di tuple, composte da una chiave e dai dati contenuti nella tupla.

Entrambi forniscono tabelle di ricerca {chiave-valore}: la differenza principale è che ETS memorizza in memoria RAM, mentre DETS nel disco.

Le tabelle sono raccolte di tuple; il primo elemento funge da chiave.

- **Set:** nessuna chiave duplicata; sono disponibili anche "set ordinati".
- **Bags:** sono consentite chiavi duplicate, ma non tuple; in "duplicate bags" possono essere presenti anche tuple duplicate.

Operazioni: creazione/apertura, inserimento, ricerca, eliminazione.

Integrated Data Management: Mnesia

Utile perché ETS ed DETS spesso non sono sufficienti, serve quindi un database aggiuntivo.

I programmi Erlang che necessitano di una gestione dati efficiente e più complessa possono contare su un sistema di database integrato: Mnesia.

Può memorizzare qualsiasi tipo di struttura dati Erlang.

In un database Mnesia, i dati sono organizzati in tabelle; le capacità del linguaggio di query

(*come in SQL*) sono disponibili nella sintassi Erlang ordinaria. Mnesia fornisce anche supporto alle transazioni.

3.10 Errori in Erlang

Erlang è progettato per supportare lo sviluppo di sistemi **fault-tolerant**, dove gli errori di runtime devono essere rilevati, gestiti e, quando possibile, corretti. Per affrontare i diversi scenari di errore, il linguaggio introduce meccanismi specifici per la gestione delle eccezioni sia nei programmi sequenziali che concorrenti.

Gestione degli errori

1. Programmi sequenziali:
 - È necessario individuare situazioni in cui un'operazione fallisce (ad esempio, una funzione che "si blocca") e definire come gestire tali casi eccezionali.
2. Programmi concorrenti:
 - A livello di processo, se un processo si arresta in modo anomalo, un altro processo deve essere in grado di rilevare l'errore e intervenire per ripristinare o continuare il lavoro.

Meccanismi per sollevare eccezioni

Erlang offre tre Built-In Functions (BIF) per sollevare eccezioni all'interno del programma:

- **error(Why)**: segnala errori gravi (ad esempio, crash interni emulati a livello di codice).
- **throw(Why)**: genera un'eccezione che il chiamante può decidere di intercettare.
- **exit(Why)**: termina il processo corrente con il motivo specificato.

Intercettazione delle eccezioni

Per gestire le eccezioni sollevate, Erlang fornisce due costrutti:

1. **try ... catch**:
 - Permette di intercettare eccezioni specifiche e gestirle secondo necessità.
 - Adotta un meccanismo simile a quello di altri linguaggi, ma è progettato in uno stile funzionale.
 - La clausola **of** è facoltativa.
 - Nota: la ricorsione in coda (tail recursion) non è consentita all'interno del blocco protetto da **try**.
2. **Expr catch**:
 - È un'operazione più diretta per catturare eccezioni generate dall'espressione **Expr**.

```
try
  1 / 0 % Operazione che genera un'eccezione
catch
  error:Reason ->
    io:format("Errore: ~p~n", [Reason]);
  throw:Reason ->
    io:format("Eccezione: ~p~n", [Reason]);
  exit:Reason ->
```

```
io:format("Uscita: ~p~n", [Reason])
end.
```

Questo approccio consente di gestire in modo esplicito diversi tipi di errore (`error`, `throw`, `exit`) con una sintassi chiara e funzionale.

Filosofia della tolleranza ai guasti in Erlang

La progettazione di un sistema fault-tolerant in Erlang si basa sul principio che gli errori sono inevitabili: i processi possono bloccarsi e le macchine possono fallire. Tuttavia, l'obiettivo non è prevenire gli errori, ma gestirli in modo efficace.

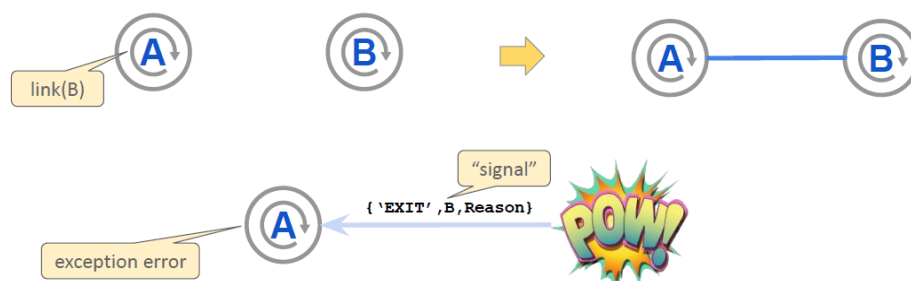
- Il comportamento di una funzione deve essere ben definito solo per input validi.
- I casi eccezionali devono automaticamente generare un'eccezione.
- La gestione dell'eccezione è responsabilità del chiamante.

In un sistema con molti processi, spesso è preferibile lasciare che un processo difettoso muoia, trasferendo il compito di recuperare dall'errore a un altro processo.

3.10.1 Link tra processi

I link e i monitor vengono utilizzati per implementare la supervisione di un processo da parte di un altro.

Organizzare un'applicazione secondo un albero di supervisione ci consente di ottenere il pieno controllo su di essa.



In Erlang è possibile stabilire una relazione speciale tra i processi per garantire una gestione coordinata dei fallimenti. Questa relazione consente di terminare un insieme di processi correlati se uno di essi fallisce. Per creare questa relazione, si utilizzano i BIF `link/1` o `spawn_link/1-3`, che collegano i processi tra loro.

I segnali di uscita, visti in precedenza, hanno lo scopo di terminare il destinatario del segnale. Tuttavia, per gestire tali segnali in modo più flessibile, possiamo trasformare il processo destinatario in un **processo di sistema**.

Processo di sistema

Un processo di sistema converte ogni segnale di uscita che riceve in un normale messaggio. Questo approccio consente di:

- Ricevere il segnale di uscita come messaggio nella mailbox del processo.
- Decidere come gestire il segnale, ad esempio riavviando il processo che si è arrestato.

Segnale kill

Il segnale di uscita con motivo **kill** è un'eccezione speciale:

- Non può essere intrappolato. È un **"arma definitiva"**, che termina incondizionatamente il processo destinatario.
- Tuttavia, il processo terminato con il motivo **kill** invierà ai suoi collegamenti il motivo **killed**. Questo comportamento permette ai processi collegati di distinguere tra un arresto normale e uno causato da un'azione forzata.

```
process_flag(trap_exit, true)
```

Segnale monitor

Un tipo di relazione più flessibile tra i processi in Erlang può essere impostato tramite **monitor/2-3** e **spawn_monitor/1-3**. A differenza dei **link**, che sono bidirezionali, i **monitor** sono **unidirezionali** e offrono maggiore controllo sulla gestione delle interazioni tra i processi.

- **Unidirezionali**: un processo può monitorare un altro processo, ma non viceversa.
- **Monitor impilabili**: è possibile avere più monitor attivi tra coppie di processi (più monitor possono essere associati a una stessa coppia di processi, creando relazioni "impilate").
- **Ricezione dei messaggi di stato**: quando un processo monitorato termina (per qualsiasi motivo), il processo di monitoraggio riceve un messaggio con il pattern: `{'DOWN', MonitorRef, process, Pid, Reason}`.

```
monitor(process, Pid)
spawn_monitor(Pid2)
```

I link e i monitor vengono utilizzati per implementare la supervisione di un processo da parte di un altro. Organizzare un'applicazione secondo un albero di supervisione ci consente di ottenere il pieno controllo su di essa.

Un comportamento OTP specifico è mirato a fornire un'implementazione robusta/flessibile di un supervisore.

Capitolo 4 - Java

Linguaggio molto più popolare di Erlang, era il più utilizzato dall'inizio del 2000 fino al 2012, è stato superato da linguaggi già esistenti, ma utilizzati per nuovi scopi, come ad esempio Python nell'analisi dei dati.

Conoscere Java completamente è molto difficile, un solo programmatore sfrutta solo alcune capacità di Java.

L'esecuzione di un programma Java segue un approccio ibrido tra compilazione e interpretazione, il codice sorgente non viene trasformato direttamente in codice macchina (*specifico per un determinato processore*), ma viene compilato in un formato intermedio chiamato **bytecode** (*indipendente dalla piattaforma*).

4.1 JVM - Java Virtual Machine

Per eseguire il programma, il bytecode viene interpretato o tradotto in codice macchina dalla Java Virtual Machine (JVM).

Quali sono i compiti della JVM?

Compilazione Bytecode in Codice Macchina:

- La JVM esegue il bytecode generato dal compilatore Java (*Java Compiler*), traducendolo in istruzioni macchina comprensibili dal sistema operativo e dal processore sottostante.

Gestione della Memoria:

- **Allocazione dinamica** della memoria per oggetti e variabili.
- **Garbage collection** per liberare la memoria dagli oggetti non più utilizzati.

Portabilità:

- Astrae le specificità del sistema operativo e dell'hardware, rendendo il codice Java "scrivi una volta, esegui ovunque" (*write once, run anywhere*).

Sicurezza ed Esecuzione Isolata:

- Verifica il bytecode per prevenire operazioni non sicure o dannose.
- Esegue il codice all'interno di una sandbox per proteggere il sistema ospitante.

Gestione delle Eccezioni e del Multithreading:

- Fornisce un supporto integrato per la gestione delle eccezioni.
- Offre un ambiente per il supporto del multithreading e la sincronizzazione.

Stack-based machine

La **JVM** è definita una **stack-based machine** perché utilizza uno **stack** (pila) come struttura dati principale per eseguire operazioni e gestire i dati durante l'esecuzione del programma.

Questo significa che la maggior parte delle istruzioni della JVM lavora direttamente con lo **stack** invece che con **registri hardware**, come avviene in altre architetture (es. *CPU register-based*).

In Java è stato deciso di utilizzare una **stack machine**, perché i programmi per queste macchine sono molto più compatti rispetto ad altri tipi di macchine (*perché non richiede di specificare i registri nelle istruzioni*).

Bytecode

Formato **intermedio** di istruzioni eseguibili generato dal compilatore Java, generata dal compilatore (*javac*), che viene eseguito dalla JVM piuttosto che direttamente dall'hardware della macchina.

- È un insieme di istruzioni binarie altamente ottimizzate, leggibili ed eseguibili dalla **JVM**.
- È **indipendente dalla piattaforma**, consentendo al programma Java di essere eseguito su qualsiasi sistema con una JVM compatibile.
- Si trova nei file **.class**, che vengono prodotti dal compilatore a partire dal codice sorgente Java.

```
int sum = a + b;
```

Diventa:

```
iload_1  
iload_2  
iadd  
istore_3
```

Ogni operazione di bytecode rappresenta un'istruzione semplice, come il caricamento di un valore in un registro, l'invocazione di un metodo o l'operazione di somma tra due valori (*una specie di assembly*).

La principale ragione per cui i progettisti di Java hanno scelto una stack-based architecture riguarda la **portabilità** e la **semplicità di implementazione**. Le stack-based machines non dipendono dall'architettura hardware sottostante (*che può variare notevolmente nei registri disponibili*), il che le rende più facilmente eseguibili su diverse piattaforme.

4.2 Java Generics

Java Generics è una **funzionalità** introdotta nella versione 5 di Java che permette di **creare classi, metodi e interfacce parametrizzati (generici)**, cioè che possono lavorare con qualsiasi tipo di dato senza specificare il tipo concreto fino al momento dell'utilizzo.

Gli obiettivi dei generics sono:

- Evitare la duplicazione del codice.
- Migliorare il controllo sulla gestione dei tipi di dati.

Java ha introdotto i "*Tipi Generici*", ovvero i tipi parametrici, anche per prevenire errori di tipo, scoprendoli al momento della compilazione.

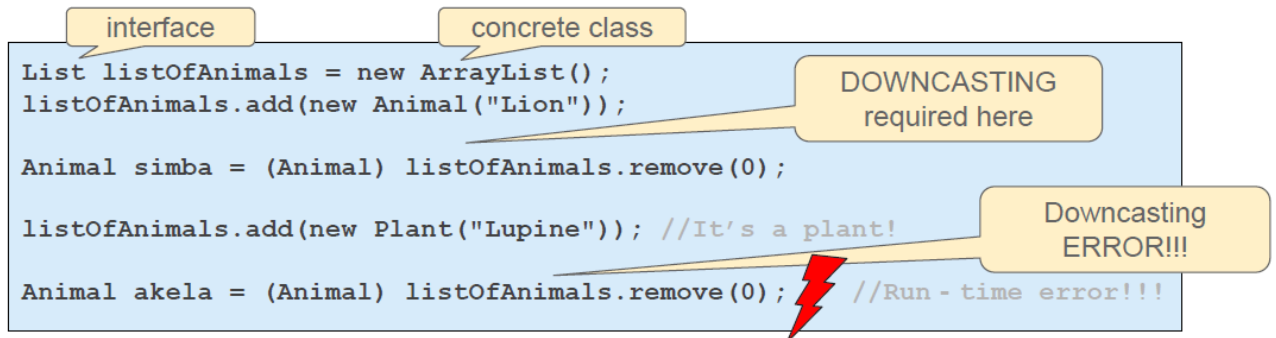
In Java, i **generici** permettono di definire il tipo di oggetti che una raccolta (*come una lista o un insieme*) può contenere, garantendo maggiore sicurezza e chiarezza del codice.

```
List<String> lista = new ArrayList<>();
```

`List<String>` specifica che la lista può contenere solo oggetti di tipo **String**, evitando errori di tipo durante l'esecuzione.

Problema della lista

Originariamente, `List` doveva contenere riferimenti a istanze di oggetti.



La lista `List listOfAnimals` può contenere qualsiasi tipo di oggetto.

- **Downcasting necessario:** Quando si rimuove un elemento, è necessario effettuare un downcasting esplicito per convertirlo nel tipo desiderato (*Animal*), il che può portare a errori a runtime.
- **Errore a runtime:** Se un oggetto di un tipo diverso (*Plant*) viene aggiunto e successivamente rimosso come se fosse un *Animal*, si genera un errore di `ClassCastException`.

Il compilatore **non può scoprire in anticipo errori** del genere perché non sa che questa lista dovrebbe contenere solo animali!

Il problema della figura precedente può essere risolto utilizzando i tipi generici (`T`).

In questo caso se ripetiamo la stessa cosa del caso precedente otteniamo un errore rilevato dal compilatore e non a runtime come nel caso precedente.

Possiamo rendere `List` parametrico rispetto al tipo dei suoi elementi, ad esempio "`T`"; quindi, `add()` accetterà solo oggetti il cui tipo è `T`, e `remove()` restituirà oggetti il cui tipo è `T`.

La lista `List<Animal>` può contenere solo oggetti di tipo `Animal`.

Vantaggi del tipo generico

- **No downcasting:** Non è necessario effettuare il downcasting quando si rimuove un elemento, poiché il compilatore sa già che si tratta di un *Animal*.
- **Errore a compile-time:** Il compilatore impedisce l'aggiunta di oggetti di tipo diverso (*Plant*), evitando errori a runtime e migliorando la sicurezza del codice.

```
List<Animal> listOfAnimals = new ArrayList<Animal>();  
listOfAnimals.add(new Animal("Wolf"));  
  
Animal akela = listOfAnimals.remove(0);  
  
listOfAnimals.add(new Plant("Lupine")); //It's a plant!
```

NO DOWNCAST
required here

Compiler-detected
ERROR!!!

4.2.1 Type Erasure

Il **Type Erasure** è il meccanismo con cui il compilatore Java rimuove le informazioni sui tipi generici (<T>) durante la compilazione.

Ciò significa che durante la **compilazione** il tipo generico (T) viene rimosso e sostituito con il suo **tipo di base** (di solito **Object** se non è specificato un limite).

Questo processo avviene per garantire la compatibilità con il bytecode più vecchio, quando i generici non esistevano ancora.

Pro del type erasure

- **Controllo dei tipi a compile-time:** Il compilatore verifica che i tipi siano utilizzati correttamente nel codice (*ad esempio, che non si aggiungano oggetti di tipo errato a una lista*).

Più o meno, dopo i controlli di sicurezza dei tipi, le operazioni eseguite sul codice sorgente sono:

- **Eliminazione dei tipi generici:** Dopo i controlli, il compilatore rimuove le informazioni sui tipi generici.

Quindi, il codice generico come:

```
Box<Integer> intBox = new Box<>();
```

viene trattato internamente dalla JVM come:

```
Box<Object> intBox = new Box<>();
```

- **Conversioni automatiche:** Il compilatore inserisce automaticamente dei **cast** per garantire la correttezza dei tipi a runtime.
- **Limite superiore:** Se una variabile generica (T) non ha vincoli specifici, viene trattata come un oggetto di tipo **Object**.

Contro del type erasure

- **Nessuna informazione sui tipi a runtime:** La JVM non conserva informazioni sui tipi generici. Quindi, non è possibile sapere quale tipo è stato utilizzato per `T` durante l'esecuzione.

Quando una classe generica come `Box<T>` viene compilata, la JVM carica il bytecode che contiene solo il codice per `Box<Object>`. In pratica, durante l'esecuzione, la JVM non conserva informazioni sui tipi specifici come `Integer` o `String`, ma gestisce solo una versione generica della classe.

- **Nessuna riflessione sui generici:** Non è possibile determinare a runtime se un oggetto è una `List<Animal>` o una `List<String>`.

Nell'esempio sotto utilizziamo il tipo generico `T` come parametro della classe, l'informazione di `T` viene persa a runtime, quindi si genera un errore rilevato dal compilatore.

```
class myClass<T> {  
    void myMethod(Object arg) {  
        if (arg instanceof T) { ... }  
        T myLocalVar = new T();  
        T[] myLocalArray = new T[100];  
    }  
}
```

At method invocation, the information on T is not available any more!

Compiler-detected ERRORS!!!

In questo esempio sotto invece, specifichiamo il tipo della lista solo a sinistra, a destra non lo specifichiamo perché viene capito dal contesto.

```
List<Plant> la1 = new ArrayList<>();  
List<Animal> la2 = new ArrayList<>();  
System.out.println(la1.getClass()); // class java.util.ArrayList  
System.out.println(la2.getClass()); // class java.util.ArrayList
```

Perché hanno la stessa classe?

Questo accade perché la type erasure rimuove l'informazione del tipo generico a tempo di esecuzione, quindi a runtime entrambe le variabili `la1` e `la2` sono trattate come istanze della stessa classe: `ArrayList`, perché l'informazione del tipo generico (`Plant` o `Animal`) viene persa a runtime.

4.3 Java Collection Framework - JCF

JCF è un insieme di **interfacce** e **classi** in Java che forniscono strutture dati e algoritmi per gestire e manipolare gruppi di oggetti.

La JCF offre un modo standardizzato per memorizzare, recuperare e manipolare dati in modo efficiente.

Le classi del JCF si trovano nel package `java.util` e nei relativi sotto-pacchetti.

Classi wrapper

I tipi primitivi (`int`, `char`, ecc.) spesso non possono essere utilizzati direttamente nelle raccolte.

Si usano le **classi wrapper** (es. `Integer`) che consentono di rappresentare tipi primitivi come oggetti.

Tipi generici

Le raccolte sono progettate per contenere **elementi omogenei** (tutti dello stesso tipo). I **generici** (es. `<T>`) garantiscono la sicurezza dei tipi, evitando errori di runtime.

Interfacce e implementazioni

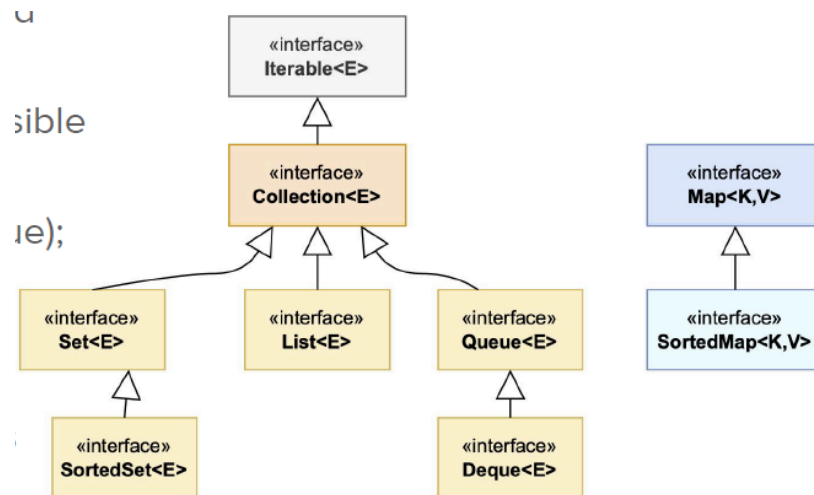
Le strutture dati sono definite da **interfacce** (es. `List`, `Set`, `Map`). Sono fornite **implementazioni di base** (es. `ArrayList`, `HashSet`) e altre più specializzate per casi d'uso specifici.

4.3.1 Gerarchie di interfacce

Il costrutto `Interface` rappresenta il modo più conveniente di lavorare con Java.

In base al modo in cui implementiamo certe strutture dati dobbiamo decidere quale implementazione di Interface è più appropriata al contesto.

Ogni raccolta è caratterizzata da funzionalità specifiche. Sono ideate interfacce per possibili tipi di dati astratti (*generici*).



4.3.2 Iterable

Nell'immagine sopra vediamo la gerarchia delle **Interface** in Java, l'Interface più alta nella gerarchia è l'**iterabile** che serve per scorrere ogni elemento di una collezione.

Si tratta di un'interfaccia che rappresenta una collezione di oggetti che può essere **iterata** (cioè attraversata) usando un **iterator**.

Poiché ogni classe che implementa **Iterable** è "iterabile", puoi usare le collezioni che la implementano in un ciclo **for-each**, che è una forma abbreviata del ciclo basato su iteratori.

```

List<String> list = new ArrayList<>();
list.add("apple"); list.add("banana");
for (String fruit : list) {
    System.out.println(fruit);
}
  
```

4.3.3 Map

Map è un array associativo, quindi un array composto da coppie <Chiave , Valore> e la Chiave può essere qualunque cosa.

Una mappa non è iterabile, possiamo accedere direttamente a qualunque suo elemento (è *come il dizionario in Python*).

In questo esempio abbiamo una mappa di stringhe e interi, nella quale la chiave è il nome del numero e il valore è il numero corrispondente.

```

Map<String, Integer> ht = new Hashtable();

ht.put("one", 1); ht.put("two", 2);

System.out.println(ht.get("two")); //--> 2

Set<String> htKeys = ht.keySet(); // set-view of keys
System.out.println(htKeys.contains("three")); //--> false

Deque<String> q = new LinkedList(htKeys);
q.addLast("four");
System.out.println(q.getLast()); //--> four

```

Interface ref variable

actual implementation class

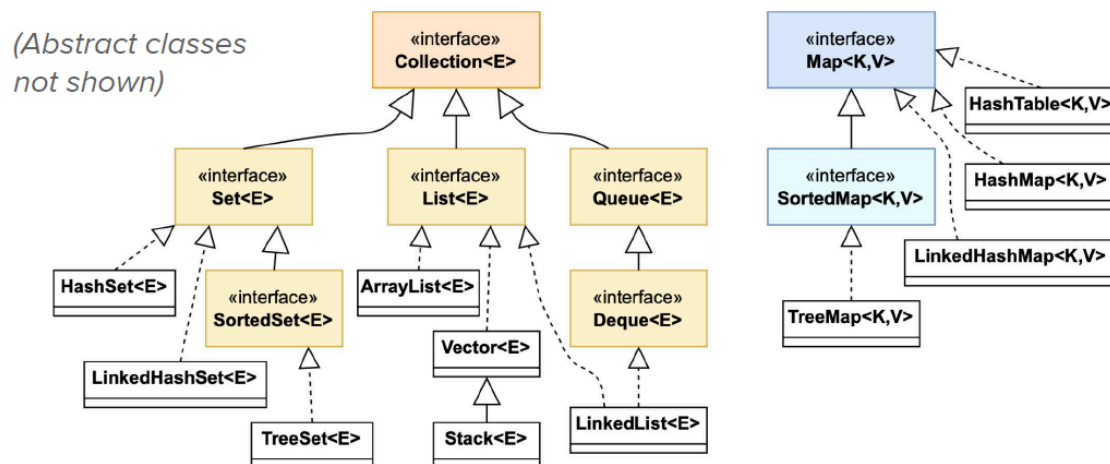
put is present in the Map interface

set in constructor of LinkedList

Methods present only in Deque

4.3.4 Implementation classes

In bianco vediamo che per la stessa interfaccia possiamo avere differenti implementazioni.



Per scegliere la giusta classe di implementazione di una certa interfaccia bisogna considerare diversi aspetti; poiché ogni classe offre un'organizzazione dei dati diversa, influenzando la complessità delle operazioni.

Si analizzano le operazioni più frequenti e si sceglie l'implementazione più efficiente in base alla complessità dei metodi.

Una buona pratica è dichiarare la variabile usando l'interfaccia anziché la classe concreta, per rendere il codice più flessibile e manutenibile.

Iterator

In questo esempio l'iteratore è di `Integer` visto che viene utilizzato su una lista di `Integer`.

Poiché `Collection` è una sottointerfaccia di `Iterable`, qualsiasi raccolta può essere attraversata tramite l'iteratore relativo.


```

List<Integer> l = new ArrayList<>();
l.add(5); l.add(7);

Iterator<Integer> iter = l.iterator();

while(iter.hasNext()) {
    System.out.println(iter.next());
}

```

This method returns the iterator

.hasNext() checks if another element is present

.next() returns the next element

forEach

In figura sotto vediamo due metodi per implementare delle iterazioni, tramite un **for** oppure tramite un **foreach**.

In certi casi serve utilizzare variabili, però invece di definire una classe specifica, viene utilizzata una classe anonima per indicare dove la variabile deve essere creata.

- Via “for (each)” statement
- Via method `Iterable.forEach`

(an implementation of Consumer must be provided)

```

// l is a list of Integers
for (Integer i : l)
    System.out.println(i);

```

Read as: “for i in l...”

```

// l is a list of Integers
Consumer<Integer> doPrint = new
    Consumer<Integer>() {
        public void accept(Integer nr) {
            System.out.println(nr);
        }
    };

l.forEach(doPrint);

```

Consumer is in java.util.function

Defined through an anonymous class

Other simpler ways to use forEach with Lambdas (not introduced yet)

4.3.5 Immutable collections

Le **immutable collections** in Java sono collezioni i cui contenuti non possono essere modificati dopo la loro creazione.

Per avere una collezione immutabile basta usare il metodo statico in figura sotto:

```

Set<String> set = new HashSet<>();
set.add("Hello"); set.add("Hi");
set = Collections.unmodifiableSet(set);

```

Un'alternativa è il metodo “of” del tipo di collezione specifico:

```

List<String> listStr = List.of("Hello", "Hi"); //immutable list
Set<String> setStr = Set.of("foo", "bar", "baz"); //immutable set

listStr.add("So long"); // -> UnsupportedOperationException

```

Altre librerie per strutture dati

In queste librerie c'è il supporto per la programmazione funzionale, è utile quando dobbiamo lavorare con strutture dati ricorsive.

Esistono altre librerie per supportare le strutture dati, oltre a JCF. Possiamo menzionare alcune scelte popolari:

- **Google Guava**: utilità di base + estensioni a JCF, con classi generiche aggiuntive; supporto alla programmazione funzionale, ecc.;
- **Collezioni Apache Commons**: parte della più ampia libreria "Apache Commons", con utilità assortite;
- **Collezioni Eclipse**: adozione di più modelli di iterazione; compatibile con JCF.

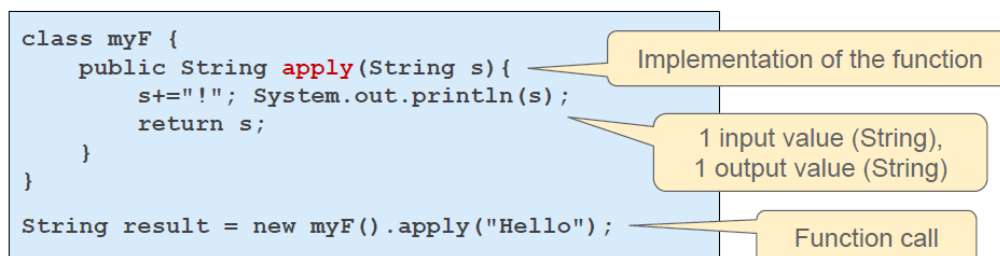
4.3.6 Interfacce e variabili funzionali

In Java, le **funzioni lambda** permettono di passare una funzione come parametro, semplificando il codice.

Funzioni rappresentate come oggetti

Essendo Java un linguaggio orientato agli oggetti, le funzioni possono essere rappresentate come oggetti di interfacce funzionali.

Il metodo principale che esegue la funzione viene spesso chiamato **apply**. Per usarlo, si crea un'istanza della funzione (o *lambda*) e si chiama metodo **apply** per eseguire l'operazione.



Questo approccio può essere concettualizzato attraverso interfacce funzionali.

Interfacce funzionali

In Java, le **interfacce funzionali** sono interfacce che hanno **un solo metodo astratto** (SAM - Single Abstract Method) che rappresenta il corpo della funzione.

@FunctionalInterface

```
interface Calculator {  
    int calculate(int a, int b);  
}  
  
// Implementazione con espressione lambda.  
Calculator add = (a, b) -> a + b;  
System.out.println(add.calculate(3, 5)); // Output: 8
```

Java offre alcune interfacce funzionali già pronte:

- `Predicate<T>` → Testa una condizione e restituisce un valore booleano.
- `Function<T, R>` → Trasforma un input di tipo **T** in un output di tipo **R**.

```
Function<String, Integer> length = s -> s.length();
```

```
System.out.println(length.apply("hello")); // 5
```

Variabili funzionali

In Java, una **variabile funzionale** è una variabile che fa riferimento a una **funzione** o a un'implementazione di una **interfaccia funzionale**. Grazie alle funzionalità introdotte con **Java 8**, le variabili funzionali possono essere create usando **espressioni lambda** o **method references** per rappresentare un comportamento.

Una variabile funzionale è dichiarata con il tipo di un'interfaccia **funzionale**. Può essere assegnata a un'espressione lambda o a un riferimento a un metodo.

```
@FunctionalInterface
interface Calculator {
    int calculate(int a, int b);
}

public class FunctionalVariableExample {
    public static void main(String[] args) {
        // Variabile funzionale usando un'espressione lambda
        Calculator add = (a, b) -> a + b;
        // Uso della variabile funzionale
        System.out.println("Sum: " + add.calculate(5, 3));
        // Output: Sum: 8
    }
}
```

Anche un metodo di classe come `Integer` potrebbe essere ragionevolmente considerato come un "valore" per tali variabili; è possibile fare riferimento a esse tramite la sintassi di riferimento al metodo (`::`).

```
Function<String, Integer> dec_int = Integer::decode;
System.out.println(dec_int.apply("10")); // Decimale: 10
```

4.3.7 Espressioni lambda

Un altro modo per fornire "valori" di funzione è tramite le **espressioni lambda**, note anche come "funzioni anonime".

Le espressioni lambda rappresentano un metodo per esprimere il target per variabili funzionali; ovvero per esprimere "cosa fare" in parametri effettivi.

```
(lista di parametri) -> { body della funzione }

(a, b) -> {
    int result = a + b;
    return result;
}
```

```
Function<Integer, Integer> myInc;  
myInc = (x) -> x+1;  
System.out.println(myInc.apply(10)); // -> 11
```

Possible target
for a lambda expression

Esempio ordinamento di un array

Quando vogliamo ordinare degli elementi, l'ordinamento deve avvenire secondo un certo criterio.

Quindi, assieme all'array possiamo passare anche il criterio come parametro, questo criterio in realtà è una funzione.

L'interfaccia `List<E>` ha il metodo `sort()`, che accetta un parametro (*un Comparator, interfaccia funzionale*) per specificare la regola di ordinamento.

Il comparatore decide, dati due elementi, quale dei due deve essere messo prima e può essere specificato da un'espressione lambda.

```
List<String> ls = new ArrayList();  
Collections.addAll(ls, "Hello", "Goodby", "Hi");  
  
ls.sort( (s1,s2) -> s1.length()-s2.length() );  
  
//ls: [Hi, Hello, Goodby] - sorted by length
```

Static method
to add elements to `ls`

Comparator specified
by a lambda expr.

4.4 Java streams

Sono una potente API per elaborare sequenze di dati in modo funzionale e dichiarativo.

Gli stream consentono di eseguire operazioni come filtraggio, trasformazione, ordinamento e riduzione su collezioni di dati (come `List`, `Set`, `Map`) o su altre sorgenti di dati, senza modificare i dati originali.

Stream

`Stream` è stata introdotta per raggiungere meglio questo obiettivo, come supporto di base per le operazioni in stile funzionale (*ad esempio, map-reduce*).

`Stream<T>`: interfaccia (*in `java.util.stream`*) per una sequenza di elementi di tipo `T`, destinata a subire una elaborazione futura.

Sono disponibili stream specifici (`IntStream`, ecc.) per i tipi primitivi.

Riguardo gli stream ci sono alcune cose da tenere in mente:

- Gli elementi di uno stream hanno origine da una o più sorgenti.
- **Gli stream sono immutabili:** alcuni dei loro metodi restituiscono nuovi stream.
- **Gli stream non contengono i loro elementi** come le raccolte!

Un oggetto `Stream` può essere ottenuto da `.stream()`, un metodo factory presente in una collezione: restituisce uno `Stream` sequenziale con tale collezione come origine.

In figura vediamo che utilizzando `.stream()` otteniamo uno stream dalla lista. Lavorare con gli stream è simile a lavorare con i database relazionali, il risultato finale può essere un singolo valore (come nel caso di `count()` nell'esempio, oppure un `resultset`).

```
List<String> ls = new ArrayList();  
Collections.addAll(ls, "Hello", "Goodbye", "Hi");  
Stream<String> ss = ls.stream();  
System.out.println(ss.count());
```

Aggregate
operation

Un'operazione di elaborazione su uno stream è chiamata operazione di aggregazione.

Le operazioni di aggregazione su uno stream possono essere concatenate per formare una pipeline. Altre possibili fonti per gli stream sono array, operazioni di I/O.

4.4.1 Stream pipeline

Una volta ottenuto un flusso da una fonte, è possibile specificare una pipeline di elaborazione su di esso.

Una pipeline è composta da 0 o più operazioni **intermedie**, e una e una sola operazione **terminale**.

Le **operazioni intermedie** trasformano un flusso in un altro flusso, alimentato all'operazione successiva;

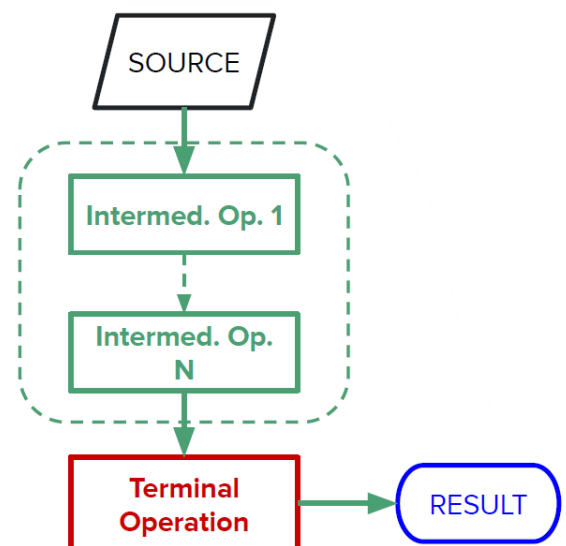
Le **operazioni terminali** producono un **risultato** o hanno un effetto collaterale.

Operazioni intermedie

Implementato tramite alcuni metodi dell'interfaccia `Stream`:

- `.limit(long maxSize)`: restituisce uno stream con solo fino a `maxSize` elementi.
- `.skip(long n)`: restituisce uno stream con gli elementi rimanenti di questo flusso dopo aver scartato i suoi primi `n` elementi.
- `.filter(Predicate<...> predicate)`: restituisce uno stream con solo gli elementi che matchano il predicato specificato.
- `.map(Function<...> mapperF)`: Restituisce uno stream costituito dai risultati dell'applicazione della funzione `mapperF` agli elementi di questo flusso.

Ovvero, `map` applica la funzione specificata a tutti gli elementi della lista.



- `.sorted()`: restituisce uno stream con gli elementi di questo flusso, ordinati secondo l'ordine naturale.

Operazioni terminali

Implementato tramite alcuni metodi dell'interfaccia Stream:

- `.count()`: Restituisce il conteggio degli elementi nello stream.
- `.min(Comparator<...> comp)`: Restituisce l'elemento minimo in base al comparatore (*letteralmente una funzione*) specificato.
- `.reduce(...)`: Sono disponibili diverse versioni dell'operazione reduce (è un'operazione terminale).

Ad esempio come argomenti ha una lista `l` e una funzione `funct2`, la funzione `reduce` lavora sugli elementi di `l` e se ad esempio `funct2` rappresenta la somma, combina i primi due elementi della lista in una somma fino ad ottenere un unico risultato finale.

- `.toArray()`: restituisce un array `Object[]` contenente gli elementi dello stream.
- `.collect(...)`: Utilizzato per raccogliere i risultati dell'elaborazione sugli elementi dello stream in un contenitore di risultati modificabile (*ad esempio una lista*).

Vediamo un esempio di una pipeline:

Data una lista `fruit`, creiamo uno stream `filteredFruit`.

La lista filtrata è creata partendo dalla prima applicando un filtro tramite un lambda in cui vengono restituiti solo i frutti il cui nome inizia con 'P'.

A quel punto si aggiunge uno step `map`, in cui applichiamo una funzione `toUpperCase` (*metodo della classe String*).

Infine si applica lo step `collect` per ottenere il risultato finale, specifichiamo il container in cui voglio mettere il risultato, ovvero una lista con `toList()`.

Otteniamo un risultato nella forma `[PEACH, PINEAPPLE]`.

```
...
List<String> fruit = Arrays.asList("Apple",
    "Peach", "Cherry", "Pineapple");

List<String> filteredFruit = fruit.stream()
    .filter(name -> name.startsWith("P"))
    .map(String::toUpperCase)
    .collect(Collectors.toList());

System.out.println(filteredFruit);
// --> [PEACH, PINEAPPLE]
```

4.4.2 Valutazione “pigra” degli stream

La valutazione dei risultati è fatta solo alla fine (*Lazy Evaluation*), ovvero quando si invoca l'ultima operazione.

Nell'esempio precedente, tutti gli step intermedi non sono eseguiti uno dopo l'altro (*questo è solo il modo in cui lo specifichiamo*) nella realtà tutti gli elementi della pipeline sono eseguiti in altro modo (*il modo in cui vengono effettivamente eseguiti dalla CPU punta ad ottimizzare*).

Nell'esempio sotto creiamo uno stream di stringhe, successivamente collezioniamo gli elementi dello stream in una lista.

```
Stream<String> fruitStream =  
    Stream.of("Apple", "Peach");  
List<String> fruitList =  
    fruitStream.collect(Collectors.toList());  
  
System.out.println(fruitList);    // --> [Apple, Peach]  
System.out.println(fruitStream.count()); // Exception!!!
```

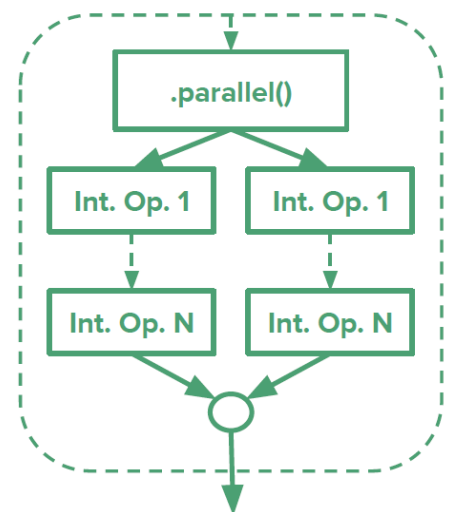
Cerchiamo di utilizzare lo stesso stream due volte ma non è possibile farlo, per questo viene restituita un'eccezione.

4.4.3 Parallelismo nella pipeline

Per ottenere un flusso parallelo da una sorgente: è possibile utilizzare `.parallelStream()` invece di `.stream()`.

Un flusso esistente può essere trasformato in parallelo con `.parallel()`.

Per trasformare flussi paralleli in sequenziali è possibile utilizzare `.sequential()`. In pratica, il flusso viene suddiviso per il calcolo parallelo e alla fine i risultati parziali vengono combinati.



4.5 Java annotations

Sono mezzi sintattici per aggiungere metadati a pezzi di codice. Le annotazioni non influenzano direttamente l'esecuzione del codice ma servono a fornire dei dettagli (*metadati*) e quindi per influenzare il comportamento durante il runtime o la compilazione.

Le annotazioni sono utili per tool esterni che devono interagire con il codice Java, ad esempio il compilatore.

Hanno molteplici scopi, ad esempio:

- **Informazioni per il compilatore:** Il compilatore può usare le annotazioni per rilevare errori o sopprimere avvisi.
- **Elaborazione in fase di compilazione e distribuzione:** Gli strumenti SW possono elaborare le informazioni di annotazione per generare codice, file XML, ecc.
- **Elaborazione in fase di esecuzione:** Alcune annotazioni possono essere esaminate a livello di programmazione in fase di esecuzione. Quindi vengono mantenute a livello bytecode.

Le annotazioni possono essere inserite in classi, interfacce, metodi, parametri di metodo, campi e variabili locali (*da Java 8 in poi*). I metadati delle annotazioni possono essere utilizzati da diversi strumenti in momenti diversi, in base alla relativa **conservazione** specifica.

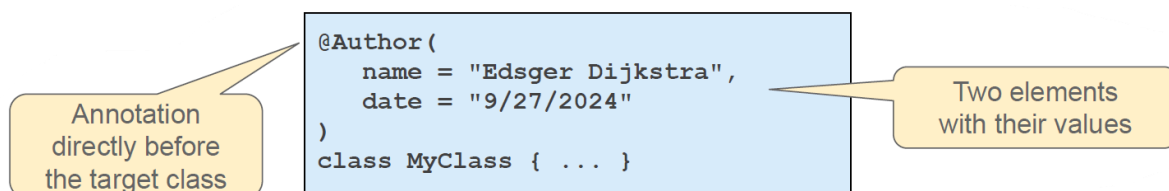
Politiche di conservazione standard

- **SOURCE:** Disponibile solo nel codice sorgente, scartato durante la compilazione. Le informazioni mantenute sono utili solo durante la compilazione.
- **CLASS:** Incluso nel file di classe ma non accessibile in fase di esecuzione.
- **RUNTIME:** Conservato in fase di esecuzione, eventualmente accessibile a livello di programmazione tramite reflection.

Le informazioni di annotazione sono espresse tramite il nome dell'annotazione e gli elementi di annotazione (*se presenti*).

Gli elementi di annotazione possono essere visti come "*campi*" dell'annotazione, con valori predefiniti che possono essere sovrascritti quando l'annotazione viene utilizzata.

Nell'esempio viene specificato l'autore e la data della classe `MyClass`.



Alcune annotazioni sono integrate, per dare istruzioni al compilatore:

- **@Deprecated:** Questa classe, metodo, ecc. non dovrebbe più essere utilizzata.

- Importante perché anche l'IDE potrebbe segnalare l'utilizzo della classe/metodo se viene utilizzato.
- **@Override**: Utilizzata sopra i metodi che sovrascrivono i metodi in una superclasse o che implementano i metodi di un'interfaccia. In caso di incongruenze, il compilatore genererà un errore.
- **@SuppressWarnings**: fa sì che il compilatore sopprima gli avvisi per un dato metodo.
- **@Contended**: `@jdk.internal.vm.annotation.Contended` viene utilizzata per evitare la falsa condivisione (*false sharing*).

Ad esempio in un sistema multicore abbiamo due variabili condivise X ed Y, la prima è utilizzata da un thread e l'altra da un altro thread, il compilatore inserisce in memoria X e successivamente inserisce Y, sono vicine in memoria quindi vengono inserite nella stessa cache line, e quindi ognuna sovrascrive l'altra nella cache e ognuno dei due thread ogni volta deve prelevare il valore della variabile dalla memoria.

Per risolvere questo problema viene risolto utilizzando due cache line separate per le due variabili.

Custom annotations

Le annotazioni personalizzate possono essere definite come **tipi di annotazione**, che possono essere definiti in modo simile alle interfacce (con **@interface**).

Nell'interfaccia a sinistra vengono definiti anche i valori di default, se quando viene utilizzata un'annotazione, non viene specificato un valore per un attributo viene assegnato il valore di default.

```
@interface ClassPreamble {
    String author();
    int currentRevision() default 1;
    String lastModified() default "N/A";
    String[] reviewers(); // an array
}
```



```
@ClassPreamble (
    author = "John Smith",
    lastModified = "4/30/2024",
    reviewers = {"Alice", "Bob"}
)
public class MyAnnotatedClass {...}
```

Capitolo 5 - Java Threads & Synchronization

5.1 Language-level Threading

Nell'implementazione della classe `Thread` ci sono tutte le funzionalità necessarie per il threading.

In Erlang quando specifichiamo il body di una funzione dobbiamo utilizzare un'altra funzione, anche con i thread in Java avviene qualcosa di simile, il body del thread corrisponde ad uno specifico metodo dedicato.

Definire un thread significa:

- Costruire tutte le strutture dati di supporto, eventualmente specificando attraverso di esse le caratteristiche del thread.
- Fornire le relative azioni di gestione.

Tutto questo, in Java, viene fatto utilizzando la classe `Thread`.

5.1.1 La classe `Thread`

La classe **`Thread`** in Java rappresenta un'unità di esecuzione indipendente all'interno di un programma, ovvero un **thread di esecuzione**. Un thread consente di eseguire più attività contemporaneamente (parallelismo o multitasking).

La classe **`Thread`** fa parte del package `java.lang` e permette di gestire la creazione, l'avvio e il controllo dei thread.

5.2 JVM Runtime Data Area

Oltre ad eseguire i programmi, tra i compiti della JVM c'è la gestione della memoria:

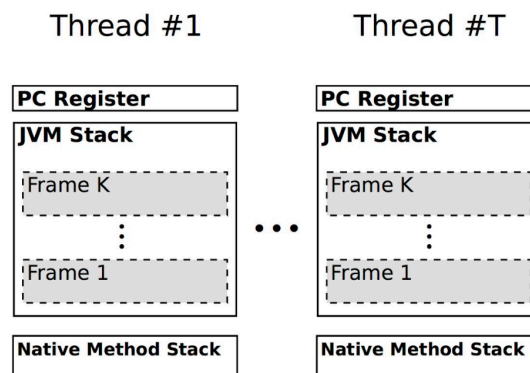
- **Allocazione dinamica** della memoria per oggetti e variabili.
- **Garbage collection** per liberare la memoria dagli oggetti non più utilizzati.

5.2.1 Stack

Ogni thread è una sequenza di istruzioni separata, quindi ogni thread ha un **program counter** e uno **stack** separato.

Lo **stack** è una zona della memoria usata per gestire i **thread di esecuzione** e per memorizzare **variabili locali**, **parametri di metodo** e informazioni di **contesto** relative ai metodi invocati.

Ogni thread ha il suo stack, in esso ogni thread effettua le chiamate di funzione, ogni operando viene inserito nello stack e le singole operazioni di bytecode opereranno sugli operandi dello stack.



In Java lo stack **memorizza variabili locali e informazioni sullo stato dei metodi in esecuzione**. È più veloce dello heap ma di dimensioni limitate e viene gestito in modo automatico dalla JVM.

Frame

In Java, un **Frame** è una struttura di dati utilizzata per rappresentare il contesto di **esecuzione** di un **metodo** all'interno dello **stack** di un thread.

Ogni volta che un metodo viene chiamato, viene creato un nuovo frame nello stack del thread, e quando il metodo termina, il frame viene rimosso.

Un frame contiene tutte le informazioni necessarie per l'esecuzione di un particolare metodo:

- **Array di variabili locali**: usato per memorizzare i parametri e le variabili locali del metodo.
- **Stack degli operandi**: utilizzato per eseguire operazioni temporanee (es. *somme, confronti, ecc.*).

- **Riferimento al runtime constant pool:** questo riferimento permette al frame di accedere alle costanti e ai riferimenti simbolici specifici della classe che contiene il metodo in esecuzione.

5.2.2 Method Area

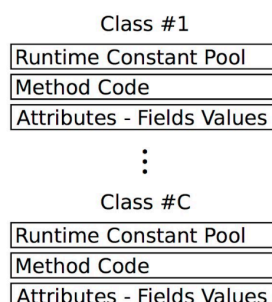
Parte della **JVM** che memorizza le informazioni relative alle classi caricate durante l'esecuzione di un programma Java.

È una zona della memoria dedicata a contenere le **strutture di dati** che descrivono i **metodi** e le **classi** caricate, come:

- **Bytecode delle classi** (ovvero i metodi compilati).
- **Metadati** (come il nome della classe, il tipo di variabili e metodi).
- **Costanti di classe** (come le costanti statiche finali).
- Il `Runtime Constant Pool`, che agisce in modo simile a una normale tabella dei simboli;

La method area è unica ed è condivisa tra tutti i thread.

METHOD AREA

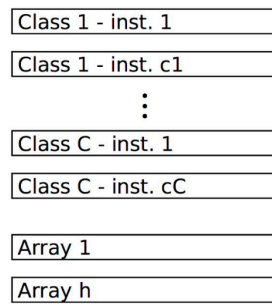


5.2.3 Heap

Parte della memoria gestita dalla JVM, **destinata a contenere gli oggetti creati durante l'esecuzione del programma.**

L'heap è utilizzato per la **memorizzazione dinamica degli oggetti**, e quindi ogni volta che un oggetto viene creato con l'operatore `new`, viene allocato nella heap.

HEAP



Lo Heap è condiviso tra tutti i thread ed è mirato a ospitare oggetti del programma.

Contiene le rappresentazioni di tutte le istanze di tutte le classi gestite nel programma, così come le rappresentazioni degli array.

Garbage collection

La **garbage collection** (GC) è responsabile della gestione dell'heap. Essa individua gli oggetti non più referenziati e li rimuove, liberando spazio.

Nelle moderne implementazioni JVM, l'heap è partizionato in sezioni gestite con policy diverse, rispetto alla garbage collection.

STRING CONSTANT POOL

Zona di **memoria ottimizzata** per conservare le **stringhe letterali** (costanti di tipo **String**) in modo che non vengano duplicate, risparmiando memoria e migliorando le performance.

Essa è una **zona speciale della memoria** nella **heap** in cui vengono conservate tutte le stringhe costanti, cioè quelle dichiarate direttamente nel codice (ad esempio, "**ciao**").

L'**interning** è un processo che consente di ottimizzare la gestione delle stringhe memorizzandole nel **STRING CONSTANT POOL** per evitare duplicati.

Quando si "interna" una stringa, la JVM verifica se una stringa identica è già presente nel **String Pool**. Se sì, restituisce un riferimento alla stringa esistente. Altrimenti, aggiunge la nuova stringa al pool.

5.3 Gestione dei thread in Java

In Java non c'è l'ereditarietà multipla, quindi non c'è possibilità di estendere ogni altra classe.

`Runnable` è un'interfaccia funzionale con un unico metodo astratto che è un target potenziale per un'espressione lambda.

Due modi di base:

- Estensione della classe `Thread`, ovvero "*Threading tramite sottoclasse*". Problema: non c'è nessuna possibilità di estendere un'altra classe.
- Utilizzo di un'istanza `Runnable` nel costruttore di `Thread`, un meccanismo più flessibile.

Generazione esplicita di thread: metodo di istanza "sbloccante" `mythread.start()`.

Definizione di un thread tramite `Runnable`

L'oggetto `Runnable` può essere definito tramite una classe che **implementa** `Runnable`, tramite una classe anonima o tramite una lambda a 0 parametri.

```
public class MyRunnable implements Runnable {
    @Override
    public void run() {
        System.out.println(Thread.currentThread().getName());
    }

    public static void main(String[] args) {
        Thread thread = new Thread(new MyRunnable());
        thread.start(); // Avvia il thread
    }
}
```

5.3.1 Terminazione di un thread

Normalmente un thread viene eseguito fino al termine delle sue istruzioni, per terminarlo senza che arrivi al termine possiamo fare qualcosa di simile a ciò che abbiamo visto in Erlang (*poison pill*), in questo caso un thread dice ad un altro di terminare.

Alcuni metodi iniziali sono stati deprecati a causa di problemi correlati:

- `stop()`: Un thread arrestato rilascia tutti i monitor bloccati, quindi può portare a incongruenze di stato.
- `suspend()`: Soggetto a deadlock: un thread può sospendersi in una sezione critica e un altro thread in grado di riprenderlo non può accedere al codice per farlo.

Soluzione: meccanismo cooperativo basato sull'**interruzione inter-thread** in cui un thread chiede a un altro di arrestarsi.

Inter-Thread Interruption

La **Inter-Thread Interruption** in Java è un meccanismo che consente a un **thread** di interrompere l'esecuzione di un altro thread.

L'interruzione di un thread non comporta direttamente l'interruzione immediata del suo lavoro, ma piuttosto segnala al thread che dovrebbe fermarsi o che è stata richiesta una sospensione del suo lavoro.

In Java, l'interruzione è gestita tramite il metodo `interrupt()` della classe `Thread`, che segnala a un altro thread che dovrebbe fermarsi o eseguire un'azione di pulizia. La gestione effettiva dell'interruzione dipende dal modo in cui il thread destinato all'interruzione risponde alla segnalazione.

Ogni `Thread` contiene un flag `interrupted` che può essere impostato da un altro thread, invocando il metodo `interrupt()` su di esso.

Un Thread può controllare il proprio flag interrotto tramite il metodo statico `Thread.interrupted()`, che cancella anche il flag (se il flag è `true` viene reimpostato a `false`).

Un esempio di quanto detto:

```
Thread thread1 = new Thread(() -> {
    while (!Thread.currentThread().isInterrupted()) {
        try {
            Thread.sleep(1000); // Pausa di 1 secondo
        } catch (InterruptedException e) {
            return; // Esce dal ciclo se il thread è interrotto
        }
    }
});
thread1.start(); // thread1 avviato.
Thread.sleep(3000);
thread1.interrupt(); // Il Thread principale interrompe thread1.
```

In genere, un thread che si accorge di essere stato interrotto viene programmato per eseguire azioni di pulizia e terminare.

Interruzioni e lock

Cosa succede se viene eseguita `interrupt()` su un thread bloccato in una `lock()`?

Dipende dal tipo di lock invocata, poiché c'è l'opzione di permettere l'interruzione o meno del thread bloccato.

Se è bloccato su una chiamata a un metodo che **supporta** l'interruzione allora:

1. L'interruzione risveglia il thread.
 - Viene lanciata un'eccezione `InterruptedException`.

- Il flag di interruzione viene automaticamente messo a `false`.

Se è bloccato su una chiamata a un metodo che **non supporta** l'interruzione allora:

1. L'interruzione **non** risveglia il thread.
2. Il thread rimane in attesa del monitor fino a quando non lo acquisisce.

Se un thread è in attesa di acquisire una lock, chiamare `interrupt()` sul thread non fa immediatamente liberare la lock. Il thread dovrà comunque completare l'attesa prima di rilasciare il controllo.

- Con `lock()`: il thread rimane bloccato e l'interruzione **non ha effetto**.
- Con `lockInterruptibly()`: l'interruzione risveglia il thread e viene lanciata una `InterruptedException`.

È particolarmente importante fornire la corretta gestione delle eccezioni di `interrupt`.

Linee guida

Oltre alle azioni di pulizia, dovremmo:

1. **Rilanciare l'eccezione**, dopo un possibile re-wrapping.

Il **re-wrapping** in contesti di gestione delle eccezioni in Java si riferisce al processo di **rilanciare un'eccezione** originaria all'interno di una nuova eccezione.

2. **Ripristinare il flag**: `Thread.currentThread().interrupt()`.

In modo da mantenere la memoria nel verificarsi dell'evento di `interrupt`.

5.4 Thread Pooling ed Executor Framework

Per organizzare il programma bisogna pensare in termini di task e non di thread, bisogna focalizzarsi sulle azioni da eseguire. Quindi è importante separare la definizione di un task e la sua esecuzione.

Principale preoccupazione nei programmi concorrenti: identificazione di azioni/task da eseguire, indipendentemente dal modo in cui verranno eseguiti.

In questi casi, il termine "task" viene utilizzato per le istruzioni relative.

Esempio: in un server, le richieste devono attivare task; task di questo tipo potrebbero essere eseguiti utilizzando il multithreading.

Separare definizione ed esecuzione

Un task può essere definito tramite una classe `Runnable`, nel metodo `run()`.

L'esecuzione del task può essere eseguita in molti modi diversi; Una soluzione generale richiede lo sviluppo di classi mirate a questo.

Tali classi devono implementare almeno la seguente interfaccia:

5.4.1 Thread pooling

I problemi di performance derivano dallo spreco di tempo nelle azioni di gestione, che può essere visto come sovraccarico nel nostro programma.

- **Context switch:** non solo richiedono tempo, ma determinano anche svuotamenti della cache che rallentano l'esecuzione successiva.
- **Contesa di blocco:** determina la serializzazione delle attività (*nessun parallelismo*), ostacolando la scalabilità: il tempo di attesa del thread dovrebbe essere ridotto.
- **Sincronizzazione della memoria:** molti thread potrebbero aver bisogno di vedere gli ultimi aggiornamenti riguardo un oggetto.

Alcune sincronizzazioni non necessarie possono essere rimosse dalla JVM (*elisione del blocco*) o tramite analisi di escape statica per individuare gli oggetti locali del thread.

Ad esempio, se abbiamo un codice come sotto in cui c'è un blocco `synchronized()` che rappresenta il meccanismo in Java per rappresentare le `lock`, ovvero un oggetto che agisce come `lock` può essere acceduto da un singolo processo alla volta.

```
synchronized (new Object()) {  
    foo();  
}
```

È raccomandabile minimizzare il tempo per creare thread.

Inoltre bisogna stare attenti al numero di thread presenti nel sistema in relazione al numero di core: se un sistema ha 16 core non ha senso creare 100 thread perchè dovendo essere schedulati su 16 core verrebbe introdotto troppo overhead.

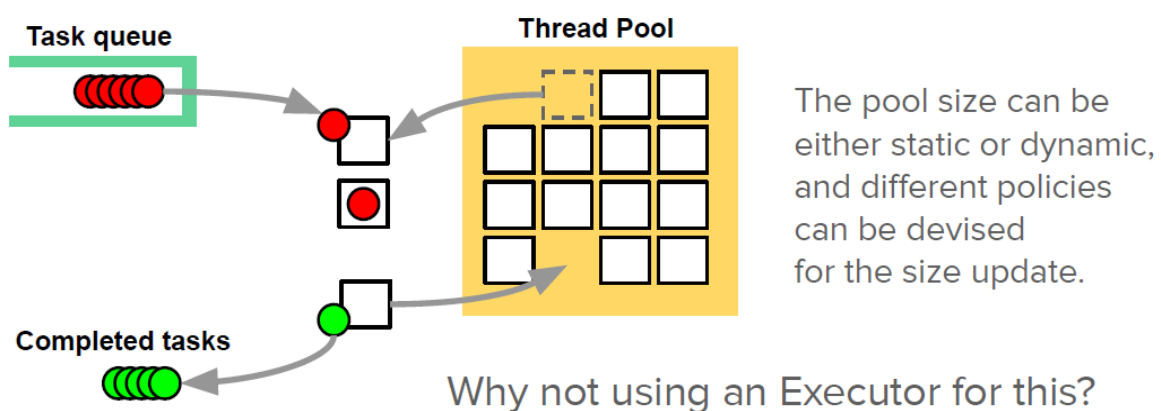
Thread Pooling

La creazione/eliminazione di un thread determina un sovraccarico computazionale, quindi l'idea è quella di non distruggere mai un thread e quindi di non doverlo mai creare di nuovo.

Supponiamo che vengano creati in anticipo una buona quantità di Thread.

Questa quantità è detta **dimensione del pool** e in certe situazioni deve essere statica (*costante*).

1. Ogni volta che c'è un nuovo task, esso viene inserito in una coda.
2. Viene preso un thread dal pool per eseguire uno dei task nella coda.
3. Una volta completato, il thread ritorna disponibile nel pool per eseguire un nuovo task.



Il punto quindi è: non distruggere il thread ma renderlo nuovamente disponibile per un'altra attività.

5.4.2 L'interfaccia `Executor`

L'interfaccia `Executor` in Java fa parte del pacchetto `java.util.concurrent` e rappresenta un'interfaccia fondamentale per la gestione di thread e attività asincrone.

Fornisce un'astrazione per eseguire task senza doversi occupare direttamente della creazione e gestione dei thread.

```
public class ExecutorExample {  
    public static void main(String[] args) {  
        Executor executor = Executors.newSingleThreadExecutor();  
        Runnable task = () -> System.out.println("ciao"); // Definisce il task  
        executor.execute(task); // Esegue il task  
    }  
}
```

Oppure se volessimo creare un'implementazione personalizzata di `Executor`.

```
public class CustomExecutor implements Executor {  
    @Override  
    public void execute(Runnable myTask) {  
        // Esegue ogni task in un nuovo thread  
    }  
}
```

```

        new Thread(myTask).start();
    }

    public static void main(String[] args) {
        Executor customExecutor = new CustomExecutor();
        // Invia due task all'executor
        customExecutor.execute(() -> System.out.println("ciao1"));
        customExecutor.execute(() -> System.out.println("ciao2"));
    }
}

```

Esempi di implementazioni di execute

In figura sotto, nel primo caso abbiamo una semplice chiamata di funzione.

```

@Override
public void execute(Runnable myTask) {
    myTask.run();
}

```

Mentre nel secondo caso viene creato un thread il cui body corrisponde a `myTask` e il thread viene mandato in esecuzione.

```

@Override
public void execute(Runnable myTask) {
    new Thread(myTask).start();
}

```

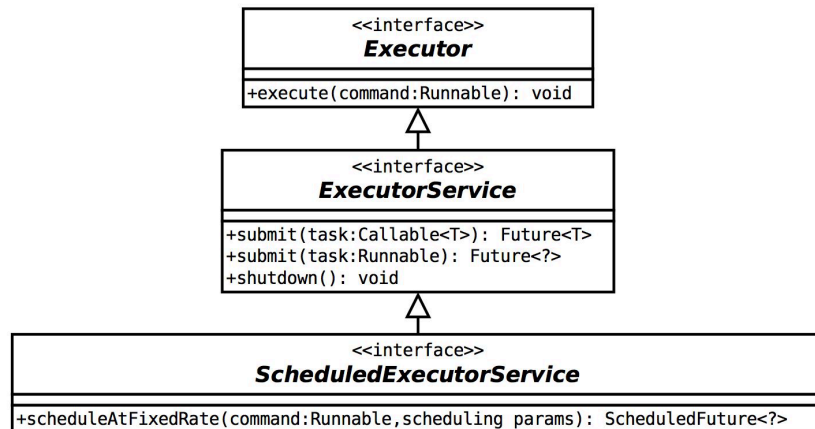
Un task può essere eseguito come parte dello stesso thread che ne richiede l'esecuzione. Oppure possiamo generare un thread completamente nuovo per eseguire un task.

Vantaggi dell'interfaccia Executor

- **Separazione delle preoccupazioni:** si separa la logica di invio dei task dall'implementazione della gestione dei thread.
- **Gestione scalabile dei thread:** gli Executor sfruttano il pooling di thread, controllo della concorrenza e gestione delle risorse.
- **Codice più leggibile e flessibile:** il framework consente di astrarre l'uso di thread in modo uniforme e gestibile.

Le classi sono:

- **Executor:** solo supporto per l'esecuzione di un task.
- **ExecutorService:** supporto per la gestione del ciclo di vita sia del task che dell'executor.
- **ScheduledExecutorService:** supporto per l'esecuzione temporizzata/periodica dei task.



5.4.3 Il Task e la classe Future

Gli **Executor** possono essere progettati per implementare una politica di esecuzione, quindi è importante distinguere l'invio e l'esecuzione delle attività.

Questa è la logica alla base dei metodi in **ExecutorService**, dove gli esecutori sono un mezzo per separare l'invio del task e la sua esecuzione.

La gestione delle esecuzioni può essere più complessa del previsto, poiché devono essere considerati alcuni aspetti aggiuntivi:

- Un risultato di attività potrebbe dover essere gestito in modo esplicito;
- Potrebbe essere richiesta la sincronizzazione per ottenere il risultato.

Definizione di un task - Interfaccia Callable

Ogni volta che un risultato di un task deve essere gestito, il task deve essere implementato in base all'interfaccia funzionale **Callable**:

```
public interface Callable<V> {
    V call() throws Exception;
}
```

Come recuperare il valore restituito dal task?

Dipende dal modo in cui viene inviata l'attività: ad esempio `submit()` restituisce un `Future<V>`, che a sua volta includerà il risultato così come sarà calcolato dall'attività (e accessibile tramite `get()`).

Recuperare il risultato di un task - Future

Un **Future** è un costrutto thread-safe utilizzato per rappresentare il risultato di un'operazione asincrona che potrebbe non essere ancora completata.

Fornisce un meccanismo per eseguire un'operazione in un thread separato e recuperare il risultato in un secondo momento.

È essenzialmente un **proxy** per un valore che sarà disponibile in futuro. Il valore per un future può essere scritto solo una volta.

La funzione incaricata di impostare il valore ad un **Future** è spesso denominata **promise**.

Diverse possibili promise possono impostare il valore di un dato **Future**, sebbene ciò possa essere fatto solo una volta per un dato **Future**.

La Promise è la controparte del **Future**, consentendo di impostare il valore. In alcuni linguaggi, **Future** e Promise sono distinti:

- **Promise**: l'entità che imposta il risultato.
- **Future**: l'entità che legge il risultato.

Callable è un'interfaccia funzionale, però in questo caso viene restituito un risultato. Il comportamento di un “*future*” è descritto dall'interfaccia relativa.

```
public interface Future<V> {
    boolean isDone();
    V get(); //also a timed version exists
    boolean cancel(boolean m);
    boolean isCancelled();
}
```

È possibile controllare se un risultato è pronto (**isDone()**), recuperarlo (**get()**), provare ad annullare l'esecuzione di un'attività, controllare se un'attività è stata annullata.

```
public class FutureExample {
    public static void main(String[] args) {
        // Creazione di un ExecutorService con un thread
        ExecutorService executor = Executors.newSingleThreadExecutor();
        // Task che calcola il quadrato di un numero
        Callable<Integer> task = () -> {
            Thread.sleep(2000); // Simula un'operazione lunga
            return 42 * 42; // Risultato
        };
        // Sottomette il task all'ExecutorService
        Future<Integer> future = executor.submit(task);
        // La promise è stata restituita, il calcolo è in esecuzione
        System.out.println("Task sottomesso. Aspetto il risultato...");
        try {
            Integer result = future.get(); // Recupera il risultato del task
            // Bloccante fino a quando il risultato è disponibile
            System.out.println("Risultato: " + result);
        } catch (Exception e) {
            System.err.println(e.getMessage());
        } finally {
            executor.shutdown(); // Arresta l'ExecutorService
        }
    }
}
```

Il metodo **submit(task)** restituisce un **Future** immediatamente. Questo rappresenta il risultato del calcolo asincrono.

5.4.4 Off-the-shelf ExecutorServices

`java.util.concurrent.Executor` ha svariati metodi built-in per creare istanze di esecutori che sfruttano il **Thread Pooling** e anche il **work stealing**.

- `newFixedThreadPool(int n)` crea un pool di `n` thread, con dimensione fissa;
- `newCachedThreadPool()` fornisce un pool di thread che crea nuovi thread quando necessario, ma riutilizzerà i thread creati in precedenza quando saranno disponibili;
- `newWorkStealingPool(int parallelism)` crea un pool con thread gestiti dinamicamente, in grado di gestire il livello di parallelismo specificato; l'ordine di esecuzione dell'attività non riflette necessariamente l'ordine di invio.

Un piccolo esempio:

```
public class FixedThreadPoolExample {
    public static void main(String[] args) {
        // Crea un pool di thread con 3 thread
        ExecutorService executor = Executors.newFixedThreadPool(3);

        // Simula 6 task da eseguire
        for (int i = 1; i <= 6; i++) {
            int taskId = i; // Variabile finale per il lambda
            executor.submit(() -> {
                try {
                    Thread.sleep(2000); // Simula un'operazione lunga
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            });
        }
        // Chiude il pool dopo l'esecuzione di tutti i task
        executor.shutdown();
    }
}
```

Mettendo `n = 3`, Limita i thread attivi, prevenendo un uso eccessivo delle risorse.

Questi thread pool hanno le seguenti caratteristiche:

- **Code di task separate:** Ogni thread ha tipicamente una propria coda di task (*in una struttura tipo deque, doppia coda*).
- **Esecuzione normale:** Un thread esegue i task prelevandoli dalla **testa** della propria coda.
- **Stealing:** Se un thread rimane senza lavoro, tenta di rubare un task **dal fondo** della coda di un altro thread.
- **Efficienza:** Questo approccio riduce il tempo di inattività dei thread (*idle time*) e distribuisce il carico di lavoro in modo dinamico.

ScheduledExecutorService

Interfaccia per un `ExecutorService` che può pianificare i comandi da eseguire dopo un ritardo dato, o da eseguire periodicamente.

Implementazione standard restituita dal metodo `factory` in "Executors":

`newScheduledThreadPool(int n)` crea un pool di `n` thread, da pianificare (spesso usato con `n=1`).

```
public class ScheduledExecutorServiceExample {
    public static void main(String[] args) {
        // Crea un ScheduledExecutorService con un pool di 2 thread
        ScheduledExecutorService scheduler =
            Executors.newScheduledThreadPool(2);

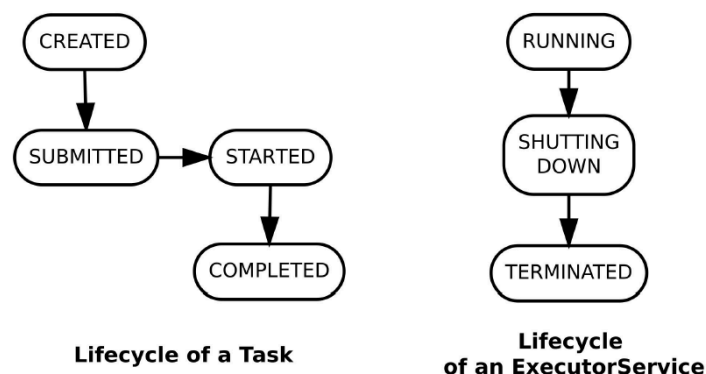
        // Task 1: Esegue un task con un ritardo iniziale di 3 secondi
        scheduler.schedule(() -> {
            System.out.println("ciao"); }, 3, TimeUnit.SECONDS);

        // Task 2: Esegue un task ogni 2 secondi, con un ritardo di 1 secondo
        scheduler.scheduleAtFixedRate(() -> {
            System.out.println("ciao"); }, 1, 2, TimeUnit.SECONDS);

        // Task 3: Esegue un task con un intervallo fisso dopo la fine di ciascuna
        // esecuzione
        scheduler.scheduleWithFixedDelay(() -> {
            System.out.println("ciao"); }, 2, 3, TimeUnit.SECONDS);

        // Arresta il scheduler dopo 10 secondi
        scheduler.schedule(() -> {
            scheduler.shutdown(); }, 10, TimeUnit.SECONDS);
    }
}
```

Come fermare lo Executor



Se vogliamo stoppare un servizio `Executor`, potrebbero esserci molti task nella coda che sono pronti ad essere eseguiti, in questo caso abbiamo lo `shutdown`, in modo da terminare gli incoming task e poi fermarsi.

Con l'opzione `shutting down`, se ci sono dei task pronti, prima vengono eseguiti, poi il thread viene terminato.

Operazioni distinte, come metodi di `ExecutorService`:

- `shutdown()`: avvia la procedura di arresto; le attività inviate in precedenza vengono completate e non verrà accettata alcuna nuova attività.
- `shutdownNow()`: vengono completate solo le attività in esecuzione.
- Controlla lo stato tramite `isShutdown()`; `isTerminated()` controlla se tutte le attività sono state terminate.
- I metodi sopra sono non bloccanti; per attendere il completamento di tutte le attività, chiama `awaitTermination()`.

5.5 Java synchronizers

La cooperazione tra thread richiede spesso azioni di sincronizzazione specifiche, da eseguire in base a un dato schema.

Un costrutto progettato per indirizzare uno specifico schema di sincronizzazione è chiamato "*Synchronizer*" e coordina il flusso di controllo dei thread in base al suo stato interno.

Java fornisce diverse classi per la sincronizzazione comune per scopi speciali. Tra queste: Latch, Barriera, Semafori, Fasori, Exchanger.

5.5.1 Latch

Permette ad un gruppo di thread di sospendere l'esecuzione fino al raggiungimento di una certa condizione (*ad esempio attendere che altri thread completino un certo numero di operazioni*).

Un'istanza di un latch può essere usata solo una volta. In java si ha il `countDownLatch`, il quale si basa su un contatore che una volta arrivato a 0 sblocca il latch.

Per fare ciò si usano due operazioni `countDown()` e `getCount()`.

Un thread attende fino al punto di rilascio nel tempo invocando `await()`.

Esiste anche una versione temporizzata del latch, che si basa sul tempo trascorso.

5.5.2 Barriera

Simile al latch. Permette ad un gruppo di thread di incontrarsi ad un certo punto. L'esecuzione continua quando tutti i thread sono arrivati alla barriera.

Un'istanza di barriera può essere usata più volte usando il comando `reset()`.

Una barriera fa sì che un gruppo di thread si incontri in un punto barriera. Un thread che raggiunge la barriera attende che arrivino tutti gli altri.

Quando arriva l'ultimo thread, tutti i thread vengono rilasciati per procedere.

L'implementazione Java di riferimento è `CyclicBarrier`.

Un oggetto `CyclicBarrier` può essere riutilizzato invocando `reset()` su di esso.

Un thread attende fino al punto di rilascio nel tempo invocando `await()` e rimane bloccato finché tutti i thread richiesti non lo fanno. C'è anche una versione temporizzata.

È possibile specificare anche un' *azione barriera* facoltativa.

5.5.3 Fasori

Versione più sofisticata di un latch/barriera, è un sincronizzatore riutilizzabile.

Similarmente a una barriera, coordina più thread attraverso più fasi, ma è più flessibile.

A differenza di `CyclicBarrier`, il numero di parti può cambiare nel tempo: è progettato per funzionare con thread dinamici e con più fasi successive.

5.5.4 Semafori

Un semaforo ha lo scopo di controllare il numero di accessi a una risorsa. Supporta quindi una generalizzazione di mutua esclusione.

In Java, la classe `Semaphore` implementa questo sincronizzatore. Il numero di accessi è controllato tramite permessi; il numero massimo di permessi disponibili è impostato al momento dell'inizializzazione.

Non dipende dal numero di thread; è un contatore di risorse.

`acquire()` è utilizzato per ottenere un permesso; se non è disponibile, si blocca.

`release()` è utilizzato per restituire un permesso al semaforo.

I nomi classici corrispondenti sono `P()` / `V()` o `down()` / `up()`.

5.5.5 Exchanger

Un exchanger ha lo scopo di far sì che due thread si aspettino l'un l'altro in un punto di sincronizzazione e scambiano oggetti. Consente a due thread di scambiarsi dati in modo diretto e sincronizzato.

In Java, è implementato dalla classe `Exchanger<V>`. `V` è il tipo di oggetto da scambiare.

Esempio di codice per un'operazione di scambio:

```
myBuffer = exchanger.exchange(myBuffer);
```

5.6 Thread-safe Data Structures

Un primo banale tentativo di definire una raccolta thread-safe è di rendere i suoi metodi "sincronizzati". Ciò può essere fatto in modo sistematico utilizzando il pattern Decorator.

Sono sicuri da utilizzare in un ambiente multithread, ovvero non avvengono corruzioni di dati a causa di data races.

Un modo standard per decorare una raccolta ordinaria può consentirci di ottenere una versione thread-safe corrispondente.

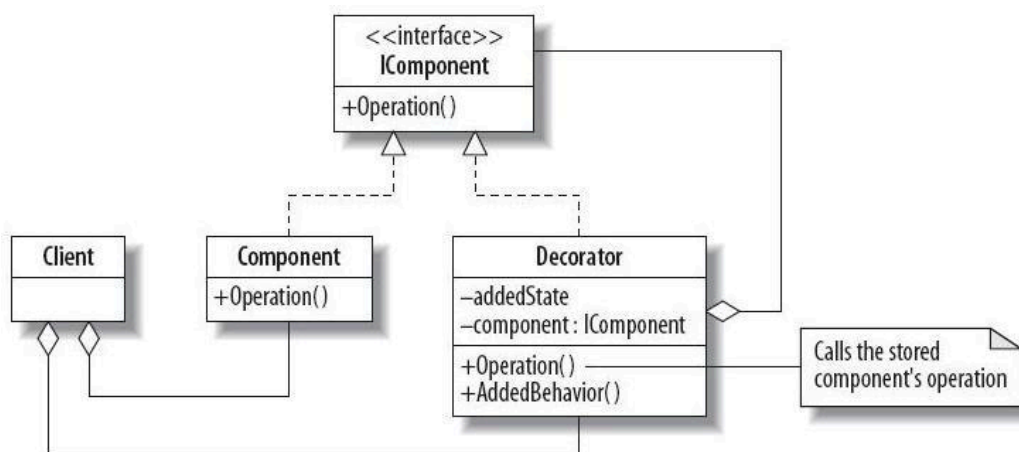
Metodi come `isEmpty()` o `size()` non sono realmente significativi in un contesto concorrente, poiché il valore restituito potrebbe non essere necessariamente accurato solo nell'operazione successiva.

5.6.1 Decorator Pattern

Vogliamo fornire un'altra implementazione in cui ogni cosa è come prima, però c'è la possibilità di aggiungere alcune informazioni aggiuntive. Trasforma ogni metodo all'interno delle collezioni in metodi sincronizzati.

Il Decorator Pattern è un **design pattern strutturale** che consente di aggiungere dinamicamente nuove funzionalità a un oggetto senza alterarne il comportamento o modificarne la struttura originale.

In Java, questo pattern si realizza avvolgendo un oggetto esistente con un nuovo oggetto chiamato **decorator**, che implementa la stessa interfaccia o eredita la stessa classe del componente originale.



```
// Interfaccia
interface Message {
    String getContent();
}

// Decoratore astratto per l'interfaccia Message.
abstract class MessageDecorator implements Message {
    protected Message wrappedMessage;
```

```

public MessageDecorator(Message message) {
    this.wrappedMessage = message;
}
@Override
public String getContent() {
    return wrappedMessage.getContent();
}
}

```

5.6.2 Collezione sincronizzata

Le synchronized collections in Java sono **collezioni thread-safe** che sincronizzano l'accesso dei thread ai loro metodi.

Questo significa che solo un thread alla volta può accedere ai metodi della collezione, prevenendo problemi di concorrenza come **race condition**.

Factory method statici specifici nella classe `java.util.Collections` del formato `.synchronizedXXX(XXX<T> ds)` restituiscono versioni sincronizzate della struttura dati semplice `ds`.

XXX sta per l'interfaccia `Collection`, o per `List`, `Map`, ecc. L'iterazione sulle raccolte sincronizzate necessita di una sincronizzazione esplicita sull'intera struttura dati, ad esempio:

```

public class SynchronizedCollectionExample {
    public static void main(String[] args) {
        // Creazione di una lista normale e di una lista sincronizzata
        List<String> synchronizedList = Collections.synchronizedList(new ArrayList<>());
        // Creazione di un executor per gestire i thread
        ExecutorService executor = Executors.newFixedThreadPool(2);
        // Task per aggiungere elementi a una lista
        Runnable addToSynchronizedList = () -> {
            for (int i = 0; i < 5; i++) {
                synchronizedList.add("Thread-2 Elemento " + i);
            }
        };
        // Esecuzione contemporanea dei task
        executor.execute(addToSynchronizedList);
        executor.execute(addToSynchronizedList);
        // Chiusura dell'executor
        executor.shutdown();
        // Aspetta il completamento di tutti i thread
        while (!executor.isTerminated()) {}
    }
}

```

In questo modo siamo sicuri che nessun altro metodo sincronizzato all'interno della collezione verrà eseguito da un altro thread.

Una raccolta "*sincronizzata*" mostra scarse prestazioni in condizioni di alta contesa.

Motivo principale: il livello di sincronizzazione è organizzato in modo coarse-grained, ovvero viene organizzato in blocchi più grandi del necessario, alcuni blocchi non sono strettamente necessari certe volte.

Soluzione: riduzione del sovraccarico di sincronizzazione, applicando qualsiasi trucco possibile.

Sono state sviluppate versioni "concorrenti" di raccolte "sincronizzate": ad esempio, `ConcurrentHashMap` è un miglioramento di `SynchronizedMap`.

5.6.3 ConcurrentHashMap

Classe del pacchetto `java.util.concurrent` di Java che fornisce un'implementazione thread-safe per le mappe.

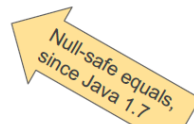
Implementa l'interfaccia `ConcurrentMap<K,V>`, che include anche operazioni di lettura e scrittura atomiche, ad esempio:

```
boolean replace(K key, V oldValue, V newValue);
```

- `key` punta ad una entry della tabella di hash.
- `oldValue` rappresenta il valore vecchio corrispondente a tale chiave (*deve corrispondere esattamente al valore che c'era precedentemente*).
- `newValue` è il valore che vogliamo inserire.

La funzione `replace()` equivalente all'esecuzione atomica di:

```
if (map.containsKey(key) && Objects.equals(map.get(key), oldValue)) {  
    map.put(key, newValue);  
    return true;  
} else return false;
```



La `concurrentHashMap` è consigliata da utilizzare ogni volta che dobbiamo implementare una struttura dati condivisa nel progetto poiché thread-safe.

5.6.4 Iteratori per collezioni concorrenti

L'iterazione su una raccolta concorrente è rischiosa poiché altri thread potrebbero accedere alla struttura dati nello stesso momento e quindi portare a delle collisioni e a risultati imprevedibili.

Gli stessi problemi si presentano con i cicli `for-each`, che vengono compilati nella forma corrispondente basata sull'iteratore.

È necessario specificare una nuova semantica per un iteratore concorrente, in modo da descriverne completamente il comportamento in caso di possibili interferenze.

Fail-Fast Iterators

Fornito dalla maggior parte delle raccolte **non concorrenti**.

Iteratori che **rilevano modifiche** (*sulla base del best effort*) **strutturali nella collezione durante l'iterazione** e generano una `ConcurrentModificationException` se queste modifiche vengono rilevate.

```
Iterator<String> iterator = list.iterator();
```

Quindi usando questi iteratori, non è consentita alcuna modifica durante lo scorrimento.

Meglio (*più sicuro*) rimuovere un elemento tramite il metodo `remove()` fornito dall'iteratore e non il metodo `remove()` fornito dalla raccolta.

In una collezione vengono mantenuti dei contatori per descrivere come lo stato evolve.

Weakly-consistent Iterators (*Fail-Safe*)

Sono forniti da collezioni concorrenti nel pacchetto `java.util.concurrent` (es. `ConcurrentHashMap`, `CopyOnWriteArrayList`).

Questi iteratori garantiscono:

1. Consistenza debole:

- Riflettono lo stato della collezione nel momento in cui l'iteratore è stato creato, più eventuali modifiche fatte durante l'iterazione.
- Non lanceranno mai una `ConcurrentModificationException`, a differenza dei fail-fast.
- Possono (*ma non è garantito che lo facciano*) riflettere qualsiasi modifica successiva alla costruzione.

2. Thread-safe:

- Consentono operazioni di lettura e modifica contemporanee da più thread.

```
ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();  
map.put("A", 1); map.put("B", 2);  
Iterator<String> iterator = map.keySet().iterator(); // Iteratore
```

Se utilizziamo questo tipo di iteratore, differenti thread possono operare su esso allo stesso tempo, in questo caso perdiamo alcune garanzie.

Se avviene una modifica durante l'iterazione da parte di un altro Thread: l'iteratore non riflette subito la modifica; lavora su una versione coerente al momento della sua creazione.

Snapshot Iterators

Tipo di iteratore utilizzato da alcune collezioni concorrenti, come `CopyOnWriteArrayList` e `CopyOnWriteArraySet`, per garantire un **comportamento thread-safe durante l'iterazione**.

L'iteratore fornisce un'istantanea dello stato della raccolta quando è stato costruito.

Chiamati così perché "*congelano*" lo stato della collezione come era all'inizio; lavorano sulla collezione come era all'inizio. In caso di modifica verrà creata una nuova copia separata e non verrà vista dall'iteratore.

Non è necessaria alcuna sincronizzazione durante l'attraversamento dell'iteratore, poiché mentre l'iteratore scorre la collezione non può cambiare.

L'iteratore non fornisce un metodo `remove()`, poiché lavoriamo come se la collezione fosse immutabile.

In tali raccolte, tutte le "operazioni mutative" vengono implementate creando una nuova copia della struttura dati sottostante, ad es. *Copy-On-Write (COW)*.

5.6.5 Blocking Queues

Le **BlockingQueue** sono un'interfaccia del pacchetto `java.util.concurrent` che rappresentano una coda thread-safe progettata per gestire la sincronizzazione tra produttori e consumatori.

Versione thread-safe ad alte prestazioni di un buffer limitato, progettato in base all'interfaccia `BlockingQueue<E>`.

Sono presenti metodi per supportare l'inserimento, estrazione e ispezione in diversi modi (*blocco, generazione di eccezioni, polling, temporizzato*).

Le blocking queue supportano il modello produttori/consumatori. Queste strutture rendono la gestione delle risorse più affidabile.

```
public class BlockingQueueExample {
    public static void main(String[] args) {
        // Creazione di una coda con capacità di 5 elementi
        BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(5);
        Thread producer = new Thread(() -> { // Produttore
            try {
                for (int i = 1; i <= 10; i++) {
                    queue.put(i); // Blocca se piena
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        Thread consumer = new Thread(() -> { // Consumatore
            try {
                while (true) {
                    Integer value = queue.take(); // Blocca se vuota
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        producer.start(); // Avvio dei thread
        consumer.start(); // Avvio dei thread
        try { // Attendere la terminazione
            producer.join(); consumer.join();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

5.7 Memory Consistency Rules

Insieme di regole che definiscono come le operazioni di lettura e scrittura su variabili condivise tra thread devono essere gestite per garantire la coerenza e la visibilità dei dati in un ambiente multi-threaded.

Queste regole fanno parte del **Java Memory Model (JMM)**, che stabilisce il comportamento delle variabili condivise tra thread e come devono essere sincronizzate per evitare inconsistenze.

5.7.1 Lazy Singleton Initialization

Un **metodo** per creare un'istanza di una classe **Singleton** solo quando è effettivamente necessaria, anziché durante l'inizializzazione della classe.

Questo approccio è utile per ottimizzare le risorse, poiché la classe viene istanziata solo quando un thread o un componente ne ha effettivamente bisogno (*Lazy*).

Un metodo factory statico deve restituire la singola istanza, che di solito è impostata con inizializzazione lazy perché l'oggetto non viene creato finché non è necessario il suo utilizzo (*non viene spreca memoria*).

La soluzione sotto non è Thread-Safe.

```
public class Singleton {
    private static Singleton ref; // l'istanza della classe.
    // Costruttore privato per prevenire istanziazione da parte di altre classi
    private Singleton() {}
    // Metodo per ottenere l'istanza
    public static Singleton getInstance() {
        if (ref == null) {
            ref = new Singleton();
            // Creazione dell'istanza quando è necessaria
        }
        return ref;}}}
```


5.7.2 Double-Checked Locking e la keyword `volatile`

Versione errata

Viene effettuato un doppio controllo di `ref`, ma in questo modo non funziona correttamente.

```
public class Singleton {
    private static Singleton ref; // l'istanza della classe.
    private Singleton() {}
    public static Singleton getInstance() {
        // Primo controllo: se l'istanza esiste già, non è necessario sincronizzare
        if (ref == null) {
            synchronized (this) {
                // Secondo controllo: all'interno del blocco sincronizzato
                if (ref == null) {
                    ref = new Singleton();
                }
            }
        }
        return ref;
    }
}
```

Il problema è la pubblicazione non sicura di `ref`, a causa della visibilità degli aggiornamenti della memoria.

Ovvero quando cambiamo qualcosa, prima di poter accedere a ciò che è stato modificato bisogna essere sicuri che la creazione sia stata effettivamente completata.

Versione corretta

La dichiarazione `volatile` è cruciale per garantire che la creazione dell'istanza di `Singleton` sia visibile a tutti i thread, evitando che i thread leggano una versione cache dell'oggetto.

`volatile` assicura che, quando l'istanza viene scritta in memoria, la scrittura sia visibile immediatamente a tutti i thread, evitando problemi di visibilità nella cache.

In parole povere, con `volatile` non è possibile accedere a `ref` senza che le operazioni di scrittura non siano completate.

```
public class Singleton {
    private static volatile Singleton ref;
    private Singleton() {}
    public static Singleton getInstance() {
        // Primo controllo: se l'istanza esiste già, non è necessario sincronizzare
        if (ref == null) {
            synchronized (this) {
                // Secondo controllo: all'interno del blocco sincronizzato
                if (ref == null) {
                    ref = new Singleton(); // Creazione dell'istanza
                }
            }
        }
        return ref;
    }
}
```

Il primo controllo (`if (instance == null)`) viene **eseguito senza sincronizzazione** per ridurre l'overhead delle operazioni di lock quando l'istanza è già stata creata.

Il secondo controllo (`if (instance == null)`) è **eseguito con sincronizzazione** per evitare che più thread creino istanze concorrenti durante la creazione dell'oggetto.

Comportamento delle variabili volatili in Java

volatile è utilizzata **per dichiarare variabili condivise tra più thread**.

Quando una variabile è dichiarata **volatile**, essa garantisce una **visibilità sicura** tra i thread, evitando che una copia della variabile venga mantenuta nella cache di un singolo thread e garantendo che i cambiamenti fatti da un thread siano immediatamente visibili agli altri thread.

È stato deciso di dare alle variabili volatili proprietà aggiuntive:

- **Garanzia di visibilità:** le letture/scritture vengono emesse direttamente nella memoria principale (*anche se sono cachate*).
- **Avviene prima delle garanzie:** Per risolvere i problemi dovuti al riordino delle istruzioni da parte del compilatore o della JVM.

Un'operazione su una variabile **volatile** impone una sorta di **barriera di memoria** che impone delle regole circa l'ordinamento delle operazioni.

- Con una scrittura (**Wvol**) su una variabile volatile, nessuna operazione di lettura o scrittura precedente (*su altre variabili nello stesso thread*) può essere posticipata dopo di essa.
- Con una lettura (**Rvol**), nessuna operazione di lettura o scrittura successiva può essere eseguita prima di essa.

In generale, è importante specificare come i costrutti del linguaggio si comportano rispetto alle operazioni di memoria.

5.7.3 Modelli di consistenza della memoria

Riguarda l'ordinamento delle operazioni di lettura e scrittura in un sistema con memoria condivisa, dove i thread/processi possono accedere simultaneamente alla memoria condivisa, ma l'ordine in cui le operazioni sono visibili ad altri thread/processi dipende dalle regole di consistenza.

L'obiettivo di un modello di coerenza della memoria è caratterizzare, con un insieme di regole, il modo in cui le operazioni verso le posizioni di memoria sono visibili ai processi. Studiato per i multiprocessori (SMP). Importante perché il programma deve conoscere come le istruzioni vengono eseguite dal sistema sottostante.

Il modello di coerenza specifica un contratto tra il programmatore e il sistema, in modo che il programmatore possa contare su alcune **garanzie sul comportamento delle operazioni di memoria**.

Meno rigoroso è il modello di coerenza, più facile sarà implementarlo nei componenti HW (e SW), in modo più efficiente.

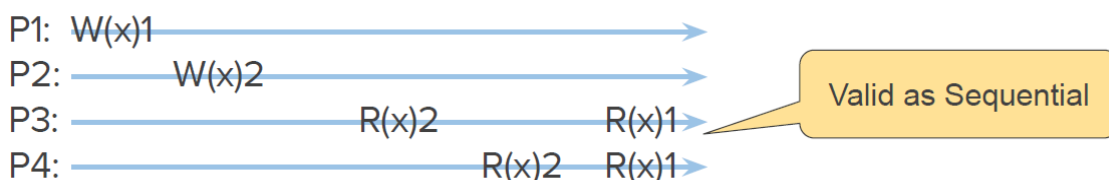
Sequential consistency

Garantisce l'ordinamento delle operazioni.

Le scritture su una variabile condivisa da parte di processi diversi devono essere viste nello stesso ordine da tutti i processori.

Generalizzato in un modello più debole denominato "*Consistenza causale*" (solo le scritture causalmente correlate devono essere viste nello stesso ordine da tutti i processi).

Le operazioni di P1 e P2 avvengono in modo concorrente, nonostante ciò, sia P3 che P4 vedono le due scritture nello stesso ordine.



PRAM Consistency

Se un thread scrive un valore in una variabile, tutti i thread successivi che leggono da quella variabile devono vedere lo stesso valore (o un valore consistente) in un ordine che ha senso in base alle operazioni che sono state fatte.

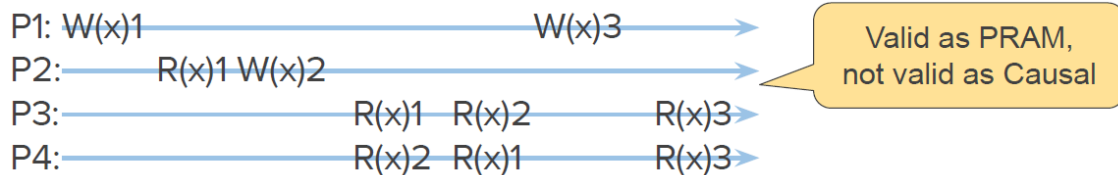
Tutti i processi vedono le scritture di un processo nell'ordine in cui sono state eseguite dal processo stesso (come se fossero state "spostate" nella RAM tramite una pipeline).

Non esiste alcuna garanzia sull'ordine in cui i diversi processi vedono le scritture da altri processi diversi (a parte la regola della "pipeline").

P3 legge i valori di $x = 1$ e $x = 2$ in ordine, mentre P4 legge prima i valori di $x = 2$ e $x = 1$ in ordine. C

Ciò che conta è che sia P3 che P4 vedano 1 prima di 3, però non necessariamente devono vedere 2 nello stesso ordine, questo perché questo valore è stato scritto da un altro processo (P2), mentre i valori 1 e 3 sono stati scritti da P1.

In generale, ogni processo deve leggere i valori scritti dallo stesso processo nell'ordine corretto.



Non deterministico: In PRAM, i thread/processi possono eseguire operazioni simultaneamente e in parallelo, il che porta a possibili risultati non deterministici.

Non ci sono garanzie circa l'ordine con cui diversi processi vedono le scritture di altri processi.

Ma una volta che vedono le scritture di un processo P1, allora di sicuro non vedranno mai un valore passato scritto P1.

5.8 JMM - Java Memory Model

Insieme di regole e specifiche che definiscono come i thread in Java interagiscono tra loro in relazione alla memoria. Esso stabilisce come le variabili devono essere visibili ai vari thread e come devono essere ordinate le operazioni di lettura e scrittura per garantire un comportamento corretto e prevedibile in un ambiente multi-threaded.

Descrive i comportamenti legali in un codice Java multi-thread rispetto alla memoria condivisa.

5.8.1 Obiettivi di JMM

Il JMM segue un modello di consistenza più debole rispetto al Sequential Consistency. In particolare, il JMM è progettato per essere un modello di consistenza che si colloca tra il Sequential Consistency e i modelli più rilassati, come il Weak Consistency.

Il **JMM** è più flessibile, permettendo **riordino delle operazioni** per migliorare le prestazioni, ma stabilisce regole di visibilità ("happens-before") che assicurano che le operazioni siano visibili in modo corretto e prevedibile tra i thread.

Happens-Before Relationship

Il JMM si basa sul concetto di "**happens-before**" per determinare l'ordine delle operazioni.

Se un'operazione A **happens-before** un'operazione B, allora tutte le modifiche fatte da A sono visibili a B.

Questo aiuta a garantire la visibilità corretta tra i thread.

Ad esempio, una scrittura sincronizzata su una variabile in un thread deve essere visibile a una lettura successiva sulla stessa variabile in un altro thread.

Visibilità delle variabili tra i thread

Ogni thread può avere una **cache locale** delle variabili (nella sua CPU), e non sempre le modifiche fatte a una variabile da un thread sono visibili ad altri thread.

Il JMM definisce le regole per la visibilità delle variabili tra i thread.

Per garantire che le modifiche fatte da un thread siano visibili agli altri, il JMM richiede l'uso di **sincronizzazione** o variabili dichiarate come **volatile**.

Ordinamento delle operazioni

La JVM o la CPU può **riordinare le operazioni** per ottimizzare le prestazioni, finché questo riordino non viola certe regole di visibilità o consistenza.

L'uso di **sincronizzazione** (come **synchronized** o **Lock**) e **variabili volatile** può impedire il riordino delle operazioni in modo che i thread vedano un comportamento coerente.

Barriere di memoria

Il JMM definisce delle **barriere di memoria** che sincronizzano le letture e le scritture tra i thread.

Se una variabile è dichiarata come **volatile**, il JMM garantisce che tutte le scritture a quella variabile siano immediatamente visibili a tutti i thread.

L'uso della **sincronizzazione** (ad esempio, con il blocco **synchronized**) crea una barriera di memoria, impedendo il riordino delle operazioni in modo che tutte le modifiche fatte prima della sincronizzazione siano visibili ai thread successivi.

Atomicità e sincronizzazione

Il JMM fornisce garanzie sulla atomicità delle operazioni di lettura e scrittura di variabili di tipo **primitivo** (ad esempio, **int**, **long**, **boolean**), ma per operazioni più complesse è necessario usare meccanismi di sincronizzazione per garantire che le operazioni siano **atomiche** e **sicure** in un ambiente multi-threaded.

5.8.2 Regole della JVM

Queste regole servono per non violare mai la relazione happens-before.

Ogni implementazione della JVM deve rispettare queste regole.

Program Order Rule

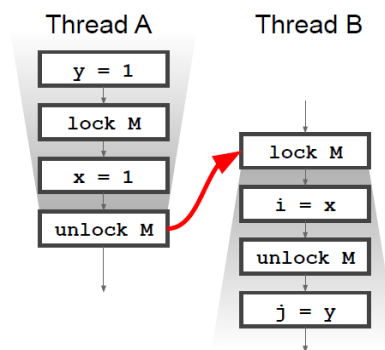
Ogni azione in un thread che happens-before qualunque altra azione in quel thread che avviene dopo nell'ordine del programma.

In altre parole:

Il runtime è libero di introdurre qualsiasi ottimizzazione di esecuzione tramite il riordino, purché il risultato del thread in isolamento sia garantito essere esattamente lo stesso di come sarebbe stato se tutte le istruzioni fossero state eseguite nell'ordine in cui le istruzioni si sono verificate nel programma (*chiamato anche ordine del programma*).

Monitor Lock Rule

Uno sblocco su un monitor avviene prima (happens-before) di ogni blocco successivo su quello stesso monitor; lo stesso vale per i blocchi espliciti.



Volatile Variable Rule

Una scrittura su un campo volatile si verifica prima (happens-before) di ogni successiva lettura dello stesso campo; la stessa semantica della memoria vale per le variabili atomiche.

Thread Start Rule

Una chiamata a `Thread.start()` su un thread si verifica prima (happens-before) di ogni azione nel thread avviato.

Thread Termination Rule

Qualsiasi azione in un thread si verifica prima (happens-before) che qualsiasi altro thread rilevi che il thread è terminato, sia tramite ritorno con successo da `Thread.join()` o tramite `Thread.isAlive()` che restituisce `false`.

Interruption Rule

Un thread che richiama `interrupt()` su un altro thread si verifica-prima (happens-before) che il thread interrotto rilevi l'interruzione (*sia generando `InterruptedException`, sia invocando `isInterrupted()` o `interrupted()`*).

Finalizer Rule

La fine di un costruttore per un oggetto si verifica-prima (happens-before) dell'inizio del finalizzatore per quell'oggetto.

Transitivity

Se A si verifica prima di B e B si verifica prima di C, allora A si verifica prima di C.

Capitolo 6 - Logical Time

Supponiamo che il nostro sistema (*composto da vari nodi*) sia composto da un certo numero di processi. In realtà il numero di processi non sempre è noto all'inizio di un'esecuzione.

I processi sono diffusi in nodi differenti connessi attraverso dei canali e possono comunicare tra loro, l'esecuzione di questi processi può essere modellata come sequenza di eventi.

Immaginiamo che all'inizio il nostro sistema target è costituito da un set (*statico*) di n processi $\{p_1, p_2, \dots, p_n\}$, ciascuno posizionato in un nodo diverso.

Gli eventi possono essere:

- **Eventi interni:** azioni significative, ovvero "*cambiamenti di stato*".
- **Eventi di comunicazione:** invio/ricezione di un messaggio (*in genere in modo asincrono, ma non necessariamente*).

6.1 Process history

Eventi di un processo

Un processo è costituito da svariati eventi.

Il k^o evento eseguito dal processo p_i è indicato come e_i^k

Ogni processo è **sequenziale**: i suoi eventi vengono eseguiti uno dopo l'altro (*sono totalmente ordinati*), tutti gli eventi di un singolo processo sono inseriti in una linea.

Cronologia di un processo

La sequenza di tutti gli eventi eseguiti da un processo è denominata come la sua **cronologia**.

$$h_i = e_i^1, e_i^2, \dots, e_i^k, \dots$$

Prefixo di una cronologia di un processo

Un **prefixo** di una cronologia di un processo corrisponde a un punto di progressione nell'esecuzione locale.

In altre parole, un **prefixo** è una rappresentazione parziale (*fino a un certo punto*) della sequenza degli eventi che costituiscono l'evoluzione del processo stesso.

Supponiamo che un processo P generi una sequenza di eventi $\{e_1, e_2, \dots, e_n\}$. Un **prefixo** di questa sequenza è una parte iniziale, come $\{e_1, e_2\}$.

6.2 Space-time Diagrams

Messaggi

Per ogni messaggio m nel calcolo, gestito in una comunicazione asincrona, possiamo definire un ordinamento degli eventi correlati:

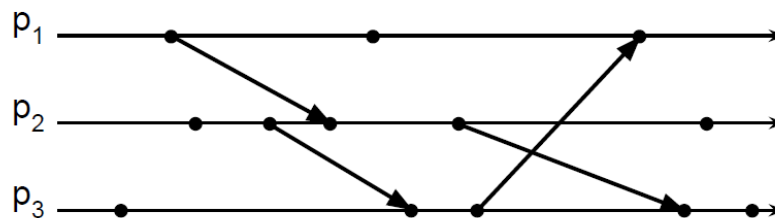
`send(m)` avviene prima di `receive(m)`.

Con altri tipi di comunicazione, ad esempio con comunicazioni “sincrone” (*rendez-vous*), queste due operazioni sono considerate come logicamente avvenute nello stesso momento, quindi non possono essere totalmente ordinate.

Diagramma spazio-tempo

Una linea rappresenta una parte degli eventi di un processo singolo.

Un'esecuzione distribuita può essere rappresentata graficamente con svariate linee parallele dove le frecce rappresentano l'invio di messaggi.



Consistent Cut

Rappresenta uno stato globale del sistema che rispetta la causalità degli eventi. È definito come un sottoinsieme di eventi che soddisfa la seguente condizione:

Se un evento e è incluso nel cut, allora tutti gli eventi che lo hanno preceduto causalmente (cioè gli eventi da cui e dipende) devono essere inclusi nel cut.

- **Causalità preservata:** Non si può includere un evento nel cut se non si sono già inclusi gli eventi che lo hanno causato.
- **Stato globale valido:** Rappresenta uno stato consistente del sistema in cui ogni processo e i loro messaggi rispettano l'ordine degli eventi.

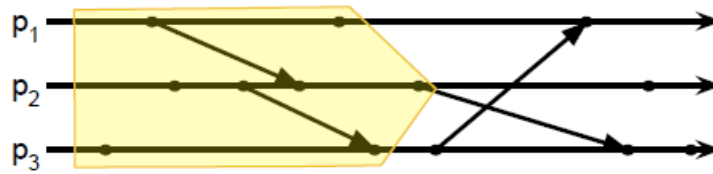
Cosa permette di fare il consistent cut?

Un taglio coerente è un concetto per formalizzare la nozione di “possibile punto di progressione” di un'intera esecuzione distribuita.

Ovvero, invece di elencare tutti gli eventi del prefisso, possiamo rappresentare un prefisso solo tramite l'ultimo evento che contiene. Ad esempio il prefisso $\{e_1, e_2\}$ può essere rappresentato da e_2 .

Il set di tutti gli ultimi eventi per ogni prefisso è la **frontiera del taglio**.

Un taglio è coerente se soddisfa la seguente proprietà: per qualsiasi messaggio m tale che $receive(m)$ sia nel taglio, anche la relativa $send(m)$ appartiene al taglio.



Ordinamento tra tagli

Un ordinamento tra i tagli è chiaramente indotto dalla relazione di inclusione degli insiemi:

Dati due tagli $C_1, C_2 \Rightarrow C_1 \rightarrow C_2 \Leftrightarrow C_1 \subset C_2$

C_2 viene dopo C_1 se e solo se il secondo è contenuto nel primo.



In particolare, alcuni tagli non potrebbero essere ordinati, come in figura sotto.



6.3 Nozione di tempo

In un sistema distribuito è impossibile avere dei clock perfettamente sincronizzati, quindi non esiste nessun clock globale in un sistema distribuito.

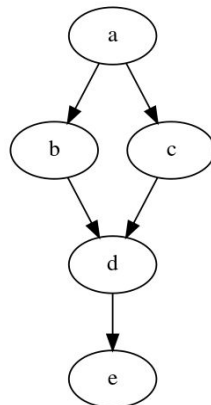
Senza un clock globale, dobbiamo capire come mettere in relazione gli eventi eseguiti in diversi punti di progressione.

Possiamo però trovare dei trucchi per raggiungere una sincronizzazione globale con una certa tolleranza.

6.3.1 Relazioni di precedenza

Il flusso di controllo di un'esecuzione distribuita è descritto dalle relazioni di precedenza tra i suoi eventi.

L'insieme di tutti i diversi tipi di precedenza può essere catturato da una singola relazione per esprimere la nozione di **causalità potenziale**.



Una singola relazione può essere definita per tenere conto di tutte le precedenza durante l'esecuzione: $\xrightarrow{HB}$ (*Happens Before*).

Può essere costruita secondo le seguenti regole:

HB0 (*transitività*)

Per gli eventi e, f, g , se $e \xrightarrow{HB} f \wedge f \xrightarrow{HB} g \Rightarrow e \xrightarrow{HB} g$.

Ovvero se e accade prima di f e f accade prima di g , allora e accade prima di g .

HB1 (*ordinamento in corso*)

Per gli eventi e, f nello stesso processo p_i ,

Se $e \xrightarrow{i} f \Rightarrow e \xrightarrow{HB} f$.

Se nel processo p_i , l'evento e accade prima di f . Allora c'è una relazione di Happens Before di e che viene prima di f .

HB2 (comunicazione asincrona)

Per qualsiasi evento $e = \text{send}(m)$ (non bloccante).

Sia l'evento corrispondente $f = \text{receive}(m)$ per lo stesso m , vale $e \xrightarrow{HB} f$.

Non posso ricevere qualcosa che non è stato ancora inviato!

Concorrenza

La definizione della relazione “*Happens Before*” ci consente anche di formalizzare la nozione di concorrenza:

Due eventi e, f sono detti concorrenti, indicati come $e||f$ se e solo se

$$\neg(e \xrightarrow{HB} f) \wedge \neg(f \xrightarrow{HB} e).$$

Ovvero se e solo se: e **non accade** prima di f ed f **non accade** prima di e ; quindi i due eventi possono avvenire anche contemporaneamente.

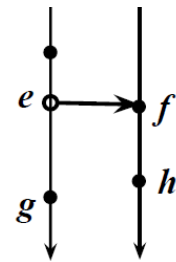
HB2 (comunicazione sincrona) “Rendez-vous”

Se abbiamo la possibilità di utilizzare il rendez-vous come modello di comunicazione, due eventi possono avvenire contemporaneamente. Nel caso in cui sia presente anche una comunicazione sincrona, un'altra regola può essere aggiunta alla definizione:

Sia l'evento $e = \text{send}(m)$ (*send bloccante*) e l'evento corrispondente $f = \text{receive}(m)$ per lo stesso messaggio m .

Per ogni evento g tale che $e \xrightarrow{HB} g \Rightarrow f \xrightarrow{HB} g$.

Per ogni evento h tale che $f \xrightarrow{HB} h \Rightarrow e \xrightarrow{HB} h$.



6.3.2 Relazioni di precedenza su variabili condivise

Anche con variabili condivise, *HB* può essere esteso in modi diversi, a seconda dell'ordinamento assunto per le operazioni di lettura/scrittura sulla stessa variabile V .

Sia $\xrightarrow{ob(V)}$ l'ordinamento totale delle operazioni sulla variabile condivisa V .

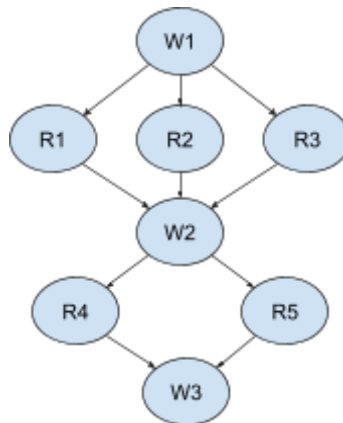
HB4 (strong)

Siano due eventi e, f corrispondenti a due operazioni su una variabile condivisa V .

Se almeno uno di essi è un evento di scrittura (*supponiamo* f), allora:

Se $e \xrightarrow{ob(V)} f$ allora $e \xrightarrow{HB} f$.

Ad esempio se abbiamo un'operazione di scrittura w_1 e tre operazioni di lettura per il valore scritto R_1, R_2, R_3 abbiamo una relazione HB tra queste letture e w_1 .



Se consideriamo il total order degli eventi e scambiamo R_1 ed R_2 non accade nulla, abbiamo lo stesso significato.

Un'altra estensione considera solo la precedenza di un'operazione di lettura rispetto alla scrittura precedente.

Numero di versione

Il numero di versione di una variabile V viene incrementato a ogni operazione di scrittura.

$v(V, e)$ indica il numero di versione di V immediatamente dopo l'evento di scrittura e .

HB4 (weak)

Sia una scrittura e ed una lettura f .

Se $v(V, e) = v(V, f) \Rightarrow e \xrightarrow{HB} f$

Se la versione osservata è la stessa, allora la scrittura avviene prima della lettura.

La relazione *HB* è un modo per tracciare la **causalità potenziale**.

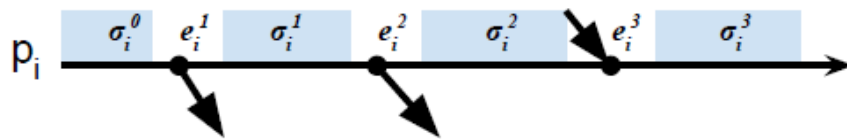
6.3.3 Eventi e stati locali

Un calcolo locale corrisponde a una sequenza di stati locali, con transizioni innescate da eventi.

Nella figura sotto abbiamo una serie di eventi nel tempo, un evento causa il cambiamento dello stato interno di un processo.

In **azzurro** in figura sono indicati gli stati locali.

Quando avviene l'evento e_i^1 , il processo andrà nello stato locale σ_i^1 .



Gli eventi sono ordinati logicamente, cioè ogni evento successivo avviene dopo un evento precedente.

HB con riflessività

Definiamo $a \xrightarrow{HB'} b$ come una relazione HB con **riflessività**.

In che senso riflessività?

$\forall a, a \xrightarrow{HB'} a$ (un evento accade prima se stesso).

La riflessività è indispensabile per stabilire che gli eventi di un singolo processo hanno senso anche isolatamente. Ad esempio, se e è un evento locale di un processo: **Garantisce che e può essere incluso in uno stato globale senza richiedere altri eventi.**

Per un singolo processo p_i , gli stati σ_i^a e σ_i^{a+1} devono essere ordinati causalmente.

Senza riflessività, non potremmo garantire che uno stato precede direttamente il successivo nel caso in cui non ci siano altri eventi concorrenti o dipendenti da essi.

Con la riflessività, possiamo affermare $\sigma_i^a \xrightarrow{\sigma} \sigma_j^b \equiv e_i^{a+1} \xrightarrow{HB'} e_j^b$

$\sigma_i^a \xrightarrow{\sigma} \sigma_j^b$ indica una transizione dallo stato locale σ_i^a al successivo σ_j^b , che è causata da un evento e_i^{a+1} , rispettando la relazione di ordine causale HB'.

In altre parole, se $e_i^{a+1} \xrightarrow{HB'} e_j^b$ possiamo concludere che $\sigma_i^a \xrightarrow{\sigma} \sigma_j^b$.

Da questa definizione, un'esecuzione distribuita può essere modellata come un ordine parziale sull'insieme di tutti gli stati S : $(S, \xrightarrow{\sigma})$.

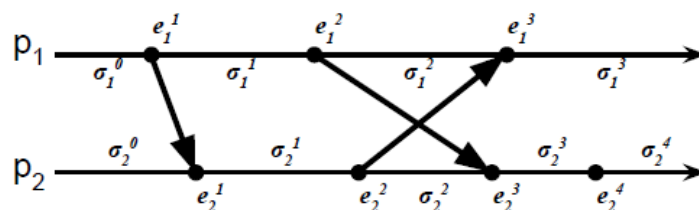
Ogni stato globale del sistema rappresenta un sottoinsieme di eventi accaduti. Poiché non tutti gli eventi sono totalmente ordinati (per via dell'assenza di sincronizzazione globale), l'insieme degli stati globali forma un **ordine parziale**.

6.3.5 Stato globale

Gli **stati globali** rappresentano le configurazioni del sistema in un dato momento. Ogni stato globale è una combinazione degli stati di tutti i processi.

Uno “*stato globale*” è una raccolta di uno stato locale per processo: $\Sigma = [\sigma_i, \sigma_j, \dots, \sigma_n]$

Uno stato globale Σ è **coerente solo** se $\sigma_i \parallel \sigma_j \forall i, j, i \neq j$. Ovvero se per ogni coppia di stati, essi saranno concorrenti.



Supponiamo di avere due processi P_1 e P_2 :

- **Processo P_1 :**
 - Stato locale $S_1 = \{e_1\}$.
 - Evento e_1 è accaduto.
- **Processo P_2 :**
 - Stato locale $S_2 = \{e_2\}$.
 - Evento e_2 è accaduto.

Caso 1: Stati locali concorrenti

Se e_1 e e_2 sono concorrenti (cioè $e_1 \not\rightarrow e_2$ ed $e_2 \not\rightarrow e_1$), allora possiamo combinare S_1 e S_2 in uno stato globale coerente:

$$S = \{e_1, e_2\}$$

Caso 2: Stati locali non concorrenti

Se e_1 e e_2 non sono concorrenti (cioè $e_1 \rightarrow e_2$), allora non possiamo includere S_2 nello stato globale senza prima includere tutti gli eventi che precedono causalmente e_2 .

Se S_1 manca di un evento necessario per rispettare la causalità, lo stato globale risultante sarebbe incoerente.

Uno stato globale è coerente solo se gli stati locali che lo compongono sono **concorrenti**, perché ciò garantisce che:

1. Non ci siano violazioni della causalità.
2. Gli eventi rappresentati siano combinati in un ordine valido.
3. Lo stato globale sia una rappresentazione consistente del sistema distribuito in un dato momento.

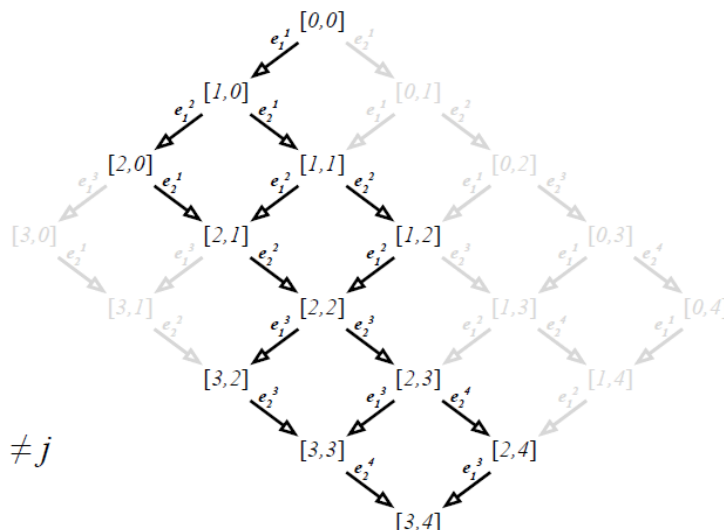
Due stati non concorrenti di fatto non possono avvenire contemporaneamente, quindi trovarli nello stato globale porterebbe ad un assurdo poiché per definizione uno deve avvenire prima dell'altro.

Grafo di raggiungibilità

Dal grafo spazio-tempo possiamo ricavare il grafo di raggiungibilità (in figura sotto).

In tale figura $[0, 0]$ è lo stato iniziale di entrambi i processi, ha gli indici del primo stato in p_1 (σ_1^0) e p_2 (σ_2^0). Con l'evento e_1^1 dallo stato $[0, 0]$ passiamo allo stato $[1, 0]$, ovvero p_1 avanza di stato, mentre p_2 rimane allo stato iniziale (p_1 (σ_1^1) e p_2 (σ_2^0)).

Da notare che con l'evento e_1^1 , il processo p_1 avanza nello stato σ_1^0 e viene generato l'evento e_2^1 nel processo p_2 .

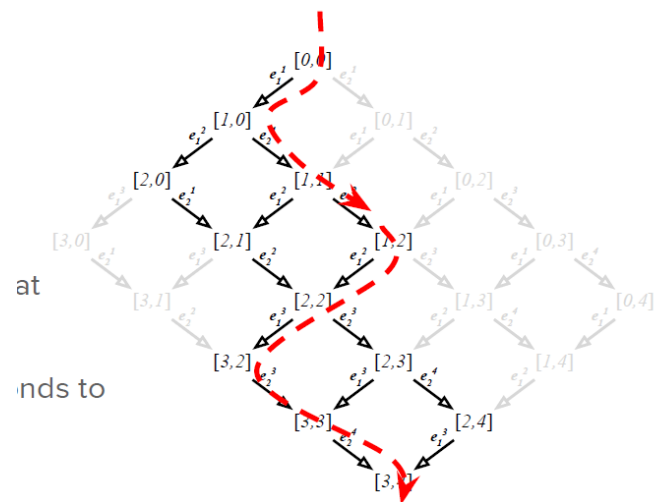


Osservazione sequenziale

Sequenza di tutti gli eventi/stati conforme al loro ordine parziale.

Un'osservazione sequenziale corrisponde a un **percorso nel grafo di raggiungibilità**.

Tutti gli ordini possibili degli eventi appartenenti alle computazioni possono essere descritti come tutti i percorsi possibili dall'inizio verso la fine.



ends to

6.4 Clock per tempo logico

Se dobbiamo occuparci degli ordinamenti relativi delle occorrenze degli eventi, possiamo sostituire il "*tempo globale*" con un indice che fa riferimento semplicemente all'ordine degli eventi. Infatti a noi non interessa molto a che ora è successo qualcosa ma più che altro ci serve sapere in che ordine collocare un certo evento rispetto ad altri eventi.

Un clock logico $C(\bullet)$ è un mezzo per mappare gli eventi su un ordine parziale.

- **Consistenza del Clock:** Se $e \xrightarrow{HB} f \Rightarrow C(e) < C(f)$.
- **Dipendenza Causale:** Se $C(e) < C(f) \Rightarrow e \xrightarrow{HB} f$.

Il codominio più semplice per $C(\bullet)$ è una sequenza di numeri crescenti.

Nel nostro caso $C(\bullet)$ è un **numero intero** e quindi può essere detto "*tempo lineare*".

6.4.1 Lamport Timestamp

Il clock logico di Lamport T_{lp} di un evento e è associato alla lunghezza della catena HB più lunga per raggiungere e .

Come assegnare tali timestamp?

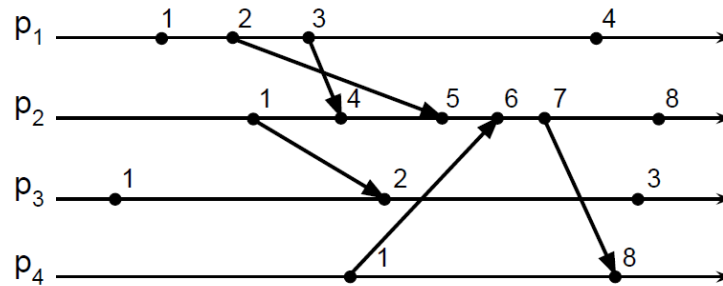
Ogni processo p_i mantiene un contatore locale $clock_i$ che viene modificato ogni volta che avviene un evento secondo le seguenti regole:

- Inizialmente $clock_i = 0$.
- Per tutti gli eventi tranne le $receive(m)$:
 - $clock_i \leftarrow clock_i + 1$ (Il clock viene incrementato di 1).
 - $T_{lp}(e) \leftarrow clock_i$
- Con le $receive(m)$ il processo ha ricevuto un messaggio da un altro processo e quindi deve verificare quale è il timestamp più aggiornato. Per questo motivo con le $send(m)$, viene trasportato anche un timestamp $t_s(m)$.
 - $t_s(m) \leftarrow clock_j$
 - $clock_i \leftarrow \max(clock_i, t_s(m)) + 1$
 - $T_{lp}(e) \leftarrow clock_i$

Il clock del processo p_i locale viene aggiornato usando il valore (incrementato) maggiore tra il clock di p_i e il clock $t_s(m)$ inviato da p_j nel messaggio m .

Esempio con i Lamport timestamp

In figura sotto vediamo che l'evento iniziale di ogni processo ha $timestamp = 1$.



Vediamo che p_1 (arrivato al clock 2 e al clock 3) invia un messaggio a p_2 .

Ma il messaggio inviato al clock 3, per una strana ragione, arriva prima di quello inviato al clock 2.

Seguendo p_2 da 1 passa subito a 4.

Questo perché in entrambe le $send(m)$ fatte in p_1 si è trasportato il timestamp del momento in cui è stato inviato il messaggio e quindi p_2 si è adattato (caso 3). Poiché:

$$clock_2 = 1 < t_{s1}(m) = 3 \Rightarrow clock_2 \leftarrow t_{s1}(m) + 1 = 4$$

Proprietà del Lamport timestamp

Il lamport timestamp è coerente?

Coerenza: Se $e \xrightarrow{HB} f \Rightarrow T_{Lp}(e) < T_{Lp}(f)$

Per contrapposizione, se $T_{Lp}(e) \leq T_{Lp}(f) \Rightarrow \neg(f \xrightarrow{HB} e)$.

Se $T_{Lp}(e) < T_{Lp}(f)$, allora e è accaduto prima di f , oppure $e || f$.

Se $T_{Lp}(e) = T_{Lp}(f) \Rightarrow e || f$.

Ma questo timestamp ha un problema:

No precede: è possibile avere $T_{Lp}(e) < T_{Lp}(f) \wedge \neg(e \xrightarrow{HB} f)$.

Ovvero può accadere che l'evento e risulti precedente all'evento f ma comunque non c'è una relazione HB tra i due eventi.

Questa cosa accade quando il mittente invia due messaggi che non sono legati in alcun modo tra loro.

Questo significa che T_{Lp} non può provare una relazione HB .

Estensioni del Lamport timestamp

L'algoritmo per i timestamp di Lamport può essere facilmente esteso per gestire la comunicazione sincrona e l'uso di variabili condivise.

6.4.2 Totally Ordered Lamport Timestamp

Meccanismo usato per ordinare gli eventi in un sistema distribuito in modo che siano comparabili in modo univoco, rispettando l'ordine causale degli eventi.

Il tempo lineare T_{Lp} può essere utilizzato per **ottenere un ordine totale** T_{Lt} su tutti gli eventi, in modo che sia coerente con la relazione HB .

Per il processo p , sia $T_{Lt} = (T_{Lp}, i)$ il quale afferma che
 $(a, b) < (c, d) \Leftrightarrow \{ (a < c) \vee [(a = c) \wedge (b < d)] \}$.

In pratica, T_{Lt} può essere un intero ottenuto come $T_{Lp} \ll (B + 1)$, $B = \lceil \log_2 n \rceil$ e n il numero di processi ($\ll B$: *uno shift a sinistra di B bit*).

6.4.3 Strong Clock Consistency - Timestamp Vettoriali

Vorremmo avere un **clock in grado di indicare la dipendenza causale**, ovvero se

$$C(e) < C(f) \Rightarrow e \xrightarrow{HB} f$$

Questo **non è possibile con il Lamport timestamp** classico poiché è possibile avere la No Precede, ovvero: $T_{Lp}(e) < T_{Lp}(f) \wedge \neg(e \xrightarrow{HB} f)$.

Una soluzione è quella di mantenere, per ogni processo, un **vettore di contatori** V_i di n (con n il numero di processi) interi, uno per ogni processo p_i nel sistema.

- $V_i[i]$ è il contatore degli eventi all'interno di p_i , ovvero l'informazione più recente che ho riguardo la progressione del contatore del processo p_i .
- $V_i[j]$ è il valore più recente di $V_j[j]$ visto da p_i , ovvero l'informazione più recente che p_i ha riguardo la progressione del contatore del processo p_j .

Ogni volta che un processo p_i invia un messaggio, il valore più recente di $V_i[i]$ viene trasportato nel messaggio e il processo destinatario aggiornerà il relativo contatore.

Algoritmo per timestamp vettoriali

A ogni evento e , il contatore vettoriale locale V_i viene aggiornato e il suo valore viene assegnato all'evento come timestamp vettoriale $T_v(e)$.

Come aggiornare i contatori vettoriali locali?

1. Per tutti i processi p_i , ad ogni operazione fatta viene effettuato $V_i[i] \leftarrow V_i[i] + 1$
2. In caso di $send(m)$: $t_s(m) \leftarrow T_v(e)$ (trasportato nel messaggio m).

L'intero vettore $T_v(e)$ viene trasportato nel messaggio m .

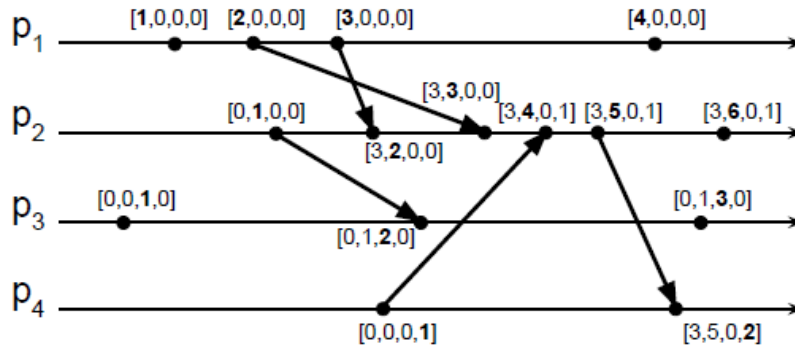
3. In caso di $recv(m)$, per ogni cella dell'array si esegue: $V_i \leftarrow \max \{ V_i, t_s(m) \}$

Il confronto nel punto 3 avviene componente per componente.

4. Infine, $T_v(e) \leftarrow V_i$.

In questo esempio vediamo che nel primo valore:

- Per p_1 [1,0,0,0].
- Per p_2 [0,1,0,0].
- Per p_3 [0,0,1,0]
- Per p_4 [0,0,0,1].



Osserviamo che p_1 non conosce mai nulla riguardo agli altri processi, perché non riceve mai un messaggio.

Nel secondo punto di p_2 , esso conosce già che p_1 è arrivato nella posizione 3 (*primo elemento della lista*), quindi non viene cambiato nulla.

Regole per il confronto tra timestamp vettoriali

Dati due timestamp vettoriali T_1 e T_2 , definiamo:

1. $T_1 \leq T_2 \Leftrightarrow T_1[k] \leq T_2[k] \forall k \in [1, n]$ *confronto elemento per elemento.*
2. $T_1 = T_2 \Leftrightarrow T_1[k] = T_2[k] \forall k \in [1, n]$ *confronto elemento per elemento.*
3. $T_1 < T_2 \Leftrightarrow (T_1 \leq T_2) \wedge (T_1 \neq T_2)$
4. $T_1 \parallel T_2 \Leftrightarrow T_1 \not\leq T_2 \wedge T_2 \not\leq T_1$

“Not Less” : $T_v(e) \not\leq T_v(f)$ significa che $\exists i \in [1, n] : T_v(e)[i] > T_v(f)[i]$

Le ultime due definizioni riguardano la precedenza e la concorrenza HB , come verrà mostrato.

Proprietà timestamp vettoriali

I timestamp vettoriali soddisfano entrambi i requisiti.

- **Consistenza del clock:** $e \xrightarrow{HB} f \Rightarrow T_v(e) < T_v(f)$.
- **(Strong C.C) Consistenza Causale Forte:** $T_v(e) < T_v(f) \Rightarrow e \xrightarrow{HB} f$.

Dimostrazione

Per contrapposizione: dimostriamo che se $\neg(e \xrightarrow{HB} f) \Rightarrow T_v(e) \not< T_v(f)$.

Quindi dimostriamo che $T_v(e) \not< T_v(f) \Rightarrow \exists i \in [1, n] : T_v(e)[i] > T_v(f)[i]$.

1. Sia e un evento in p_i , e f un evento in p_j .
2. Supp. che $\neg(e \xrightarrow{HB} f)$ sia vera.
3. Data (2), **non esiste** una relazione HB tra e ed f .
4. Appena prima di e , p_i aumenta il suo $V_i[i]$, quindi $T_v(e)[i] \leftarrow t$.
5. Data(2) non c'è modo per il valore t di propagarsi a p_j prima che l'evento f accada.
6. Quindi il valore conservato da p_j al momento di $f \rightarrow T_v(f)[i] < t$.
7. Quindi $T_v(e) \not< T_v(f)$.

□

6.4.4 Cuts & Vector Timestamps

I timestamp vettoriali ci permettono anche di **verificare la coerenza di un taglio**.

Sia $F(C) = \{f_1, \dots, f_n\}$ l'insieme di eventi di frontiera di un taglio C .

Sia il **timestamp del taglio** $T_v(C) = \max\{T_v(f_1), \dots, T_v(f_n)\}$

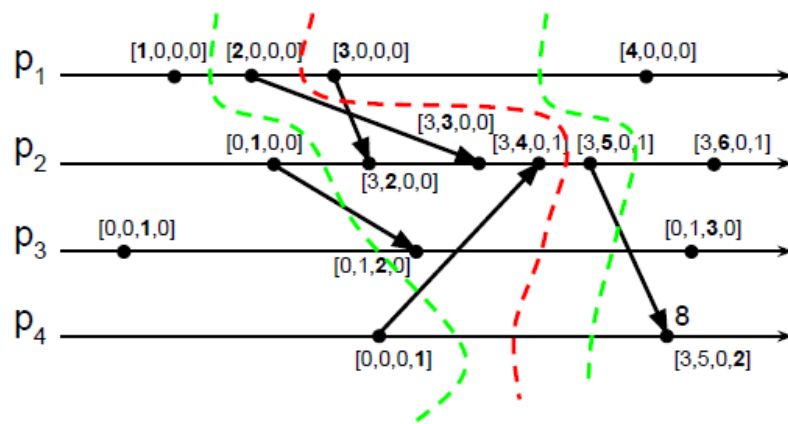
Il taglio C è coerente se $\forall i, T_v(C)[i] = T_v(f_i)[i]$.

Il valore massimo i^o valore deve sempre essere sul processo p_i .

Se così non fosse, significherebbe che un altro processo diverso da p_i ha un valore $T_v(f_i)[i]$ maggiore di quello detenuto da p_i stesso. E ciò implicherebbe che un altro processo nel taglio conosce il futuro di p_i .

Se controlliamo il taglio **verde**, vediamo che la proprietà è applicata.

Nel taglio **rosso** non è applicata perché p_2 conosce il futuro di p_1 , visto che al suo punto 3, ha valore 3 come primo elemento dell'array, mentre p_1 ha 2 come primo elemento al suo punto 2.



Capitolo 7 - Supporto per la com. in sist. distr.

Differenza tra comunicazione diretta e indiretta

Nella **comunicazione diretta** ogni parte coinvolta conosce direttamente l'altra entità con cui sta comunicando.

Nella **comunicazione indiretta** la parte coinvolta non sa precisamente chi riceverà un messaggio ma userà un intermediario.

Comunicazione diretta

Ci sono tre opzioni fondamentali:

- I **protocolli di richiesta/risposta**.
- **RPC - Remote Procedure Call**: Modo per fornire esecuzione remota del codice, un'entità chiede ad un'altra di eseguire il suo codice con richieste esplicite.
- **RMI - Remote Method Invocation**: In questo caso il paradigma OO è stato inserito nella soluzione fornita.

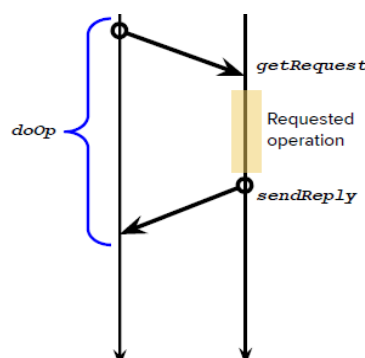
7.1 Protocolli di richiesta/risposta

I **protocolli di richiesta/risposta** sono progettati per supportare lo scambio di messaggi (a basso livello) nei sistemi client-server.

Questi protocolli sono di solito sincroni: dopo aver inviato una richiesta, il client attenderà una risposta dal server.

La risposta ha anche il ruolo di acknowledgement ad indicare che la richiesta è stata ricevuta con successo.

Questi protocolli si basano su altri tipi di protocolli di trasporto che supportano la nozione di connessione. In questo caso, noi considereremo UDP come protocollo di trasporto di riferimento.



Una forma generale del protocollo può essere modellata usando tre primitive:

doOp

```
public byte[] doOp(RemoteRef s, int opId, byte[] args)
```

- **RemoteRef s**: Riferimento al server remoto a cui far eseguire tale operazione.
- **int opId**: Tipo di operazione da eseguire.

doOp è implementata come l'invio di un messaggio al server e bloccarsi in attesa di una risposta.

getRequest

```
public byte[] getRequest()
```

Primitiva bloccante che ritorna un messaggio di richiesta in attesa nella casella di posta del server.

sendReply

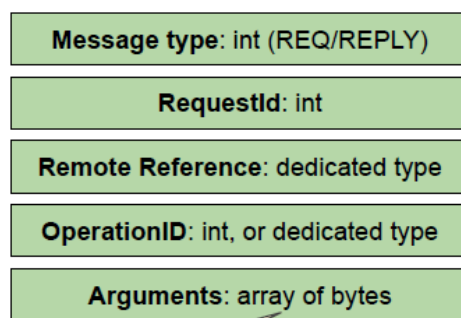
Primitiva per inviare la risposta al client.

```
public void sendReply (RemoteRef c, byte[] reply)
```

- **RemoteRef c**: Riferimento al client a cui rispondere.

Formato del messaggio

L'indirizzo del mittente è dato da IPAddr + Port.



RequestId identifica il messaggio ed è associata alla richiesta. E' compito del livello middleware quello di assegnare un ID specifico ad ogni richiesta.

Message Type fa in modo che il ricevitore sappia cosa fare con il messaggio appena ricevuto.

Arguments è un array di byte (*non sappiamo in anticipo quanti parametri abbiamo*), il server che riceve il messaggio deve essere in grado di capire come ricostruire i parametri.

Marshalling

Perché gli argomenti li interpretiamo come un array di byte generici?

- Non necessariamente componenti differenti (*client e server*) vengono implementati con lo stesso linguaggio.
- I dati scambiati potrebbero essere altamente strutturati (*ad esempio, oggetti, insieme al loro stato*).

Le due parti devono concordare su come codificare i dati scambiati, in modo da ricostruire correttamente tutti gli elementi di dati.

Per questo è stato introdotto il “*Marshalling*”, ovvero rendere un tipo di dato leggibile in tutti i linguaggi.

La trasformazione della rappresentazione di memoria di un oggetto in un formato di dati adatto alla trasmissione è detta *marshalling* (e *un-marshalling* l'operazione opposta).

7.2 Failure Model

Dato che stiamo usando UDP bisogna gestire:

- Perdita di messaggi.
- Riordino dei messaggi ricevuti.

Verifica se un processo è alive

Riguardo i processi considereremo il modello **fail-stop** (*no Byzantine*).

Nel modello fail-stop, gli altri processi o nodi rilevano il guasto dell'altro processo e possono attivare un meccanismo di recupero.

Byzantine: Assumiamo che il server possa comportarsi in modo diverso da quello che avevamo supposto.

7.2.1 Strategie per gestire le difficoltà

Per gestire le difficoltà è possibile applicare le seguenti strategie:

Timeout

Per verificare se il server è inattivo, viene impiegato un timeout nell'implementazione di `doOp()` inviata dal client.

Un processo non-responsivo può essere interpretato come non-alive (*non è detto che lo sia veramente*) dopo un certo periodo di tempo.

Se ritorna alive nuovamente verrà presa un'azione necessaria.

Ritrasmissione della richiesta

Per compensare la possibile perdita di messaggi di richiesta.

Filtraggio dei duplicati

Possono arrivare una o più copie dello stesso messaggio, quindi i duplicati devono essere rilevati e scartati.

Se sappiamo l'identificatore del client più l'identificatore del messaggio richiesto possiamo sapere se il messaggio ricevuto è un duplicato oppure no.

7.2.2 Riesecuzione vs caching

In caso di ri-trasmissione della richiesta da parte del client, ci sono due politiche possibili da applicare lato server: una è la **ri-esecuzione**, quindi rieseguire la richiesta, mentre l'altra è il **caching lato server**, in modo che questo riconosca il client che effettua le richieste.

Il caching è un metodo per rendere più efficiente il lavoro del server, però è possibile solo per richieste **idempotenti**.

Ad un certo punto il server ha molte informazioni in cache e deve capire quali informazioni rimuovere perché non più utili e quali lasciare.

Il client può avvertire il server che può cancellare certe informazioni perché non riceverà più messaggi di quel tipo da esso.

Idempotenza

Si riferisce ad una funzione f e ad molteplici chiamate successive della funzione.

*Se f è chiamata due volte in successione con gli stessi argomenti: la seconda chiamata **non avrà effetti collaterali** e il valore di ritorno sarà lo stesso della prima chiamata.*

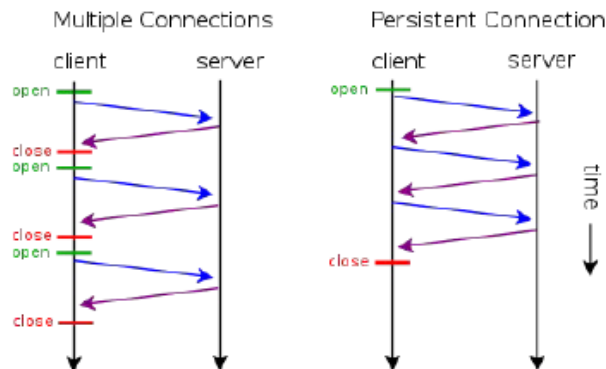
Naturalmente, questa è una proprietà significativa per le funzioni con effetti collaterali: le funzioni pure (ovvero funzioni senza effetti collaterali e con risultati deterministici) sono idempotenti per natura.

Quindi una funzione idempotente è "safe" da riprovare.

7.3 Esempio di prot. Richiesta-Risposta - HTTP

HTTP utilizza TCP, e di solito abbiamo una sola connessione per una richiesta e la relativa risposta.

In questa soluzione abbiamo un problema di efficienza perché viene perso molto tempo ad aprire e chiudere le connessioni.



Una soluzione migliore è quella di effettuare richieste multiple all'interno di una singola connessione (*HTTP keep-alive*).

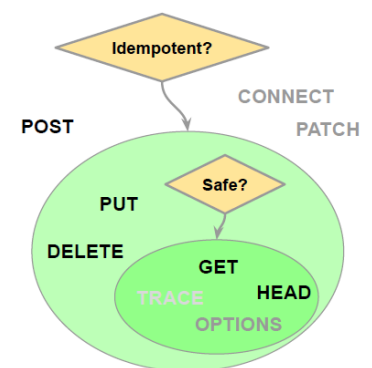
L'indirizzamento del server avviene tramite URI/URL, che tengono conto anche dell'operazione richiesta, e anche dei parametri.

Tipologie di messaggi di HTTP

Per fare in modo che il client comunichi con il server possiamo definire vari tipi di messaggio.

Se la sua richiesta è idempotente gli effetti collaterali di molte richieste identiche sono gli stessi di una singola richiesta, quindi il server può effettuare il caching e il client può restringere l'insieme di metodi HTTP utilizzabili (*in questo caso si usa GET*).

Altrimenti deve essere utilizzato il *POST* che è al di fuori di tale insieme.



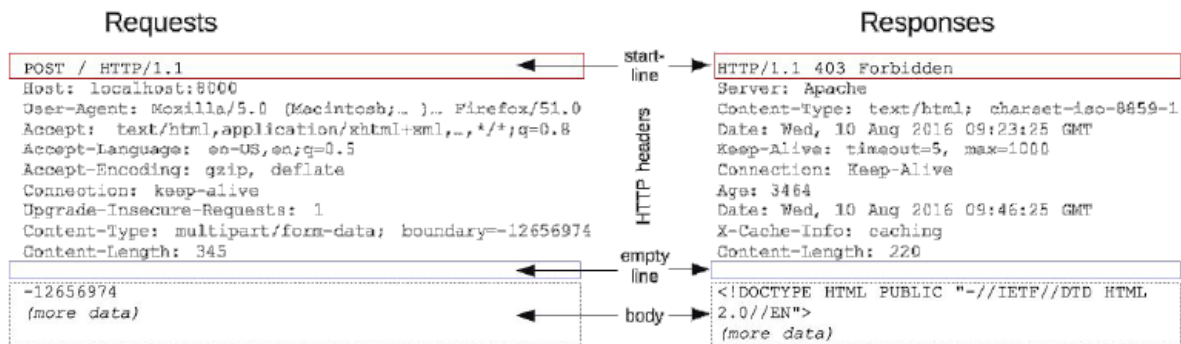
Quindi la differenza tra *POST* e *GET* è principalmente il fatto che *GET* è capace di supportare operazioni idempotenti. Mentre *POST* no.

Esempio di operazione non idempotente: c'è un effetto collaterale non nullo, se effettui un acquisto due volte, spendi il doppio.

Nullipotenza (o "sicurezza"): i messaggi determinano il recupero, nessun cambiamento di stato sul server.

La struttura è diversa per i messaggi *REQ* e *RESPONSE* (*questi ultimi hanno STATUS, per tenere conto di possibili errori*).

Gli stessi campi hanno possibili significati diversi (o possono essere usati in modo diverso), a seconda del tipo/metodo.

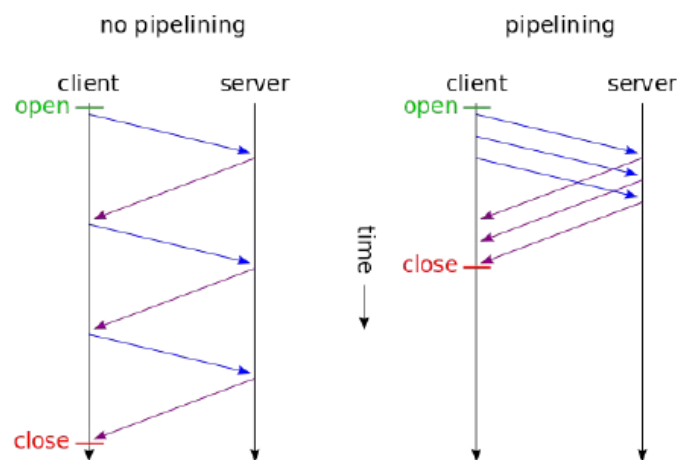


Pipelining

Per quanto riguarda le prestazioni in HTTP abbiamo il pipelining che consiste nell'inviare richieste al server senza attendere le relative risposte come avviene nel caso a sinistra nell'immagine.

Con il pipelining viene risparmiato molto tempo, la soluzione senza pipelining ha un overhead proporzionale al numero di richieste.

Ovviamente, questo non può essere fatto in caso di richieste non idempotenti, perché una richiesta non idempotente può provocare effetti che influenzano le altre risposte.



Marshalling in HTTP

In ogni messaggio HTTP viene specificato il tipo di dato tramite le specifiche MIME (text/plain, text/html, ecc).

7.4 RPC - Remote Procedure Call

RPC è una forma di comunicazione (*a livello di applicazione*) interprocesso (*non tra nodi/macchine/sistemi operativi*) tra un client e un server.

Una RPC è l'esecuzione di una procedura nel server con la restituzione dei risultati al client.

In generale i linguaggi adottati dal client e dal server possono essere differenti e ovviamente questa cosa va tenuta in conto.

Nel nostro programma quando vogliamo effettuare una RPC non dobbiamo specificare tutti i dettagli possibili, vogliamo mantenere queste chiamate il più possibile simili a chiamate ordinarie nel codice.

- Programmazione basata sull'interfaccia → Astrazione, "*portabilità*".
- È possibile scegliere/implementare diverse semantiche di chiamata.
- Trasparenza, rispetto all'accesso (*locale vs remoto*), posizione, ecc. (*concorrenza, replica, errore, mobilità, prestazioni, scalabilità*).

7.4.1 IDL - Interface Definition Language

Ci sono dei linguaggi appositi per le RPC detti **IDL** (*Interface Definition Language*) che permettono di far operare tra loro linguaggi differenti tramite appunto queste RPC.

Serve questo strumento perché i parametri, tra linguaggi diversi, vengono mappati in modo diverso. C++ e Java codificano un oggetto in modo diverso e rischiano di non comprendersi.

La definizione di un'interfaccia è compilata tramite un tool che restituisce dei moduli (*in linguaggi diversi*) da schierare sui vari nodi per supportare la comunicazione.

Un punto cruciale è quello del passaggio dei parametri: le controparti non condividono lo spazio di indirizzamento perché sono su macchine differenti, quindi il passaggio per riferimento non ha senso.

Gli indirizzi normali non possono essere passati come parametri, invece, come in pl/SQL, di solito un parametro può essere specificato come

- IN (solo input).
- OUT (solo output).
- INOUT (input in Request, output in Response).

Il passaggio dei parametri nelle RPC è sempre per valore.

7.4.2 Chiamate remote

Le chiamate locali di solito hanno una semantica di tipo “*exactly-once*” (*l'operazione viene effettuata una sola volta e con successo*), ma essa non è possibile in un ambiente distribuito.

Per le chiamate remote **in presenza di comportamenti errati** possono esserci i comportamenti espressi in tabella.

Call semantics	<i>REQ retransmission</i>	<i>Duplicate filtering</i>	<i>Re-exec vs Re-transmit</i>	<i>Examples</i>
Maybe	NO	Not applicable	Not applicable	-
At-least-once	YES	NO	Re-execution	SUN (ONC) RPC
At-most-once	YES	YES	Re-transmission	CORBA, RMI, gRPC default

Filtraggio dei duplicati, ritrasmissione o ri-esecuzione è fatto tutto a livello middleware.

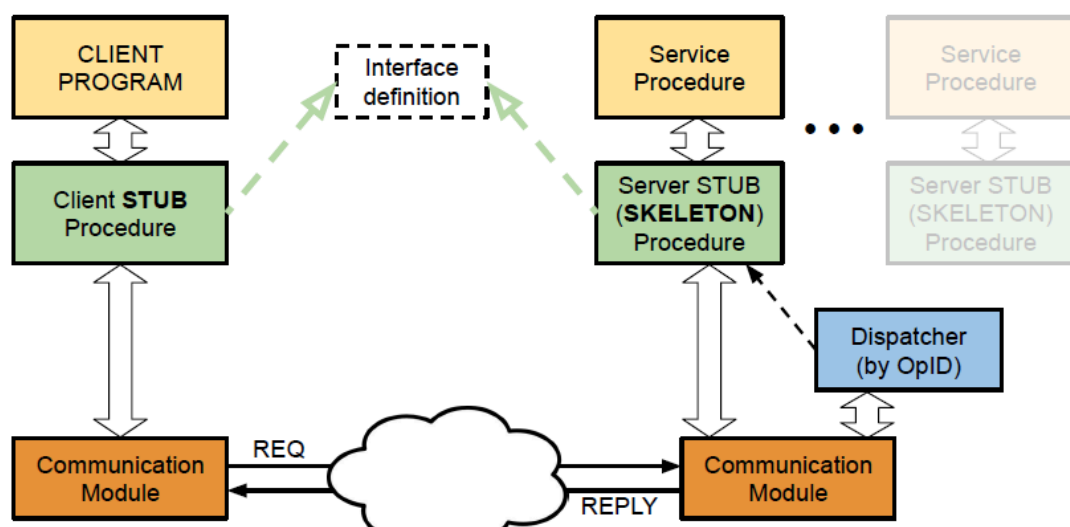
In caso di trasmissione della stessa richiesta per una seconda volta:

- **Re-execution:** La risposta viene ricalcolata e re-inviata; il server non ignora i duplicati e tratta tutte le richieste come se fossero uniche.
- **Re-transmission:** La risposta viene riproposta la risposta calcolata prima.

7.4.3 Implementazione di una RPC

Il resto (*tranne il programma client e il service procedure*) è implementato nel lato middleware.

Tutto ciò che non è **client program** e **service procedure** è considerato middleware.



Client STUB

Componente che agisce come un *proxy* per il client, nascondendo i dettagli della comunicazione remota.

Quando il client effettua una chiamata a una funzione remota, la richiesta viene intercettata dal **client stub**. Questo si occupa di:

- Serializzare i parametri (*marshalling*).
- Inviare la richiesta al server tramite il modulo di comunicazione.
- Ricevere, deserializzare la risposta (*unmarshalling*) e restituirla al codice client.

Dispatcher

Nel server ci sono molte procedure che possono essere eseguite.

Il **dispatcher** è il componente responsabile di gestire le richieste ricevute dal modulo di comunicazione e instradarle verso la funzione o procedura corretta da eseguire sul server.

Esso ispeziona il codice della richiesta e decide quale codice (quale funzione dello skeleton) eseguire per rispondere a tale richiesta.

Server STUB (Skeleton)

Il **server stub** è il corrispondente del client stub, ma sul lato server.

- Riceve la richiesta dal modulo di comunicazione.
- Deserializza i parametri ricevuti (*unmarshalling*).
- Invoca la funzione effettiva sul server (implementata nel server).
- Serializza il risultato (*marshalling*) e lo invia al client.

Cosa fa nello specifico?

1. Lo **skeleton** prende la sequenza di byte dal modulo di comunicazione.
2. Ricava i parametri e li converte nel linguaggio locale (*un-marshalling*) in cui è implementata la **service procedure**.
3. Invia i parametri alla **service procedure**.

Interface Definition

Tutto inizia dalla definizione della interfaccia, la quale diventa utilizzabile in ogni client a prescindere dal linguaggio con cui è stato scritto.

Specifica la firma delle funzioni: nomi, parametri, tipi di dati e valori restituiti. Agisce come un contratto tra client e server, assicurando che entrambi abbiano una visione coerente delle funzioni disponibili.

Spesso viene definita utilizzando un linguaggio specifico (ad esempio, IDL - *Interface Definition Language*).

Dai file IDL vengono generati automaticamente i codici del client stub e il server stub.

Schema di funzionamento

1. **Programma client:** Il programma client chiama una RPC utilizzando il codice esposto dal client STUB.
2. **Client stub:** Serializza i parametri e invia una richiesta al server tramite il modulo di comunicazione.
3. **Modulo di comunicazione (client):** Trasmette la richiesta al server tramite rete.
4. **Modulo di comunicazione (server):** Riceve la richiesta e la passa al server stub.
5. **Server stub:** Deserializza i parametri e invoca la funzione effettiva sul programma server.
6. **Dispatcher:** Rileva l'operazione desiderata e invia un valore al programma server.
7. **Programma server:** La funzione calcola il risultato e lo restituisce al server stub.
8. **Server stub:** Serializza il risultato e lo invia al modulo di comunicazione.
9. **Modulo di comunicazione (server):** Invia la risposta al client.
10. **Modulo di comunicazione (client):** Riceve la risposta e la passa al client stub.
11. **Client stub:** Deserializza la risposta e la restituisce al programma client.

Esempio IDL

Sia il file .idl:

```
interface Calculator {  
    // Dichiarazione di una funzione remota che somma due numeri interi.  
    int add(int a, int b);  
}
```

Il file IDL è utilizzato per generare automaticamente sia il **client stub** che il **server skeleton**. Questo processo è gestito da un **compilatore IDL**, che interpreta la definizione dell'interfaccia e produce il codice necessario per la comunicazione tra client e server.

Il client stub contiene le funzioni "proxy" (*vuote ma con i parametri e il nome impostato*), che nascondono i dettagli della comunicazione (*come la serializzazione e il trasporto*).

Il server skeleton contiene le funzioni "dispatcher", che ricevono la richiesta, la inoltrano all'implementazione del metodo e gestiscono la risposta.

7.5 gRPC - Google RPC

gRPC (*Google Remote Procedure Call*) è un framework open-source sviluppato da Google per implementare comunicazioni efficienti e scalabili tra sistemi distribuiti.

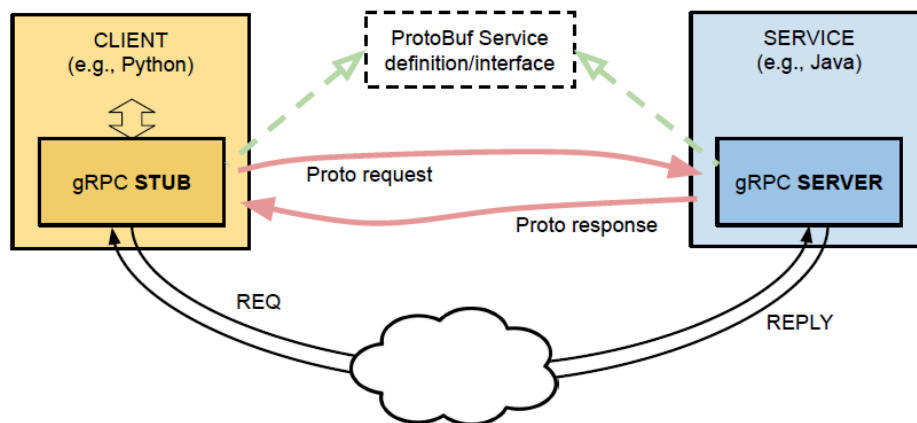
È basato sul protocollo **HTTP/2** e utilizza **Protocol Buffers** (*Protobuf*) come IDL.

Idealmente avremo un tool per ottenere lo `STUB` e lo `SKELETON` (*server*) partendo dalla definizione dell'interfaccia.

ProtoBuf è usato anche per definire strutture dati/messaggi in modo indipendente dal linguaggio, così come la sua serializzazione.

Considereremo `ProtoBuf` come linguaggio per definire un'interfaccia.

Lo scenario in gRPC può essere rappresentato come nella seguente figura, il punto cruciale è la definizione di interfacce comuni tra client e server fatta tramite `ProtoBuf`.



Sul lato server, il server implementa questa interfaccia ed esegue un server gRPC per gestire le chiamate client.

Sul lato client, uno stub con la stessa interfaccia fornisce l'accesso ai metodi del servizio remoto, nascondendo i dettagli della comunicazione.

7.5.1 ProtoBuf

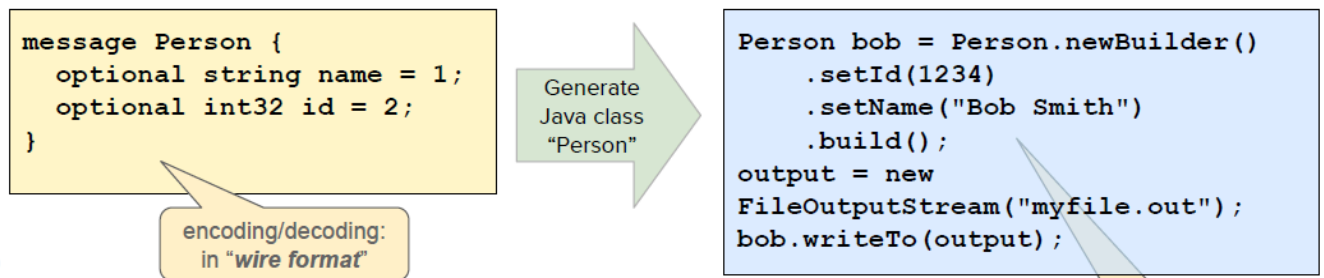
In `ProtoBuf` le strutture dati sono `message` perché quando le informazioni da un processo vengono passate ad un altro, vengono passate come strutture dati.

Il `message` risiede in un file `.proto`.

Da questo file (*tramite un tool*) viene creata una apposita classe Java detta `Person` (*utilizzare uno speciale codice sorgente generato per scrivere e leggere facilmente tali dati da e verso una varietà di flussi di dati*).

Lo stesso approccio si adatta alle esigenze di comunicazione nei sistemi client/server.

I `message` fanno riferimento ad una codifica speciale detta **wire format**.



I **message** sono **immutabili** e possono essere creati tramite il builder relativo. Un builder ha metodi "setter" per i campi messaggio, che possono essere concatenati.

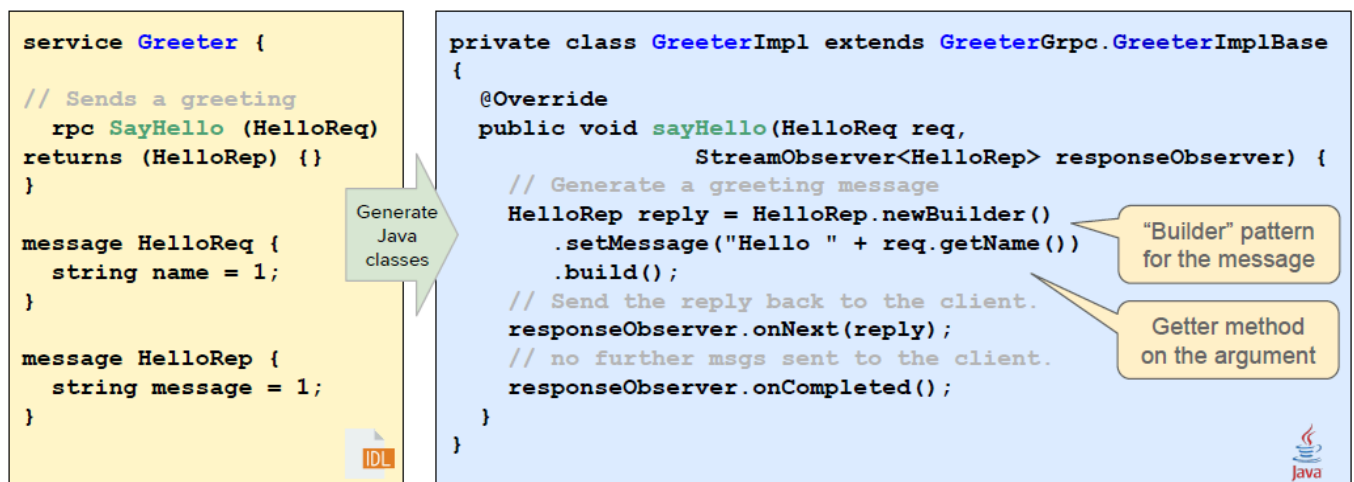
Per creare l'istanza del messaggio uso il metodo `newBuilder()`. Il builder viene informato dei valori da utilizzare come ID e come nome, successivamente chiama il metodo `build()`.

Il messaggio nel file `.proto` avvantaggia la serializzazione (*infatti si vede che specifica il numero esatto di bit per gli interi*).

Parte Server

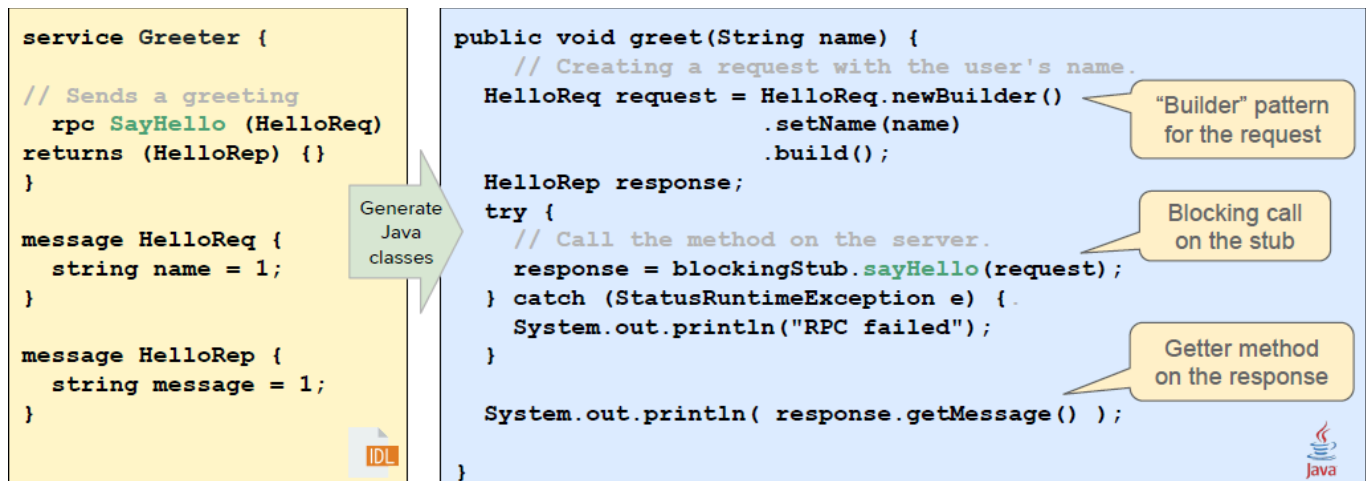
Per implementare il server bisogna specificare l'interfaccia (**sezione in giallo**).

Dopo aver specificato l'interfaccia vengono create automaticamente delle classi, una di queste è la seguente:



Parte Client

L'implementazione del client utilizza la stessa definizione dell'interfaccia utilizzata dal server.



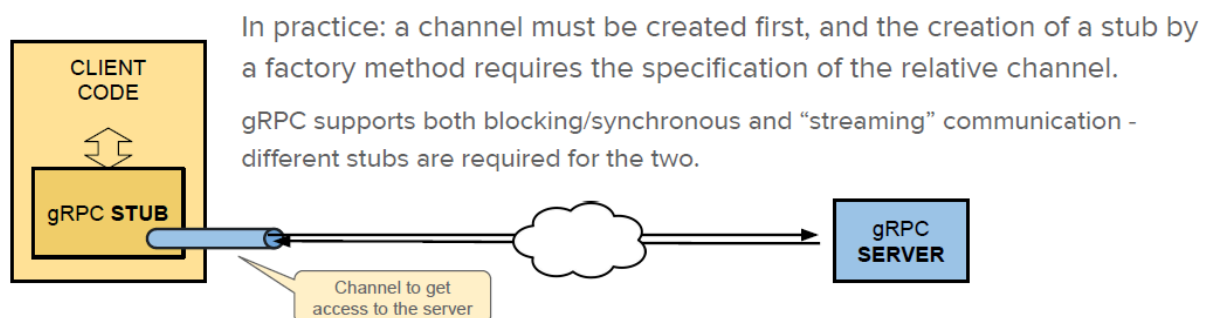
Essendo una chiamata remota qualcosa potrebbe andare storto, quindi viene inserito un blocco try-catch per rilevare le eccezioni.

7.5.2 Localizzare il Server gRPC

Il **gRPC STUB** del client comunica con il **gRPC Server** attraverso un canale che rappresenta la connessione al server a un indirizzo IP e una porta specifici.

In pratica: un canale deve essere creato prima e la creazione di uno stub tramite un metodo factory richiede la specifica del canale relativo.

gRPC supporta sia la comunicazione bloccante/sincrona che quella "streaming": sono richiesti stub diversi per entrambe.



7.6 RMI - Remote Method Invocation

RMI è la tecnologia standard di comunicazione utilizzata anche da altre tecnologie Java. Si tratta di invocare metodi di un oggetto remoto che quindi non si trova nella nostra macchina.

Il sistema che può essere considerato il primo ad applicare il paradigma della programmazione ad oggetti è stato CORBA.

CORBA, acronimo di Common Object Request Broker Architecture, è uno standard sviluppato dall'OMG (*Object Management Group*) per supportare l'invocazione remota di metodi (*Remote Method Invocation - RMI*) tra oggetti distribuiti in un contesto di rete. CORBA permette a programmi scritti in diversi linguaggi di programmazione e su piattaforme differenti di comunicare e interagire come se fossero parte di un unico sistema integrato.

Consente programmazione tramite interfacce, semantica delle chiamate, problemi di trasparenza.

In RMI non affrontiamo il problema dell'eterogeneità dei linguaggi, poiché l'implementazione si basa sulle interfacce Java, che sono parte integrante del linguaggio e vengono utilizzate direttamente per definire i contratti tra client e server.

7.6.1 Riferimenti remoti

Dato che stiamo lavorando con oggetti remoti, dobbiamo localizzare questo oggetto remoto e chiamare metodi su di esso.



La loro creazione avviene da un modulo dedicato che idealmente dovrebbe contenere almeno i campi in figura sotto. Bisogna poter identificare la macchina remota (*IP + porta*), deve contenere la **data di creazione** ed il **numero di oggetto** e infine l'**interfaccia dell'oggetto remoto**.

Un riferimento remoto permette l'accesso diretto ad un oggetto in un nodo diverso da quello corrente come se fosse nel nodo corrente.

Un riferimento remoto è **unico in tutto il sistema distribuito** ed è creato dal RRM (*Rem. Ref. Module*), ovvero un componente dedicato.

Idealmente, la struttura di un riferimento remoto è quella mostrata nell'immagine sotto:

32 bits	32 bits	32 bits	32 bits	
IP Address	Port	Creation time	Object nr.	Interface of r. obj

Interfaccia remota

Un oggetto remoto si differenzia da uno locale se implementa una specifica interfaccia, chiamata *interfaccia remota*.

Un'*interfaccia remota* è un'interfaccia che dichiara un set di metodi che possono essere richiamati da un sistema JVM remoto (*client*).

Un'interfaccia è "*remota*" se almeno estende (*eventualmente indirettamente*) `java.rmi.Remote`, che è un'interfaccia marcatore (*nessun metodo*).

Ogni dichiarazione di metodo in un'interfaccia remota (*o nelle sue super-interfacce*) deve essere una dichiarazione di metodo remoto quindi che utilizza un riferimento remoto.

Comportamenti inaspettati

Quando chiamiamo un metodo locale non ci aspettiamo che qualcosa vada storto, mentre quando chiamiamo un metodo remoto potrebbero capitare comportamenti inaspettati.

Per questo motivo ogni metodo remoto deve includere `java.rmi.RemoteException` nella sua clausola `throws`, assieme ad altre possibili eccezioni che potrebbero essere necessarie (*che non devono estendere* `java.rmi.RemoteException`);

Utilizzo di una RMI

Per utilizzare una RMI basta avere l'interfaccia remota e non la classe di implementazione, la quale può rimanere nascosta.

Passaggio di parametri

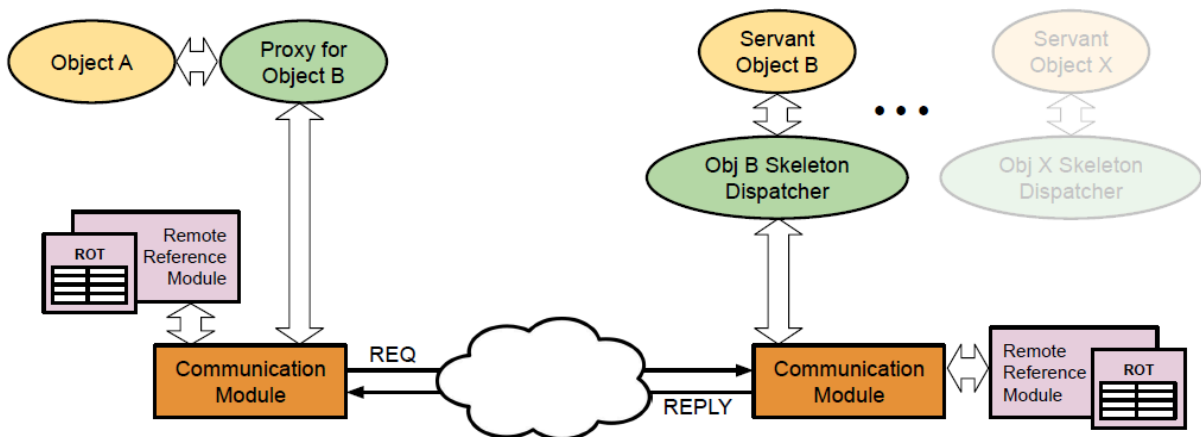
In RMI, per i metodi remoti di oggetti remoti, gli argomenti sono IN, il valore di ritorno è l'unico OUT.

I parametri in questo contesto sono di due tipologie:

- **Parametro Ordinario** "per valore"; deve essere sottoposto a marshalling, quindi è necessario implementare `Serializable`.
- **Riferimenti remoti**; devono essere tipizzati come `Remote` (*interfaccia*).

7.6.2 Implementazione di una RMI

In figura vediamo che gli oggetti singoli specifici del server sono indicati con *Servant Object*, mentre gli oggetti del client sono indicati come *Object*.



L'interfaccia ora è la Java Remote Interface.

Oggetto Proxy

Il client andrà ad interagire con il Proxy, ovvero una controparte dell'oggetto remoto "vero" (*praticamente uno Stub*), responsabile del **marshalling/unmarshalling** e dell'**invio al server** attraverso il modulo di comunicazione.

Il proxy è costruito sulla base dell'interfaccia remota (*richiede rmic per Java < 5*)

A differenza di RPC, il client interagisce con un oggetto vero e proprio tramite le tecniche classiche della programmazione ad oggetti.

Il proxy, dato che è stato costruito partendo dall'interfaccia, nasconde la complessità delle comunicazioni remote.

Modulo di comunicazione

Sia nel client che nel server si ha un modulo di comunicazione, il suo scopo è implementare la comunicazione all'interno della rete.

ROT e RRM

Il RRM (Remote Reference Module) controlla la Remote Object Table (ROT), che è una mappa contenente:

- I riferimenti agli oggetti remoti registrati.
- Gli identificativi univoci associati a ciascun oggetto.

Nella ROT c'è un riferimento all'oggetto remoto della calcolatrice, registrato come "CalcolatriceRemota".

Il RRM identifica che la richiesta è per questo oggetto e la inoltra alla sua implementazione (ad esempio, il metodo somma della classe *CalcolatriceImpl*).

Dispatcher e Skeleton

C'è un Dispatcher e uno Skeleton per ogni oggetto remoto offerto dal server.

7.6.3 Esempio di funzionamento

1. Il client invoca un metodo sul proxy (client stub).

Il proxy serializza i parametri e interagisce con il modulo di comunicazione.

2. Il modulo di comunicazione trasmette la richiesta.
3. Il server riceve la richiesta tramite il suo modulo di comunicazione.
4. Il modulo di comunicazione chiede a RRM il riferimento locale del dispatcher/skeleton coinvolto.
5. Il RRM individua l'oggetto remoto nella ROT.

La corrispondenza tra riferimenti locali e remoti da usare in realtà è tenuta nella ROT.

6. Il dispatcher, in base a `opId`, determina il metodo da invocare sullo skeleton.
7. Lo skeleton esegue l'unmarshalling e invoca il metodo corretto sul servant corretto.
8. Il risultato viene restituito e il proxy lato client riceve il risultato.

7.6.4 Servants

I servants sono dei processi lato server e ognuno di essi è un'istanza di una classe che implementa una interfaccia remota.

Una classe può:

- Implementare un numero qualsiasi di interfacce remote.
- Può estendere un'altra classe che implementa una o più interfacce remote.
- Può avere anche metodi non-remoti ma utilizzabili solo localmente.

Il servant deve essere esportato ai client remoti.

Se la classe estende `UnicastRemoteObject`, questa cosa è fatta in modo implicito, altrimenti l'operazione deve essere fatta invocando un metodo:

```
UnicastRemoteObject.exportObject (Obj)
```

Ricerca di un oggetto remoto

L'accesso ad un servant può essere fatto solo se il riferimento è disponibile.

Serve un *rmiregistry* per cercare i riferimenti remoti dai nomi (*vale a dire, agisce come un servizio di nomi*), il quale si comporta come un name registry.

1. Un servant si collega al *rmiregistry* rendendo il suo riferimento remoto disponibile in esso.
2. Qualsiasi client può cercare il relativo riferimento remoto in *rmiregistry*.

7.6.5 RMI e Multithreading

Quando viene invocato un RMI, come viene eseguito?

- **Thread-per-request:** Ogni richiesta è gestita da un thread separato.
- **Thread-per-connection:** Ogni thread è dedicato ad una connessione con un client.
- **Thread-per-object:** Le richieste a un singolo oggetto servant sono gestite su un thread dedicato.

Questa cosa non è decisa dal singolo programmatore ma è una cosa definita per l'intero sistema.

In verità, si possono ideare altre combinazioni di queste policy creando modalità molto personalizzate.

"Un metodo inviato dal runtime RMI a un'implementazione di oggetti remoti (un server) può o non può essere eseguito in un thread separato. Le chiamate provenienti da diverse macchine virtuali client verranno eseguite in thread diversi.

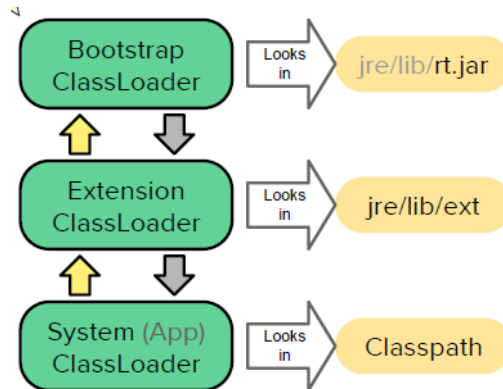
Dalla stessa macchina client non è garantito che ogni metodo verrà eseguito in un thread separato".

Quindi la cosa importante per il programmatore è che il codice relativo alla processazione di una richiesta sia thread-safe (in modo che vada sempre bene) e quindi deve contare della concorrenza per le risorse.

7.7 Class Loading in Java

7.7.1 Distributed Class Loader

I class loaders sono organizzati in una gerarchia basata sul delegare il lavoro al padre.



Quando arriva una richiesta di caricamento, il primo che la prende in carico è il bootstrap loader.

Ogni classloader chiede prima al “*padre*” di caricare la classe, nel caso non potesse allora proverà a fare il lavoro da solo.

Il modulo più alto (*bootstrap loader*) è parte della JVM e ha il compito di guardare in `jre/lib/rt.jar` che contiene le classi di base della Standard Library.

Se la classe non è trovata lì allora interviene l’extension ClassLoader; se anche lui fallisce ci penserà il System ClassLoader il quale guarda i file registrati nel Classpath.

Proprietà di “*loading*”: unicità, visibilità.

Caricamento esplicito: `cl.loadClass(...)`, `Class.forName(...)`.

Class Loading in un sistema distribuito

Il **class loading distribuito** in Java è un meccanismo che consente a un’applicazione (*client*) di caricare classi da fonti remote (*server*) durante l’esecuzione.

A cosa serve un class loader distribuito?

In un sistema distribuito che sfrutta le RMI, è possibile che il client non abbia la definizione della classe di un oggetto remoto (*il file .class*).

In questi casi, il Distributed ClassLoader consente di **scaricare la definizione della classe** (*il file .class*) da un server.

Questo viene gestito da un classloader distribuito che si occupa di **localizzare e scaricare la definizione della classe necessaria**.

Il download avviene tramite HTTP e il server da cui scaricare l’oggetto remoto è identificato tramite un URL, il quale è contenuto nel riferimento remoto stesso.

Class loading distribuito e RMI

Se un client invoca un metodo remoto su un server e il server restituisce un oggetto di una classe sconosciuta al client.

Il codice della classe può essere caricato dinamicamente da una risorsa remota (*ad esempio, un server HTTP o un file system remoto*).

Il sistema utilizza la proprietà di sistema `java.rmi.server.codebase`, che specifica l'URL da cui caricare le classi remote.

```
System.setProperty("java.rmi.server.codebase", "http://example.com/classes/");
```

7.8 Garbage Collection

In un sistema Java standalone, la garbage collection è il **processo automatico con cui la JVM libera la memoria non più utilizzata**, recuperando lo spazio occupato da oggetti che non sono più raggiungibili.

Una singola esecuzione della garbage collection non blocca il sistema: può essere "interleaved" con il programma principale.

Questa procedura è **fondamentale per gestire la memoria in Java**, dato che il linguaggio non richiede all'utente di liberare esplicitamente la memoria (*come accade ad esempio in C o C++*).

Ci sono modi differenti per organizzare la garbage collection, possiamo raggruppare tutti gli algoritmi di garbage collection in due classi principali.

7.8.1 Approccio mark and sweep

Ovvero approccio "*marca e spazza*" usato tipicamente nelle implementazioni ordinarie della JVM.

1. Identifica tutti gli oggetti che non sono più usati (*raggiungibili*) nel sistema.
2. Vengono marcati.
3. Successivamente, gli oggetti marcati vengono eliminati.

Iniziando dai nodi principali possiamo raggiungere tutti i nodi, alcuni nodi presenti nel sistema non sono raggiungibili, questi sono il target per la garbage collection.

L'algoritmo inizia da un **pool di oggetti radice** che è quello principale, ci sono vari oggetti radice:

- Variabili locali e metodo main.
- Main thread.
- Variabili statiche della classe main.

L'algoritmo (*ce ne sono vari, ma ne vedremo solo uno*) per l'attraversamento dell'intero grafo è detto **tri-color marking**.

Algoritmo Tri-color marking

Questo algoritmo non richiede lo stop dell'esecuzione del programma ma è un processo concorrente al programma stesso.

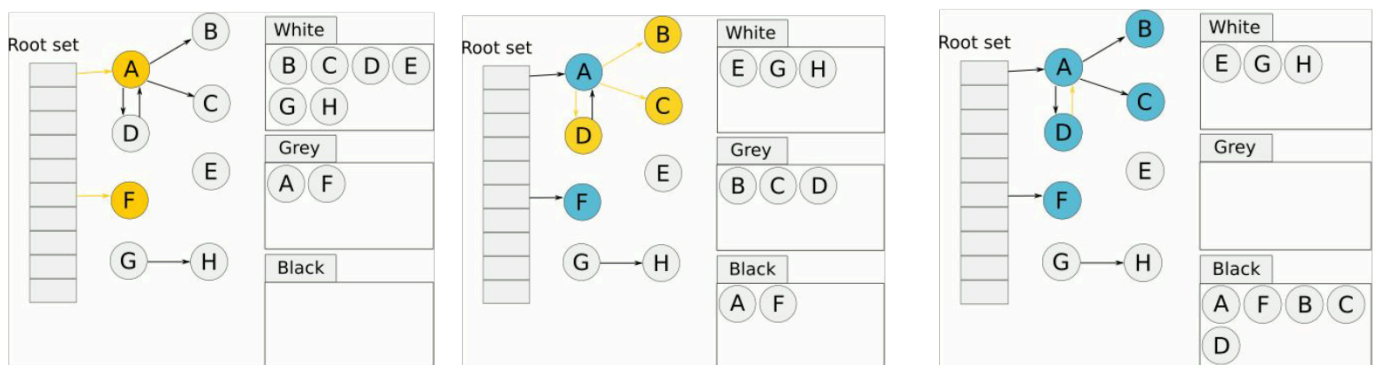
La marcatura richiede l'attraversamento del grafico complessivo degli oggetti.

L'algoritmo divide gli oggetti in tre set diversi:

- **Bianco:** Oggetti candidati da essere “spazzati”, ovvero eliminati.
- **Grigio:** (*ancora da spostare in bianco o nero*) Sono oggetti raggiungibili dalle radici e non ancora scannerizzati per possibili riferimenti ad oggetti nella categoria bianca.
- **Nero:** Oggetti intoccabili (*non candidati per la raccolta*), raggiungibili dalle radici e non possono essere tolti dalla memoria. Gli oggetti in questa categoria non possono avere riferimenti ad oggetti nella categoria bianca.

All'inizio il set nero è vuoto e il set grigio contiene tutti gli oggetti raggiungibili dagli oggetti radice, infine il set bianco contiene tutto il resto.

1. Finché il set grigio non è vuoto:
 - a. Prendiamo un oggetto X dal grigio e spostiamolo in nero.
 - b. Muoviamo in grigio tutti gli oggetti in bianco che hanno riferimenti con X.
2. Quando il set grigio è vuoto, tutti gli oggetti in bianco possono essere eliminati.



In questo esempio abbiamo a sinistra l'insieme root (A, F).

1. **Inizializzazione:** Inseriamo A ed F in grigio e tutto il resto in bianco.
2. Spostiamo A ed F nel set nero.
3. B, C e D hanno un riferimento con un oggetto nella categoria nera (A), quindi li spostiamo nella categoria grigia.
4. Adesso spostiamo gli oggetti B, C e D in nero perchè sono nella categoria grigia.
5. Il set grigio è vuoto, quindi tutti gli oggetti rimasti in bianco (E, G, H) vengono eliminati.

7.8.2 Performance della garbage collection

Problemi dell'approccio mark and sweep

Gli oggetti sono creati velocemente e traversare il grafo dei riferimenti può rappresentare un compito molto gravoso e lento.

Un modo per ottimizzare è quello di fare in modo che oggetto mantenga un **contatore di riferimenti (in arrivo)**;

- Riferimenti oggetto aggiunti/rimossi → il contatore viene incrementato/decrementato.
- Il GC può rimuovere tutti gli oggetti il cui contatore = 0

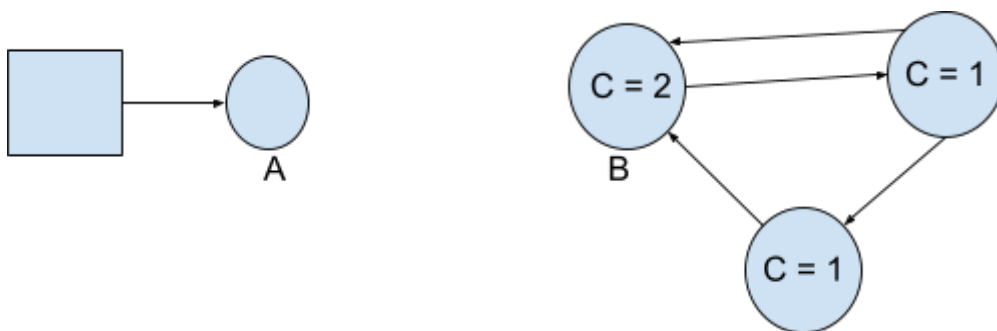
Problema: **possibili riferimenti ciclici** portano a perdite di memoria! Pertanto, devono essere eseguiti alcuni controlli periodici per questo.

Se un oggetto usa l'altro, allora i due oggetti avranno sempre il contatore maggiore di 0 anche se non sono mai usati.

Questa è una delle debolezze dell'approccio Mark and Sweep.

Ad esempio abbiamo un oggetto (*oggetto A*) che ne crea un altro (*oggetto B*).

Successivamente non mantiene il riferimento, quindi non potendo raggiungere B e quelli a cui B è collegato, B e gli altri andrebbero rimossi, però quei tre oggetti hanno un contatore differente da 0, quindi non verranno rimossi.



Il GC locale è solitamente implementato utilizzando approcci Mark&Sweep.

Oggetti diversi mostrano diversi modelli di utilizzo (*in genere, si osserva un'elevata "mortalità infantile"*), e questo influisce sulle prestazioni del GC.

Per questo motivo, gli oggetti sono divisi in "popolazioni" (*ad esempio, giovani e anziani*) in base al loro comportamento, in modo da ottimizzare il processo complessivo (*solo gli oggetti più giovani sono impostati su bianco*).

Come determinare se un oggetto appartiene ad una popolazione o ad un'altra? All'inizio un oggetto appartiene alla popolazione giovane, se sopravvive per un tempo prestabilito verrà spostato in un'altra popolazione.

7.8.3 DGC - Distributed Garbage Collection

Meccanismo che consente di gestire in modo automatico la rimozione degli oggetti remoti non più utilizzati.

In base al conteggio dei riferimenti; il GC locale e il GC distribuito devono collaborare!

Il client informa sempre il server sui riferimenti remoti che sta utilizzando (*perché li richiede attraverso il class loading*).

Quindi il server mantiene, per ogni entrata della ROT, la lista di client che hanno riferimenti remoti ad un certo oggetto (*il servant*).

Questa lista di oggetti deve essere aggiornata ogni volta che un riferimento remoto è creato, duplicato o rimosso.

Le funzionalità del DGC sono specificate in una interfaccia Java chiamata:

```
java.rmi.dgc.DGC
```

Utilizzata per il componente lato server del sistema complessivo.

Questa interfaccia ha due metodi:

- `dirty(<list of rem refs>, ... , <Client JVM_ID>)`
- `clean(<list of rem refs>, ... , <Client JVM_ID>)`

Ognuna delle due ha svariati parametri.

- `dirty()` consente al client di informare il server sul fatto che quel client JVM utilizza quella lista di riferimenti remoti. Questa invocazione deve essere eseguita dal runtime RMI appena prima di costruire i proxy per gli oggetti remoti corrispondenti.
- `clean()` serve per informare il server che il client non utilizzerà più i riferimenti remoti per un certo oggetto (*rimozione di JVM_ID dagli elenchi relativi di "holder"*).

Appena una lista diventa vuota e nessun riferimento locale è presente per un oggetto, quell'oggetto può essere rimosso dallo heap.

I problemi che nascono dal DGC sono:

1. Data race tra `dirty(X , ... )` e `clean(X , ...)` quando la lista dei detentori dell'oggetto `X` hanno un singolo elemento.
2. Eventuali crash dei client possono prevenire che una lista diventi vuota, portando ad un memory leak (*perdita di memoria*). Poiché il client non è in grado di dire che non userà un certo oggetto e quindi il server è obbligato a tenere quell'oggetto lì per sempre, portando ad una perdita di memoria a lungo termine.

Problema 1 - `clean(X, ...)` and `dirty(X, ...)` race

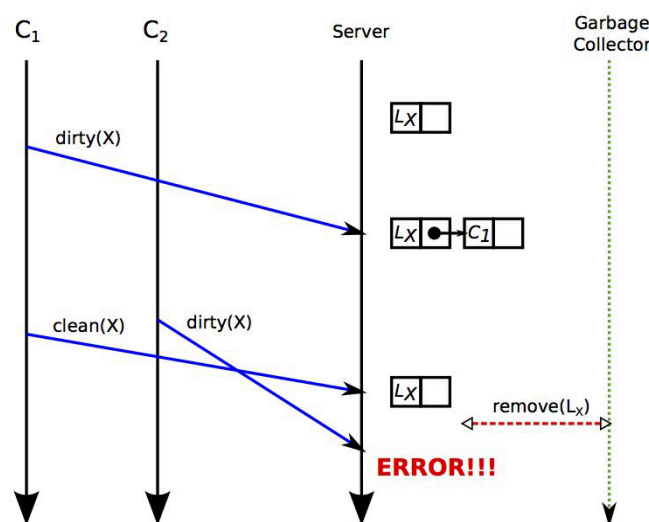
Supponiamo che il server mantenga un oggetto remoto `X` e siano due client `C1` e `C2` che vogliono utilizzare `X`.

All'inizio il server ha la lista corrispondente ai riferimenti remoti di X .

1. C1 invia al server: `dirty(X, ...)`.
2. La prima ad arrivare è quella di C1 e il server aggiorna la lista degli owner di X .
3. C2 invia al server: `dirty(X, ...)`.
4. C1 invia una `clean(X, ...)` ma la `dirty(X, ...)` di C2 non è ancora arrivata.
5. L'oggetto potrebbe venire rimosso dal garbage collector.

Supponiamo che X sia rimosso.

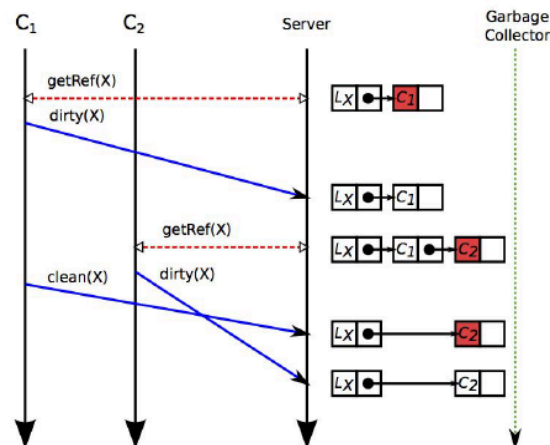
6. La `dirty(X, ...)` di C2 arriva e **viene sollevata un'eccezione** perché X è stato rimosso.



Una soluzione è quella dei **segnaposto**.

Se C2 vuole fare una `dirty(X, ...)`, significa che C2 in qualche modo conosce l'esistenza di X e quindi ha potuto prenderne il riferimento da qualche parte.

Possiamo fare in modo che il server dia la **ownership di un oggetto nel momento in cui ne viene chiesto il riferimento.**



Adesso nel momento in cui viene richiesto il riferimento viene creato un "*placeholder*", ovvero una copia di *X* fittizia.

Il segnaposto elenca i nodi in attesa del corrispondente *dirty(X, ...)* a venire. Il placeholder fa in modo che, in caso sia presente, l'oggetto non venga pulito.

Problema 2 - Crash del client

Questo problema può essere risolto cambiando la semantica di *dirty()*.

Possiamo modificare il significato in modo che il client indichi al server per quanto tempo utilizzerà una risorsa (*leasing*). Adesso *dirty()* restituisce un tempo di lease.

Il client può rinnovare il tempo inviando di nuovo una *dirty()*.

Oltre il tempo di leasing, il server considera come se fosse stata ricevuta una *clean()*, quindi la risorsa è deallocata dal client.

Sunto

Gli oggetti remoti sono istanze di classi che implementano l'interfaccia *java.rmi.Remote*.

Quando un client ottiene un riferimento a un oggetto remoto, viene mantenuta una connessione logica tra il client e il server, chiamata **remote reference**.

Lease per i riferimenti remoti:

- Ogni riferimento remoto ha un "contratto" temporale, chiamato **lease**, che specifica per quanto tempo il client dichiara di voler utilizzare l'oggetto remoto.
- Periodicamente, il client deve rinnovare il lease con una *dirty()*.
- Se un client non rinnova il lease (ad esempio, perché il client è terminato o ha perso il riferimento), il server considera l'oggetto remoto non più in uso.

Raccolta degli oggetti remoti:

- Quando tutti i riferimenti remoti a un oggetto non sono più attivi, il DGC rimuove l'oggetto remoto dalla memoria del server.

- Questo avviene in modo analogo al garbage collection locale, ma considerando anche riferimenti distribuiti.

Capitolo 8 - Servizi Web

Perchè un client deve chiedere l'aiuto di un server per effettuare una certa operazione? per ottenere una certa risorsa, ad esempio necessità di ulteriore RAM per addestrare una rete neurale.

Il client contatta il server per un servizio che fornisca qualcosa che il client non ha. Ci sono protocolli come ad esempio HTTP che consentono comunicazione machine-to-machine.

Inoltre sono state sviluppate versioni di HTTP che sono lightweight, ovvero non includono un gran overhead.

Lo scopo nei servizi web è l'interoperabilità.

Le WS sono app client/server che comunicano tramite HTTP. Come descritto dal W3C, le WS forniscono un mezzo standard di interoperabilità tra applicazioni software in esecuzione su una varietà di piattaforme e framework.

Se vogliamo specificare come accedere ad un servizio dobbiamo scrivere una sorta di API, in questo caso bisogna specificare API secondo interazioni web.

I programmi che forniscono servizi semplici possono interagire per fornire servizi a valore aggiunto più complessi (*ad esempio: "mashup" nelle app Web*).

Dalle API ordinarie alle API Web del server.

8.1 Modalità per sviluppare servizi web

Ci sono due modi per costruire un servizio web:

8.1.1 Big web services

Si concentrano sull'accesso al servizio, mediante il protocollo SOAP a livello di applicazione, codifica XML e protocolli aggiuntivi. Standard W3C per WS.

Messaggi che contengono un contenuto XML utilizzando il protocollo SOAP, con codifica XML dei tipi di dati.

Indirizzamento di aspetti non funzionali: transazioni, sicurezza, ecc.

Un servizio può essere descritto in modo formale, questo comporta vari vantaggi perché possiamo confrontare le diverse descrizioni (*tramite WSDL*).

La flessibilità è data al costo di aumentare la complessità utilizzata nell'intero sistema.

La complessità dello sviluppo può essere ridotta utilizzando gli IDE, però non è sufficiente.

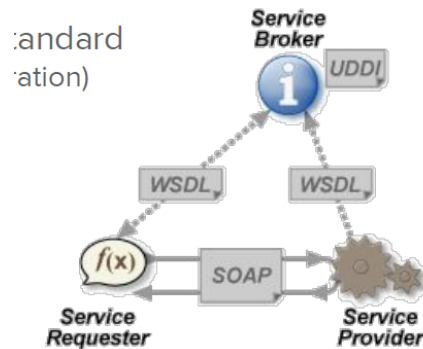
API Java: JAX-WS, che rende SOAP trasparente ai programmatori.

Lo sviluppo del sistema può essere organizzato in vari modi, uno di questi è WSDL first, l'altro approccio è Implementation first, in quest'ultimo approccio il file WSDL può essere derivato automaticamente dalla classe.

C'è la possibilità di pubblicare tutti i file WSDL per consentire a tutti i client di accedere ad essi.

Broker Repository: utilizzo dello standard UDDI (*Universal Description Discovery and Integration*).

Mirato a sistemi SOA (*Service-Oriented Architecture*), in realtà, non molto popolare al giorno d'oggi.



8.1.2 RESTful web services

Si concentrano sull'accesso alle risorse, JSON o XML.

Segue lo stile architetturale REST la quale non segue un vero standard, è un modo per organizzare la comunicazione, ciò che viene comunicato sono informazioni sullo stato corrente.

L'idea è semplice, abbiamo una risorsa nel server e la richiesta HTTP restituisce la descrizione di uno stato, tale messaggio ha lo scopo di ottenere tale risorsa.

Tutte le entità gestite dal servizio sono considerate **risorse**, ognuna identificata da un URL univoco.

Esempio: <http://example.com/users/123> rappresenta un utente con ID 123.

Vincoli architetturali REST

- Architettura client-server.
- **Stateless-ness**: Ogni richiesta al server è **indipendente** e non mantiene alcuno stato sul server. Tutte le informazioni necessarie devono essere incluse nella richiesta.
- **Cacheability**: Le risposte possono essere contrassegnate come "cacheable" per migliorare le prestazioni.
- **Sistema a strati**: Nessun modo per un client di distinguere il server da un intermediario.
- **Codice su richiesta**: Possibilità di trasferire codice (*ad esempio JavaScript...*).

- **REST Uniform Interface:** L'interazione tra client e server avviene tramite un'interfaccia uniforme basata su standard HTTP, semplificando la comunicazione.

REST uniform interface

Le risorse vengono manipolate da un set fisso di operazioni, specificate dal "*metodo*" del messaggio HTTP:

- **PUT** crea una nuova risorsa e **DELETE** ne elimina una.
- **GET** recupera lo stato corrente di una risorsa in una qualche rappresentazione. Le risorse sono disaccoppiate dalla loro rappresentazione: il loro contenuto è accessibile in molti formati, come HTML, XML, testo normale, PDF, JPEG, JSON, ...
- **POST** trasferisce un nuovo stato su una risorsa.

REST API

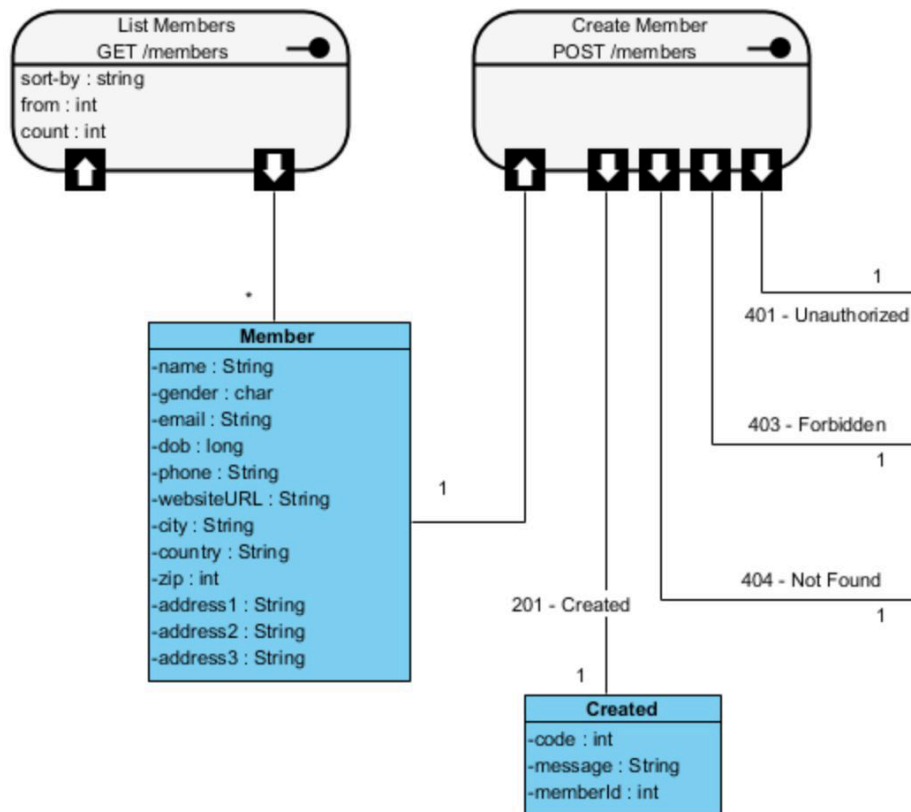
Interfaccia di programmazione che consente a diversi sistemi di comunicare tra loro tramite il protocollo HTTP, seguendo i principi dell'architettura REST.

Il design di un API per un servizio esposto tramite REST è molto importante dal punto di vista del software engineering, per questo sono stati introdotti vari description language (DL) come RSDL (*RESTful Service Description Language*), RAML (*RESTful API Modeling Language*), OpenAPI.

Utilizzare annotazioni per decorare una "*classe di risorse*" per gestirla come una risorsa web REST. Alcune annotazioni tipiche:

- **@Path** - Un percorso URI relativo che indica il "posizionamento" della classe.
- **@GET** - Annotazione per il metodo della classe per elaborare le richieste GET.
- **@Produces** - Specifica il tipo MIME del contenuto prodotto.

In caso di server web RESTful c'è JAX-RS che descrive ogni funzionalità necessaria.



8.1.3 JSON

È un formato indipendente dal linguaggio molto diffuso utilizzato per scambiare dati in sistemi interoperabili.

È un formato testuale, scritto secondo la notazione degli oggetti di JavaScript. In JS: `JSON.parse(text)`, `JSON.stringify(obj)`.

Object: raccolta non ordinata all'interno di `{ }` di coppie separate da virgole composte da nome (stringa) : valore.

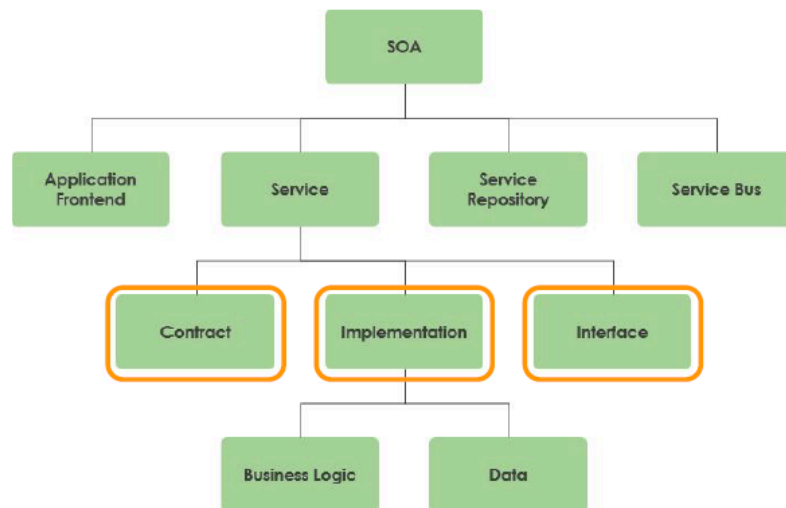
```

{
  "firstname": "Tom",
  "lastname": "Smith",
  "age": 20,
  "exams": ["Calculus", "Algebra", "Algorithms"]
}
  
```

Come si può vedere, JSON come XML è autodescrittivo e facile da comprendere, ma supporta anche gli array.

8.2 SOA - Service Oriented Architecture

Stile architetturale che organizza un sistema software come un insieme di **servizi indipendenti**, ognuno progettato per eseguire una funzione specifica e ben definita. L'obiettivo principale di SOA è favorire la **riusabilità**, la **scalabilità** e la **manutenibilità** del software, consentendo ai servizi di comunicare tra loro attraverso protocolli standard.



Interfaccia

Il punto di accesso al servizio ed è la rappresentazione tecnica del contratto, attraverso cui i client interagiscono con il servizio.

- **Cosa definisce l'interfaccia:**
 - I metodi o endpoint esposti (es. *URL di un'API REST, operazioni SOAP*).
 - I tipi di dati scambiati (es. *JSON, XML*).
 - Le convenzioni di chiamata (es. *parametri query, intestazioni HTTP*).
- **Differenza dal contratto:**
 - Il contratto è una descrizione di cosa fa il servizio, mentre l'interfaccia è **l'implementazione tecnica** di come accedere al servizio.

Contratto

Il **contratto** definisce ciò che un servizio offre e come i client possono interagire con esso. È una specifica chiara e formale delle funzionalità esposte dal servizio.

- **Operazioni disponibili:** Quali azioni il servizio può eseguire.
- **Parametri:** Input richiesto e output previsto per ogni operazione.
- **Protocolli e formati:** Come deve avvenire la comunicazione (es. *HTTP, SOAP, JSON, XML*).
- **Requisiti di sicurezza:** Autenticazione, autorizzazione, ecc.

In un web service REST, il contratto potrebbe essere rappresentato dalla documentazione API (es. tramite OpenAPI/Swagger).

In un servizio SOAP, il contratto è espresso tramite un file **WSDL** (Web Services Description Language).

Implementazione

Il codice effettivo che realizza il servizio, ovvero il "come" il servizio esegue le operazioni definite nel contratto.

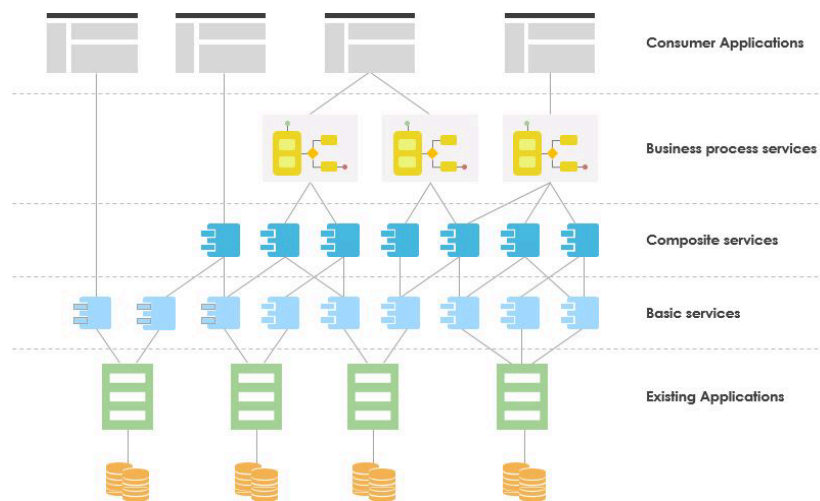
- **Dettagli dell'implementazione:**
 - Linguaggio di programmazione utilizzato.
 - Logica applicativa e accesso ai dati.
 - Algoritmi e gestione delle operazioni richieste dal contratto.
- **Separazione:**
 - L'implementazione è **indipendente** dal contratto e dall'interfaccia. I dettagli su come il servizio funziona sono nascosti ai client.

Jakarta EE

Le applicazioni distribuite sono diventate sempre più complesse, questo ha portato alla definizione di Jakarta EE (*Enterprise Edition*) per consentire all'applicazione di diventare facile da sviluppare e da effettuare il deployment.

Piattaforma open-source per lo sviluppo di applicazioni aziendali basate su Java.

È progettata per semplificare la creazione di applicazioni scalabili, robuste e sicure per ambienti aziendali, come applicazioni web, microservizi e sistemi distribuiti.

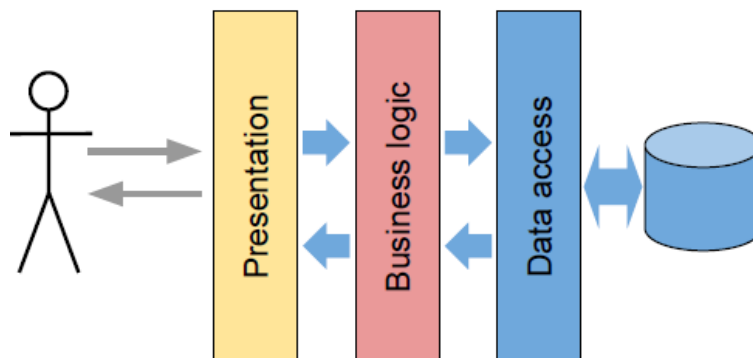


Si tratta di un set di specifiche per definire un intero framework per il supporto di un'ampia varietà di funzionalità per applicazioni Java distribuite, affidandosi a un runtime appropriato (*server web, server applicativi, ecc.*).

Il programmatore può utilizzare le funzionalità fornite dal sistema sottostante. Più l'applicazione diventa complicata abbiamo bisogno di maggiore supporto dal middleware.

Architettura di una applicazione web

L'architettura di un'applicazione web è composta in genere da tre **tier**: in genere schierati su macchine diverse (*l'interazione avviene tramite la rete e protocolli standard come HTTP*).



Un'architettura client-server in cui:

1. **Presentazione:** la GUI con cui l'utente interagisce.
2. **Business Logic:** la logica di processo funzionale.
3. **Business Data:** interfaccia per l'accesso ai dati/archiviazione dati sono sviluppati e mantenuti come moduli indipendenti, possibilmente su piattaforme separate.

In questo tipo di applicazioni **tutti i componenti possono essere organizzati in container separati.**

I container si occupano di gestire e supportare i loro componenti, fornendo all'esterno le funzionalità offerte dai componenti stessi.

I container hanno interfacce di richiesta e di fornitura (*providing*).

Applicazioni web

La prima osservazione è la differenza tra il contenuto statico e dinamico.

Una risorsa può essere qualcosa memorizzata nel file system che può essere acceduta. Una risorsa può essere statica (*non cambia nel tempo*) o dinamica.

Se abbiamo una pagina web che restituisce la scritta "hello world" in questo caso è un contenuto statico, mentre se ad esempio abbiamo un servizio che restituisce le previsioni meteorologiche in tempo reale, questo è un contenuto dinamico.

Il protocollo HTTP si occupa di richiedere una risorsa e di ottenerla.

Spetta al server fornire la risorsa, recuperandola (*contenuto statico*) o generandola al volo tramite un programma (*contenuto dinamico*). Il server come può distinguere il tipo di una risorsa? Ispezionando il contenuto del relativo URL.

8.3 CGI - Common Gateway Interface

Standard che permette l'interazione tra un server web e uno script esterno (*detti anche script CGI*) per generare contenuti dinamici in risposta a richieste HTTP.

CGI definisce un'interfaccia che il server web utilizza per invocare questi programmi e passare loro i dati necessari (*come le variabili di input da un modulo HTML*), e quindi ricevere l'output che deve essere restituito come risposta HTTP.

Esempio di CGI

Un semplice script Python per restituire una pagina HTML:

Supponiamo che un utente invii una richiesta a uno script CGI con l'URL:

`http://example.com/cgi-bin/greet.py?name=Alice&age=30`

Questo script Python legge i parametri `name` e `age` dalla query string e genera una risposta HTML.

```
#!/usr/bin/env python3
import cgi # Modulo per elaborare i dati inviati
import html # Modulo per sanitizzare l'output
# Creazione dell'oggetto FieldStorage per accedere ai parametri
form = cgi.FieldStorage()
# Estrazione dei parametri con valori di default in caso di assenza
name = form.getvalue('name', 'Guest') # Se 'name' non è fornito, usa 'Guest'
age = form.getvalue('age', 'unknown') # Se 'age' non è fornito, usa 'unknown'

name = html.escape(name) # Sanitizzazione per evitare attacchi XSS
age = html.escape(age) # Sanitizzazione per evitare attacchi XSS

print("Content-Type: text/html\n") # Invio dell'header HTTP
# Generazione della risposta HTML
print(f"""
<html>
    <head><title>Greeting Page</title></head>
    <body>
        <h1>Welcome, {name}</h1>
        <p>Your age is: {age}</p>
    </body>
</html>
""")
```

Utilizzo di CGI

1. **Richiesta del client:** Quando un utente fa una richiesta tramite il browser (*ad esempio, cliccando un link o inviando un modulo*), questa viene inviata al server web.
2. **Elaborazione da parte del server web:** Se la richiesta riguarda uno script CGI, il server web invoca il programma da eseguire (*per esempio uno script in Perl, Python, PHP o un eseguibile C*).
3. **Esecuzione del programma CGI:** Lo script CGI viene eseguito dal server e può eseguire qualsiasi tipo di operazione, come interagire con un database, elaborare dei dati o eseguire calcoli complessi.
4. **Output:** Il programma CGI restituisce l'output (*di solito in formato HTML, ma anche JSON, XML o altri formati*) al server come risposta HTTP alla richiesta.
5. **Risposta al client:** Il server web invia quindi il contenuto generato dal programma CGI al browser dell'utente, che lo visualizza.

Problemi di CGI

Il problema di questo approccio era di **prestazioni** perchè ogni richiesta comporta l'avvio di un nuovo processo, il che può risultare inefficiente in scenari ad alto traffico, dove le risorse vengono sprecate nell'avvio di nuovi processi per ogni richiesta.

Inoltre c'è anche un problema di sicurezza: l'esecuzione di script esterni sul server web può rappresentare un rischio di sicurezza se non gestita correttamente.

CGI in pratica

Gli script CGI sono solitamente posizionati nella directory speciale `cgi-bin`. I parametri vengono passati tramite URL.

`http://xyz.com/cgi-bin/myscript.pl/my/pathinfo?a=1&b=2`

Per risolvere il sovraccarico di generazione del processo → Approcci **FastCGI**

Una soluzione per ottenere miglioramenti delle prestazioni sostanziali è quello di eseguire internamente al server web il codice per la generazione del contenuto, ad esempio all'interno dello stesso processo, possibilmente utilizzando il multithreading.

Negli ambienti Microsoft: ASP classico, in Jakarta EE: Servlet e tecnologie correlate.

Servlet: il codice eseguito come conseguenza ad un certo processo viene mantenuto assieme alla JVM. In questo modo ogni overhead è stato rimosso, questo metodo è diventato lo standard fino ai primi anni 2000.

8.4 Tomcat Web Server

Tomcat è un **contenitore di servlet** open source sviluppato dalla **Apache Software Foundation**.

Viene utilizzato per eseguire applicazioni web basate su **Java** e implementa le specifiche di Java Servlet e JavaServer Pages (JSP).

Tomcat è spesso usato per ospitare applicazioni web in ambienti di produzione e sviluppo, ed è una delle soluzioni più popolari per eseguire applicazioni web Java.

Tomcat ha alcuni componenti interni:

- **Catalina**: È il cuore di Tomcat e gestisce il ciclo di vita delle servlet. Catalina è l' "Engine", contenitore Servlet; si riferisce anche a un Realm (DB di nomi utente, password e ruoli relativi). Inoltre è responsabile dell'instradamento delle richieste HTTP, dell'esecuzione delle servlet e della gestione dei vari processi di configurazione. È la parte che implementa la specifica **Servlet API**.
- **Coyote**: È un socket HTTP/HTTPS che permette a Tomcat di comunicare con il client via HTTP. Coyote gestisce la ricezione delle richieste HTTP, le passa a Catalina per l'elaborazione e poi restituisce la risposta al client. Gestisce anche i protocolli come HTTP/1.1 e AJP (*Apache JServ Protocol*).
- **Jasper**: È il motore di compilazione (*container*) delle **JavaServer Pages (JSP)**. Quando viene richiesta una JSP, Jasper la compila in una servlet, che poi viene eseguita da Catalina per generare la risposta dinamica. Jasper implementa la parte JSP della specifica Java EE.

Elemento "*Service*": combinazione di almeno un Connector che condividono un singolo componente Engine per l'elaborazione delle richieste in arrivo.

In Tomcat un servizio è una combinazione di connettori.

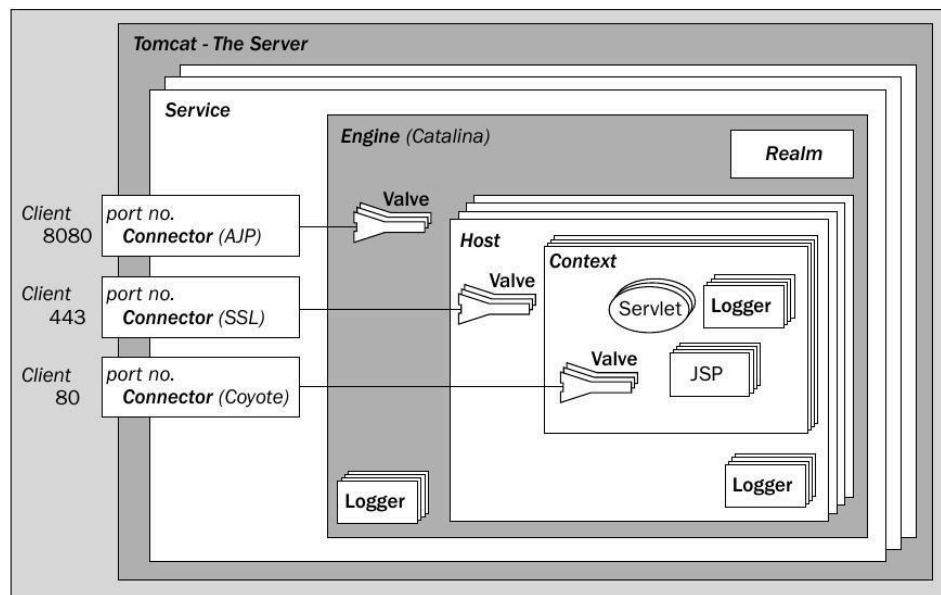
8.4.1 Architettura di Tomcat

Tomcat è il web server di riferimento per Java.

Composto da vari connettori connessi a porte differenti, ogni engine può lavorare con host multipli, all'interno dello stesso host possono esserci applicazioni multiple.

I valve sono elementi da inserire nella pipeline di elaborazione REQ.

Altri elementi nidificati: (*Session*) Manager, (*W.apps class*) Loader, Listener(s), ecc.



8.4.2 Host and Context

Host

Un **Host** rappresenta un **contenitore virtuale** che definisce un'istanza di un **virtual host**. È un'unità logica che consente di gestire più siti web o applicazioni web separati all'interno dello stesso server Tomcat.

Un **Host** rappresenta un singolo dominio (ad esempio, www.miosito.com) ed è una configurazione che mappa un dominio a una determinata applicazione web.

```
<Host name="www.miosito.com" appBase="webapps" unpackWARs="true" autoDeploy="true">
  <Context path="/" docBase="/var/www/miosito" />
</Host>
```

Osservando solo l'URL, con tutto ciò che viene dopo il primo "/" indica varie possibili applicazioni, osservando la prima parte dell'URL dopo il primo "/" identifichiamo il contesto specifico a cui tale URL è riferito.

In Tomcat, i **Host** sono definiti all'interno di un elemento `<Host>` nel file di configurazione `server.xml`.

Ogni **Host** è associato a un nome di dominio specifico e a una directory in cui si trovano le applicazioni web per quel dominio.

Context

Un **Context** è un'istanza di un'applicazione WEB all'interno di un **Host**.

Ogni **Context** rappresenta una singola applicazione web e definisce il suo ambiente e le sue risorse all'interno del server Tomcat.

Un **Context** è associato a un percorso URL specifico. Ad esempio, se un'applicazione è raggiungibile tramite l'URL `http://example.com/myapp`, il **Context** gestisce tutte le richieste che arrivano a quel percorso (`/myapp`).

Quindi un context rappresenta una specifica applicazione web offerta da un host.

```
<Host name="example.com" appBase="webapps">
  <Context path="/myapp" docBase="myapp" />
  <Context path="/app1" docBase="/var/www/app1" />
</Host>
```

- `path="/app1"`: è il **Context Path**, indica il percorso per l'applicazione web all'interno dell'Host.
- `docBase="/var/www/app1"`: è la directory fisica in cui si trova l'applicazione web.
- `example.com` è l'host in questione.

Quindi l'URL per accedere a quella risorsa sarà:

`http://www.example.com/myapp`

Un **Context** è associato a una directory in cui si trovano i file dell'applicazione (tipicamente una directory con un file **WEB-INF**).

In Tomcat, i **Context** sono definiti nel file `context.xml` o direttamente nel `server.xml` all'interno della configurazione del **Host**.

WAR - Web Application aRchives

Una **web application** in Tomcat è una directory o un file WAR (*Web Application Archive*) che contiene tutti i file necessari per l'esecuzione di un'applicazione web, come HTML, CSS, JavaScript, JSP (*Java Server Pages*), servlets, e le configurazioni necessarie.

```
myapp.war
├─ WEB-INF/
│   ├─ web.xml          # Deployment descriptor
│   └─ lib/              # Librerie JAR necessarie per l'applicazione
│       └─ classes/      # Classi compilate (com.example.MyServlet, ecc.)
└─ index.jsp            # Una pagina JSP (esempio di risorsa)
```

```
└─ somepage.jsp      # Un'altra pagina JSP
└─ images/           # Eventuali immagini o risorse statiche
    └─ logo.png
```

Context-root

Il **context-root** è il percorso URL di base associato a un'applicazione web all'interno di un **server** come **Tomcat**. Definisce la parte iniziale dell'URL che è mappata a una determinata applicazione web. In altre parole, è il punto di accesso all'applicazione.

Ad esempio, se un'applicazione ha il **context-path** `/app1`, la sua **root** sarà raggiungibile con l'URL `http://www.miosito.com/app1`.

WEB-INF

La directory **WEB-INF** è una **directory speciale** che contiene il file di configurazione `web.xml`, le classi Java e altre risorse usate dal server (*non direttamente accessibili agli utenti*).

- `classes/`: Una sottodirectory in **WEB-INF** che contiene i file `.class` compilati, ovvero le classi Java dell'applicazione (per esempio, le servlet). Queste classi vengono caricate dal server servlet quando è necessaria l'esecuzione di una servlet.
- `lib/`: Una sottodirectory in **WEB-INF** che contiene tutte le librerie JAR necessarie per l'applicazione. Queste librerie possono includere file JAR che contengono servlet, JSP, o altre dipendenze esterne.

File di configurazione web.xml

Il **file di configurazione** di un'applicazione web Java ed è posizionato nella cartella **WEB-INF/** di una webapp.

- Definisce **Servlet** e i loro URL mapping.
- Configura **filtri**, **listener** e **parametri di inizializzazione**.
- Imposta la **pagina di benvenuto** dell'applicazione.
- Specifica impostazioni di **sicurezza** e gestione delle sessioni.

```
<web-app xmlns="http://java.sun.com/xml/ns/javaee" version="3.0">
  <servlet>
    <servlet-name>MyServlet</servlet-name>
    <servlet-class>com.example.MyServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>MyServlet</servlet-name>
    <url-pattern>/hello</url-pattern>
  </servlet-mapping>
  <welcome-file-list>
    <welcome-file>index.html</welcome-file>
  </welcome-file-list>
</web-app>
```

Nell'esempio sopra:

- Il Servlet `MyServlet` è registrato con il nome `MyServlet`.
- È associato all'URL `/hello`, quindi sarà chiamato quando si accede a `http://localhost:8080/myapp/hello`.
- `index.html` è la pagina predefinita se nessuna specifica risorsa è richiesta.

Avvio di una applicazione web

All'arrivo di una richiesta HTTP, il server TomCat deve selezionare l'applicazione web a cui è rivolta.

L'applicazione Web viene **selezionata da Catalina** in base alla corrispondenza del **longest prefix match dell'URI della richiesta con il context path di ogni contesto definito**.

È compito dell'engine quello di organizzare tutte le applicazioni web hostate da un particolare web server (*istanza di TomCat*).

Se un utente invia una richiesta per `http://localhost:8080/myapp`, Catalina:

1. Cerca l'**host** `localhost`.
2. All'interno dell'host, cerca un **Context** che mappi il percorso `/myapp`.
3. Se il **Context** esiste, la richiesta è diretta all'applicazione corrispondente. Altrimenti, restituisce un errore.

Nel file system, una directory (`appBase` - per impostazione predefinita, "webapps") viene utilizzata per conservare tutto il materiale per le applicazioni Web.

Ogni sottocartella corrisponde ad una web app.

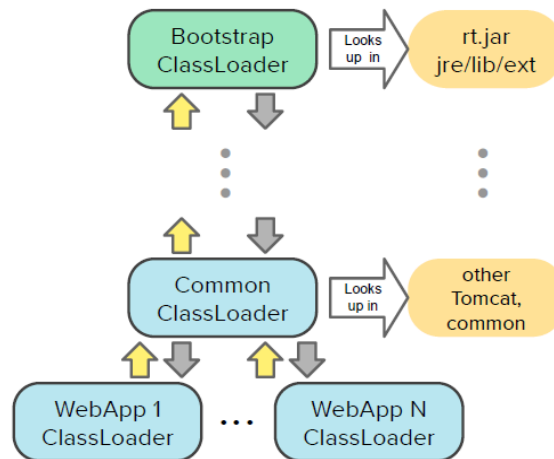
Ogni singola applicazione Web corrisponde a un nome di file di base. Regole per ottenere il nome di file di base, dato il percorso del contesto:

- Se il percorso del contesto è `"/` → `"ROOT"`.
- Se il percorso del contesto non è `"/` allora il nome di base è uguale al percorso del contesto con il `"/` iniziale rimosso e qualsiasi `"/` rimanente sostituito con `"#"`.

8.4.3 Class Loader

Il **ClassLoader** di Tomcat è una componente fondamentale per il caricamento dinamico delle classi Java in un'applicazione web.

Tomcat utilizza un **ClassLoader gerarchico** che consente di **caricare le classi in modo isolato per ogni contesto web** (*così che un'applicazione non possa interferire con un'altra*) e permette una gestione efficiente delle risorse di sistema.



Ogni classe caricata è identificata dal suo nome e anche dal classloader che l'ha caricata, quindi la stessa classe utilizzata in due web app potrebbe essere caricata due volte.

Bootstrap ClassLoader

Il **Bootstrap ClassLoader** è il primo nella gerarchia e si occupa di caricare le classi di base della JVM, come le classi di `java.*` e altre classi fondamentali.

Responsabile anche del caricamento delle librerie di sistema necessarie all'avvio di Tomcat, come quelle presenti in `lib/bootstrap.jar` e altre risorse di base.

Questo classloader non può essere configurato o esteso da un'applicazione web. È sempre presente in memoria fin dall'avvio di Tomcat.

Common ClassLoader

Il **Common ClassLoader** carica le librerie condivise tra tutte le applicazioni web ospitate da Tomcat.

Queste librerie si trovano di solito nella cartella `lib/` all'interno della directory di installazione di Tomcat (*non all'interno di una singola webapp*).

Le classi caricate da questo loader sono accessibili a tutte le webapp di Tomcat, quindi, se due o più applicazioni web dipendono dalla stessa libreria, queste possono condividere il caricamento delle stesse classi.

Le librerie di **Tomcat** stesse, come i connettori, i gestori di sessioni, i servlet, ecc., sono caricate tramite il **Common ClassLoader**.

`CATALINA_HOME/common/lib/` → serve per cercare i file `.jar`

`CATALINA_HOME/common/classes/` → serve per cercare i file `.class`

WebApp ClassLoader

Ogni webapp ha il suo **ClassLoader**, che è responsabile del caricamento delle classi specifiche per quella webapp. Ogni applicazione web ha una sua cartella `WEB-INF/lib/` (per le librerie) e `WEB-INF/classes/` (per le classi Java).

Quando un'applicazione web viene avviata, Tomcat crea un class loader specifico per quell'applicazione, che carica le sue classi e librerie, isolandole dagli altri contesti web. Questo significa che le classi di una webapp non possono accedere direttamente alle classi di un'altra webapp, riducendo così il rischio di conflitti tra applicazioni.

Il **WebApp ClassLoader** ha una gerarchia:

1. **Ricerca:** Prima cerca le classi nel proprio contesto (i file JAR in **WEB-INF/lib** e le classi in **WEB-INF/classes**).
2. **Delega:** Cerca eventuali classi nei classloader superiori.

C'è un'importante differenza rispetto al classloader tradizionale di Java.

Un WebApp Classloader prima cerca e **poi** nel caso delega (solo per le librerie Java core... ma tale comportamento può essere modificato).

Tomcat directory

```
/tomcat
├─ bin/                # Script per avviare e fermare Tomcat
├─ conf/               # File di configurazione di Tomcat
│   └─ server.xml      # Configurazione principale di Tomcat
│   └─ web.xml         # Configurazione globale delle applicazioni web
├─ lib/               # Librerie comuni di Tomcat
│   └─ catalina.jar    # Librerie core di Tomcat
│   └─ servlet-api.jar # API per le servlet
│   └─ altri jar di supporto
├─ logs/              # Log di Tomcat
│   └─ catalina.out    # Log principale
├─ webapps/           # Cartella contenente le webapp
│   └─ myapp/          # Prima applicazione web
│       └─ WEB-INF/
│           └─ web.xml  # File di configurazione specifico dell'applicazione
│           └─ lib/     # Librerie JAR specifiche dell'applicazione
│               └─ classes/ # Classi Java compilate per l'applicazione
│               └─ index.jsp # Altri file pubblici (JSP, HTML, immagini, ecc.)
│   └─ anotherapp/     # Seconda applicazione web
│       └─ WEB-INF/
│           └─ web.xml
│           └─ lib/
│               └─ classes/
│               └─ home.jsp
│   └─ ROOT/           # Applicazione predefinita (se presente)
│       └─ WEB-INF/
│           └─ web.xml
│           └─ lib/
│               └─ classes/
│               └─ index.jsp
└─ work/              # Cartella di lavoro temporanea di Tomcat
    └─ catalina/
```

8.5 Servlet

Un **Servlet** è una **classe Java** che gestisce le richieste HTTP e produce risposte dinamiche all'interno di una applicazione web. È una delle componenti fondamentali di Jakarta per lo sviluppo di applicazioni web.

I servlet sono parte integrante delle specifiche **Java EE (Enterprise Edition)** e sono gestiti dal **container Servlet** (ad esempio, *Tomcat* è un famoso servlet container).

Funzionamento di un servlet

I Servlet sono eseguiti all'interno di un **container servlet** (come **Tomcat**), che gestisce il ciclo di vita e l'esecuzione delle servlet.

1. Caricamento e istanziazione:

Quando viene ricevuta una richiesta HTTP per una determinata risorsa (un URL mappato a un servlet), il container servlet (TomCat) crea un'istanza del servlet (a meno che il servlet non sia configurato come "scoped" per avere più istanze).

2. Chiamata al metodo `service()`:

Una volta che il servlet è stato caricato, il container servlet invoca il metodo `service()` del servlet per gestire la richiesta HTTP.

`service(HttpServletRequest request, HttpServletResponse response)`

Il metodo `service()` riceve due oggetti come argomenti:

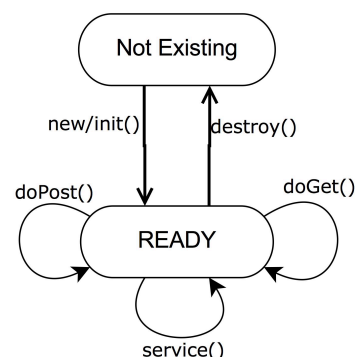
- **HttpServletRequest**: contiene le informazioni sulla richiesta HTTP, come i parametri, le intestazioni, i cookie e i metodi (*GET*, *POST*, ecc.).
- **HttpServletResponse**: fornisce il modo per inviare una risposta al client (ad esempio, una pagina HTML, un file, un messaggio di errore, ecc.).

3. Elaborazione della richiesta:

Dall'oggetto **HttpServletRequest** il servlet estrae i dati dalla richiesta (come parametri inviati nel corpo della richiesta o nell'URL) e poi esegue una logica di elaborazione scritta dallo sviluppatore dell'applicazione (ad esempio, accesso a un database, calcoli, ecc.).

4. Generazione della risposta:

Una volta completata l'elaborazione, il servlet invia una risposta al client tramite l'oggetto **HttpServletResponse**.



Il servlet può anche configurare il codice di stato HTTP della risposta (*ad esempio, 200 OK, 404 Not Found, ecc.*).

5. Chiusura:

Quando il servlet non è più necessario o quando l'applicazione è terminata, il container invoca il metodo `destroy()` per consentire al servlet di liberare le risorse, come connessioni al database o thread in uso.

Esecuzione di un servlet

In genere si applica il modello **thread-per-richiesta** e quindi ci possono essere problemi di sincronizzazione.

Mappatura servlet-URL

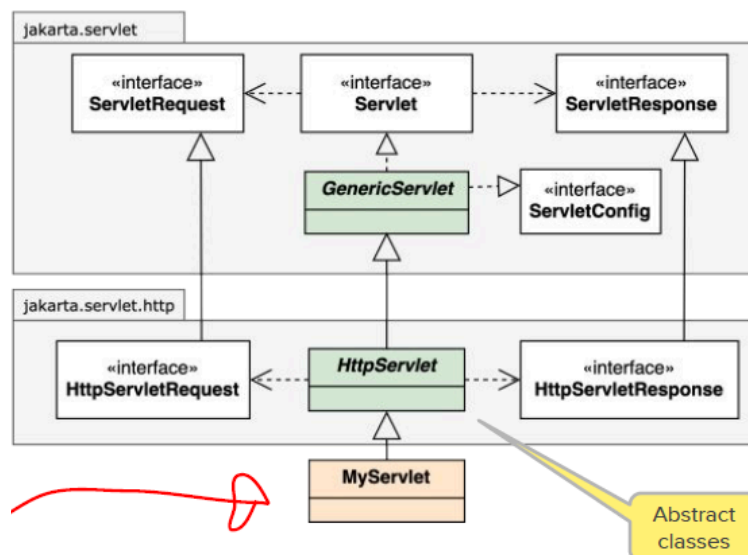
Il container è responsabile della gestione del ciclo di vita dei servlet e della mappatura degli URL ai servlet corrispondenti.

Questa mappatura è definita nel file `WEB-INF/web.xml` e può essere fatta in maniera standard o tramite regole modificate da noi, specificabili direttamente.

Interfaccia di Servlet

Concretamente, un servlet è una classe che implementa l'interfaccia `jakarta.servlet.Servlet`.

In figura la freccia tratteggiata corrisponde ad “*utilizza*”.



In pratica: i nostri servlet personalizzati devono estendere `GenericServlet` o, solitamente, `HttpServlet`.

Quante istanze di una classe `Servlet` (in una singola webapp)? Almeno una.

Ha senso tenere 10 oggetti servlet per una specifica classe?

Se abbiamo un modello thread-per-request allora non c'è nessuna necessità di avere diversi oggetti servlet.

Richieste e risposte in un Servlet

Per facilitare la gestione delle richieste e delle risposte HTTP, queste sono rappresentate come oggetti `HttpServletRequest` e `HttpServletResponse` che il **container Servlet** può facilmente elaborare durante la pipeline di gestione delle richieste.

1. Creazione degli Oggetti Richiesta e Risposta

Il container servlet (TomCat) riceve una richiesta HTTP e costruisce i due oggetti: uno rappresenta la **richiesta** e l'altro rappresenta la **risposta**.

- `jakarta.servlet.http.HttpServletRequest`
- `jakarta.servlet.http.HttpServletResponse`

2. Accesso ai Parametri di Richiesta

Indipendentemente dal metodo utilizzato per la richiesta (GET o POST), i parametri all'interno dell'oggetto di richiesta sono accessibili tramite l'interfaccia `ServletRequest`, che offre i seguenti metodi:

- `getParameterNames()`: fornisce l'elenco dei nomi dei parametri.
- `getParameter(name)`: restituisce il valore di un parametro specifico, identificato dal nome.
- `getParameterValues()`: restituisce un array con tutti i valori di un parametro, utile se il parametro ha più valori (*come in un campo di tipo checkbox*).

Le informazioni nelle intestazioni HTTP possono essere recuperate nello stesso modo, utilizzando metodi forniti dall'oggetto richiesta `HttpServletRequest`.

3. Creazione del Corpo della Risposta

Per inserire dati nel corpo del messaggio di risposta HTTP, `HttpServletResponse` fornisce due metodi principali:

- `getWriter()`: restituisce un oggetto `PrintWriter` per inviare dati di tipo testo (come HTML, JSON o XML).
- `getOutputStream()`: restituisce un oggetto `ServletOutputStream` per inviare dati di tipo binario (*come file o immagini*).

Entrambi i flussi di output (Writer e OutputStream) devono essere chiusi dopo l'uso per liberare risorse.

4. Impostazione del Tipo di Contenuto e Intestazioni

Per impostare il tipo di contenuto della risposta, è possibile utilizzare `setContentType(<MIME_type>)` su `HttpServletResponse`, ad esempio per definire il tipo MIME (`text/html`, `application/json`, ecc.).

5. Configurazione dei Servlet

La configurazione dei servlet all'interno dell'applicazione web può essere effettuata in due modi:

- **Tramite il file di configurazione `web.xml`**, specificato nel descrittore di distribuzione:
 - `servlet-name` → `servlet-class`
 - `servlet-name` → `url-pattern`
- **Direttamente nel codice**, utilizzando annotazioni posizionate sopra la definizione della classe del servlet.

```
@WebServlet(  
    name = "MyAnnotatedServlet",  
    urlPatterns = {"/foo", "/bar", "/pippo*"}  
)  
public class MyServlet extends HttpServlet {  
    // servlet code  
}
```

Alcune annotazioni sono scartate immediatamente se alcuni metadati non sono presenti nel file `.class`.

In Tomcat, la gestione ordinaria dei contenuti statici (*ad esempio, pagine html, immagini, ecc.*) viene eseguita utilizzando un servlet "di sistema" dedicato (*il sistema si comporta sempre allo stesso modo*):

DefaultServlet, che viene chiamato per qualsiasi URL REQ che fa riferimento a contenuti statici. Classe di implementazione:

`org.apache.catalina.servlets.DefaultServlet`, che estende la classe astratta `HttpServlet`.

Modi specifici per accedere ai contenuti dipendono dalle sue impostazioni, ad esempio, è possibile specificare se l'elenco delle directory è consentito o meno.

Una serie di query dallo stesso client può essere semanticamente correlata, ovvero, può corrispondere a diverse interazioni nel tempo appartenenti a una conversazione specifica.

L'insieme di richieste provenienti da un client specifico verso lo stesso server sono chiamate sessioni.

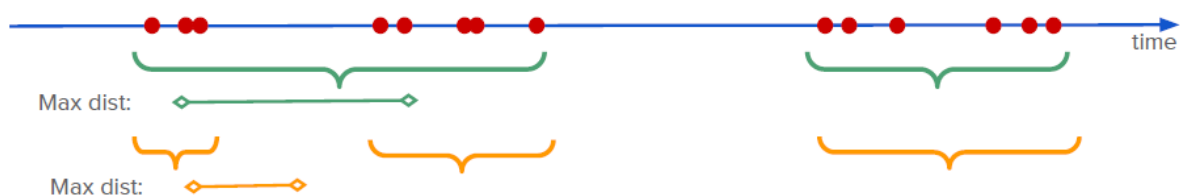
8.6 Tracciamento delle sessioni

Poiché HTTP è un protocollo senza stato (stateless), ogni richiesta che arriva al server viene trattata come indipendente dalle altre.

Le sessioni permettono di "collegare" tra loro più richieste fatte dallo stesso client, mantenendo informazioni come lo stato dell'utente, i dati inseriti in un modulo, il contenuto di un carrello acquisti, e altre preferenze o informazioni legate a una specifica interazione con il sito.

Anche per non far ripetere il login all'utente.

Le sessioni possono essere definite impostando una distanza temporale massima tra richieste adiacenti; valori diversi determinano diversi "cluster".



Nella figura sopra notiamo che con la **distanza massima verde** abbiamo due gruppi, mentre con **distanza massima arancione** abbiamo 3 gruppi. Questa distanza massima temporale deve essere impostata dal server web.

In base al valore specifico del tempo di idle abbiamo gruppi differenti, in applicazioni web differenti possiamo utilizzare diversi idle time. Questo è uno dei parametri che solitamente impostiamo per le nostre applicazioni web.

Le sessioni possono essere specificate impostando un valore di timeout, da reimpostare quando arriva una nuova richiesta nella stessa sessione.

Il server web deve essere in grado di identificare le sessioni (*assegnando "SessionID"*), e di determinare se una richiesta appartiene a una determinata sessione.

Come passare il SessionID al client?

Le sessioni sono rappresentate tramite oggetti specifici:

```
interface HttpSession
```

Che devono essere condivisi tra tutti i servlet a cui accede lo stesso client.

A livello di programmazione, gli oggetti sessione possono essere ottenuti dalle richieste:

```
HttpSession mySess = req.getSession(boolean create);
```

Se la richiesta non appartiene a nessuna sessione, il metodo restituisce `null`.

8.6.1 Tecniche di tracciamento delle sessioni

Il tracciamento della sessione è motivato dalla necessità di associare informazioni sullo stato alle sessioni.

Come associare una richiesta alla sessione relativa?

Questa informazione deve essere presente nelle richieste del client.

Cookie di sessione

Questa è la tecnica standard in cui viene inserito un identificatore randomico nel cookie inviato dal server verso il client. Il client nella prossima richiesta invierà tale identificatore nel cookie all'interno della sua richiesta.

Un cookie ha un tempo di espirazione che dipende dall'idle time di tale applicazione web (*ricordarsi che il web server implementa il middleware richiesto*).

Il problema di questa soluzione è che il client può rifiutare la gestione dei cookie e quindi non possiamo contare solo su questa soluzione.

Il nome del cookie usato da TomCat è `jsessionid`.

Campo HTML nascosto

Un campo nascosto è un tag HTML con attributo `hidden = true`.

Ovvero un campo input di un form usato per inviare alcune informazioni al server al momento della conferma del form. Il browser non lo mostra e l'utente non può vederlo o modificarne il valore.

L'idea è che quando il server invia il contenuto HTML al client, inserisce in un campo nascosto l'identificatore della sessione. Quando l'utente clicca un bottone per inviare informazioni al server, questo identificatore viene inviato indietro al server.

Riscrittura URL

In questo caso in una pagina HTML abbiamo un campo con attributo `href` per creare un link.

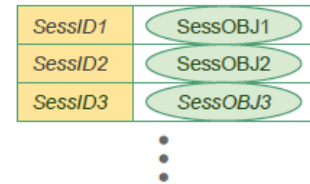
All'interno di un link per passare da una pagina ad un'altra inseriamo un parametro il cui valore è l'identificatore della sessione.

8.6.2 Gestione degli oggetti sessione

Tenere traccia delle sessioni è necessario perché si ha la necessità di associare informazioni di stato alle sessioni.

Un oggetto `Session` specifico deve essere utilizzato per ogni sessione in corso.

Il web server contrassegna ogni nuova sessione con un "SessionID" univoco; gli oggetti sessione possono essere mantenuti in un array associativo, con SessionID come chiavi.



Il recupero di questi oggetti è reso facile da una **tabella di hashing con chiave il SessionID** che rende l'accesso molto efficiente.

Qualsiasi messaggio di richiesta che appartiene a una determinata sessione, deve essere associato al SessionID relativo: questo consente al servlet container di recuperare l'oggetto sessione corretto da qualsiasi servlet.

In Tomcat, ogni app distinta (*contesto*) ha un oggetto "*manager*" incaricato di gestire gli oggetti sessione.

Metodi get e set

Le informazioni sullo stato negli oggetti `Session` vengono mantenute come un set di attributi, ciascuno identificato da una chiave, a cui si accede tramite metodi:

```
public void setAttribute(String name, Object obj);
```

```
public Object getAttribute(String name);
```

`name` è la chiave che verrà usata per accedere alla tabella di hash (*// SessionID*).

Invalidare una sessione

Le sessioni possono essere invalidate ad esempio per effettuare il logout:

- `mySess.invalidate()` → La sessione viene invalidata immediatamente.
- `mySess.setMaxInactiveInterval(int interval)` → La sessione viene invalidata dopo un intervallo massimo tra richieste successive.

Il valore di default di `interval` è definito in `web.xml` tramite il tag `<session-config>` con `<session-timeout>`.

Il file `web.xml` regola l'esecuzione di applicazioni **web Java**. Si tratta del deployment descriptor utilizzato nei servlet container (*come Apache Tomcat*) per configurare parametri di avvio e comportamento delle applicazioni web basate su Java EE.

8.7 Servlet e richieste HTTP

In questo esempio abbiamo una servlet `HelloWorld` che estende una servlet HTTP ovvero `HttpServlet`.

Il metodo `doGet()` serve ogni volta che la richiesta raggiunge il server web e verrà eseguita nel container del servlet.

Per riempire il corpo della risposta può essere utilizzato `out` (oggetto di tipo `PrintWriter`).

```
import ...

public class HelloWorld extends HttpServlet {
    private String msg;

    public void init() throws ServletException {
        msg = "Hello World";
    }

    public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("<h1>" + message + "</h1>");
    }
}
```

8.7.1 Filtri Servlet

Un **Filtro Servlet** è un componente in Java che consente di **intercettare e modificare** le richieste e le risposte tra un client (di solito un browser) e il server web (come Tomcat).

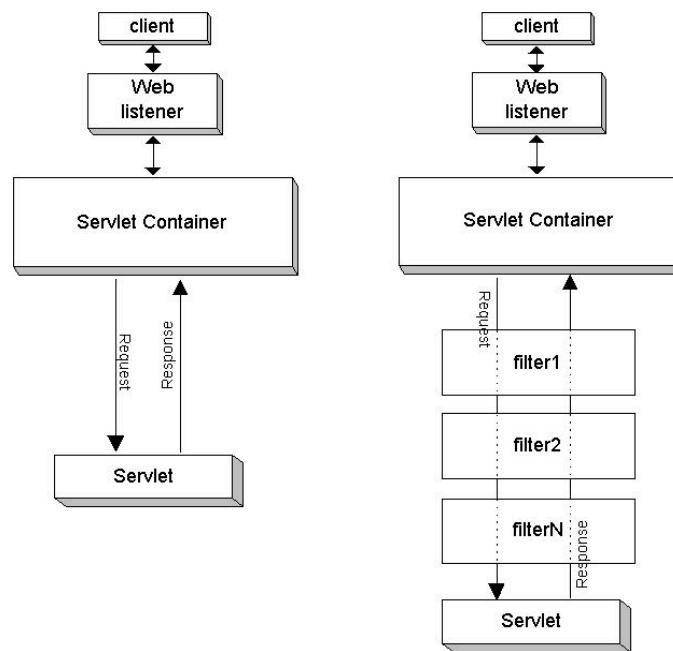
I filtri servlet possono essere utilizzati per eseguire operazioni di **pre-elaborazione su una richiesta o post-elaborazione su una risposta**, prima che venga gestita dal servlet o inviata al client.

I filtri servlet possono anche essere messi in cascata, ovvero possono essere inseriti filtri in sequenza

Per fare queste cose utilizzeremo delle API in `jakarta.servlet`, le interfacce `Filter`, `FilterChain` e `FilterConfig`.

I filtri servlet possono essere configurati per applicarsi a **specifici URL, servlet**, o anche a tutte le richieste, a seconda della configurazione nel file `web.xml` o tramite annotazioni.

L'applicazione o meno di un filtro a una richiesta dipende dalla corrispondenza tra l'URL della richiesta e il modello specificato per il filtro.



A sinistra vediamo lo scenario senza filtri. Mentre a destra vediamo lo scenario con filtri: i filtri si posizionano tra il servlet container e il servlet e si possono concatenare. Lo sviluppatore deve scrivere il codice del filtro e specificare la posizione nella catena di filtri.

Come implementare un filtro

Un filtro servlet personalizzato deve estendere la classe `Filter` e deve implementare i seguenti metodi:

```

init();

destroy();

doFilter(ServletRequest req, ServletResponse resp, FilterChain chain);

```

Il corpo di `doFilter()` implementa l'azione da intraprendere nel filtro.

Per passare la richiesta al componente successivo dobbiamo utilizzare:

```

chain.doFilter(req, resp);

```

Per mappare i filtri è possibile applicare lo stesso metodo dei servlet (*in web.xml*) o utilizzare annotazioni, ad esempio:

```

@WebFilter("/protected/*")
public class MyFilter implements Filter {
    @Override
    public void doFilter(
        ServletRequest request, ServletResponse response,
        FilterChain chain) throws IOException, ServletException {
        // Pre-processing della richiesta
        chain.doFilter(request, response); // Passa al prossimo filtro o al servlet
        // Post-processing della risposta
    }
}

```

8.8 Collaborazione tra Servlet

In un'applicazione web, è possibile utilizzare più servlet che collaborano per generare il contenuto della risposta. Questa collaborazione avviene tramite l'interfaccia `RequestDispatcher`.

Il **RequestDispatcher** è un'interfaccia in Java, parte della Servlet API, che consente di **inoltrare** o **reindirizzare** una richiesta HTTP da un servlet a un altro servlet.

Questa funzionalità è utile per gestire la logica di invio delle richieste a diversi componenti dell'applicazione senza che l'utente debba essere consapevole del percorso effettivo.

Un oggetto `RequestDispatcher` può essere ottenuto dal metodo `request.getRequestDispatcher(<targetResource>)` dell'oggetto `ServletRequest`.

L'argomento `<targetResource>` specifica il servlet o la risorsa con cui vogliamo collaborare.

Esistono due modalità principali di collaborazione:

1. **Inclusione del contenuto:** un servlet può includere il contenuto di un'altra risorsa nella propria risposta.
2. **Delegazione della gestione della richiesta:** un servlet inizia a processare la richiesta, costruisce una parte della risposta e poi la passa a un'altra risorsa per completare il lavoro.

Inclusione - Include

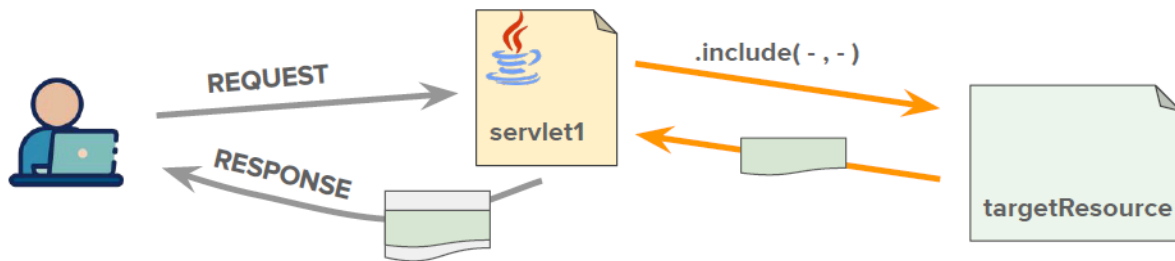
Il risultato prodotto da un'altra risorsa (*servlet, jsp, html*) può essere incluso nella risposta che viene assemblata dal servlet corrente:

```
rd = request.getRequestDispatcher(<targetResource>);  
  
rd.include(request, response);
```

`rd` indica il request dispatcher.

Il `targetResource` fornisce un pezzo di informazione da inviare a `servlet1`, il quale lo include nella risposta da inviare al client.

Quindi la risposta è generata e inviata dal Servlet che ha chiamato la `include`.



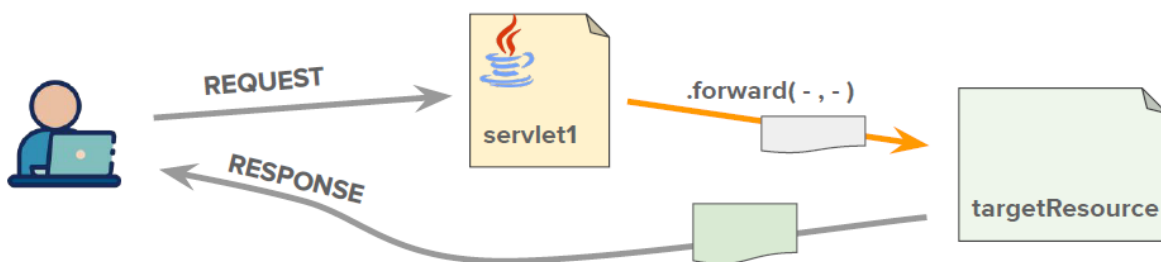
Forward

Invia la richiesta e la risposta al componente destinato (un altro servlet, JSP, o risorsa).

La risposta finale è quella generata dal componente di destinazione, quindi il servlet che chiama `forward()` non invierà la propria risposta al client.

```
rd = request.getRequestDispatcher(<targetResource>);
rd.forward(request, response);
```

In questo caso viene utilizzato `forward` per inviare la richiesta al `targetResource` che restituisce la risposta direttamente al client senza passare da `servlet1`.



8.8.1 Condivisione di informazioni tra Servlet

Poiché i servlet possono collaborare, potrebbero dover **condividere informazioni**.

Gli **scope object** sono degli ambiti (scopes) in cui è possibile memorizzare e condividere informazioni tra diversi componenti di un'applicazione web, come servlet, JSP o altri oggetti.

Questi scope determinano la durata e la visibilità degli oggetti che vengono memorizzati in essi. I principali scope utilizzati dai servlet per condividere informazioni sono:

Gli scope object forniscono metodi come `setAttribute` e `getAttribute` per memorizzare e recuperare dati.

```
// Servlet 1

HttpSession session = request.getSession();

session.setAttribute("username", "john_doe");
```

```
// Servlet 2
```

```
HttpSession session = request.getSession();
```

```
String username = (String) session.getAttribute("username");
```

8.8.2 Organizzare le applicazioni con i Servlet

I servlet sono una tecnologia di base che non impone alcun vincolo su come dovrebbe essere strutturata un'intera applicazione.

Una cosa da capire è che non sempre avere totale libertà è un bene. Altrimenti programmare in assembly sarebbe magnifico e lo farebbero tutti.

Servono delle regole per evitare di perderci, serve una struttura da seguire.

Magic Servlet

Si tratta di una cattiva pratica (*antipattern*) e succede quando problemi su diversi aspetti dell'applicazione vengono trattati all'interno dello stesso metodo.

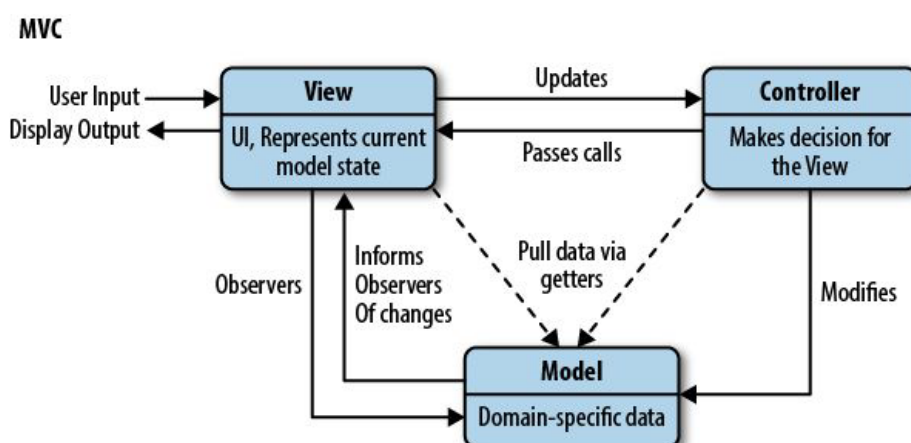
Spesso dal punto di vista delle prestazioni del software, alcune soluzioni non sono buone, identificare antipattern è cruciale perché se li identifichiamo è una buona idea riorganizzare in modo da evitare i problemi.

Occorre quindi definire ruoli specifici, in modo che un'attività/preoccupazione specifica faccia riferimento a un componente software distinto.

MVC - Model View Controller

Un buon modo per organizzare i servlet di una applicazione è quello di applicare MVC.

- **Model:** Gestisce i dati.
- **View:** Gestisce l'interazione con l'utente.
- **Controller:** Coordina le interazioni.



L'utente interagisce con la view, tramite questa è possibile aggiornare il controller che prende decisioni modificando il model.

La view può osservare le informazioni correnti disponibili nel modello.

8.9 JSP - Jakarta Server Pages

Le servlet, pur essendo uno dei primi approcci allo sviluppo di applicazioni web con Java, **risultano difficili da mantenere** perché uniscono strettamente la logica di business (*scritta in Java*) e la presentazione (*tutta la parte relativa allo HTML e quindi alla GUI*).

Questo porta a un codice poco leggibile e difficile da modificare.

Template

Meccanismo per separare la logica dell'applicazione dalla presentazione, permettendo ai servlet di concentrarsi sulla gestione della logica e ai file di template (*come JSP o specifici framework di templating*) di occuparsi della visualizzazione.

Ecco un esempio di JSP, ovvero `welcome.jsp`:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Welcome Page</title>
  </head>
  <body>
    <h1>Benvenuto, ${username}!</h1>
  </body>
</html>
```

Come si può vedere, il template è **scritto in HTML**.

Dentro un file HTML possiamo inserire il codice utilizzando **testo speciale**.

E questo è il servlet di riferimento:

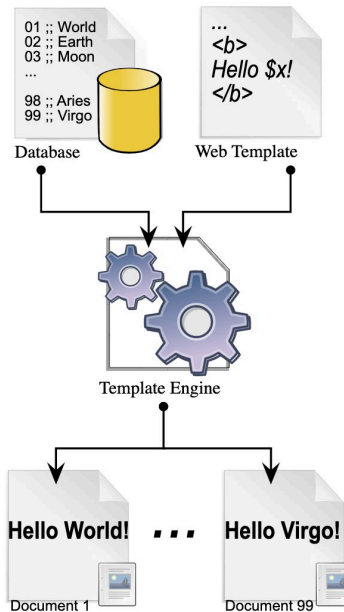
```
@WebServlet("/welcome")
public class WelcomeServlet extends HttpServlet {
    protected void doGet(
        HttpServletRequest request,
        HttpServletResponse response
    ) throws ServletException, IOException {
        // Imposta un attributo da passare alla JSP
        request.setAttribute("username", "John Doe");
        // Inoltra la richiesta alla pagina JSP
        RequestDispatcher dispatcher =
            request.getRequestDispatcher("/WEB-INF/views/welcome.jsp");
        // La risposta è generata tramite il file JSP.
        dispatcher.forward(request, response);
    }
}
```

Questo template può essere processato da un engine di processazione dedicato. Molti sistemi di template sono sviluppati per generare contenuti customizzati

Funzionamento di un template engine

La logica viene inserita usando tag speciali **`${ }`**, poi il motore di template processa questi file e genera contenuti dinamici.

Jakarta Server Pages (JSP): è il motore di template standard di Java EE



I sistemi di template web sono progettati per generare contenuti personalizzati e pagine web dinamiche.

Il **motore di template** è responsabile di:

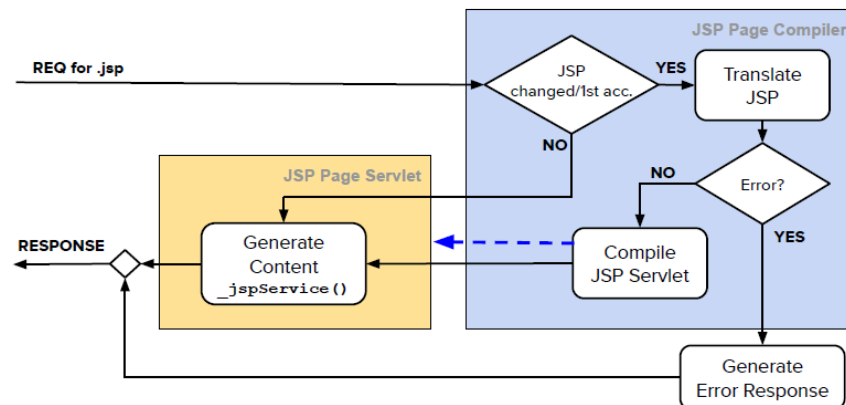
1. Analizzare gli **elementi segnaposto `${ }`** (*placeholder*) nel template, che rappresentano dati o logica da processare.
2. Creare il documento finale basandosi su:
 - Una collezione di dati (*come un dataset*).
 - Eventuali logiche definite nel template stesso.

Questo schema di elaborazione può essere applicato a diversi livelli, come nella generazione dinamica di contenuti complessi o nella personalizzazione di interfacce utente.

8.9.1 Funzionamento di JSP

Tecnologia Java che permette di creare pagine web dinamiche miscelando codice Java con HTML.

L'immagine sotto illustra il ciclo di vita di una pagina JSP all'interno di un server web. Mostra come una richiesta HTTP per una pagina `welcome.jsp` venga gestita, tradotta, compilata e servita come risposta dinamica.



1. Richiesta della pagina JSP

- Quando un client invia una richiesta HTTP per una pagina `.jsp`, questa viene inoltrata al **JSP Page Compiler**.

2. Verifica dello stato della pagina JSP

- Il compilatore controlla se la pagina è stata modificata rispetto all'ultima compilazione o se si tratta del primo accesso alla pagina:
 - Sì**: Se è stata modificata o non è mai stata compilata prima, si avvia il processo di traduzione (*abbastanza lento*).
 - No**: Se la pagina è già stata compilata e non ci sono cambiamenti, viene utilizzata la versione precedentemente compilata.

3. Traduzione della JSP

- Il file JSP viene **convertito** in un servlet (JSP Page Servlet).

Questo passaggio converte il codice JSP (HTML con scriptlet Java) in codice Java puro.

4. Compilazione della servlet

- La servlet generata viene compilata in bytecode (*file .class*) eseguibile sulla JVM.

5. Esecuzione del codice JSP (Servlet)

- La servlet compilata viene eseguita dal container JSP.

Il metodo `_jspService()` viene invocato per generare il contenuto dinamico in base ai dati e alla logica della pagina.

6. Generazione della risposta

- Il contenuto generato dalla servlet viene inviato al client come risposta HTTP, di solito in formato HTML.
- La risposta viene inoltrata al client attraverso il connettore HTTP (*Coyote*).

8.9.2 Struttura di un template HTML

Un template è essenzialmente un documento HTML a cui possono essere aggiunti elementi di scripting per renderlo dinamico.

Modi per integrare script nei template JSP:

- **Scriptlet:** inserimento di **codice Java** delimitato da `<% ... %>`.

Il codice racchiuso nei tag di scriptlet viene inserito nel metodo `_jspService()` durante la compilazione del file JSP.

Questo metodo si occupa di gestire la logica di business associata alla pagina.

- **Espressione:** Inserimento di una espressione Java all'interno di `<%= ... %>`.

L'espressione Java all'interno viene calcolata, viene ottenuta la stringa corrispondente al risultato e viene inserita nella posizione corretta nel template.

`<%= new Date() %>` → visualizza la data corrente.

In pratica tutto il codice HTML viene trasformato in codice Java per ottenere il risultato corretto.

```
<p>Listing of the first natural numbers:</p>
```

```
<% for (int i = 1; i <= 5; i++) { %>
```

```
    <p>This number is <%= i %>.</p>
```

```
<% } %>
```

```
<p>OK.</p>
```

Oggetti Impliciti in JSP

Quando si utilizza JSP, l'ambiente mette a disposizione **oggetti impliciti** che facilitano lo sviluppo:

- **request**: rappresenta la richiesta HTTP e consente di accedere ai parametri inviati dal client.
- **response**: gestisce la risposta HTTP inviata al client.
- **out**: un oggetto `PrintWriter` per scrivere direttamente nel corpo della risposta HTML.
- **session**: consente di accedere ai dati relativi alla sessione utente.
- **application**: offre accesso al contesto condiviso tra più pagine JSP (oggetto `ServletContext`).
- **page**: sinonimo dell'oggetto `this`, che rappresenta la pagina corrente.

Questi oggetti possono essere utilizzati direttamente nel codice JSP per accedere a dati o configurazioni utili.

8.9.3 Separazione delle Responsabilità nei JSP

Per migliorare la gestione del codice e seguire il principio della **separazione delle responsabilità**, nei JSP è possibile utilizzare strumenti aggiuntivi:

1. **JavaBeans**: componenti Java riutilizzabili per incapsulare la logica di business e i dati.
2. **JSTL (JavaServer Pages Standard Tag Library)**: una libreria di tag standard che consente di implementare strutture di controllo e manipolare dati senza usare scriptlet.
3. **Expression Language (EL)**: un linguaggio semplice per accedere ai dati delle applicazioni, come i valori memorizzati nei JavaBeans, rendendo il codice più leggibile.

Esempio di utilizzo di EL:

```
<p>Benvenuto, ${utente.nome}!</p>
```

8.10 Sviluppo di WebApp: Struttura dei Progetti

Le applicazioni web Java sono spesso sviluppate seguendo standard come quelli di **Maven**, un tool di gestione e costruzione dei progetti.



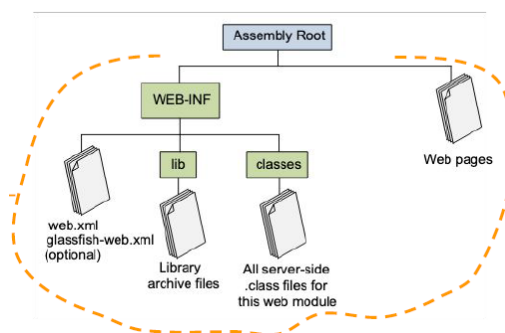
Maven impone una struttura predefinita per i file, che include:

- Una directory per il codice sorgente Java (chiamata spesso **java**).
- Una directory per le risorse statiche e i file di configurazione (**webapp**).

File WAR

Questa organizzazione semplifica il packaging e il deployment.

Le applicazioni vengono distribuite come file **.war** (Web Application Archive), che includono tutti i file necessari per l'esecuzione.



8.10.1 JSP e Architettura MVC

I JSP sono spesso utilizzati per implementare la **vista** nell'architettura **Model-View-Controller (MVC)**, separando la presentazione dalla logica di controllo.

Tuttavia, per applicazioni complesse, si adottano framework più avanzati che integrano tecnologie come Servlet e JSP per semplificarne l'uso.

I sistemi risultanti sono noti come **web framework**, e la maggioranza di essi sono progettati per prendere MVC come riferimento.

- **Java** → Spring Boot, Jakarta EE.
- **Python** → Django, Flask.

Se abbiamo JSP non dobbiamo sostituire le servlet, il programmatore può decidere come utilizzare queste tecnologie.

Dobbiamo aumentare il livello di astrazione, questo è sempre relativo alla perdita di alcuni gradi di libertà sulla manutenzione della nostra applicazione.

JSF - JakartaServer Faces

Lo standard originale proposto è il JSF (**JakartaServer Faces**).

Si tratta di un framework Java per lo sviluppo di applicazioni web basate su componenti. Fa parte della specifica **Jakarta EE** e fornisce un'architettura MVC per creare interfacce utente dinamiche in modo strutturato.

Capitolo 9 - Applicazioni enterprise

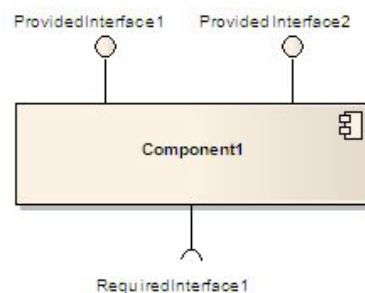
Vediamo le tecnologie usate per realizzare applicazioni di grandi dimensioni.

L'uso di oggetti Java semplici (*POJO*) in applicazioni enterprise può comportare problemi come:

- Mancanza di trasparenza nel deployment (*non vogliamo che il codice non dipenda dal fatto che l'applicazione viene progettata in una piattaforma specifica*).
- Dipendenze implicite difficili da gestire.

Soluzione: adottare un **approccio basato su componenti**, in cui i diversi moduli comunicano attraverso **interfacce definite** (*fornite e richieste*). Questo facilita la gestione e l'integrazione dei componenti.

In figura vediamo la rappresentazione di un componente software, in questo caso ha interfacce differenti che dipendono da che cosa deve ricevere dall'esterno (*required interface*) e che cosa deve esporre verso l'esterno (*provider interface*).



9.1 Application Servers

Sono **soluzioni middleware** progettate per semplificare lo sviluppo di applicazioni enterprise.

Il suo ruolo principale è **gestire, eseguire e distribuire applicazioni server-side**, fornendo servizi essenziali come:

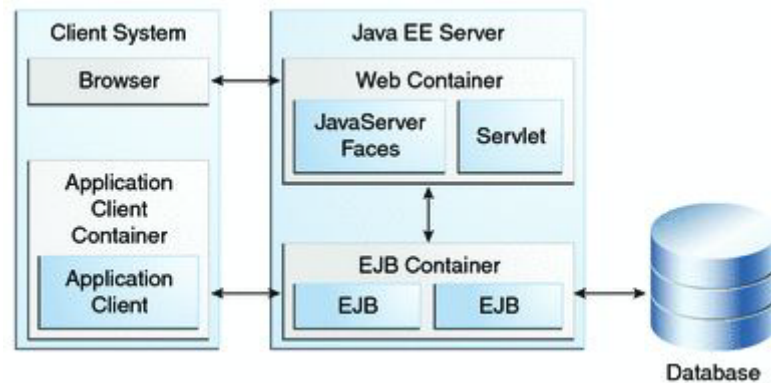
- Gestione delle connessioni e delle richieste HTTP
- Elaborazione della logica di business
- Gestione delle sessioni utente
- Accesso ai database tramite JDBC o ORM
- Sicurezza e autenticazione
- Scalabilità e gestione del carico

Permettono agli sviluppatori di concentrarsi sulla logica di business, delegando al server tutto il resto.

Java EE / Jakarta EE → **Apache Tomcat**, WildFly (ex JBoss), GlassFish, Payara

9.2 JEE Server

La seconda immagine rappresenta l'architettura generale di un'applicazione enterprise basata su Java EE, evidenziando i principali componenti e come interagiscono.



Client System

Browser: Interfaccia utente che invia richieste HTTP al server e riceve risposte HTML.

Application Client Container: In alcuni casi, l'applicazione client può essere un'applicazione standalone Java che interagisce con il server tramite protocolli come RMI o servizi web.

Java EE Server

Il cuore del sistema che gestisce le richieste dei client ed è diviso in due diversi container.

Web Container

Contiene tutte le componenti dedicate alle applicazioni web come i Servlet.

Include framework come **JavaServer Faces (JSF)**, che facilita lo sviluppo di interfacce utente MVC.

EJB Container

Gestisce gli **Enterprise Java Beans (EJB)**, che implementano la logica di business e forniscono servizi come transazioni, sicurezza e persistenza.

Comunica con il database per recuperare o salvare i dati.

Database

È il sistema di memorizzazione dei dati utilizzato dall'applicazione.

Le **EJB interagiscono direttamente con il database** per eseguire **operazioni CRUD** (Create, Read, Update, Delete).

Java Bean

Classe Java che segue una serie di convenzioni per rappresentare **oggetti riutilizzabili** con **stato** (dati) e **comportamento** (metodi).

A cosa serve?

- **Conservare dati** in applicazioni web (es. moduli utente, prodotti).
- **Passare informazioni tra componenti Java** (Servlet, JSP, JSF).
- **Facilitare l'integrazione con framework Java** (Spring, JSF, Hibernate).

9.3 Spring Framework

Framework modulare che **semplifica lo sviluppo** di applicazioni Java Enterprise.

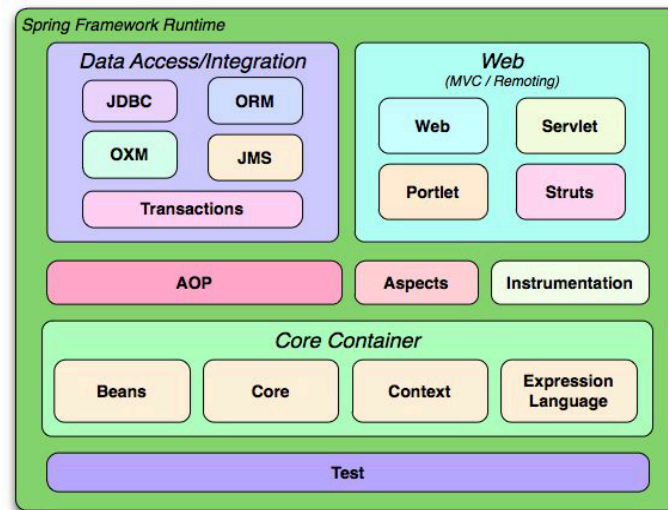
Offre un insieme di strumenti per gestire **dipendenze, sicurezza, transazioni e accesso ai database**, facilitando la creazione di applicazioni scalabili e modulari.

```
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        // Avvia il server Web Tomcat.
        SpringApplication.run(DemoApplication.class, args);
    }
}

@RestController
@RequestMapping("/hello") // URL
class HelloController {
    @GetMapping // Metodo GET
    public String sayHello() {
        return "Ciao da Spring Boot!";
    }
}
```

L'esempio sopra: avvia un server web (Tomcat) e gestisce una richiesta **GET** allo url **/hello**, restituendo **"Ciao da Spring Boot!"**.

9.3.1 Architettura di Spring



Principi di Spring

- **Inversion of Control (IoC):** il framework gestisce il ciclo di vita degli oggetti e la loro configurazione, liberando gli sviluppatori da dettagli complessi.
- **Dependency Injection (DI):** un oggetto riceve le sue dipendenze da un container esterno invece di crearle autonomamente, viene delegata la configurazione ed il lifecycle degli application Java object a dei container.

Core Container

Il **Core Container** di Spring è la parte fondamentale del framework Spring, che si occupa di gestire la creazione e l'organizzazione degli **oggetti (bean)** all'interno dell'applicazione. In pratica, è il "cuore" di Spring che fornisce tutti i servizi necessari per il funzionamento di un'applicazione Spring, come la **gestione delle dipendenze** e il **ciclo di vita degli oggetti**.

- **Gestisce i Bean:** Crea e configura gli oggetti (chiamati "bean") definiti nel contesto dell'applicazione.
- **Iniezione delle Dipendenze:** Fornisce automaticamente agli oggetti le dipendenze di cui hanno bisogno (come un database o un altro servizio).
- **Controllo del Ciclo di Vita:** Gestisce la creazione, l'inizializzazione e la distruzione dei bean.
- **Configura l'Applicazione:** Può essere configurato con file XML, annotazioni o codice Java.

Tutta la complessità si sposta dalla scrittura del codice all'impostazione della **configurazione del sistema**.

Il lavoro del programmatore viene semplificato, poiché non deve più gestire manualmente la creazione e la configurazione degli oggetti. Tuttavia, la **gestione della configurazione** stessa può diventare complessa, specialmente in applicazioni di grandi dimensioni.

Spring vs Jakarta

Spring non è un **competitor diretto di JEE**, ma offre un modo **più flessibile** di organizzare un'applicazione **enterprise**.

Sebbene entrambi i framework possano essere usati per costruire applicazioni di grandi dimensioni, Spring si distingue per la sua **flessibilità** e per l'adozione di **design pattern** moderni che favoriscono la **modularità** e la **manutenibilità**.

In altre parole, mentre JEE è uno standard ufficiale che definisce un insieme di specifiche per lo sviluppo enterprise, Spring fornisce **una soluzione più dinamica e modulare**, adattabile alle necessità specifiche di ciascun progetto.

Spring e JMS

Spring fornisce anche il supporto per **JMS (Jakarta Messaging Service)**, che consente di implementare la **comunicazione asincrona** tra componenti dell'applicazione.

JMS è una tecnologia che consente di inviare e ricevere messaggi tra **produttori e consumatori di messaggi**, utilizzando un **broker di messaggi** (ad esempio, ActiveMQ o RabbitMQ). Questo tipo di comunicazione è utile per creare applicazioni **scalabili e decoupled**, dove i componenti possono comunicare in modo **indiretto** attraverso il broker, invece di comunicare direttamente tra loro.

In un sistema **asincrono**, la comunicazione non avviene in tempo reale, ma i messaggi vengono inviati e poi letti dal destinatario quando è pronto, senza bloccare altre operazioni.

Le transazioni possono creare accessi multipli a database multipli.

Comunicazione indiretta indica che ogni componente è in grado di comunicare con un altro componente attraverso un broker intermedio, in questo paradigma abbiamo un modo completamente asincrono di comunicare.

Ricordiamoci che una factory è un modo di accedere ad un oggetto senza chiamare esplicitamente una new. invece di utilizzare la new chiediamo all'oggetto factory.

Spring 101

Spring è considerato una soluzione agile rispetto a Jakarta EE, adatta sia a piccole applicazioni che a sistemi enterprise complessi.

Le funzionalità Spring possono essere utilizzate per creare qualsiasi programma Java, e vengono messe in atto soluzioni specifiche per le applicazioni web (*tramite Web MVC*).

In JEE ci sono componenti necessari per soddisfare molti vincoli, questo rende lo sviluppo molto complesso, in Spring un approccio differente fu proposto, l'approccio è POJO based.

POJO (*Plain Old Java Object*): per lavorare con i container tipici di JEE tipicamente dobbiamo avere certi vincoli, se non presenti non c'è un motivo vero per dipendere dai container JEE.

Configurare un'applicazione è complicato, può essere fatto in modi differenti, un modo è quello di utilizzare le annotazioni.

Caratteristiche di Spring:

- **POJO-based**: supporta classi Java ordinarie senza vincoli particolari.
- **Modularità**: è possibile utilizzare solo i moduli necessari, ad esempio Spring Web MVC per applicazioni web.
- **Annotazioni**: permettono di definire ruoli e configurazioni in modo semplice ed elegante.

Spring 102

Gli oggetti gestiti da un Spring IoC Container (*secondo le informazioni di configurazione*) sono chiamati bean.

L'interfaccia `org.springframework.context.ApplicationContext` rappresenta il container IoC di Spring e ha il compito di istanziare, configurare e assemblare i bean.

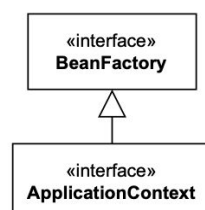
Opzioni per i metadati di configurazione:

- Classi componenti annotate.
- Classi di configurazione con metodi di factory.
- File XML esterni/Scripts Groovy.

Un Bean è un componente gestito automaticamente da un framework, in Spring abbiamo gli Spring Bean.

Spring utilizza il **Factory Pattern** per la creazione dei bean. Questo pattern permette di delegare l'istanza degli oggetti a metodi specifici o configurazioni centralizzate, facilitando la gestione e l'inizializzazione dei componenti.

`ApplicationContext` è un'interfaccia, però in Spring abbiamo classi implementative differenti (*ci sono diverse implementazioni dell'interfaccia `ApplicationContext` in Spring, ognuna progettata per scenari specifici*).



Spring Beans & Annotations

Un bean di Spring può essere ottenuto utilizzando due annotazioni principali:

- **@Bean**: Annotazione a livello di metodo per specificare che l'oggetto restituito da tale metodo deve essere gestito come un bean.
- **@Component**: Utilizzata per contrassegnare una classe come componente.

Spesso invece di utilizzare **@component**, possiamo utilizzare altre annotazioni con una semantica differente, ad esempio possiamo utilizzare:

- **@controller**: bean responsabile di gestire le richieste degli utenti.
- **@service**: bean che contiene la logica di business.
- **@Repository**: per le classi DAO o simili, con accesso a data repository.

9.3.2 Spring MVC

Spring MVC è utilizzato per sviluppare applicazioni web secondo il pattern **MVC**.

- **Servlet centrale**: **DispatcherServlet** fornisce un algoritmo comune per l'elaborazione delle richieste.
- **Delegati configurabili**: Il controller scelto crea il modello e specifica la vista; la vista selezionata renderizza la risposta.

Spring MVC è un modo classico per gestire le richieste ed anche un modo per effettuare il processing asincrono.

REQ processing in Spring MVC

Quando una richiesta in entrata (*REQ*) viene inviata a **DispatcherServlet**:

1. **HandlerMapping**: Determina quale controller specifico utilizzare per la richiesta.
2. **Controller (Handler)**: Crea il model e restituisce indicazioni sulla view da utilizzare.
3. **ViewResolver**: Determina la view effettiva utilizzando il nome restituito dal controller.
4. **View**: Si occupa del rendering del model passato.

Nota: Tali bean appartengono a un **WebApplicationContext** specifico.

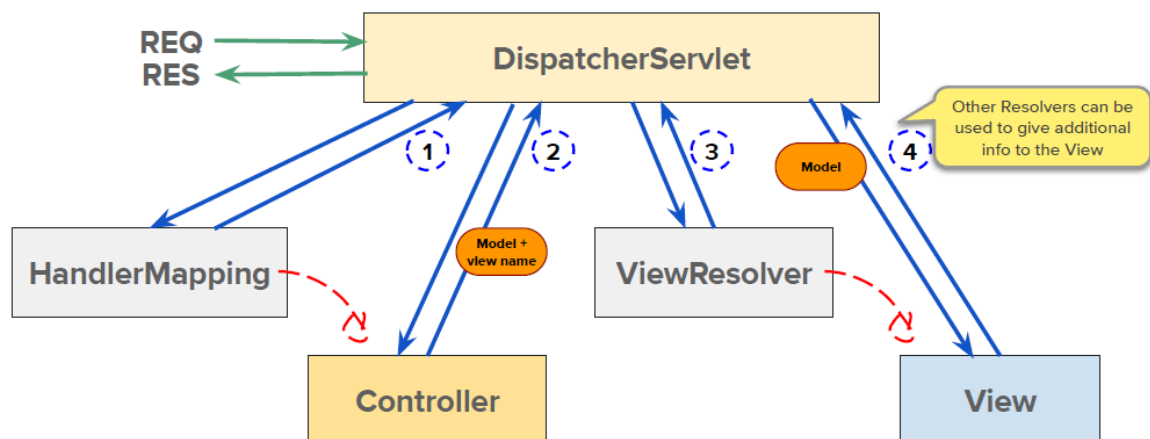
Possiamo avere **applicationContext** multipli, il **dispatcherServlet** deve decidere che controller scegliere.

In base a come il controller è codificato, potremmo avere bisogno del **HandleAdapter**.

Schema di REQ processing in Spring MVC

1. **DispatcherServlet riceve una richiesta (REQ):**
La richiesta HTTP è gestita dal **DispatcherServlet**, che funge da front controller.
2. **HandlerMapping determina il controller:**
Utilizzando **HandlerMapping**, Spring determina quale controller deve elaborare la richiesta specifica.
3. **Il Controller elabora la richiesta:**
Il controller esegue la logica aziendale, crea un modello e restituisce il nome della vista da visualizzare.
4. **ViewResolver seleziona la vista:**
Il **ViewResolver** prende il nome della vista restituito dal controller e determina quale vista effettiva deve essere utilizzata (*ad esempio, un template JSP o Thymeleaf*).
5. **La vista renderizza il modello:**
La vista finale renderizza il modello dati e restituisce una risposta (RES) all'utente.

L'immagine mostra il flusso di lavoro di **Spring MVC**, dove diversi **bean** vengono utilizzati per ciascuna fase della gestione di una richiesta. Ogni bean è responsabile di una parte specifica del processo, ed è rappresentato in un **livello differente**.



Spring Boot

Sviluppare applicazioni Spring è impegnativo a causa di:

- Necessità di configurazione estensiva (*tramite annotazioni/XML, ecc.*).
- Gestione complessa delle dipendenze (*molti moduli*).
- Curva di apprendimento ripida.

Soluzione: Il principio “**Convention over configuration**” → **Spring Boot**:

Sviluppando un'applicazione può capitare di sviluppare molte configurazioni, quindi si hanno molte scelte da intraprendere, l'idea è quella di ridurre il numero di scelte e prendere la scelta più ragionevole (*Convention over Configuration*).

- Configurazione semplificata.
- Server web embedded (*applicazioni stand-alone, senza problemi di deployment*).
- Template pronti all'uso per diversi tipi di applicazioni.

Spring Boot è una visione “*opinionated*” di Spring significa che **Spring Boot** segue un approccio basato su **opinioni predefinite** per semplificare lo sviluppo delle applicazioni.

In altre parole, Spring Boot prende decisioni di default su come configurare e organizzare un'applicazione Spring, questo rende lo sviluppo più veloce ed efficace, riducendo il bisogno di configurazione manuale e semplificando la complessità di creare applicazioni Spring.

Spring Boot 101

In Spring Boot, abbiamo una classe **entry-point** con il metodo **main()** annotata con **@SpringBootApplication**, che include automaticamente:

- **@Configuration**: Definisce la classe come sorgente di definizioni di bean.
- **@EnableAutoConfiguration**: Avvia automaticamente la configurazione dei bean in base al classpath. Ad esempio, se `spring-webmvc` è presente, viene configurata un'app web con `DispatcherServlet`.
- **@ComponentScan**: Permette a Spring di cercare altri componenti e configurazioni.

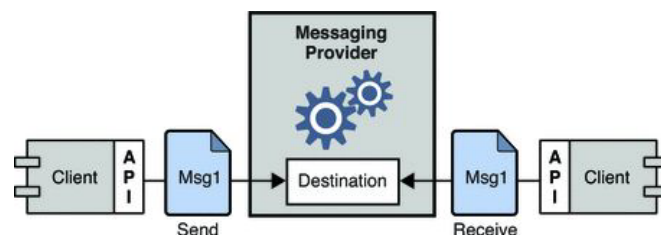
Il metodo `main()` utilizza tipicamente `SpringApplication.run()` per avviare l'applicazione.

9.4 JMS - Jakarta Message System

MOM - Message Oriented Middleware

Un **MOM** è un tipo di software che facilita la comunicazione tra applicazioni o componenti distribuiti tramite l'invio e la ricezione di **messaggi**.

In altre parole, un MOM fornisce una **struttura** che consente a diverse applicazioni, o parti di un'applicazione, di comunicare tra loro in modo **asincrono**, senza dover essere direttamente collegate tra loro.



Il MOM funge da broker e quindi permette un disaccoppiamento: consente ai produttori e ai consumatori di messaggi di non conoscere i dettagli reciproci, ma solo il sistema di messaggistica.

1. **Produttore di messaggi:** Un'applicazione o componente che crea un messaggio.
2. **MOM:** Il software che riceve, memorizza e smista i messaggi ai destinatari corretti.
3. **Consumatore di messaggi:** Un'applicazione o componente che riceve i messaggi dal broker e li elabora.

9.4.1 Funzionamento di JMS

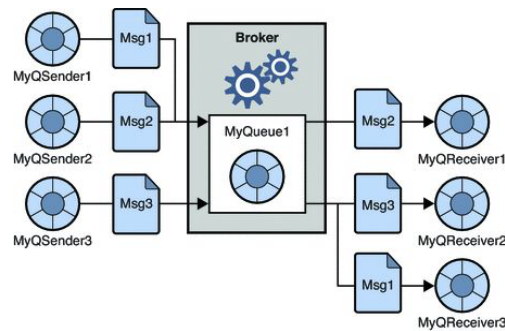
Si tratta di una **API** che permette la **comunicazione asincrona** tra applicazioni distribuite attraverso l'invio e la ricezione di **messaggi**.

1. **Comunicazione asincrona:** Inviare e ricevere messaggi senza che il mittente debba aspettare una risposta immediata dal destinatario.
2. **Due modelli di messaggistica:**
 - **Point-to-Point (P2P):** Un messaggio viene inviato a una **coda** (queue), e un solo consumatore può riceverlo.
 - **Publish/Subscribe (Pub/Sub):** Un messaggio viene inviato a un **topic**, e può essere ricevuto da più consumatori (abbonati).
3. **Affidabilità:** JMS garantisce che i messaggi vengano consegnati, anche in presenza di guasti temporanei, memorizzando i messaggi finché non sono ricevuti correttamente.
4. **Decoupling:** I produttori e i consumatori sono indipendenti, cioè non si conoscono direttamente e non dipendono l'uno dall'altro.

Questo approccio è particolarmente utile per **integrare sistemi eterogenei** e supporta la comunicazione di gruppo, come la trasmissione multipla o in broadcast, e la gestione dei membri del sistema.

Modello punto-punto

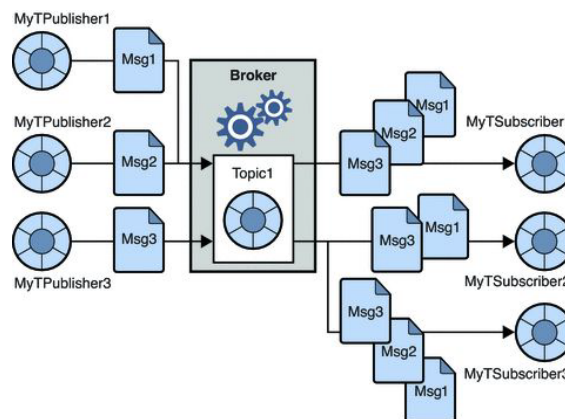
Nel modello punto-punto, si può avere uno scenario semplice con un mittente e un destinatario.



Oppure uno più complesso con più mittenti e destinatari, che possono anche condividere connessioni. Non vi sono ipotesi sull'ordine con cui i messaggi vengono consumati.

Modello publish-subscribe

In questo caso abbiamo una **repository dei messaggi** in cui vengono inseriti i messaggi in arrivo secondo un **topic**.



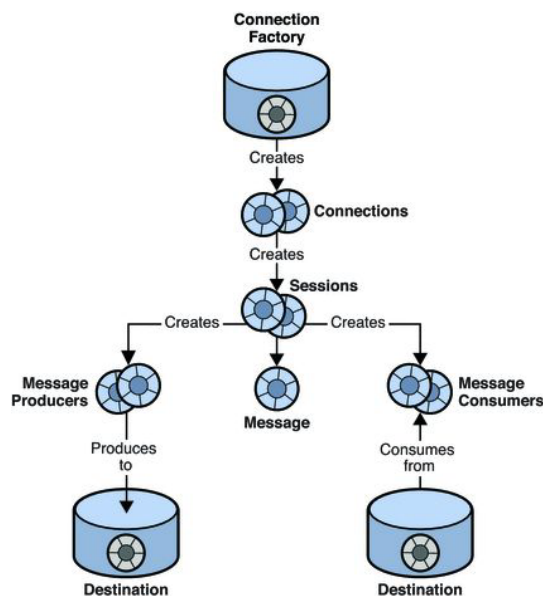
Ci sono vari client che si registrano come subscriber, i messaggi appartenenti ad un certo topic vengono inoltrati a tutti i subscriber registrati a tale topic.

Il publisher, non conosce a quanti ne a quali subscriber arriverà quel messaggio.

9.4.2 JMS programming objects

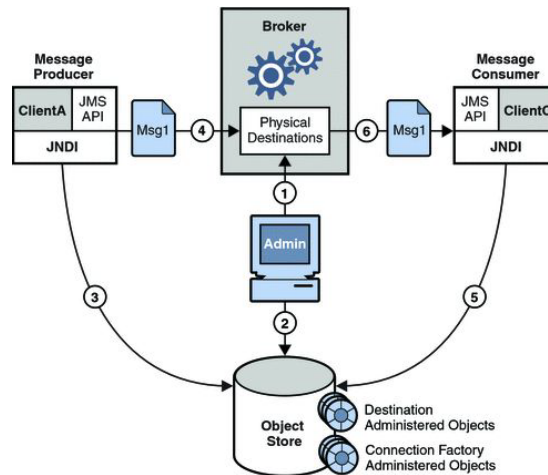
Per implementare la messaggistica JMS, si utilizzano diversi oggetti di programmazione.

1. **Connection Factory:** È responsabile della creazione delle **connessioni** al broker di messaggi. La connessione rappresenta il canale di comunicazione tra il client e il broker.
2. **Sessione:** Gestisce la **conversazione** tra il client e il broker. All'interno di una sessione vengono create e inviate le comunicazioni, come la gestione delle transazioni.
3. **Producer:** È l'oggetto che **invia** i messaggi a una **destinazione** (coda o topic) nel broker di messaggi.
4. **Consumer:** È l'oggetto che **riceve** i messaggi dalla destinazione. Il consumer si sottoscrive a una coda o un topic per elaborare i messaggi in arrivo.
5. **Messaggio:** Un messaggio JMS è composto da tre parti principali:
 - **Intestazione (Header):** Contiene informazioni di controllo, come il tipo di messaggio e le proprietà di routing.
 - **Proprietà (Properties):** Sono metadati opzionali che possono essere aggiunti al messaggio per fornire informazioni aggiuntive.
 - **Corpo (Body):** Contiene i dati effettivi del messaggio, cioè l'informazione che viene trasmessa.
6. **Destinazione:** È il **punto fisico** di arrivo del messaggio. Può essere una **coda (queue)** o un **topic**. La destinazione rappresenta dove il messaggio viene inviato e da dove il consumer lo riceve.



JMS administered objects

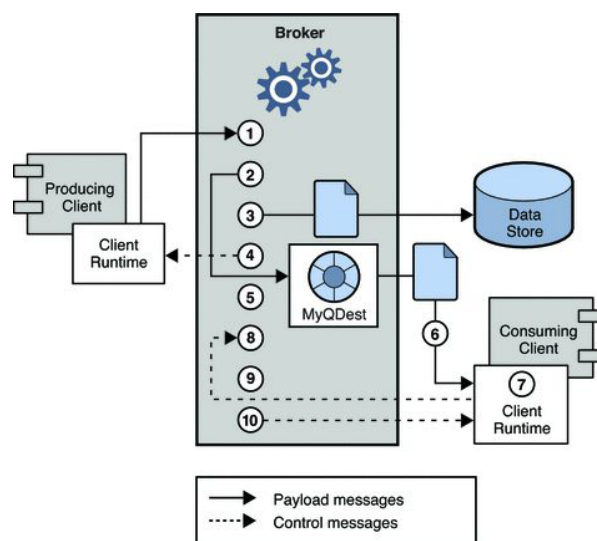
Gli oggetti amministrati, come le connection factories e le destinazioni, vengono solitamente creati tramite strumenti di amministrazione sull'Application Server. Per accedervi, JMS utilizza JNDI (*Java Naming and Directory Interface*) per cercare questi oggetti amministrati.



Message Delivery Steps

Il processo di consegna di un messaggio prevede diversi passaggi:

1. Il messaggio viene inviato tramite una connessione al broker, che lo legge e lo posiziona nella destinazione appropriata.
2. Successivamente, il broker determina l'instradamento del messaggio e lo invia alla connessione di destinazione.
3. Se il messaggio è persistente, viene archiviato in un data store.
4. Il broker conferma la ricezione al client.
5. Il runtime del client consegna quindi il messaggio e invia una conferma al broker. Se il messaggio era persistente, viene cancellato dal broker dopo la conferma del client.
6. Infine il broker invia al client un messaggio di "ack processed".



9.5 EJB - Enterprise JavaBeans

Gli **EJB** sono un tipo specifico di componente in **Jakarta EE** e offrono solo alcune **funzionalità specifiche** che vengono gestite dal **container EJB**.

Questo approccio consente di mantenere i **moduli client** più leggeri, poiché **gran parte della logica viene eseguita sul server**, alleggerendo il compito del programmatore che altrimenti dovrebbe costruire componenti per conto suo.

Riutilizzabilità: È possibile costruire nuove applicazioni riutilizzando EJB esistenti, il che facilita la manutenzione e lo sviluppo.

Ci sono tipi differenti di EJB, come i **session beans** e i **message driven beans**.

9.5.1 Session Beans

Un tipo di **EJB**, gestisce la logica di business per le applicazioni distribuite.

Gestiscono la logica di business e sono utilizzati per gestire operazioni che non richiedono persistenza.

Possono essere **stateful** (mantengono lo stato tra le chiamate) o **stateless** (non mantengono lo stato tra le chiamate).

Stateful Session Beans

Mantengono lo stato tra le invocazioni dei metodi, rendendo la sessione specifica per ciascun client.

Lo stato dipende dalle interazioni con il client e viene mantenuto per tutta la durata della sessione.

Interfaccia:

```
@Remote // Interfaccia remota.
public interface AccountSessionBeanRemote {
    void deposit(double amount);
    void withdraw(double amount);
    double getBalance();
}
```

E l'implementazione:

```
@Stateful
public class AccountSessionBean implements AccountSessionBeanRemote
{
    private double balance = 0; // Stato del bean.
    @Override
    public void deposit(double amount) {
        balance += amount;
    }
}
```

```

@Override
public void withdraw(double amount) {
    if (balance >= amount) {
        balance -= amount;
    } else {
        System.out.println("Fondos insuficientes");
    }
}

@Override
public double getBalance() {
    return balance;
}
}

```

Questi bean conservano i valori delle variabili di istanza tra le chiamate del client.

La sessione con il bean non coincide necessariamente con la sessione web dell'applicazione. Quando il client termina la sessione o il bean viene rimosso, lo stato viene perso.

Un bean stateful non è idempotente.

Stateless Session Beans

In questo caso l'idea di base è quella di non fornire un supporto ad uno stato della conversazione con il client.

Il client può cambiare lo stato del bean ma non è garantito che sia recuperato alla prossima invocazione.

Ogni invocazione è indipendente, rendendoli più scalabili e adatti per applicazioni con molti client.

Il server applicativo ottimizza mantenendo un **pool di session bean**: quando avviene una richiesta il primo session bean disponibile verrà utilizzato per fornire il servizio.

Sono più efficienti rispetto ai Stateful Session Beans in applicazioni con molti client.

Inoltre, possono implementare servizi web (*grazie all'idempotenza*) mentre uno stateful session bean non può a causa di problemi di idempotenza, le chiamate successive alla prima non possono essere idempotenti.

Singleton Session Beans

In questo caso le session bean hanno uno stato, ma è unico e condiviso tra diversi clienti.

Esistono come un'unica istanza per applicazione, mantenendo lo stato tra le chiamate dei client, ma non tra i riavvii del server.

Solitamente deve essere thread safe, quindi in questi casi bisogna utilizzare le tecniche fornite dalla Java Standard Library.

Esiste una sola istanza per applicazione e questa viene condivisa tra tutti i client. Mantengono lo stato tra le invocazioni ma non garantiscono la persistenza dello stato tra i riavvii del server. Possono essere utilizzati per implementare endpoint di servizi web.

9.5.2 Message Driven Beans

Sono dedicati agli oggetti di business e hanno a che fare con il processare le informazioni.

Eseguono la logica di business in risposta a messaggi, piuttosto che a chiamate di metodi. Sono utilizzati per implementare l'elaborazione asincrona.

Ci riferiamo alle sessioni non relative solo alle applicazioni web, la sessione è relativa alla conversazione tra due componenti non necessariamente passando attraverso una connessione web.

```
public interface OrderProcessor {
    void processOrder(String order);
}

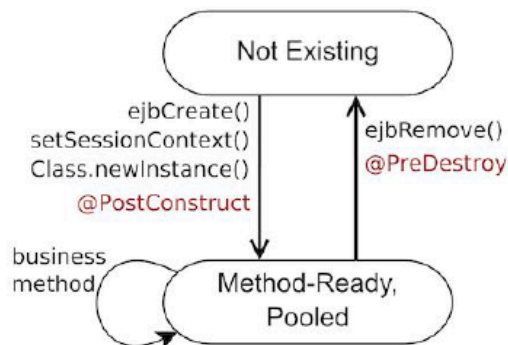
@MessageDriven(
    mappedName = "java:/jms/queue/OrderQueue", // Nome della coda JMS
    activationConfig = {
        @ActivationConfigProperty(
            propertyName = "destinationType",
            propertyValue = "javax.jms.Queue"
        )
    }
)
public class OrderMDB implements OrderProcessor {

    @Override
    public void processOrder(String order) {
        // Qui andrebbe la logica di business per elaborare l'ordine
    }

    // Metodo chiamato quando un messaggio è ricevuto
    public void onMessage(Message message) {
        try {
            if (message instanceof TextMessage) {
                String order = ((TextMessage) message).getText();
                processOrder(order); // Processiamo il messaggio
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

9.5.3 Ciclo di vita di uno Stateless Session EJB

In questo caso consideriamo “*Ditto for Singletons (not pooled!)*”.



Creazione (Instantiation):

Quando un client invoca un metodo su un **Stateless EJB**, il container EJB crea un'istanza del bean se non esiste già una disponibile.

Il container EJB **gestisce automaticamente la creazione** del bean e non è necessario che il programmatore intervenga in questo processo.

Inizializzazione (Initialization):

Quando il bean viene creato, il container può eseguire l'inizializzazione tramite il metodo annotato con `@PostConstruct` (se presente), che consente di eseguire alcune operazioni di configurazione iniziale.

Il bean viene quindi reso disponibile per ricevere invocazioni da parte dei client.

Utilizzo (Usage):

Il bean è stateless, quindi ogni invocazione è indipendente dalle altre, e il bean non memorizza alcun stato tra le invocazioni.

Pooling: per non ripetere le prime due fasi, il container EJB può **riutilizzare istanze** di un per rispondere a più invocazioni da parte di client diversi.

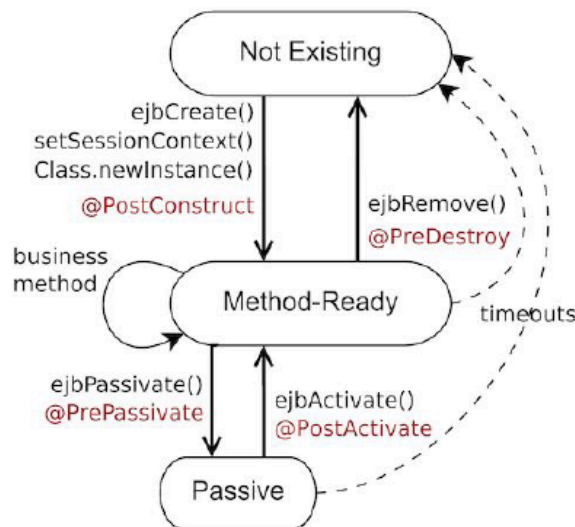
Distruzione (Destruction):

Poiché un **Stateless EJB** non conserva uno stato tra le invocazioni, il container può **distruggere le istanze** quando non sono più necessarie, senza bisogno di conservarne i dati.

La distruzione del bean può avvenire al termine della sessione del client o quando il container decide di rilasciare risorse per migliorare le prestazioni.

Prima della distruzione, se presente, può essere invocato il metodo `@PreDestroy` per eseguire la pulizia finale.

9.5.4 Ciclo di vita di uno Stateful Session EJB



La prima parte dei due schemi (*quella in alto*) è circa la stessa, gli oggetti non sono raccolti in un pool come nel caso precedente.

Creazione (Instantiation):

Un client invoca un metodo di creazione sul **Session Bean**. Questo può essere fatto tramite l'interfaccia di business del bean.

Il container EJB crea una nuova istanza di **Stateful Session EJB** per il client. La creazione dell'istanza avviene generalmente su richiesta del client.

Inizializzazione (Initialization):

Dopo la creazione, il bean può essere inizializzato, ad esempio, tramite un metodo annotato con `@PostConstruct` se specificato, che viene eseguito subito dopo la creazione.

A questo punto, l'istanza è pronta per essere utilizzata dal client. Il bean può anche eseguire configurazioni iniziali, come l'inizializzazione di variabili o la connessione a risorse esterne.

Utilizzo (Usage):

Durante questa fase, il client può fare ripetute invocazioni sul bean.

Lo stato del bean viene mantenuto tra le chiamate, permettendo al bean di "ricordare" i dati specifici del client.

Le invocazioni successive del client utilizzano lo stesso oggetto bean, che conserva lo stato tra le chiamate.

Passivazione (Passivation):

Quando il container EJB rileva che un **Bean** non è più in uso o che la sua sessione con il client è scaduta (o ha bisogno di liberare risorse per motivi di performance), il bean può essere **passivato**.

Durante la passivazione, lo stato del bean viene **salvato** su disco (*rimosso dalla RAM*) o in un'area di memoria per permettere di liberare risorse senza perdere i dati.

In fase di passivazione, vengono salvate anche tutte le informazioni necessarie per "scongellare" (*riattivare*) l'EJB in futuro, ripristinando lo stato esattamente come era prima.

La passivazione è un meccanismo automatizzato gestito dal container EJB. Una volta passivato, il bean è sospeso e non può essere utilizzato fino a quando non viene "riattivato".

Se un EJB passivato rimane inattivo per un certo periodo di tempo (*definito nel container EJB*), viene distrutto per liberare ulteriori risorse nel disco.

Riattivazione (Activation):

Quando il client invoca nuovamente il bean dopo che è stato passivato, il container **riattiva** l'istanza recuperando il suo stato salvato durante la passivazione.

Una volta riattivato, il bean è di nuovo pronto per l'uso, e il client può continuare a interagire con esso come prima.

La riattivazione è gestita automaticamente dal container EJB.

La passivazione e riattivazione comportano un certo overhead, poiché **richiedono operazioni di I/O con il disco** per salvare e recuperare lo stato.

Rimozione (Removal):

Quando la sessione del client termina o quando il bean non è più necessario (ad esempio, il client invoca il metodo `remove()`), il bean entra nella fase di **rimozione**.

Durante questa fase, il container EJB distrugge l'istanza del bean e libera le risorse che erano state allocate.

Prima della distruzione, il container può invocare il metodo annotato con `@PreDestroy` per eseguire eventuali operazioni di pulizia.

Dopo la rimozione, il bean non è più disponibile per essere utilizzato.

9.6 Ricerca di risorse remote in Applicazioni Enterprise

I servizi di directory, come LDAP, associano nomi e indirizzi di rete, facilitando l'accesso a risorse distribuite (*EJB, o altro*) lungo la rete. Sono utilizzati per gestire informazioni come coppie nome/utente e password.

Utilizziamo i **directory services**: ovvero delle repository per mantenere le coppie:

```
< nome , indirizzo IP , ... >
```

Possiamo associare informazioni aggizionali per ogni match.

Differentemente da un RDBMS, un directory service è specializzato nel gestire strutture dati tipicamente usate in ambienti di rete.

LDAP - Lightweight Directory Access Protocol

Questo servizio è utilizzato per mantenere coppie `username-password` per scopi di autenticazione. Lo stesso può avvenire con i DBMS standard.

9.6.1 JNDI - Java Naming and Directory Interface

Abbiamo un servizio che dobbiamo costruire secondo una API specificata in JNDI.

L'idea dietro JNDI è la seguente: Chiediamo a questo servizio (*istanziato nel nostro server applicativo*) come recuperare un oggetto.

Un nome è associato ad un oggetto direttamente oppure tramite un riferimento, per cercare un oggetto.

Il **JNDI** è un'API Java che consente di cercare dati o oggetti tramite un nome all'interno di un servizio di directory. Ogni server applicativo Java EE include un servizio JNDI. I nomi sono organizzati gerarchicamente e associati agli oggetti in modo diretto o tramite riferimenti.

Per prima cosa ci servono le informazioni per accedere all'oggetto, quindi possiamo specificare in una tabella di hash tutte le informazioni necessarie per allocare il servizio, dopo questo creiamo l'oggetto del contesto iniziale, una volta che abbiamo questo contesto, lo utilizzeremo per le operazioni di lookup.

Hashtable è una struttura dati che memorizza coppie chiave-valore. In questo caso, viene utilizzata per specificare i parametri necessari per allocare il contesto iniziale JNDI.

I parametri inseriti con `contextArgs.put(...)` possono includere informazioni come:

- **Nome dell'host del server JNDI**
- **Porta del servizio JNDI**
- **Credenziali di accesso (username/password)**
- **Classe factory del contesto iniziale** (es.
`com.sun.jndi.rmi.registry.RegistryContextFactory`)

Questi parametri sono fondamentali per localizzare il servizio JNDI corretto all'interno dell'infrastruttura di rete.

InitialContext è la classe principale dell'API JNDI che rappresenta l'accesso iniziale al servizio di directory.

Quando viene creata un'istanza di **InitialContext**, questa utilizza i parametri forniti nell'**Hashtable** per connettersi al servizio JNDI.

myContext rappresenta il contesto JNDI dal quale si possono effettuare operazioni di lookup (ricerca) per localizzare risorse come EJB, datasource, code JMS, ecc.

Specifichiamo un nome simbolico e poi otteniamo un riferimento all'oggetto, dobbiamo effettuare il downcast all'interfaccia corretta. Solitamente ci sono nomi standard per gli EJB.

lookup(String name): Questo metodo viene utilizzato per cercare un oggetto nel servizio di directory JNDI. La stringa passata al metodo ("**com.mysite.ejb.MyBean**") rappresenta il **nome JNDI** dell'EJB.

Se la ricerca ha successo, il metodo restituisce un oggetto che deve essere **castato** al tipo appropriato (in questo caso, **MyBean**).

Il nome "**com.mysite.ejb.MyBean**" segue una convenzione che può variare in base alla configurazione del server. Solitamente rappresenta un percorso gerarchico che identifica l'EJB all'interno del servizio JNDI.

I nomi JNDI devono essere unici all'interno del contesto e rispettare le convenzioni definite dal server applicativo.

```
Hashtable contextArgs = new Hashtable();

contextArgs.put( ... ) // insert all req. params to locate the
service

Context myContext = new InitialContext(contextArgs);

MyBean myBean = (MyBean)
myContext.lookup("com.mysite.ejb.MyBean");
```

Glassfish

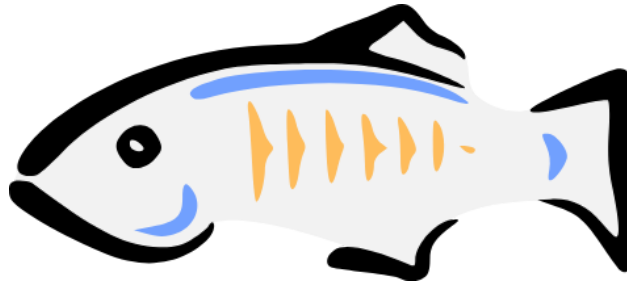
Possiamo gestire JDBC come risorsa, in esso troviamo un pool di risorse.

GlassFish è un **application server** open-source per la piattaforma **Java EE (Enterprise Edition)**, sviluppato inizialmente da Sun Microsystems e ora mantenuto dalla comunità Eclipse Foundation sotto il nome **Eclipse GlassFish**. È utilizzato per lo sviluppo, il deploy e la gestione di applicazioni Java EE.

Caratteristiche principali:

- **Supporto completo per Java EE/Jakarta EE:** GlassFish implementa tutte le specifiche Java EE, come EJB, JPA, servlets, JMS e servizi web.

- **Server modulare:** Basato su **OSGi**, consente di caricare solo i moduli necessari.
- **Admin Console:** Interfaccia web intuitiva per configurare e gestire l'applicazione.
- **Alte prestazioni:** Supporta clustering, load balancing e connessioni sicure.
- **JNDI supportato:** Gestisce il lookup di risorse come EJB e datasource.



Capitolo 10 - Algoritmi distribuiti fondamentali

Algoritmo base

L'**algoritmo principale** che i nodi della rete eseguono per risolvere il problema specifico.

Definisce il **comportamento normale** del sistema distribuito, come l'elaborazione dei dati, la comunicazione tra nodi e il raggiungimento di un obiettivo comune.

Ad esempio un algoritmo di routing.

Algoritmo di controllo

Algoritmo ausiliario che serve per garantire il corretto funzionamento dell'algoritmo base, quindi è eseguito contemporaneamente all'algoritmo base.

Monitora e gestisce situazioni particolari come **errori, guasti, sincronizzazione e recupero**.

Esempi: rilevamento di **deadlock**, gestione di **errori di comunicazione**, meccanismi di **ripristino dopo un fallimento**, algoritmi di **snapshot globale**.

Initiator

Il nodo (o il processo) che **avvia l'esecuzione dell'algoritmo** all'interno del sistema distribuito.

L'initiator è particolarmente rilevante in contesti in cui un'azione deve essere avviata in un sistema senza un coordinatore predefinito.

Il primo evento di un initiator può essere interno oppure una send.

Algoritmo centralizzato

Algoritmo in cui **una singola entità centrale** (*un nodo o un processo*) è responsabile della gestione e del coordinamento dell'intero sistema.

Gli altri nodi delegano a questo nodo centrale le decisioni critiche, che possono riguardare l'accesso a risorse condivise, la gestione della sincronizzazione o il controllo del flusso dei dati.

Un algoritmo è centralizzato se ha **un unico initiator** che avvia l'esecuzione dell'algoritmo.

Algoritmo decentralizzato

Algoritmo in cui **non esiste un singolo nodo centrale** responsabile del coordinamento. Il controllo è distribuito tra più nodi, i quali comunicano tra loro e prendono decisioni collettive.

Un algoritmo è decentralizzato se ha **molteplici initiator indipendenti**, ognuno dei quali può avviare l'esecuzione indipendentemente dagli altri.

10.1 Wave algorithms

Classe di **algoritmi distribuiti** utilizzati per **diffondere (e/o raccogliere) informazioni** attraverso una rete e garantire che ogni nodo sia raggiunto prima della terminazione del processo.

Sono spesso impiegati in contesti come **elezione di un leader, esplorazione della rete e terminazione distribuita**.

Tipicamente la richiesta nasce da uno o più initiator, che poi, come un onda (*un'onda di informazioni*) coinvolge tutti gli altri nodi e ciascun nodo deve restituire le informazioni richieste.

- **Propagazione ad onda:** L'informazione (*partendo dall'initiator*) attraversa la rete in modo graduale.
- **Terminazione garantita:** L'algoritmo è finito, ovvero ha un numero finito di step e prima o poi di sicuro terminerà.
- **Eventi di decisione:** Contiene uno o più eventi di decisione in cui vengono prese decisioni basate sui dati raccolti.

Eventi di decisione

Momenti nei quali un processo in un sistema distribuito **prende una decisione** (es. elezione di un leader, commit di una transazione, consenso in un algoritmo distribuito).

Per ogni evento di decisione d e processo p_i , vale $c \xrightarrow{HB} d$ per un certo c in p_i .

Grazie alla relazione HB, possiamo stabilire **quali eventi influenzano la decisione**.

Esempio di algoritmo Wave - Anello

Supponiamo di avere una topologia ad anello come in figura, l'initiator può essere un p_x qualsiasi.

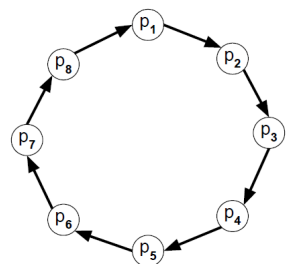
Questo algoritmo è decentralizzato, perché dicendo che non mi importa quale initiator inizia, implica che ce ne possa essere più di uno.

Il codice initiator:

```
send(toNeighbor, OutToken) // Invia il token al vicino secondo l'anello.  
receive(fromNeighbor, inToken) // Riceve il token dal vicino secondo l'anello.  
// Se non ci sono messaggi nella coda, si mette in attesa.  
decide() // Prende una decisione in base alle informazioni ricevute.
```

Il codice di un nodo non-initiator:

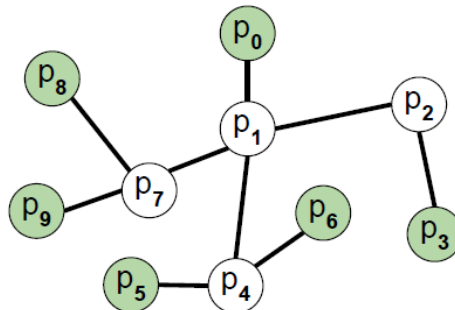
```
receive(fromNeighbor, inToken) // Riceve il token dal vicino secondo l'anello.  
doSomething() // Fa qualcosa con le informazioni ricevute.  
send(toNeighbor, OutToken) // Invia il token al vicino secondo l'anello.
```



Il messaggio è inviato e questo verrà propagato e ad un certo tempo tornerà indietro verso il nodo initiator (*completa un giro dell'anello*).

Esempio di algoritmo Wave - Tree Wave

I nodi sono organizzati in una topologia ad albero connesso senza loop.



Supponiamo che ogni nodo conosca i suoi vicini.

L'initiator può essere una foglia qualunque (*quindi l'algoritmo è decentralizzato*) e invia il token al proprio "padre".

L'algoritmo inizia dalle foglie dell'albero e procede verso la radice, raccogliendo informazioni o propagando messaggi man mano che si sale nell'albero.

In questo caso il codice dello initiator è lo stesso di ogni altro nodo.

Il nodo foglia invierà direttamente il messaggio all'unico neighbor, dopo questo, il prossimo step è la ricezione di un messaggio dallo stesso neighbor.

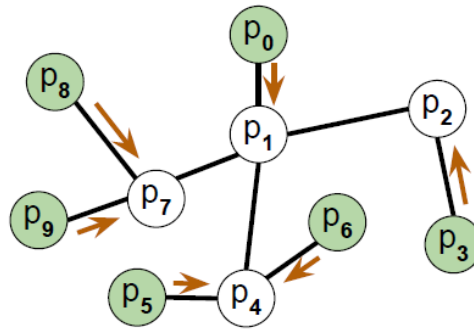
```
bool recv[n_neigh] //array initialized [false]

// loop executed n_neigh - 1 times
while len(filter(false, recv)) > 1:
    receive(fromNeighbor, inToken)
    recv[fromNeighbor] = true

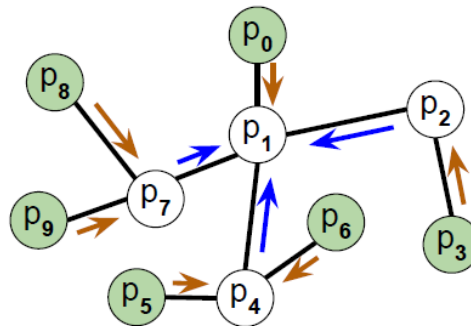
//toNeighbor: the only proc with recv == false
send(toNeighbor, outToken)
receive(toNeighbor, inToken)
recv[toNeighbor] = true //not necessary...
decide()
//this way, only two processes decide. Possibly:
//foreach n in neighbors\toNeighbor:
//    send(n, outToken)
```

1. Ogni nodo non-foglia esegue per `#neighbours - 1` volte il ciclo `while` dove aspetta di ricevere un token da ogni nodo da cui si aspetta di ricevere qualcosa (vicino).
2. Non appena un messaggio da un neighbor viene ricevuto, l'entry corrispondente viene impostata a `true`.
3. Il loop termina quando avremo solo una posizione che vale `false`, corrispondente all'unico processo da cui non abbiamo ricevuto un messaggio.
4. Quel processo è il padre a cui gli viene inviato un messaggio.

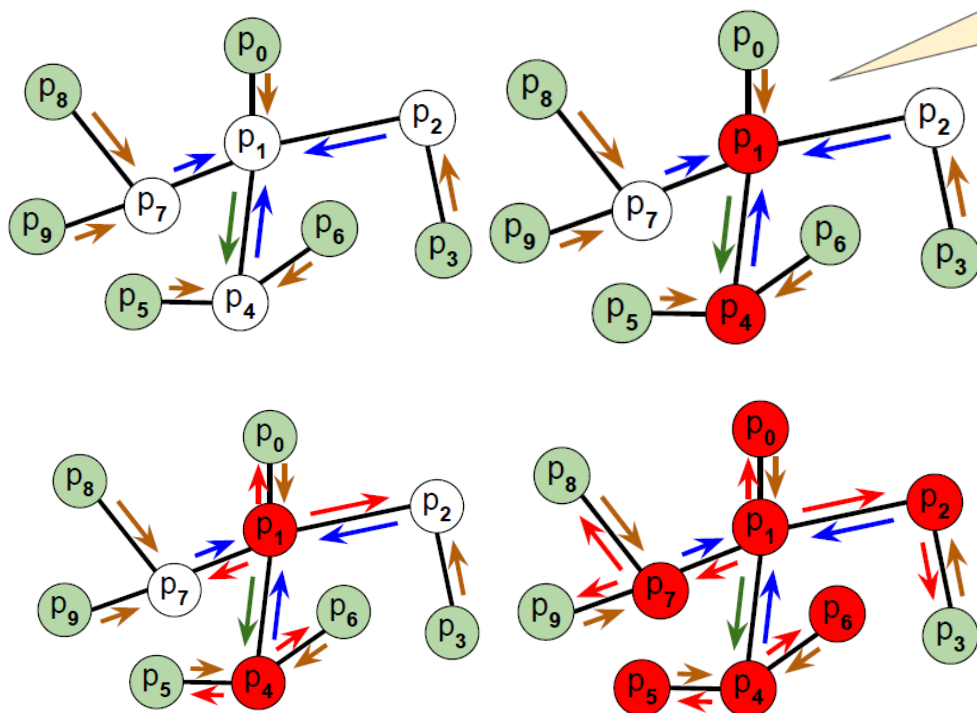
In figura osserviamo che p_7 era bloccato all'inizio, vediamo che riceve due messaggi (da p_8 e p_9).

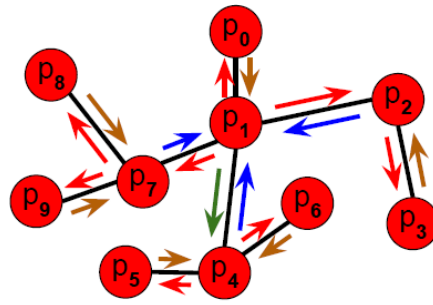


Nel **prossimo step** vediamo che p_1 riceve i messaggi, sia p_1 che p_4 sono in grado di raggiungere il punto di decisione nel codice (*decide*).



E così via, tutti i nodi ricevono i messaggi.





Alla fine l'informazione è distribuita tra tutti i nodi.

Traversal algorithms

Un **algoritmo di traversamento** è un tipo di algoritmo a onda centralizzato dove l'iniziatore invia un token che attraversa tutta la rete, toccando ogni nodo.

Tutti i processi ricevono il token almeno una volta e infine torna all'iniziatore, che esegue un evento decisionale basato sulle informazioni raccolte.

Questi algoritmi vengono spesso utilizzati per esplorare reti, raccogliere dati o determinare lo stato globale di un sistema distribuito.

10.2 Snapshot di una computazione distribuita

Uno **snapshot** in un sistema distribuito rappresenta una sorta di "fotografia" dello stato globale del sistema in un preciso momento dell'esecuzione.

Da cos'è composto uno snapshot

- **Stati locali dei processi:** ciascun processo registra il proprio stato interno, che può includere variabili, contatori o qualsiasi altra informazione rilevante.
- **Messaggi in transito nei canali di comunicazione:** questi messaggi sono stati inviati ma non ancora ricevuti, e rappresentano lo stato dei canali tra i processi.

Snapshot consistente

Per essere considerato **consistente**, uno snapshot deve essere catturato dopo un **taglio consistente** (*consistent cut*). Un taglio è consistente se preserva l'ordine causale degli eventi, ossia:

Se un evento **a** precede un evento **b** ($a \rightarrow b$), allora, se **b** è incluso nello snapshot, anche **a** deve esserlo.

Questo garantisce che lo snapshot catturi uno stato coerente del sistema, evitando situazioni in cui eventi futuri vengano registrati senza i loro eventi passati corrispondenti.

Perché ci interessano gli snapshot? - Predicati stabili

Gli snapshot sono utilizzati principalmente per verificare alcune proprietà fondamentali dei sistemi distribuiti, in particolare i **predicati stabili**.

Un predicato è stabile se, ad un certo punto nel tempo diventa vero e rimane tale per tutta la durata dell'esecuzione.

Esempi di predicati stabili includono:

- **Deadlock:** Situazione in cui più processi rimangono bloccati in attesa di risorse che non verranno mai rilasciate.
- **Terminazione:** Rilevazione che tutti i processi hanno completato la loro esecuzione.

Perché ci interessano gli snapshot? - Checkpointing

Gli snapshot servono anche per implementare meccanismi di **checkpointing**, che consentono di salvare uno stato consistente del sistema.

Questo stato può essere utilizzato per **ripristinare l'esecuzione in caso di guasti** (*rollback*), migliorando l'affidabilità del sistema.

Snapshot algorithms

Uno snapshot è fatto tramite un apposito algoritmo.

Un algoritmo di snapshot è un **algoritmo di controllo** che monitora l'esecuzione di un altro algoritmo distribuito in esecuzione nello stesso sistema.

L'esecuzione dei due algoritmi è in contemporanea e quindi intervallata in qualche modo, questo perché l'algoritmo di snapshot non deve interrompere il sistema.

Dobbiamo coordinare tutti i nodi per raccogliere le informazioni ad un certo punto nel tempo.

In sistemi reali c'è un nodo speciale che deve raccogliere queste informazioni e decidere quali azioni prendere.

L'aspetto complesso è che, nei sistemi distribuiti, non esiste un clock globale che sincronizzi i processi. Pertanto, la sincronizzazione si basa sul **tempo logico**, che ordina gli eventi in base alla loro causalità.

10.2.1 Chandy-Lamport Algorithm

L'algoritmo di **Chandy-Lamport** è il più noto per catturare snapshot consistenti.

Ipotesi dell'algoritmo

Un algoritmo distribuito è corretto solo sotto certe ipotesi, se queste vengono cambiate non è garantito che l'algoritmo si comporti come ci aspettiamo.

- **Canali FIFO:** I messaggi vengono ricevuti nello stesso ordine in cui sono stati inviati.

Se il canale non fosse FIFO, il ricevitore potrebbe ricevere dei messaggi in un ordine non corretto.

- **Grafo fortemente connesso:** Tutti i processi devono poter comunicare tra loro direttamente o indirettamente.

Le azioni devono essere guidate nei nodi da un marker speciale nei messaggi.

Sotto queste ipotesi, più nodi possono avviare questo algoritmo contemporaneamente, sicuramente questi due o più eventi (*in base al numero di nodi*) saranno concorrenti.

Funzionamento dell'algoritmo

1. **Inizializzazione**
 - Un **processo initiator** decide di avviare lo snapshot e **salva il proprio stato locale**.
 - Invia un **messaggio marker** a tutti i suoi vicini nei canali di comunicazione.
2. **Registrazione dello stato locale**
 - Quando un processo **riceve il marker per la prima volta**, salva il proprio stato locale e inoltra il marker agli altri processi.
 - Se il processo riceve **altri marker successivi**, ignora la fase di salvataggio perché lo snapshot è già stato catturato.
3. **Gestione dei messaggi in transito**
 - Ogni processo continua a registrare i messaggi in arrivo su un canale **fino a quando non riceve un marker su quel canale**.
 - Una volta ricevuto il marker su tutti i canali in ingresso, lo snapshot è completo.

Il primo rettangolo è relativo all'**initiator**: se il marker è **False**, viene impostato a **True** e viene inviato un messaggio ad ogni processo.

```
procedure takeSnapshot()  
  if not marked then  
    marked = true  
    foreach c in outchannels send(c, marker)  
    <record local state + incoming channels' state>
```

Nell'immagine sotto troviamo il codice relativo ad un singolo processo:

1. Attende il **marker**.
2. Salva in un array i canali da cui ha ricevuto i messaggi marker.
3. Se ha ricevuto un messaggio da ogni canale in ingresso, allora lo snapshot locale è terminato.

```
procedure onMarker(channel c)
  takeSnapshot()
  chmark[c] = true
  if chmark[ic] for each incoming channel ic then
    <local snapshot termination>
```

Se invece riceve un messaggio base, se è **marked** e non aveva ricevuto il marker per esso, allora appende tale messaggio al vettore **chstate**.

Se **marked == true** significa che da qualche parte ha ricevuto un marker e quindi lo snapshot è iniziato.

Il canale FIFO garantisce che alla ricezione del marker, tutti i messaggi provenienti da quel canale sono stati ricevuti.

```
procedure onBasicMsg(channel c, msg m)
  if marked and not chmark[c] then
    chstate[c].append(m) //state for chann. c
```

Complessità del messaggio

Ogni canale FIFO richiede l'invio di un solo marker, la complessità del messaggio è espressa come $\theta(E)$, dove E è il numero di canali.

Complessità di tempo

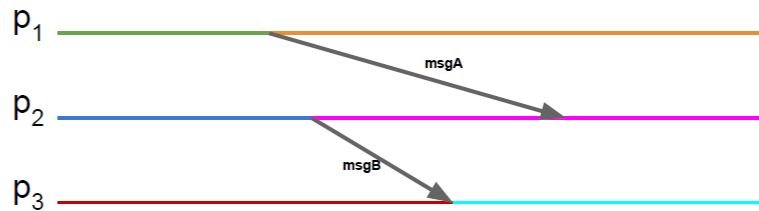
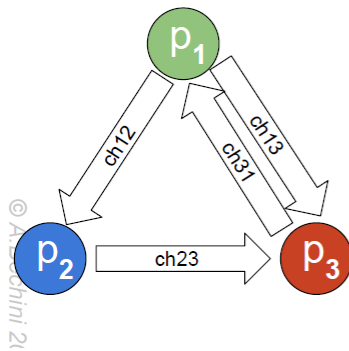
Il tempo necessario (*nel caso peggiore dipende dalla lunghezza del percorso*) per raggiungere tutti i nodi. Ovvero il tempo necessario per percorrere il percorso più lungo.

Il valore della complessità di tempo vale $O(D)$, dove D è il diametro della rete (*la distanza massima tra due processi*).

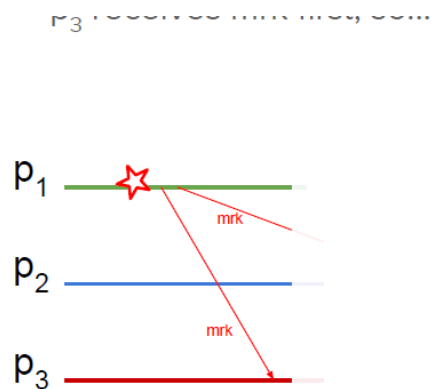
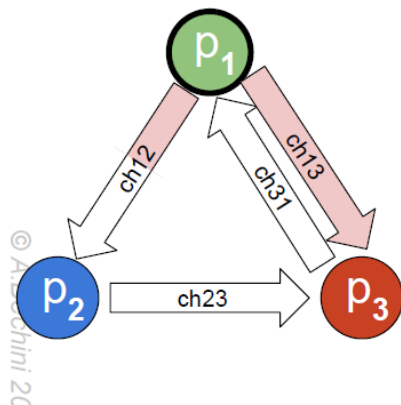
Esecuzione algoritmo Chandy-Lamport

In figura notiamo che abbiamo 4 canali disponibili e 3 processi, nel diagramma temporale a destra vediamo l'esecuzione totale.

Three processes; in the basic algorithm, p_1 sends msgA to p_2 and p_2 sends msgB to p_3 , as shown.

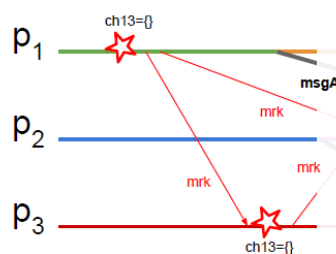
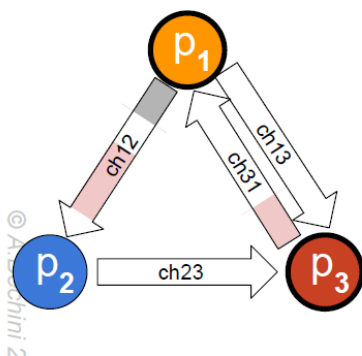


1. p_1 decide di effettuare uno snapshot, viene inviato un messaggio marker (*mrk*) negli altri canali (*ch12* e *ch13*).

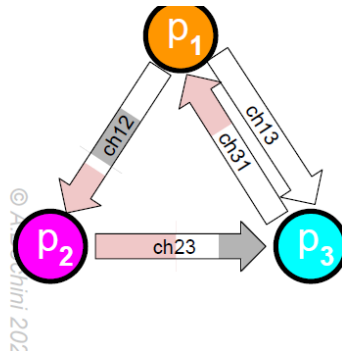


2. p_3 effettua lo snapshot e invia il marker nel canale *ch31* (diverso da *ch13*).
3. p_1 alla ricezione del marker da p_3 , termina l'esecuzione dell'algoritmo (non ha altri canali di ingresso).

p_3 records its state (red), and sends *mrk* to p_1 . Meanwhile, p_1 sends basic msgA to p_2 and changes state ($\rightarrow$ orange). Then...



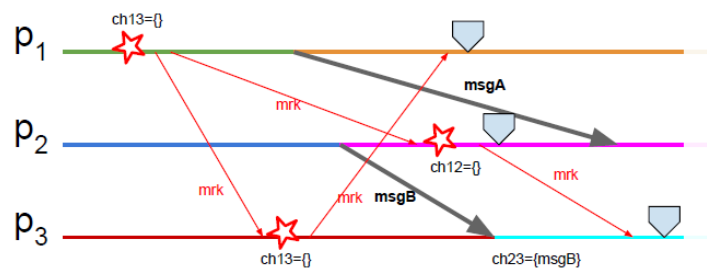
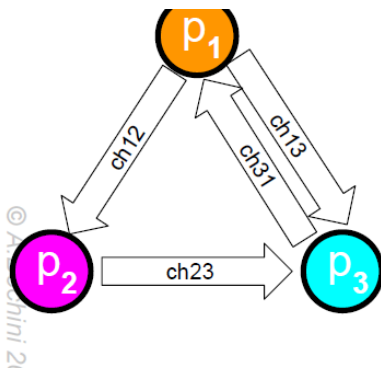
4. p_2 riceve il marker dall'unico canale con p_1 , ed effettua lo snapshot.
5. p_2 , invia il marker nell'unico canale di uscita verso p_3 .
6. p_2 termina l'algoritmo (non ha altri canali di ingresso).



7. p3 riceve un messaggio non marker da p2, **lo salva** e va avanti.

Questo messaggio è considerato in transito.

8. p3 riceve il marker dall'unico canale con p2, ed non effettua nuovamente lo snapshot (ha già `marked == true`).
9. Non ci sono altri canali in ingresso, quindi anche in p3 termina l'algoritmo.



Per lo stato dei canali, non c'è nessun messaggio in tutti i canali tranne che in ch23.

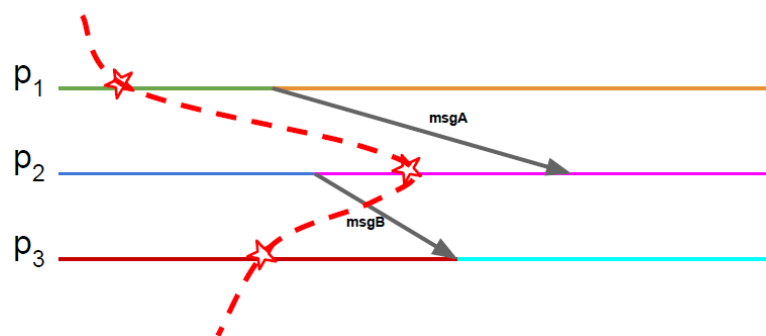
I messaggi al volo sono quelli che inviano un evento che appartiene al taglio corrispondente e ricevono informazioni fuori dal taglio.

Lo snapshot si ricorderà del messaggio inviato da p2 ma non di quello inviato da p1.

The obtained snapshot corresponds to the consistent cut shown below;

anyway, the overall system has not experienced such a state in any specific point in (global) time...

p_1 (green circle)
 p_2 (pink circle)
 p_3 (blue circle)
 $ch_{12} = \{\}$
 $ch_{13} = \{\}$
 $ch_{23} = \{msgB\}$
 $ch_{31} = \{\}$



Correttezza dell'algoritmo Chandy-Lamport

La correttezza di uno snapshot implica che esso corrisponda a un **taglio consistente** della computazione.

La dimostrazione si divide in due casi principali:

1. **Caso 1: a e b si trovano nello stesso processo.**

Questo caso è semplice e intuitivo. Se due eventi appartengono allo stesso processo e **a** precede **b**, allora **a** sarà necessariamente incluso nello snapshot se **b** lo è. Questo perché lo stato del processo viene registrato in modo sequenziale.

2. **Caso 2: a e b si trovano in processi diversi.**

Supponiamo che:

- **a** si trovi nel processo p_i e rappresenti l'invio di un messaggio m_x .
- **b** si trovi nel processo p_j e rappresenti la ricezione dello stesso messaggio m_x .

Se **b** è incluso nello snapshot (*cioè è un evento "pre-snapshot"*), allora il processo p_j non ha ancora ricevuto il messaggio di marker (**mrk**) al momento della ricezione di m_x . Questo implica che il processo p_i , che ha inviato m_x , non ha ancora inviato il marker al momento di **a**.

Poiché i canali di comunicazione sono FIFO, i messaggi arrivano nell'ordine in cui sono stati inviati. Quindi, se **b** è pre-snapshot, anche **a** deve esserlo.

10.2.2 Algoritmo Lai-Yang

L'algoritmo **Lai-Yang** è un esempio di approccio che non richiede l'ipotesi FIFO.

Garantisce che lo **snapshot catturato sia coerente**, includendo sia lo stato dei processi che i messaggi in transito.

Messaggi in transito

I messaggi normali **ricevuti dopo il marker**.

Stato locale di un canale

Definito come l'**insieme dei messaggi in transito** nel momento in cui lo snapshot viene catturato.

Lo stato di un canale c_{ij} , che collega il processo p_i a p_j , può essere calcolato come:

$$state(c_{ij}) = sent_msgs_i[j] \setminus received_msg_j[i]$$

In questo modo, anche se i messaggi non arrivano in ordine FIFO, possiamo ricostruire lo stato corretto del canale.

Funzionamento di base

Immaginiamo che i processi **cambino colore** quando fanno lo snapshot:

- Un processo è **VERDE** quando ancora non ha registrato il suo stato.
- Un processo diventa **ROSSO** quando registra il suo stato locale per lo snapshot.

To perform the local snapshot:

```
procedure takeSnapshot()  
  marked = true  
  <record local_state considering also all logged  
    outgoing and incoming messages >
```

Any process p sending out
any msg m from ch. c :

```
procedure wrappedSend(channel c, msg m)  
  m.color = green if not marked else red  
  send(c,m)  
  log(c,m) // keep record of outgoing msgs
```

Any process p receiving
any msg m on ch. c :

```
procedure wrappedReceive(channel c, msg m)  
  if msg.color == red then  
    takeSnapshot()  
  receive(c,m)  
  log(c,m) // keep record of incoming msgs
```

Ulteriore miglioramento

Se un processo non invia messaggi dopo aver registrato il proprio stato, è necessario attendere che tutti gli altri completino il loro snapshot. Una soluzione è introdurre un messaggio di controllo (*trigger*) che spinga gli altri processi a completare la registrazione.

Questo algoritmo utilizza un trucco per ottenere snapshot consistenti indipendentemente dall'ordine di consegna dei messaggi.

Esempio di funzionamento

Immaginiamo che i processi P1, P2 e P3 comunichino tra loro scambiandosi messaggi. Vogliamo **catturare uno snapshot consistente** dello stato globale del sistema.

1. Inizialmente tutti i processi sono **verdi** e questo colore viene inserito in ogni messaggio inviato.
2. P1 (*initiator*) decide di avviare lo snapshot.
3. P1 **colora il proprio stato in rosso**, indicando che sta catturando lo snapshot.
4. P1 invia un **messaggio marker** (*indicando l'inizio dello snapshot*) a P2 e P3.

Questo marker è come un segnale che dice: "*Fermati un attimo e scrivi su un foglio la tua situazione attuale!*"

5. P2 riceve il marker per la **prima volta**, colora il suo stato in **rosso** ed esegue lo snapshot.

Lo snapshot contiene:

- Stato locale del processo.
- I messaggi normali che ha ricevuto prima del marker.

6. P2 propaga il marker ai suoi vicini.

7. Supponiamo che prima di ricevere il marker, P3 riceva un messaggio normale, allora quest'ultimo diventa parte dello stato locale di P3.

8. P3 riceve il marker per la **prima volta**, colora il suo stato in **rosso** ed esegue lo snapshot.

9. P3 propaga il marker ai suoi vicini.

10. Quando tutti i processi hanno ricevuto il marker e registrato il proprio stato locale, lo snapshot è completato.

Elenchiamo alcune proprietà:

- I messaggi normali **ricevuti prima del marker**, sono considerati **parte dello stato locale**.
- I messaggi normali **ricevuti dopo il marker**, sono considerati **messaggi in transito** e sono registrati separatamente.

Quindi elenchiamo tutti i casi:

- Se un **processo ROSSO** riceve un messaggio da un **processo VERDE**, quel messaggio è **in transito** (*registrato separatamente*).

- Se un **processo VERDE** riceve un messaggio da un **processo ROSSO**, quel messaggio viene valutato normalmente (*registrati nello stato locale*).
- Se un **processo VERDE** riceve un messaggio da un **processo VERDE**, quel messaggio viene valutato normalmente (*registrati nello stato locale al momento dello snapshot*).
- Se un **processo ROSSO** riceve un messaggio da un **processo ROSSO**, quel messaggio non è considerato nello snapshot.

Perché elimina il vincolo FIFO?

Perché registra esplicitamente i **messaggi in transito**.

Quando un processo diventa **rosso** (cioè registra il suo stato), **segna separatamente i messaggi che riceve da processi ancora verdi**.

I messaggi provenienti da processi verdi devono essere registrati nello snapshot, anche se separatamente.

Questi messaggi vengono considerati "in transito" nello snapshot, indipendentemente dall'ordine di consegna nei canali.

Poiché le send saranno dentro il cut ma le receive no.

Questo significa che **anche se i messaggi arrivano fuori ordine**, l'algoritmo garantisce comunque uno snapshot consistente **senza bisogno di un ordine FIFO**.

10.3 Algoritmi di broadcast e convergecast

Nei sistemi distribuiti ogni processo raccoglie informazioni localmente, ma per prendere decisioni globali è necessario condividere e combinare tali dati.

Una strategia comune prevede l'utilizzo di due operazioni complementari: il **broadcast** e il **convergecast**.

Broadcast

Consiste nell'invio di un messaggio da un nodo sorgente a tutti gli altri nodi della rete.

Una tecnica semplice per implementarlo è il **flooding**, in cui ogni nodo inoltra il messaggio ricevuto a tutti i propri vicini.

Tuttavia, il flooding presenta due svantaggi rilevanti:

- Genera una notevole ridondanza, poiché lo stesso messaggio può essere trasmesso ripetutamente lungo gli stessi collegamenti.
- In reti di grandi dimensioni, il numero di messaggi può crescere in maniera esponenziale, causando un sovraccarico della rete.

Per superare questi problemi, si ricorre solitamente alla costruzione di uno **spanning tree**, ovvero un albero di copertura della rete che garantisce che ogni nodo riceva il messaggio una sola volta, riducendo così il numero di trasmissioni e l'uso delle risorse.

Convergecast

Operazione inversa al broadcast, prevede invece che, dopo aver ricevuto il messaggio, ogni nodo elabori le informazioni locali e le invii lungo i rami dell'albero verso il nodo sorgente.

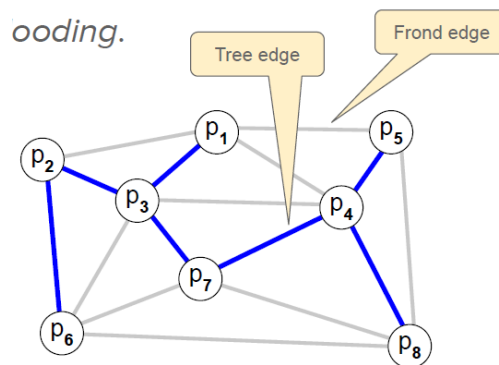
In questo modo **si aggregano i dati raccolti nei vari nodi**, facilitando decisioni globali basate sull'intero insieme delle informazioni del sistema.

È importante che il percorso inverso sia ben definito per evitare ritardi o perdite di dati; in alcuni contesti, i dati vengono elaborati lungo il percorso, ad esempio calcolando somme o medie.

10.3.1 Spanning Tree

Un approccio per fare broadcasting (e anche il convergecast).

Lo spanning tree (*albero di copertura*) è una struttura che, a partire da un grafo connesso, seleziona un sottoinsieme di collegamenti in modo da **connettere tutti i nodi senza formare cicli**.



In altre parole, è un albero che include ogni nodo della rete e garantisce che esista un percorso unico tra ogni coppia di nodi.

Questa proprietà lo rende particolarmente utile:

- **Efficienza:** Ogni nodo riceve un messaggio broadcast solo una volta, riducendo drasticamente il numero totale di messaggi.
- **Nessun ciclo:** I messaggi non si ripetono lungo lo stesso percorso, evitando ridondanze.

Proprietà dello spanning tree

Dobbiamo elencare alcune proprietà:

- Qualunque grafo G connesso (e senza direzione) con n nodi e E canali, ha almeno uno spanning tree.
- Qualunque spanning tree di G ha n vertici e $(n - 1)$ collegamenti.
- Uno spanning tree è sempre **minimamente connesso** (la rimozione di un qualsiasi collegamento comporta il non essere più connesso, e quindi neanche uno spanning tree).
- Uno spanning tree è **maximally acyclic**, ovvero aggiungere un qualsiasi collegamento comporta la creazione di almeno un loop.

Assumiamo che il canale di comunicazione tra due nodi è FIFO.

Edge dell'albero (spanning tree edges): Questi archi formano la struttura dello spanning tree e sono utilizzati per la comunicazione.

Fronde edge: Sono gli archi del grafo originale che non fanno parte dello spanning tree. Non vengono utilizzati per la comunicazione principale, ma possono servire come percorsi alternativi in alcune applicazioni avanzate o per garantire la connettività in caso di guasti.

Costruzione di uno spanning tree

Possiamo descrivere uno spanning tree attraverso una struttura dati, tenuta da ogni processo coinvolto (*ogni processo è un nodo del grafo*).

Ogni nodo nel grafo ha un **padre** (`var parent`) e una lista di **figli** (`List children`).

La struttura dati viene creata in maniera distribuita tramite un **algoritmo traversal**.

1. Il processo iniziale (*initiator*) avvia il processo di costruzione (*broadcast*) e diventa la **radice** dell'albero.
2. Invia un messaggio `forward` ad ogni suo vicino.

Adesso inizia la fase **broadcast**.

I `forward` vengono propagati per esplorare la rete.

3. Quando un nodo riceve il primo `forward`, identifica come padre il mittente di tale messaggio.
4. Inoltra il `forward` ai propri vicini.
5. ...
6. Una volta che un nodo ha inoltrato il messaggio a tutti i vicini e non ne riceve altri da processare, invia un `backward` al proprio genitore.

Adesso inizia la fase **convergecast**.

7. Il genitore identifica come figli i nodi che hanno inviato un `backward`.

Il `backward` segnala che il proprio ramo è stato completamente esplorato.

8. I `backward` si propagano lungo l'albero fino a raggiungere la radice, che potrà così determinare che la costruzione dell'albero è terminata.

Durante il convergecast, i messaggi di risposta vengono raccolti lungo il percorso inverso, tornando alla radice.

L'initiator è la radice dell'albero e non deve conoscere l'intero sistema, deve conoscere solo i suoi figli, ovvero i suoi vicini.

Messaggi di forward/backward e le fronde del grafo

Ogni arco non appartenente allo spanning tree (*fronda*) collega due nodi già raggiunti dall'esplorazione.

Come fanno a capire quali archi sono delle fronde?

I due nodi si scambiano il **forward**.

Però entrambi i nodi hanno già ricevuto un **forward** (dal loro rispettivo genitore) quindi capiscono che l'arco da cui hanno ricevuto il **forward** è ridondante.

Alla fine del broadcast, i due nodi si scambiano il **backward**, e con la ricezione di quest'ultimo i due nodi stabiliscono che l'arco è ridondante.

Quindi su ogni arco fronda, vengono scambiati 4 messaggi.

Esempio pseudocodice spanning tree

In figura abbiamo lo pseudocodice dello spanning tree, per ogni vicino, un processo invia un **fwd** (*forward*), questo messaggio contiene informazioni anche del mittente.

L'initiator esegue il seguente codice:

```
procedure startSpanning()
  parent=pid //pid: local proc. index
  children=empty; expected = n_neighbors
  foreach p in neighbors send(p, fwd)
```

Quando un processo riceve un **fwd**, esegue il seguente codice:

```
procedure onForward(proc sender)
  if parent == 1 then //first fwd received
    parent=sender; children=empty; expected=n_neighbors-1
    if expected == 0 then //reached a "dead end"
      send(parent, bck(list( (pid, myval) ))
    else
      foreach p in neighbors\sender send(p, fwd)
  else send(sender, bck(empty)) //ie "I'm not your child"
```

Il processo controlla se ha già un padre, se non è così (è il primo **fwd** che riceve) il sender viene preso come padre.

Se ha già un padre (ha già ricevuto un **fwd** da qualcuno) allora invia immediatamente un **bck** e il link diventa una fronda.

Il processo assegna **expected = n_neighbors - 1** (esclude il padre).

- Se **expected == 0** : invia un **bck** al padre contenente i PID dei figli.
- Altrimenti: invia un **fwd** ai vicini (esclude il padre).

Quando un processo riceve un **bck**, esegue il seguente codice:

```
procedure onBackward(proc sender, msg m)
  expected -= 1; val_list.append(m.val_list)
  if m.val_list not empty then children.append(sender)
  if expected == 0 then //no more bck from children
    val_list.append( (pid,myval) )
    if parent != pid then send(parent, bck(val_list))
  else <termination; use val_list>
```

Decrementa **expected** e aggiunge il PID del sender alla lista dei figli.

Se il **bck** è vuoto, significa che il link è una fronda.

Se **expected** = 0, il processo ha ricevuto il **bck** da tutti i figli e quindi invia il **bck** al padre.

Complessità in termini di messaggi dello Spanning Tree

Sia E il numero di archi del grafo e n il numero di nodi.

- **Numero di archi dell'albero** (*spanning tree edges*): $S = (n - 1)$.
- **Numero di archi non appartenenti all'albero** (*frond edge*): $F = (E - S)$.

In ogni arco dello spanning tree si inviano 2 messaggi (1 **forward** + 1 **backward**) →
 $N_S = 2S = 2(n - 1)$.

In ogni fronda si inviano 4 messaggi (2 **forward** + 2 **backward**) →
 $N_F = 4F = 4(E - S) = 4(E - (n - 1)) = 4E - 4(n - 1)$.

Numero totale di messaggi inviati:

$$N_T = N_F + N_S = 4E - 2(n - 1) \sim O(E).$$

Con $E \in [n - 1, \frac{n(n-1)}{2}]$ quindi nel **caso peggiore** $N_T \sim O(n^2)$.

Complessità temporale dello Spanning Tree

Sia D il diametro della rete, ovvero la distanza (in numero di hop) massima tra due nodi della rete.

Nella costruzione dello spanning tree, **tutti i processi vengono raggiunti e un backward ritorna all'iniziatore.**

Nel **caso peggiore**, un **backward** dovrà percorrere proprio il percorso massimo, quindi: $O(D)$.

10.4 Algoritmi di mutua esclusione

Serve per garantire che solo un processo alla volta possa accedere a una risorsa condivisa.

L'accesso ad una risorsa condivisa avviene in una specifica sezione del codice detta "sezione critica" (**Critical Section, CS**).

Una sezione critica è delimitata dalle operazioni `req_access()` (*richiesta di accesso*) e `rel_access()` (*rilascio della risorsa*).

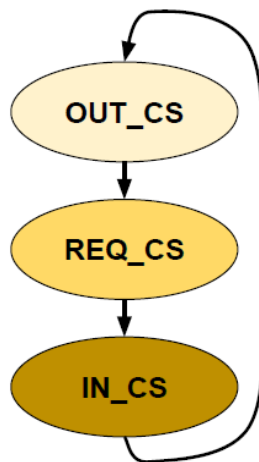
Safety e liveness

Un sistema ha delle proprietà che possono essere suddivise in **safety** e **liveness**.

- **Sicurezza (ME1)**: Al massimo un processo può essere nella CS contemporaneamente.
- **Liveness (ME2)**: Ogni richiesta di accesso o rilascio della CS verrà prima o poi soddisfatta (*non sappiamo quando un processo entrerà nella sezione critica, però sicuramente in futuro ci accederà*).

Stati di un Processo rispetto alla Sezione Critica

- **OUT_CS**: Il processo **non ha ancora richiesto di entrare** nella CS.
- **REQ_CS**: Il processo **ha richiesto l'accesso ma non è ancora entrato** nella CS.
- **IN_CS**: Il processo **è entrato** nella CS.



Tipologie di algoritmi di mutua esclusione

1. **Algoritmo con permessi Individuali**: Ogni processo concede il permesso di accedere alla CS solo per sé stesso.
2. **Algoritmi basati su Token**: Un unico token è condiviso tra tutti i processi e solo il quello che possiede il token può accedere alla CS.

3. **Algoritmo con permessi con Arbitro:** Esiste almeno un processo (*arbitro*) che risolve i conflitti tra le richieste. I permessi vengono restituiti all'uscita dalla CS.

10.4.1 Algoritmo Ricart-Agrawala

Questo algoritmo appartiene alla categoria dei **permessi individuali** e consente a un solo processo alla volta di accedere alla CS (ME1).

Si basa sull'invio di messaggi per sincronizzare i processi coinvolti (*ovvero interessati alla CS in questione*).

L'idea di base è la seguente: per entrare in una CS, un processo deve chiedere il permesso a tutti gli altri processi coinvolti mediante un messaggio **REQ** e aspettare il relativo **OK** da ogni altro processo.

Risoluzione dei conflitti

Ovviamente possono esserci conflitti con i **REQ** (*due processi lo mandano nello stesso momento*).

Risolviamo con i timestamp di Lamport (*totally-ordered*) e con i PID dei due processi.

Alla **REQ** viene inserito il timestamp e la richiesta con il timestamp minore ha priorità.

$$(t_{sa}, i) < (t_{sb}, j) \Leftrightarrow (t_{sa} < t_{sb}) \vee [(t_{sa} = t_{sb}) \wedge (i < j)]$$

In caso di timestamp sincronizzati, viene fatto un confronto dei PID.

Implementazione dell'algoritmo

Per inviare la **REQ**, deve essere eseguita la funzione:

```
procedure req_access() // pid: local pid
  mystate = REQ_CS
  pending_oks = N-1 //all the others
  myreq_lt = myclock+1 // l.timest. for my req
  req.ts = (myreq_lt, pid) //set msg timest.
  foreach p in req set send(p, req)
  wait_until (pending_oks == 0 )
  mystate = IN_CS // then, CS
```

Tenere traccia dello stato corrente del processo è importante perché in base allo stato vengono tenuti diversi comportamenti.

Semplificazione: la variabile interna **myclock** rappresenta il Lamport timestamp della richiesta del processo.

Se ci sono **n** processi, allora un processo deve collezionare **n-1 OK**.

Ogni volta che un **OK** viene ricevuto, la variabile **pending_oks** viene decrementata (*la variabile tiene conto degli OK che devono ancora essere ricevuti*).

La funzione **onReq** viene eseguita ogni volta che viene ricevuto un **REQ**.

Possono esserci diversi processi che richiedono l'accesso concorrentemente allo stesso tempo; per questo motivo nella funzione **onReq** il processo deve aggiornare il suo clock interno **myclock** secondo la regola dei timestamp di lamport.

```
procedure onReq(proc sender, msg m)
  myclock = max(myclock, m.ts)
  my_prior = (mystate != OUT_CS) and (myreq_lt,pid) < m.ts
  if not my_prior then send(sender, ok)
  else req_delayed.append(sender)
```

Se il processo ha una priorità minore rispetto ad altri, invia lo **OK** per avvertire chi ha una priorità maggiore.

Il processo invia la **OK** in una (or) di queste due condizioni:

- Ha già eseguito la CS, ovvero si trova nello stato **OUT_CS**.
- **Conflitto di REQ**: Se il processo ha una priorità minore (*quindi un timestamp maggiore*) $(t_{sa} < t_{sb}) \vee [(t_{sa} = t_{sb}) \wedge (i < j)]$

Altrimenti **non invia OK** (*l'altro processo non può andare avanti*) e mette il **sender** in una coda per tenere traccia dei processi a cui il processo dovrà poi inviare **OK** quando uscirà dalla CS.

Quindi prima o poi il **sender** riceverà **OK** e verrà sbloccato (**ME2**).

```
procedure onOk(proc sender, msg m)
  pending_oks = pending_oks-1
```

La funzione **onOk** viene eseguita ogni volta che viene ricevuto un **OK**, semplicemente decrementa il valore di **pending_oks** (*il numero di OK necessari ad entrare nella CS*).

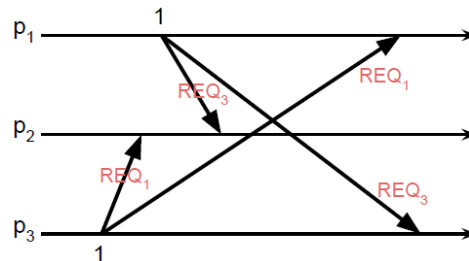
In uscita dalla CS il processo esegue:

```
procedure rel_access()
  mystate = OUT_CS
  foreach p in req_delayed: send(p, ok)
  req_delayed = empty
```

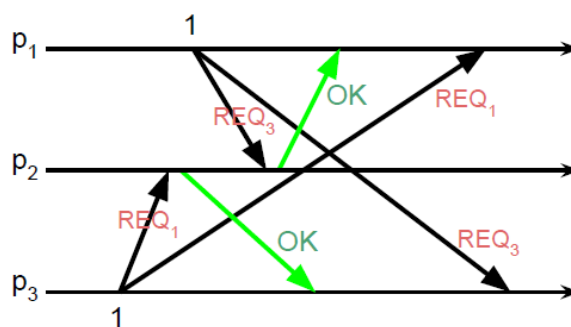
Dove invia **OK** a tutti i processi che ha messo in attesa precedentemente.

Esempio di esecuzione dell'algoritmo

In questo esempio osserviamo cosa succede se p_1 e p_3 provano ad entrare nella sezione critica in modo concorrente.



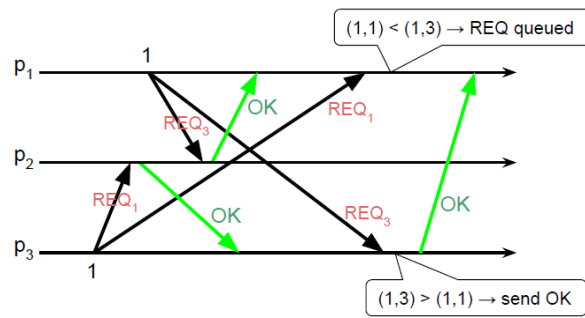
1. p_2 riceve REQ1 da p_3 .
2. p_2 risponde con OK, perchè p_2 non è interessato nell'ingresso alla CS (OUT_CS).
3. p_2 riceve REQ3 da p_1 .
4. p_2 risponde con OK, perchè p_2 non è interessato nell'ingresso alla CS (OUT_CS).



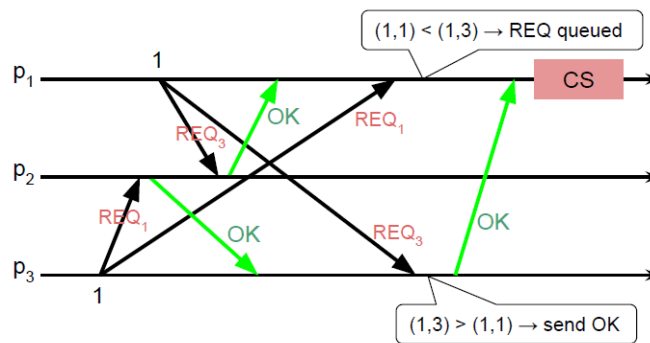
5. p_1 riceve REQ1 da p_3 .
6. p_1 **non** risponde con OK, perché:
 - p_1 è interessato nell'ingresso alla CS (REQ_CS).
 - p_1 ha priorità maggiore $(1, 1) < (1, 3)$.

p_1 mette p_3 nella sua coda.

7. p_3 riceve REQ3 da p_1 .
8. p_3 risponde con OK, perchè p_1 ha priorità maggiore $(1, 1) < (1, 3)$.

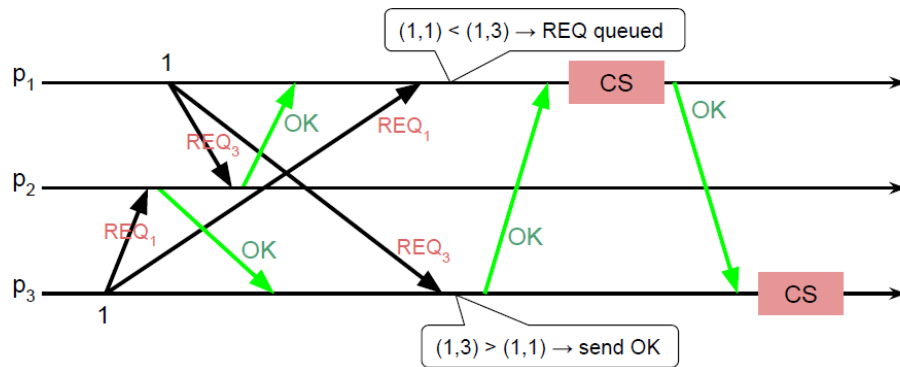


9. p1 esegue la sezione critica (ha ricevuto i due OK che gli servivano).



10. p1 termina l'esecuzione nella sezione critica, invia OK a p3.

11. p3 può entrare nella sezione critica ed iniziare la sua esecuzione.



Complessità del numero di messaggi dell'algoritmo Ricart-Agrawala

Ogni ingresso nella sezione critica richiede:

- L'invio di $n - 1$ REQ.
- La ricezione di $n - 1$ OK.

Con n processi si avranno $2(n - 1)$ messaggi.

Complessità del tempo dell'algoritmo Ricart-Agrawala

Supponendo di avere uno **spanning tree** per il broadcast dei REQ e il convergecast degli OK.

Partendo da un qualsiasi initiator, tutti i processi devono essere raggiunti, quindi nel caso peggiore **il costo in termini di tempo dipende dal percorso più lungo possibile** (*numero di hop*) tra due processi generici nella rete.

Quindi, dipende dal diametro D della rete (*cioè il numero massimo di hop tra due nodi*).

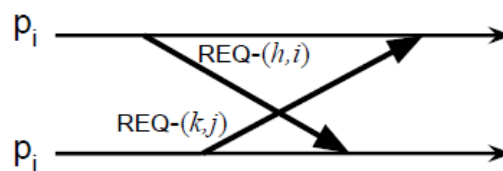
Complessità Temporale: $O(D)$.

ME1 - Correttezza dell'Algoritmo Ricart-Agrawala

Dimostriamo (*per assurdo*) che l'algoritmo garantisce **ME1**: "al massimo un processo alla volta può essere nella sezione critica".

Cerchiamo di trovare uno scenario in cui due processi siano nella CS contemporaneamente. Perciò illustriamo due possibili scenari, entrambi contraddittori:

Scenario 1: I processi inviano la **REQ** prima di ricevere quella dell'altro.



Supponiamo che il timestamp della **REQ** di $p_i = (h, i)$ e quello della **REQ** di $p_j = (k, j)$.

Supponiamo che p_i abbia priorità maggiore a p_j , ovvero $p_i < p_j \Leftrightarrow (h, i) < (k, j)$.

1. p_j riceve la **REQ** da p_i .

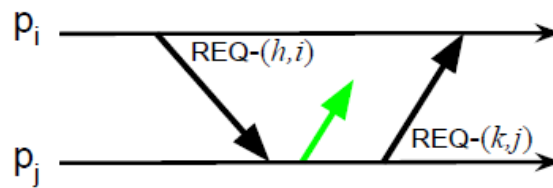
Secondo le regole dell'algoritmo, p_j confronta i due timestamp.

2. Poiché $(h, i) < (k, j)$, p_j invia un messaggio di **OK** a p_i .
3. Tuttavia, p_i riceve successivamente la **REQ** di p_j .
4. Poiché $(h, i) < (k, j)$, p_i **non invia OK** a p_j , dato che p_i è già in attesa di entrare nella CS e ha la priorità.

Quindi p_j non può ottenere il consenso necessario per entrare nella CS.

Questo contraddice l'ipotesi iniziale che vede entrambi i processi dentro la CS.

Scenario 2: Un processo invia **OK** prima di inviare **REQ**.



Supponiamo che p_j abbia inviato **OK** a p_i prima di inviare la propria richiesta (**REQ**).

Supponiamo che p_i abbia priorità maggiore a p_j , ovvero $p_i < p_j \Leftrightarrow (h, i) < (k, j)$.

1. p_j riceve la **REQ** di p_i con timestamp h , aggiorna il proprio clock logico
 $myclock_{(j)} = \max(myclock_{(j)}, h)$.

Questo garantisce che $myclock_{(j)} \geq h$.

2. p_j invia **OK** a p_i , e quest'ultimo entra nella CS.
3. p_j invia la propria **REQ**, il timestamp associato sarà $k = (myclock_{(j)} + 1) > h$.
4. p_i riceve la **REQ** di p_j con timestamp (k, j) , confronta i timestamp e capisce di avere maggiore priorità $(h, i) < (k, j)$.
5. p_i non invia **OK** a p_j , perché il proprio stato non è **OUT_CS** e ha priorità maggiore.
6. Pertanto, p_j non ottiene il consenso necessario per entrare nella CS.

Questo contraddice l'ipotesi iniziale che vede entrambi i processi dentro la CS.

Entrambi gli scenari portano a una contraddizione, dimostrando che non è possibile che due processi siano contemporaneamente nella CS. **Mutua esclusione garantita.**

ME2 - Assenza di Deadlock e Starvation

L'algoritmo garantisce ME2 (**liveness**), ovvero:

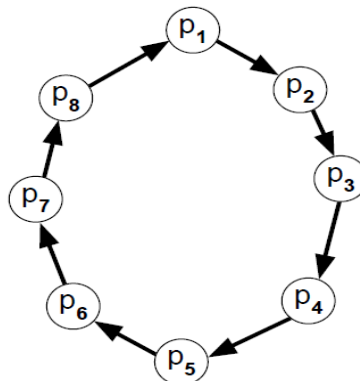
- **Assenza di deadlock:** Non si possono verificare cicli di attese reciproche. Ogni richiesta viene processata e le risposte vengono inviate seguendo un ordine deterministico (*basato sui timestamp*).
- **Assenza di starvation:** Ogni processo, una volta inviata la propria richiesta, riceverà eventualmente tutte le risposte necessarie (*OK*) e potrà accedere alla CS. Questo è garantito dall'ordine totale imposto sui timestamp.

In sintesi, ogni processo attende al massimo il tempo necessario affinché tutti gli altri processi con richieste precedenti completino il loro ciclo nella CS.

In questo caso è importante rispettare dei pattern, perché se non vengono rispettati non è garantito che le cose vengano eseguite nel modo giusto.

10.4.2 Token Ring Mutex

Vediamo un esempio di algoritmo di mutua esclusione basato su un token. I processi sono organizzati (*logicamente*) in un anello diretto.



Un **token** (*messaggio speciale e unico nella topologia*) viene passato lungo l'anello, permettendo solo al possessore di accedere alla CS.

- **req_access**: Cambia lo stato del processo a **REQ_CS**.
- **onToken()**: Se il processo è in **REQ_CS**, entra nella CS; altrimenti passa il token al successivo processo.
- **rel_access()**: Cambia lo stato del processo a **OUT_CS** e passa il token al prossimo processo nell'anello.

Vediamo gli standard di **correttezza** di questo sistema:

- **ME1**: Garantita perché **esiste un solo token** e quindi solo un processo alla volta può possederlo e quindi accedere alla CS.

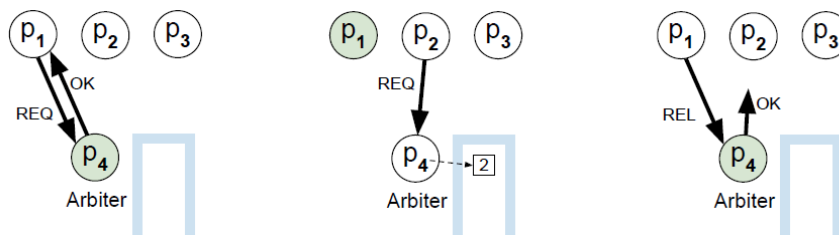
- **ME2**: Garantita dal **passaggio continuo del token**, di sicuro un nodo prima o poi riceverà il token ed eseguirà la sezione critica.

Ovviamente la **ME2** è valida solo se la durata della sezione critica è finita.

10.4.3 Mutex Centralizzato

Nel sistema è presente un processo speciale detto **arbitro** che gestisce le richieste per la CS da parte di tutti gli altri processi.

1. I processi inviano le richieste **REQ** di entrata nella sezione critica all'arbitro.
2. Se la risorsa è :
 - **Occupata**: La richiesta viene inserita in una coda d'attesa.
 - **Libera**: La richiesta viene subito soddisfatta e l'arbitro risponde con **OK**.
3. L'arbitro, secondo una sua politica (*fairness*), pesca le richieste dalla coda conferendo il diritto di entrare nella CS, inviando **OK** al processo pescato.



Questo sistema è molto semplice ma l'arbitro rappresenta un **single point of failure**.

10.5 Classificazione dei Fallimenti

Fino ad ora abbiamo supposto che i processi non abbiano problemi. Ma nel mondo reale i processi possono incontrare svariati problemi.

I fallimenti vengono gestiti secondo un modello e a seconda del modello, uno o più componenti ne sono affetti.

Fail-stop

Il modello più utilizzato.

Il processo si blocca e rimane bloccato. Gli altri processi possono rilevare questo stato.

Influenza un processo.

Crash

Il processo si blocca e rimane bloccato. Gli altri processi potrebbero non essere in grado di rilevare questo stato.

Influenza un processo.

Omission (canale)

Il messaggio viene perso.

Influenza un canale e anche il processo ricevente in caso non potesse gestire questo errore.

Byzantine

L'errore è gestito secondo comportamenti arbitrari.

Influenza un processo e il canale di comunicazione.

10.5.1 Modelli sincroni

Si basano su una strategia in cui le operazioni vengono eseguite in modo sequenziale e bloccante, ovvero un'operazione non procede fino a quando non è stato gestito un eventuale errore o ricevuta una conferma dal nodo remoto.

Chiamata bloccante

Un nodo invia una richiesta a un altro nodo e **attende la risposta** prima di procedere.

Slot temporali

Se il mittente non riceve una risposta entro un certo periodo di tempo (*timeout*), può effettuare un nuovo tentativo.

Tiene conto di una sequenza di slot temporali fissi secondo il tempo globale, e ogni operazione deve essere eseguita in ciascun processo entro uno slot.

Irrealisticità

Questo modello è chiaramente irrealistico perché richiede la sincronizzazione dei nodi.

10.6 Algoritmi per l'elezione di un leader

I processi scelgono uno di loro che da ora in poi dovrà svolgere dei compiti speciali, spesso relativi alla coordinazione.

Ruoli durante l'elezione

Per scegliere un leader nel sistema, ogni processo p_i ha un identificatore unico `UIDi`.

Durante un'elezione, un processo può avere due ruoli:

- **Partecipante**: Coinvolto attivamente nell'elezione.
- **Non partecipante**: Non coinvolto o ha terminato la partecipazione.

Un processo in alcuni casi può avere le seguenti variabili:

- `leaderi`: Contiene l'identificatore del leader.
- `donei`: Booleano per indicare se l'elezione è terminata.

Proprietà

- **LE1 (Safety)**:
 - Viene eletto al massimo un solo leader non guasto (*non-crashed*).
 - Gli stati `electedi` e `donei` rimangono stabili.
 - Alla terminazione, il leader è stato eletto.
- **LE2 (Liveness)**:
 - Un processo verrà, prima o poi, eletto.
 - L'elezione è nota a tutti i processi non guasti (*non-crashed*).

Nota importante: È stato dimostrato che in un sistema anonimo (senza `UID`) e con una topologia regolare di rete, il problema dell'elezione del leader non è risolvibile.

10.6.1 Algoritmo di Chang & Roberts (C&R)

Presuppone una topologia ad **anello unidirezionale**, senza che i canali siano FIFO.

Il leader sarà il processo con **UID** massimo.

Funzionamento

1. L'elezione viene avviata da un initiator che invia un messaggio **ELECTION** lungo l'anello.
2. Ogni processo che vede arrivare un **ELECTION** invia a sua volta un **ELECTION** con il proprio **UID**.

Il messaggio **ELECTION** contiene lo **UID** del mittente.

3. Ogni processo blocca ogni **ELECTION** con un **UID** inferiore **al proprio** e inoltra quelli con **UID** maggiore.

Data la regola (2) solo lo **UID** massimo può completare un giro dell'anello e vincere.

4. Il processo che vede ritornare il proprio **ELECTION** capisce di essere il leader.

Non importa quanti initiator ci sono nel sistema, il leader stesso è quello che scopre di essere il leader.

5. Il leader invia un messaggio **LEADER** per informare gli altri.

```
procedure startElection()
  mystate = participant
  election.UID = myUID
  send(election)
```

startElection() è il metodo che consente di avviare il meccanismo di elezione, dobbiamo mantenere l'informazione **mystate** per capire se partecipiamo all'elezione oppure vogliamo inviare solo un messaggio.

```
procedure onElection(msg m)
  if myUID < m.UID then
    mystate = participant
    send(m)
  elif myUID > m.UID then
    if mystate == nonparticipant
      mystate = participant
      election.UID = myUID
      send(election)
  elif myUID == m.UID then
    leader.UID = myUID
    send(leader); elected = true
```

onElection() viene utilizzata per controllare se il processo è il leader o meno.

Nel metodo `onElection()` se il nostro `UID` è minore di un `UID` ricevuto allora passiamo l'elezione ad un nostro vicino. In caso contrario il nostro processo è un possibile leader, ci sono due casi possibili se non è un partecipante viene settato come partecipante.

Se il processo riceve un messaggio con il proprio `UID`, significa che quel messaggio che ha inviato ha completato il giro, quindi il processo è il leader, è l'unico che conosce di essere il leader, allora invia un `LEADER` message per informare gli altri che lui è il leader.

```
procedure onLeader(msg m)
  leader = m.UID; done = true
  if myUID != m.UID then
    elected = false
    send(m)
```

Se però il processo riceve un messaggio `LEADER` allora viene eseguita la `onLeader()`: se lo `UID` ricevuto è diverso dal proprio, invia il messaggio al vicino, altrimenti non fa nulla.

Ovvero appena il `LEADER` torna al leader il giro si ferma.

Complessità del numero di messaggi

Caso migliore:

Vi è un **unico initiator** che avvia l'elezione, ed è anche il processo con il `UID` più alto.

- `ELECTION` viaggia una sola volta intorno all'anello (*ogni processo lo riceve e lo inoltra*), richiedendo n messaggi.
- `LEADER` viaggia una sola volta intorno all'anello (*ogni processo lo riceve e lo inoltra*), richiedendo n messaggi.

Costo totale: $N_m = n + n \sim O(n)$.

Caso peggiore: Tutti i processi sono initiator e i `UID` sono ordinati in modo decrescente sull'anello.

- Il processo con il `UID` massimo invia n messaggi `ELECTION` (*uno per ciascun processo*)
- Quello con il secondo `UID` invia $n - 1$ messaggi (*il futuro leader è l'unico che non lo inoltra*)
- Quello con il terzo `UID` invia $n - 2$ messaggi (*il futuro leader e il precedente nodo sono gli unici non lo inoltrano*)
- ...
- L'ultimo processo che invia un solo messaggio.

Il numero di `ELECTION` è: $n + (n - 1) + (n - 2) + \dots + 1 = \frac{n(n+1)}{2}$

Successivamente, il leader invia n messaggi `LEADER`.

Costo totale: $O(n^2)$.

Caso medio:

L'elezione viene avviata da un sottoinsieme di processi in modo casuale, la complessità media è $O(n \log n)$.

Questa stima si basa sull'analisi probabilistica del numero di messaggi **ELECTION** e **LEADER**, che dipende dalla distribuzione degli **UID** e dal numero di iniziatori.

Complessità del tempo

Caso migliore

Un unico initiator è quello con il **UID** più alto invia un solo giro di messaggi **ELECTION** intorno all'anello.

Questo richiede $O(n)$ in termini di tempo (*una comunicazione da processo a processo su un anello*).

Caso peggiore

Initiator è il processo con il secondo **UID** più alto e si trova immediatamente prima del processo con il **UID** massimo nell'anello (quindi sarà l'ultimo a ricevere **ELECTION**).

Il processo con il secondo **UID** più alto invia $n - 1$ messaggi **ELECTION** e poi, quando il messaggio raggiunge il processo con il **UID** massimo, quest'ultimo invia un altro giro di n messaggi.

Tempo totale: $(n - 1) + n \sim O(n)$.

Gestione dei crash durante l'elezione

Quando i processi possono fallire, l'algoritmo deve gestire in modo robusto queste situazioni, garantendo comunque che il leader venga eletto.

Ogni processo ha una forma differente del problema, il problema non consiste nel trovare chi ha **UID** maggiore, ma di trovare quale processo correntemente attivo ha lo **UID** più elevato.

Ipotesi:

1. **Consegna affidabile dei messaggi:**
 - Ogni messaggio inviato da un processo corretto viene consegnato.
2. **Sistema sincrono:**
 - Esiste un limite superiore noto sul tempo di latenza dei messaggi.
3. **Rilevamento dei guasti:**
 - I fallimenti dei processi possono essere rilevati tramite meccanismi come i **timeout**.

Presupposti:

1. Ogni processo conosce gli **UID** degli altri processi. Ogni nodo fa sapere il proprio **UID** agli altri.

2. Ogni processo può comunicare con gli altri.
3. Il leader deve essere il processo con il **UID** più alto tra i processi attivi (*cioè non guasti*).

Problematiche principali:

1. **Processi guasti durante l'elezione:**
 - Se un processo si arresta mentre inoltra un messaggio, è necessario che i processi rimanenti rilevino il fallimento e riprendano l'elezione.
2. **Dipendenza dalla conoscenza dei **UID**:**
 - Per garantire che il leader sia il processo con il **UID** massimo tra i processi attivi, è necessario mantenere una mappatura aggiornata dei processi attivi (*quelli che non sono falliti*).

Gestione dei crash:

- Quando un processo rileva che il leader corrente è guasto (*ad esempio tramite un timeout*), può:
 1. Avviare una nuova elezione inviando un messaggio **ELECTION** agli altri processi.
 2. Se altri processi rispondono, seguirà la logica dell'algoritmo per eleggere il processo con il **UID** più alto tra quelli attivi.
 3. Se non riceve risposte, si autoproclama leader.

In base all'istante di tempo, il leader può essere differente a causa dei crash.

Caratteristica	C&R Standard	C&R con Crash
Robustezza ai guasti	Non prevista	Rilevamento dei guasti con timeout
Presupposti	Topologia ad anello	Topologia più generale, processi connessi
Leader scelto	UID massimo	UID massimo tra i processi attivi
Complessità messaggi	$O(n)$ (caso migliore), $O(n^2)$ (caso peggiore)	Dipende dai fallimenti (<i>può aumentare</i>)
Sicurezza (LE1)	Garantita	Garantita
Vitalità (LE2)	Garantita	Garantita (<i>se il sistema non è troppo guasto</i>)

10.6.2 Bully Algorithm

L'algoritmo del Bullo è una tecnica di **elezione di un leader** in sistemi distribuiti.

Viene usato per scegliere un processo coordinatore tra un gruppo di processi, anche in presenza di fallimenti.

Si basa sull'assunzione che ogni processo abbia un **UID** unico e che i processi conoscano gli **UID** di tutti gli altri.

Tipi di messaggi

L'algoritmo utilizza tre tipi principali di messaggi:

- **ELECTION:**
 - Serve per annunciare l'avvio di un'elezione e viene inviato da un qualsiasi processo initiator.

Una nuova elezione inizia quando il leader corrente viene decretato fallito.

- **ANSWER:**
 - È una risposta a un messaggio **ELECTION**.
 - Indica che il destinatario è vivo e disponibile a partecipare al processo di elezione.
- **LEADER:**
 - Serve per annunciare che un processo è stato eletto leader.

Meccanismo di base

1. Un processo p rileva che il leader corrente è inaccessibile (*ad esempio, tramite un timeout*).

Timeout: se il leader non risponde dopo un certo intervallo, allora è fallito.

2. p invia **ELECTION** a tutti i processi con **UID superiore al proprio**.
 - I processi con **UID** maggiore rispondono con **ANSWER**.
 - I processi con **UID** minore saranno informati tramite un altro messaggio.

I processi con **UID** maggiore, dopo aver risposto, avviano un'elezione propria (*partono dal punto 1*).

3. Se p **riceve ANSWER**, attende per un tempo T' addizionale per un **LEADER** proveniente dal nuovo leader.
 - Se T' scade, p avvia un nuovo turno di elezione (*si riparte dal punto 1*).
4. Se p **non riceve ANSWER** entro un certo intervallo di tempo T , significa che nessun processo con **UID** superiore al suo è attivo (tolleranza ai guasti) e quindi lui sarà il leader.

- p si proclama leader e invia **LEADER** a tutti i processi con **UID** inferiore.
5. Quando un processo riceve un messaggio **LEADER** da un processo con **UID** superiore, aggiorna la propria variabile locale **leader** per indicare il nuovo leader.

Scalabilità e semplicità

Inviare il messaggio solo ai processi con **UID** superiore riduce il numero complessivo di messaggi rispetto a un approccio in cui ogni processo lo invia a tutti gli altri.

Comportamento da bullo:

Non importa che il leader corrente è ancora attivo, quando un nuovo processo si avvia e possiede il **UID** più alto, può decidere di autoproclamarsi leader senza verificare lo stato del leader attuale.

Questo comportamento "aggressivo" ha ispirato il nome dell'algoritmo.

Proprietà dell'algoritmo - Safety (LE1)

Non possono esistere due leader attivi contemporaneamente, perché i processi con **UID** inferiore cedono sempre al leader con **UID** superiore.

Proprietà dell'algoritmo - Liveness (LE2)

L'algoritmo garantisce che un leader venga sempre eletto, purché i messaggi vengano consegnati correttamente e il sistema non sia completamente guasto.

Esempio di esecuzione dell'algoritmo

In questo esempio i cerchi rappresentano dei processi, quelli **verdi** sono attivi, quelli **grigi** no.

Supponiamo che inizialmente il **processo grigio** fosse il leader, però dopo un certo tempo, il **primo processo verde** contatta quello **grigio** e scopre che non è più attivo.



Allora il primo processo contatta solo i processi con **UID** maggiore del proprio (*il secondo ed il terzo*) inviando **ELECTION**.



Il secondo ed il terzo processo rispondono al primo con un messaggio **ANSWER**.



Ma questi due processi non sanno chi è attivo e chi no.

Allora inviano i messaggi **ELECTION** ai processi alla loro destra (*terzo e quarto*) per capire chi è il leader.



Il terzo processo non riceve risposta dal quarto, mentre il secondo riceve un **ANSWER** dal terzo.



A questo punto il terzo processo è il leader (*non ha ricevuto nessun ANSWER*), quindi vengono informati gli altri processi attivi (*primo e secondo*) che lui è il leader con un **LEADER**.



Quando il processo inattivo torna attivo, non conosce chi è il leader, quindi invierà un **ELECTION** e inizierà una nuova elezione.

Complessità - Numero di messaggi

Caso migliore: L'initiator è il processo con il **UID** più alto.

Questo processo invia $n - 1$ messaggi **LEADER** per notificare il proprio ruolo a tutti gli altri processi. Non abbiamo messaggi **ELECTION**.

Complessità: $N_m = n - 1 \sim O(n)$.

Caso peggiore: L'initiator è il processo con il **UID** più basso.

Deve inviare un **ELECTION** a tutti i processi con **UID** superiore, quindi $N_{E1} = n - 1$.

Ogni processo superiore (tutti gli altri) avvia a sua volta un'elezione:

$$N_{E2} = n - 2 + N_{E3} = n - 3 + \dots + N_{En} = 0$$

Complessità: $N_m \sim O(n^2)$

Complessità - Tempo

Caso migliore: L'initiator è il processo con il **UID** più alto.

L'elezione richiede un solo giro di messaggi **LEADER**, completandosi in $O(n)$.

Caso peggiore: L'initiator è il processo con il **UID** più basso.

Richiede più round di comunicazione per propagare i messaggi **ELECTION** e aspettare risposte, completandosi in $O(n^2)$.

Vantaggi e svantaggi

- **Semplice** da implementare in sistemi distribuiti.
- Garantisce **LE1** e **LE2**.
- Supporta il **rilevamento di guasti** tramite timeout.
- **Inefficiente con molti processi**, a causa della complessità $O(n^2)$ nel caso peggiore.
- La strategia "da bullo" può causare **elezioni non necessarie**, aumentando il carico di rete.

10.7 Multicast

10.7.1 Le basi della comunicazione di gruppo

La **comunicazione di gruppo** è un paradigma fondamentale nei sistemi distribuiti, dove i processi sono organizzati in **gruppi**.

Invece di indirizzare messaggi a un singolo processo, si inviano messaggi a tutti i membri di un gruppo (*broadcast*) oppure ad un insieme dei membri di un gruppo (*multicast*).

Le operazioni principali della comunicazione di gruppo sono due:

`Xmulticast(group g, msg m)`

Indica l'invio di un messaggio `m` a tutti i membri del gruppo `g`.

La `X` è un segnaposto che specifica il tipo di multicast (es. *Basic*, *Reliable*, *FIFO*, ecc.), che varia in base alle garanzie richieste sull'ordinamento dei messaggi.

`Xdeliver(msg m)`

Quando un messaggio arriva a destinazione, viene "*consegnato*" al processo ricevente.

Può essere possibile ottenere il mittente del messaggio dalla ricezione di `m`.

Si utilizza il termine `deliver` invece di `receive` perché:

- I messaggi possono arrivare fuori ordine a causa di ritardi nella rete.
- I sistemi usano una **hold-back queue** per riordinare i messaggi in base a regole specifiche prima della consegna.

Incoming MSG → HOLD_BACK

Quando un messaggio arriva, viene inizialmente inserito nella **hold-back queue**.

Questa coda serve per trattenere i messaggi temporaneamente, **consentendo al sistema di ordinarli o verificarne la validità** prima che siano elaborati.

HOLD_BACK → DELIVERY

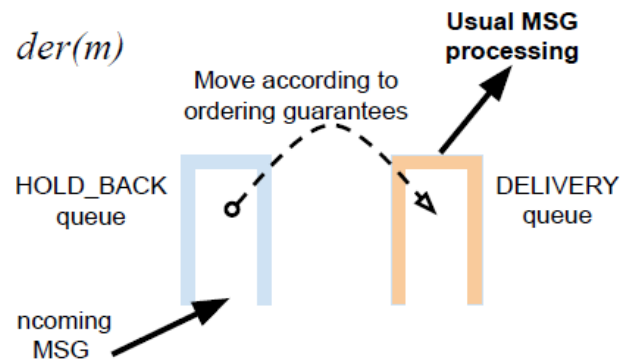
I messaggi vengono spostati alla delivery queue una volta che soddisfano le **garanzie di ordinamento**.

Le garanzie di ordinamento possono dipendere da algoritmi specifici, come il mantenimento dell'ordine causale o totale nei sistemi distribuiti.

DELIVERY → Usual MSG processing

Dopo essere stati riordinati e validati nella delivery queue, i messaggi vengono consegnati al livello applicativo per la normale elaborazione ("usual MSG processing").

Questo garantisce che i messaggi arrivino al livello applicativo nell'ordine previsto, anche se la rete li ha consegnati in modo disordinato.



Un trucco simile è stato visto nel Lai-Yang algorithm.

10.7.2 BMulticast - Basic Multicast

La forma più semplice di comunicazione multicast, con poche garanzie:

I processi possono fallire solo a causa di **crash** (*nessun comportamento bizantino*).

Meccanismo di base

Il mittente esegue:

```
Bmulticast(group g, msg m)
```

Il mittente invia il messaggio **m** separatamente a ciascun membro del gruppo **g** (*incluso se stesso*) tramite una `send(p, m)`.

Ogni processo destinatario esegue:

```
Bdeliver(m)
```

Ovvero consegna il messaggio a sé stesso.

Se il mittente non fallisce durante l'invio, ogni processo corretto riceverà il messaggio.

Non garantisce né ordine né affidabilità in caso di fallimenti del mittente.

10.7.3 Rmulticast - Reliable Multicast

Il **Reliable Multicast** aggiunge garanzie più forti rispetto al Basic Multicast, soddisfacendo tre proprietà principali:

1. **Integrità:**
 - Un processo corretto invia un messaggio multicast al massimo una volta.
2. **Validità:**
 - Se un processo corretto invia un messaggio multicast, esso sarà prima o poi consegnato a sé stesso. Questa è auto-validità, è significativa solo con la prossima proprietà.
3. **Accordo (o Atomicità):**
 - Se un processo corretto riceve (*deliver*) un messaggio, allora **tutti** i processi corretti del gruppo riceveranno quello stesso messaggio.

Implementazione di Rmulticast

$Rmulticast(g, m) \rightarrow Bmulticast(g, m)$

Invece la deliver è diversa:

$Rdeliver(m) \rightarrow Bdeliver(m) + (only\ once)\ Bmulticast(g, m)$

Il processo riceve il messaggio tramite **Bdeliver**, poi ri-invia il messaggio al gruppo tramite **Bmulticast** (*ma solo una volta, la prima volta che otteniamo un messaggio*).

Quindi ogni processo riceverà molteplici copie di m .

Per distinguere il primo messaggio dalle copie successive, vengono utilizzati ID univoci per messaggio.

Agreement garantito: Dopo un **Bdeliver**, ogni processo corretto invia il messaggio a tutti gli altri processi corretti (**Bmulticast**).

Un processo corretto non **Rdeliver** (non va a buon fine) il messaggio solo se nessun altro lo ha **Bdeliver-ato**.

Infatti, dovrebbe esserci una relazione **HB** da ogni altra **Bdeliver** ed il **Rdeliver** locale.

10.7.4 Altre semantiche di Multicast

Esistono semantiche di consegna che garantiscono ordini specifici nei messaggi:

FIFO Multicast

Garantisce che se un processo corretto p invia due messaggi m_1 e m_2 in quest'ordine in un gruppo g , allora ogni destinatario riceverà m_1 prima di m_2 .

Utilizzato per **preservare l'ordine relativo dei messaggi** inviati da un singolo processo.

Causal Multicast

Preserva la relazione di causalità tra i messaggi:

Se m_1 causa (**HB**) m_2 , allora ogni destinatario nel gruppo g riceverà m_1 prima di m_2 .

La causalità implica FIFO, ma non viceversa.

Implementazione: Usa **vector timestamps** per tracciare la causalità tra eventi.

Total Order Multicast

Garantisce che tutti i processi consegnino i messaggi nello **stesso ordine globale**.

Ad esempio, se un processo corretto p consegna m_1 prima di m_2 , allora ogni altro processo farà lo stesso.

Utile per replicare dati o implementare sequenze coerenti in database distribuiti.

Dettagli Implementativi

Vector Timestamps per COrmulticast (Causal Order)

- Ogni processo mantiene un vettore di timestamp.
- I messaggi vengono inseriti in una **hold-back queue** fino a quando il loro ordine causale è soddisfatto:
 - Controllo sul vettore del messaggio rispetto al vettore del clock locale.

Queue per Total Order Multicast

- Si può utilizzare un **sequencer** centrale o un protocollo distribuito per assegnare un numero d'ordine univoco a ciascun messaggio.
- I processi ordinano i messaggi in base a questi numeri prima della consegna.

Confronto tra semantiche:

Proprietà	Basic	Reliable	FIFO	Causal	Total
Integrità	✓	✓	✓	✓	✓
Validità	✓	✓	✓	✓	✓
Accordo (Atomicità)	X	✓	✓	✓	✓
Ordine FIFO	X	X	✓	✓	✓
Ordine Causale	X	X	X	✓	✓
Ordine Totale	X	X	X	X	✓

La comunicazione di gruppo è un meccanismo potente che bilancia affidabilità, efficienza e complessità. Le semantiche specifiche vengono scelte in base ai requisiti applicativi, considerando i costi computazionali e di rete.

Capitolo 11 - Sincronizzazione dei clock fisici

Ogni nodo di un sistema distribuito possiede un proprio **clock interno**, che deve essere sincronizzato il più possibile con un **tempo di riferimento**.

Esistono diversi standard di riferimento temporale:

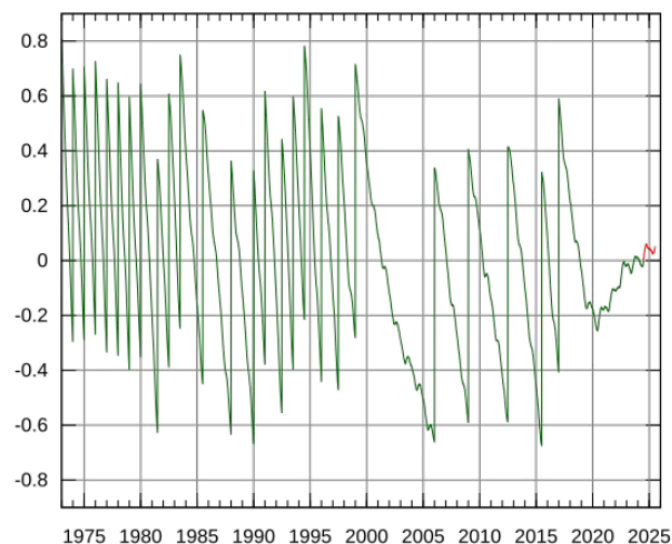
- **UTC (*Coordinated Universal Time*)**: è il riferimento principale per la comunità scientifica, basato sul **Tempo Atomico Internazionale**. Non è collegato a nessun riferimento astronomico.
- **UT1**: Successore del Greenwich Mean Time (*GMT*), è un riferimento "basato sul moto solare".
- **Unix (o *POSIX*) time**: rappresenta il numero di secondi trascorsi dalle 00:00:00 UTC del 1° gennaio 1970, esclusi i secondi intercalari (*leap seconds*).

UTC - Leap Seconds

A causa del rallentamento e delle irregolarità della rotazione terrestre, UTC non è del tutto affidabile: UT1 si discosta dal tempo UTC.

Per questo motivo, UTC ha bisogno di alcune correzioni.

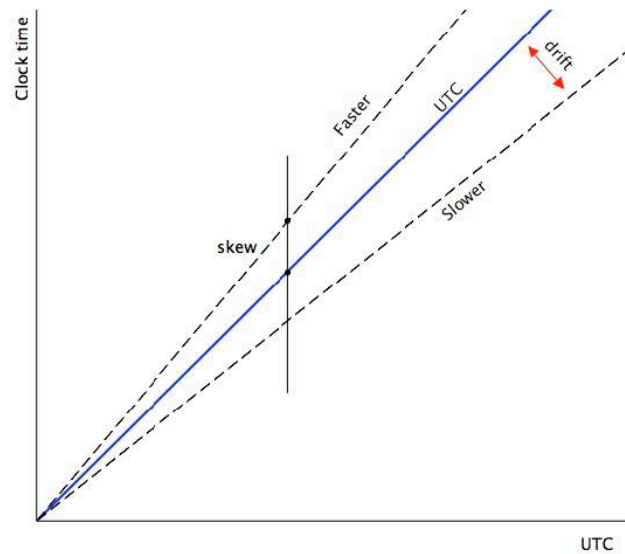
Per correggere questa discrepanza, vengono introdotti i **secondi intercalari**, indicati come segmenti verticali nei grafici temporali.



Disallineamento dei Clock

Due clock possono differire in due modi:

- **Clock Skew:** differenza istantanea tra i valori letti da due clock (due timer partiti in due momenti diversi).
- **Clock Drift:** differenza nella velocità (tasso) di avanzamento di due clock (due timer che avanzano con due velocità diverse).



Diremo che due clock sono sincronizzati solo se il loro clock skew è minore di un certo valore D .

Supponiamo che il nostro clock locale sia la linea tratteggiata.

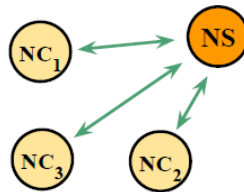
11.1 Sincronizzazione esterna, interna e ripristino

Sincronizzazione esterna

Ogni nodo si sincronizza rispetto a una **fonte di riferimento affidabile** $S(t)$, assicurando:

$$|S(t) - C_i(t)| < D, \forall NC_i$$

In tal caso si dice che i nodi sono sincronizzati esternamente con accuratezza D .

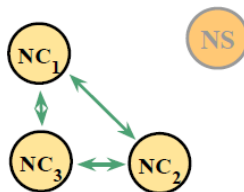


Sincronizzazione interna

I **nodi di un gruppo** concordano tra loro in modo che:

$$|C_i(t) - C_j(t)| < D, \forall NC_i, NC_j$$

In tal caso si dice che i nodi sono sincronizzati internamente con accuratezza D .



Relazione tra sincronizzazione esterna e interna

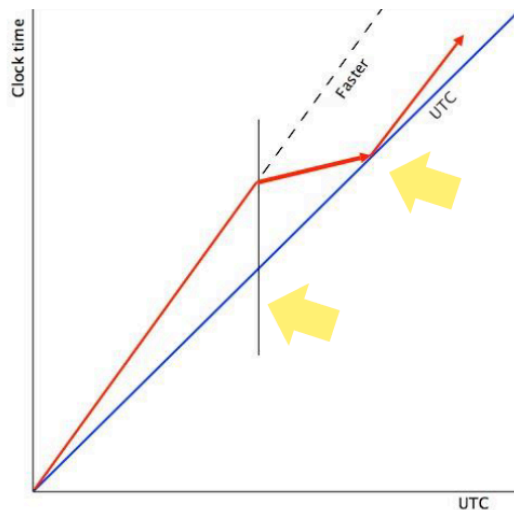
Se un sistema è sincronizzato esternamente con accuratezza D , allora i suoi clock interni concordano con una differenza massima di $2D$.

Ripristino di un Clock

Quando si corregge un valore per un clock, è essenziale garantire la **monotonicità temporale** (cioè, il tempo non deve mai "tornare indietro nel tempo").

Perciò, anziché impostare indietro il clock (ovvero *diminuire il clock skew*), si rallenta il suo ritmo (*diminuiamo il clock drift*) fino a raggiungere il valore corretto.

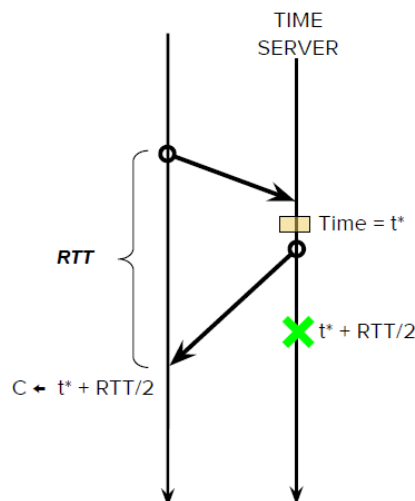
Se il nostro clock è troppo veloce, come in figura, e ad un certo tempo vediamo che il valore corretto dovrebbe essere un altro, non è possibile correggerlo immediatamente perchè non è corretto portare il clock indietro, quindi viene rallentato (o in alcuni casi fermato) fino ad ottenere il valore corretto.



11.1.1 Sincr. esterna - Algoritmo dei Cristiani †

L'algoritmo sfrutta il **Round Trip Time (RTT)**, cioè il tempo di andata e ritorno di un messaggio tra client e server.

1. Un client invia una richiesta di sincronizzazione al server di riferimento.
2. Il server risponde con il proprio timestamp attuale.
3. Il client stima il tempo di rete (*basandosi sullo RTT*) e corregge il proprio orologio **assumendo che la latenza sia simmetrica** (ovvero il tempo di andata è uguale a quello del ritorno).



Latenza simmetrica

Se RTT è molto piccolo ($\ll 1$), si può assumere che i tempi di latenza per la richiesta e la risposta siano simmetrici.

Calcolo lato client

Se il server inviasse il tempo corrente misurato al momento di quando riceve la risposta, il client otterrebbe un valore non aggiornato.

Sia t^* il timestamp attuale del server.

In tal caso, il client può misurare solo RTT, quindi calcola il tempo corretto in questo modo:

$$t_{client} \leftarrow t^* + \frac{RTT}{2}$$

Più basso è lo RTT e più precisa sarà la sincronizzazione, per questo conviene fare più tentativi per ottenere un RTT più basso.

11.1.2 Sincr interna - Algoritmo di Berkeley

Soluzione **centralizzata** progettata per gruppi di computer UNIX:

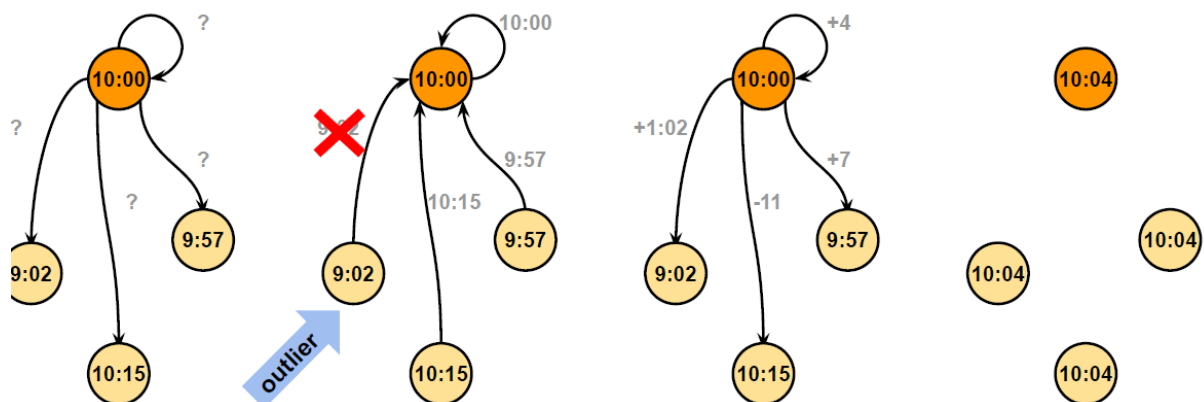
1. Un processo centrale (**time daemon**) opera come **master** e raccoglie i valori dei clock dagli altri nodi.
2. Ogni nodo invia il proprio valore temporale; il time daemon annota i tempi insieme anche ai rispettivi RTT (*come nell'algoritmo dei cristiani*).
3. Il time daemon calcola una media dei valori raccolti.
4. Il time daemon invia a ogni nodo la correzione che deve essere applicata.

Perché nel punto 4 non viene inviato il tempo in broadcast?

Perché arriverebbe ad ogni nodo in punti differenti nel tempo (*anche nel punto 4 vi è un messaggio e lo RTT può differire di molto dall'altro*), quindi ci sarebbero clock skew differenti per ogni nodo.

Inviando solo il valore di correzione, il reset del clock avviene in modo time-insensitive, in base a quando ogni nodo riceve la correzione.

eventuali outlier non vengono considerati nel calcolo della media, ma gli sarà comunque inviata una correzione per loro.



Applicare correzioni negative non si può fare, quindi il nodo che riceve una correzione negativa (*significa che il suo clock è troppo avanti nel tempo*) applicherà una delle tecniche descritte in precedenza (lo rallenta fino al valore desiderato).

11.2 NTP - Network Time Protocol

Permette ai client di sincronizzarsi con UTC attraverso Internet:

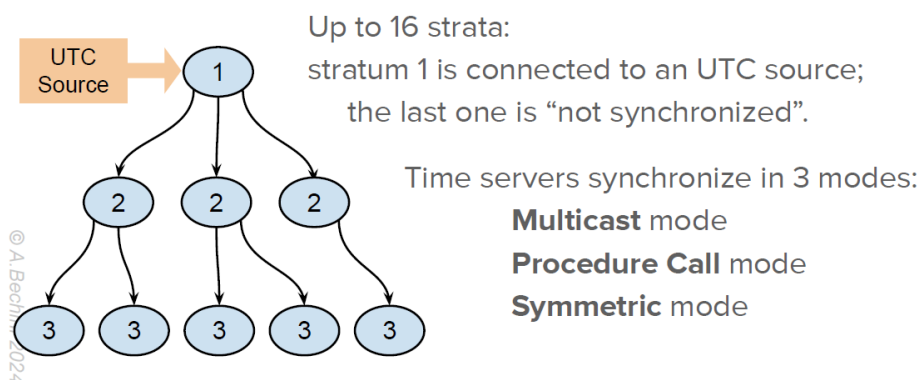
- **Scalabilità** per grandi reti, perché bisogna considerare ogni nodo su internet.
- Tecniche statistiche per filtrare i dati.
- **Autenticazione** per evitare problemi relativi ad interferenze.
- **Gerarchia di server temporali** distribuiti su Internet.
- **UDP** per inviare i messaggi.

Gerarchia di Server Temporali (Strati)

I server NTP sono organizzati in **strati (strata)**, fino a un massimo di 16:

- Lo strato 1 è direttamente connesso a una fonte UTC.
- Ogni server di uno strato è sincronizzato (*approccio client-server, come l'algoritmo dei cristiani*) con un server dello strato precedente.
- Lo strato finale è considerato "*non sincronizzato*".

Più un nodo è vicino alla sorgente UTC ufficiale, più sincronizzato è; perché gli algoritmi non sono perfetti e quindi più ci si allontana e più impreciso diventa il riferimento.



11.2.1 Modalità di Sincronizzazione in NTP

NTP offre tre modalità principali di sincronizzazione tra server e nodi:

Modalità Multicast

Uno o più server inviano messaggi di sincronizzazione (*time multicast*) a più nodi della rete.

Non vengono utilizzate correzioni message delay, quindi è particolarmente adatta a reti locali (LAN), dove gli RTT sono molto piccoli e poco variabili.

I nodi che ricevono il multicast aggiornano (sostituiscono il valore) i propri clock basandosi sull'ora ricevuta dal server.

In questa modalità si presuppone un RTT molto basso e variabile, quindi non viene applicata una precisa compensazione delle latenze, rendendola meno accurata rispetto ad altre modalità.

Modalità Procedure Call

Più accurata rispetto al multicast, poiché considera la **compensazione delle latenze**, ma rimane comunque molto efficiente.

Un client invia una richiesta al server basandosi sul **Christian's algorithm**.

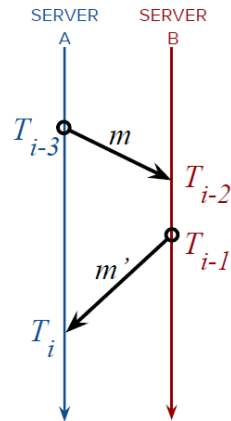
Questa modalità è più adatta per ambienti in cui si richiede una buona accuratezza senza un costo computazionale eccessivo.

Modalità Simmetrica

La modalità **più precisa**, ma anche la **più costosa** in termini di risorse.

Viene utilizzata principalmente tra server appartenenti agli strati più alti della gerarchia NTP, dove è essenziale una sincronizzazione estremamente accurata (*per evitare di propagare l'errore agli strati successivi*).

Una **coppia di server** si scambiano due messaggi contenenti i timestamp al momento dell'invio.



Ognuno dei due server ha quindi i 4 timestamp.

Poi stimano lo **scostamento reale** (offset o) tra i propri clock. Lo scostamento stimato (o_i) si calcola con la formula:

$$o_i = \frac{1}{2}(T_{i-2} - T_{i-3} + T_{i-1} - T_i)$$

Quanto è precisa questa formula? Ovvero vediamo l'errore commesso a quanto ammonta.

Dal grafico sopra vediamo bene a quanto ammontano queste due grandezze:

$$\begin{cases} T_{i-2} = T_{i-3} + t_m + o \\ T_i = T_{i-1} + t_{m'} - o \end{cases}$$

Quindi l'offset è calcolabile come:

$$\begin{cases} o = T_{i-2} - T_{i-3} - t_m \\ o = T_{i-1} - T_i + t_{m'} \end{cases}$$

Possiamo esprimere RTT in modo certo come il tempo in cui i due server non hanno lavorato.

$$RTT = t_m + t_{m'} = \Delta T_A - \Delta T_B = (T_i - T_{i-3}) - (T_{i-1} - T_{i-2})$$

$$o = \frac{1}{2} (T_{i-2} - T_{i-3} + T_{i-1} - T_i - t_m + t_{m'}) = o_i + \frac{1}{2} (t_{m'} - t_m)$$

Infine:

$$o_i - \frac{1}{2} RTT \leq o \leq o_i + \frac{1}{2} RTT$$

Correzione del Tempo

Ovviamente, più basso è RTT e più precisa è la misurazione, quindi conviene fare molteplici misurazioni usando quella con RTT minore.

Per correggere il tempo, i server raccolgono più coppie di valori $\langle o_i, RTT \rangle$. Ogni coppia include:

Tra tutte le coppie raccolte, viene scelto il valore che garantisce la maggiore accuratezza, ossia quello con il **minore RTT** o quello statisticamente più affidabile.

Laboratorio 1 - Federate learning

Federated learning è il problema in cui abbiamo un modello generale in un server sul quale abbiamo tutti i dati. In questo caso ci sono due problemi, uno di sicurezza e uno di quantità (*troppi dati in un punto*).

Modello locale

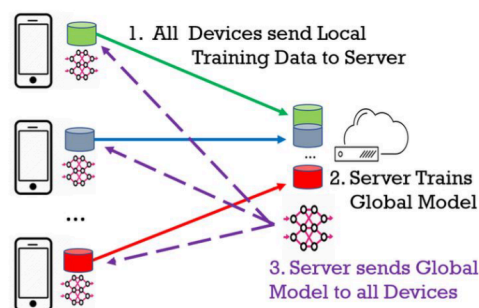
Il modello può essere allenato localmente ma il problema è che lavoriamo solo con i nostri nodi.



(a) Local: share nothing

Modello centralizzato

I dati vengono immagazzinati in un singolo server. Il server allena un unico modello che invia a tutti i client.



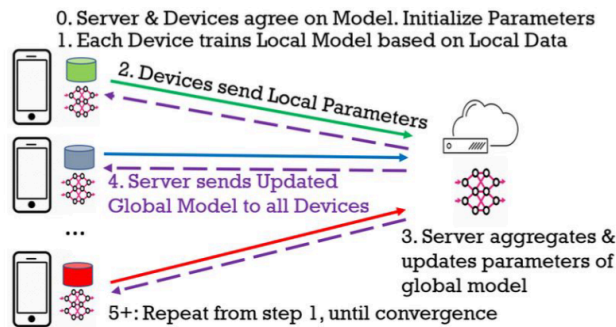
(b) Centralized: share training data

Federated Learning

Il server orchestra l'allenamento e condivide i parametri iniziali del modello.

Il *federated learning* è una tecnica di apprendimento automatico che consente di addestrare modelli su dati distribuiti su diversi dispositivi o organizzazioni senza spostare i dati in un'unica posizione centrale.

Invece di inviare i dati al server centrale, ciascun dispositivo o entità (*come uno smartphone o un ospedale*) addestra localmente il modello e poi invia solo i parametri aggiornati al server, che aggrega i risultati e aggiorna il modello globale. Questo approccio protegge la privacy e riduce i rischi di violazioni dei dati.



(c) Federated: share model parameters, not data

Il federate learning si divide in due sotto-categorie:

Horizontal federate learning

Il dataset è splittato in base ai dati.

I dati sono distribuiti tra diverse entità che condividono lo stesso tipo di caratteristiche o attributi.

Ad esempio, se due ospedali in città diverse hanno dati simili su pazienti (*come età, peso, malattie, ecc.*), ma non sugli stessi individui, possono collaborare usando HFL. Ogni entità addestra il modello sui propri dati locali, e il modello globale viene aggiornato aggregando i risultati.

Client A			Client B				
	x_1	x_2	y		x_1	x_2	y
1				4			
2				5			
3				6			

Vertical federate learning

Il dataset è splittato in base alle funzionalità. Le entità che collaborano hanno informazioni complementari sugli stessi individui, ma non sugli stessi attributi.

Ad esempio, una banca o un'azienda di e-commerce possono avere dati diversi sugli stessi clienti (*ad es. dati finanziari e dati di acquisto*). In questo caso, il VFL permette di combinare informazioni senza condividere i dati direttamente, mantenendo la privacy delle singole caratteristiche di ogni entità.

Client A			Client B				
	x_1	x_2	x_3		x_4	x_5	y
1				1			
2				2			
3				3			

Un problema potrebbe essere che i nodi per comunicare devono avere un formato di messaggio comune, un secondo problema potrebbe essere quello della sicurezza.

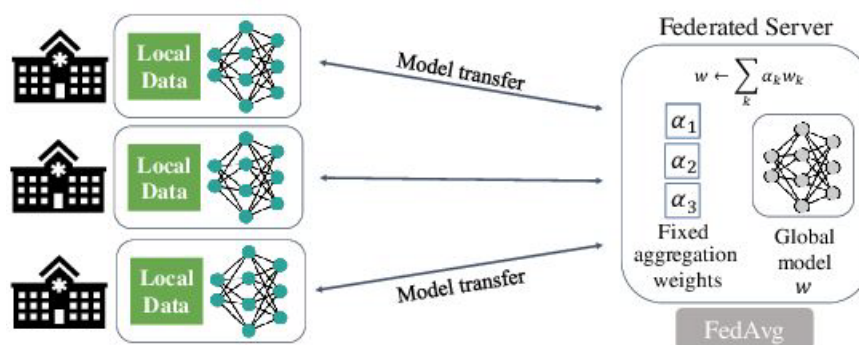
Se il server invia i pesi al client per effettuare il training, il client non conosce per quanto tempo deve effettuare il training.

Un altro problema è capire come avviene la selezione dei partecipanti.

L1.1 Approccio algoritmico

All'inizio il federate learning è stato progettato per allenare le reti neurali, quindi per questa operazione servono dei pesi.

Possiamo applicare questo tipo di training a tutti i modelli?



Come possiamo vedere dalle formule dell'algoritmo **fuzzy c-mean**:

1. Initialize $U = [u_{ij}]$ matrix, $U^{(0)}$
2. At k -step: calculate the centers vectors $C^{(k)} = [c_j]$ with $U^{(k)}$:

$$c_j = \frac{\sum_{i=1}^N u_{ij}^\lambda * x_i}{\sum_{i=1}^N u_{ij}^\lambda}$$

3. Update $U^{(k)}, U^{(k+1)}$

$$u_{ij} = \frac{1}{\sum_{k=1}^C \left(\frac{\|x_i - c_j\|}{\|x_i - c_k\|} \right)^{\frac{2}{\lambda-1}}}$$

4. If $\|U^{(k+1)} - U^{(k)}\| < \epsilon$ then STOP; otherwise return to step 2

where ϵ is a termination criterion between 0 and 1.

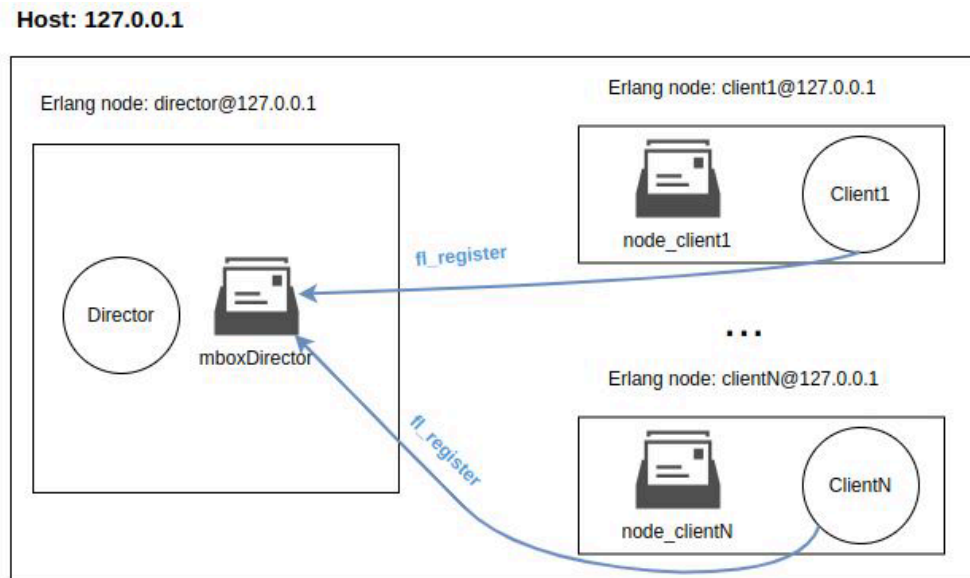
Ogni volta che dobbiamo modificare un algoritmo per produrre la versione federated, bisogna studiare per prima cosa come cambiare la formula.

L1.2 FedLang

FedLang è un middleware che separa la parte di comunicazione dalla parte di machine learning.

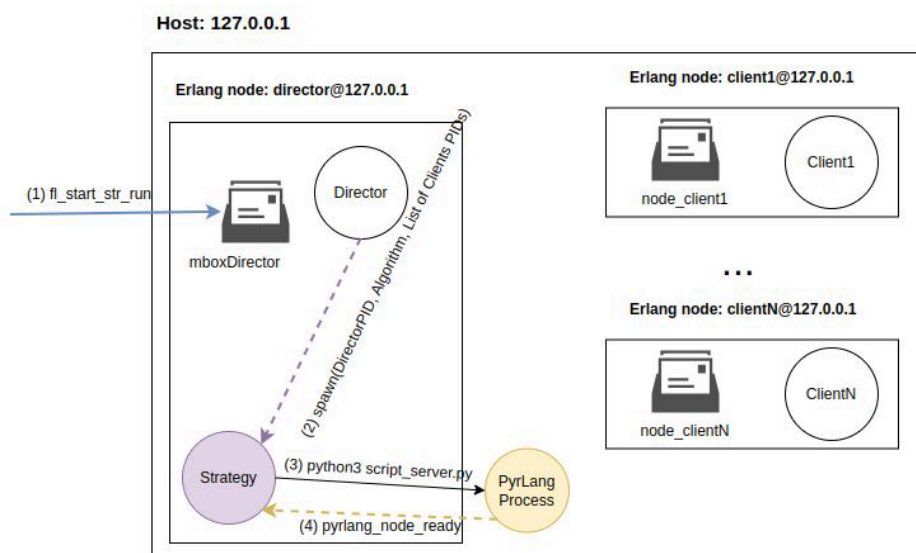
Come possiamo trasmettere tipi di dati complessi da un nodo all'altro? Bisogna serializzare i dati, ovvero creare delle loro rappresentazioni.

C'è il **director** in esecuzione che è un server che tiene traccia di tutti i partecipanti. Utilizzando il comando `epmd -names` è possibile osservare tutti i nodi Erlang in esecuzione.



Erlang viene utilizzato per gestire la parte di comunicazione, mentre per il machine learning viene utilizzato Python.

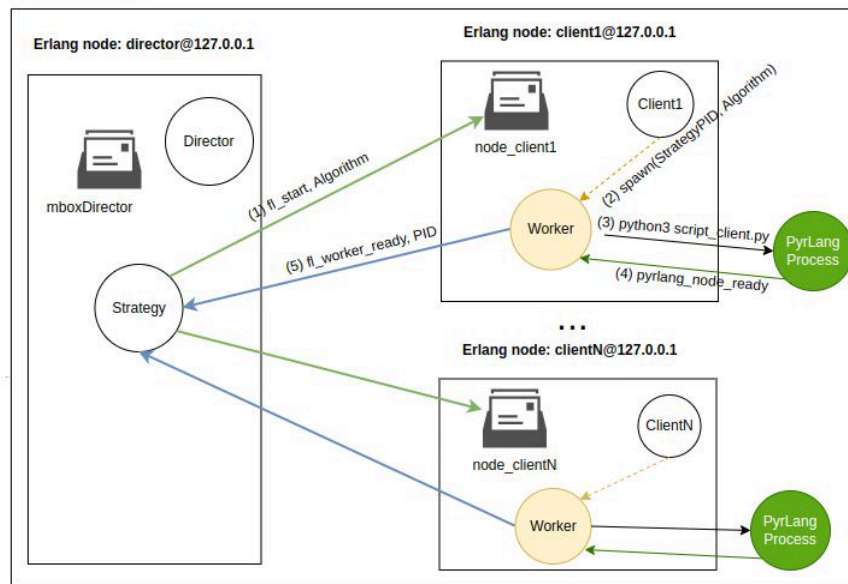
Per ogni richiesta da un client viene creato in Erlang un nuovo processo **Strategy** in parallelo.



Lo strategy server che è incaricato di aggregare le informazioni inviate dai client.

Ogni client quando riceve un messaggio dallo strategy, crea un processo Worker, associato ad esso ci sarà un processo Pyrlang.

Host: 127.0.0.1



Quando eseguiamo l'esperimento vengono creati tre nodi Pyrlang che implementano la parte di machine learning, al termine della federation non ha senso tenere questi processi, quindi vengono rimossi.

Laboratorio 2 - GRPC

Python

- Apriamo il terminale e andiamo nella cartella in cui è presente il codice Python.
- Rimuoviamo i due file con `rm -f hello_world`.
- Attiviamo l'ambiente virtuale `pipenv shell`.
- Generiamo la classe GRPC con il comando su github.
- Per eseguire il server eseguiamo il comando `python greeter_server.py`.
- Per eseguire il client eseguiamo il comando `python greeter_client.py`.

Java

- Il proto file in Java è lo stesso visto per Python.
- Utilizzando i plugin in Maven generiamo le dipendenze necessarie.
- Eseguiamo il server da IntelliJ.
- Eseguiamo la parte Python da terminale e controlliamo che non ci siano altri server con `ps ax | grep python`.
- Cambiamo la porta visto che è la stessa ed eseguiamo.

Il messaggio va inviato a `192.168.146.14 8999`.

Laboratorio 3 - Apache Maven

Approccio Divide Et Impera: Invece di avere un unico grande progetto, possiamo splittarlo in funzionalità più piccole.

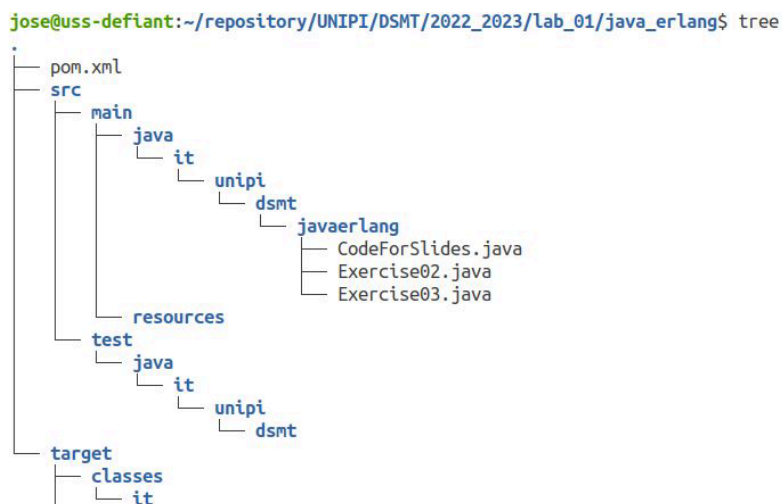
Vedremo come fa Maven ad ottenere le dipendenze da internet e come le gestisce localmente.

Come organizziamo tutte le persone che devono sviluppare parti di codice che devono unire per ottenere il prodotto finale.

L'immagine sotto mostra un esempio di albero creato da Maven, dove vengono mostrate tutte le dipendenze del progetto.

L3.1 Il file pom.xml

Nel file POM sono presenti le informazioni generali sul progetto: Project coordinate = groupId + artifactId + version. Inoltre sono presenti le impostazioni, l'ambiente e le relazioni.



```
<?xml version="1.0" encoding="UTF-8" ?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd" >
```

```
<modelVersion>4.0.0</modelVersion>
<groupId>it.unipi.dsmt.javaerlang</groupId>
<artifactId>java_erlang</artifactId>
<version>1.0.0-SNAPSHOT</version>
```

```
<name>${artifactId}-${version}</name>
```

```
<dependencies>
  <dependency>
    <groupId>com.ericsson</groupId>
    <artifactId>erlang-jinterface</artifactId>
    <version>1.13.1</version>
  </dependency>
</dependencies>
```

```
<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <maven.compiler.source>11</maven.compiler.source>
  <maven.compiler.target>11</maven.compiler.target>
</properties>
```

```
<build>
  <finalName>${artifactId}</finalName>
</build>
</project>
```

General project information.

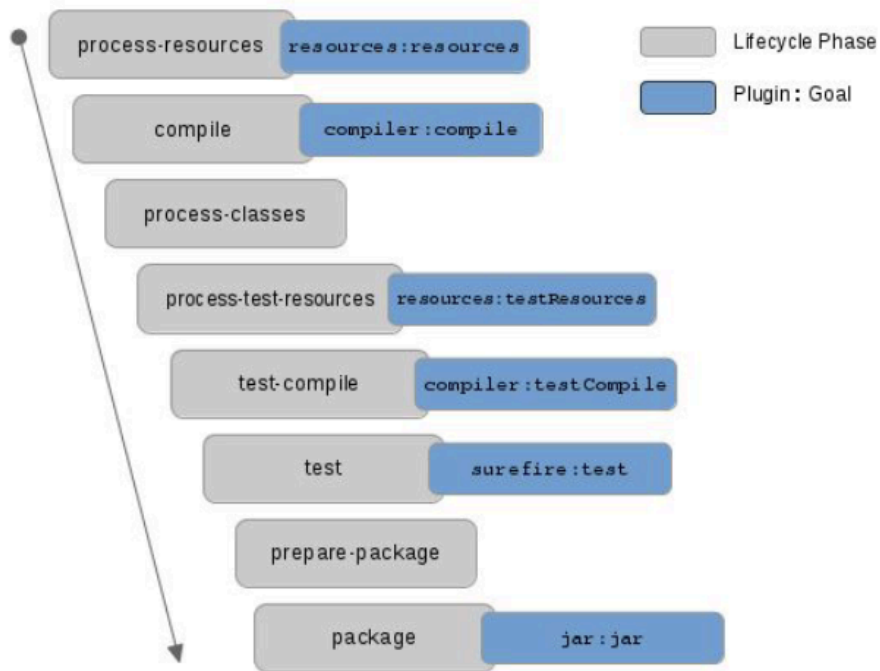
Project coordinate = groupId + artifactId + version

The Maven trinity!

POM Relationships.

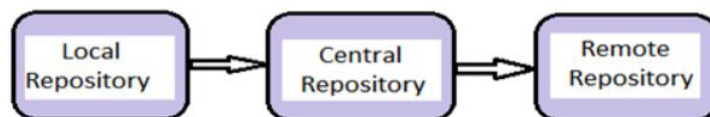
Build settings and environment section here.

Ogni volta che eseguiamo un lifecycle i goals sono eseguiti.



L3.2 Maven repository

Quando specifichiamo una dipendenza nel file `pom.xml`, Maven per prima cosa l'esistenza della suddetta dipendenza nella repository locale, se non c'è prova a controllare se la dipendenza esiste da una repository centrale, infine da una repository remota.



Quindi la precedenza ce l'ha la repository locale, la quale è posizionata nella cartella `$HOME/.m2`.

Se nella repository locale è assente ma nella repository remota è presente allora avverrà un download e poi memorizzata nella repository locale in modo da evitare un altro download in futuro.

Se abbiamo un'applicazione con 10 dipendenze, come possiamo creare l'eseguibile?

Laboratorio 4 - Jinterface

Jinterface è simile a Pyrlang e serve per avviare un nodo Erlang in Java.

Jinterface deve essere importato tramite Maven, la quale ultima versione non è presente nella repository centrale; se facciamo in modo che l'IDE installi la dipendenza ci darà errore.

Dobbiamo crearla e installarla manualmente tramite la CLI di Maven.

Per prima cosa bisogna aggiornare il database locale e questo comando crea un database locale.

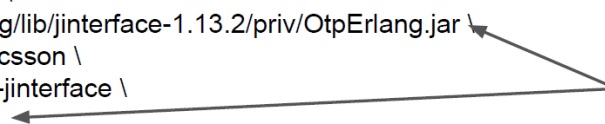
```
sudo updatedb
```

Ora bisogna capire dove è stata installata la nostra dipendenza.

```
sudo locate OtpErlang.jar
```

Bisogna installare la dipendenza direttamente da terminale con il seguente comando.

```
mvn install:install-file \
  -Dfile=/usr/lib/erlang/lib/jinterface-1.13.2/priv/OtpErlang.jar \
  -DgroupId=com.ericsson \
  -DartifactId=erlang-jinterface \
  -Dversion=1.13.2 \
  -Dpackaging=jar \
  -DgeneratePom=true
```



DO NOT forget to update the path of this jar file and the version.

In questo esempio vediamo l'implementazione di una mailbox.

```
import com.ericsson.otp.erlang.*;

...

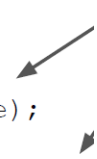
String cookie = "abcde";

String nodeId = "client_node@127.0.0.1";

String mailBoxId = "MyMailBox";

OtpNode node = new OtpNode(nodeId, cookie);

OtpMbox mailBox = node.createMbox(mailBoxId);
```




```
import com.ericsson.otp.erlang.*;
```

```
...
```

```
String serverMailBoxId = "ServerMailBox";
```

```
String serverNodeId = "server_node@127.0.0.1";
```

```
OtpErlangObject[] content = new OtpErlangObject[2];
```

```
content[0] = mailBox.self();
```

```
content[1] = new OtpErlangString("James");
```

```
OtpErlangTuple message = new OtpErlangTuple(content);
```

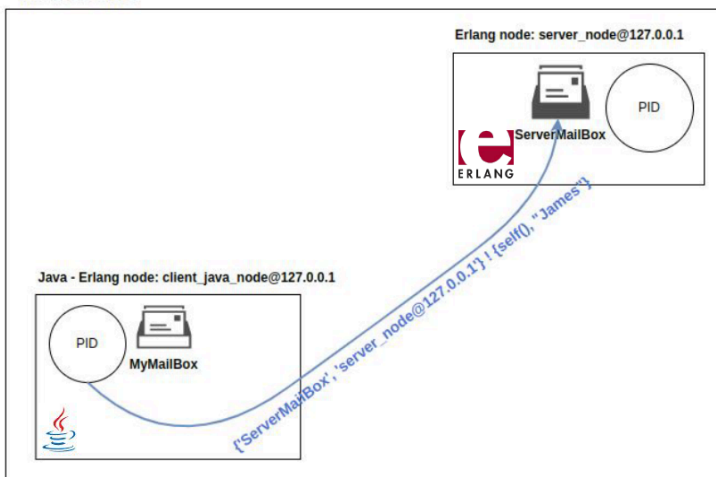
```
mailBox.send(serverMailBoxId, serverNodeId, message);
```

In this example we want to send a Tuple of two elements to an Erlang process.

Note that the PID value is sent.

Since we don't have the PID of the remote process, we have to make use of its MailBoxId and NodeId.

Host: 127.0.0.1



```
jose@uss-defiant:~$ ps ax | grep epmd
33645 pts/0  S+      0:00 grep --color=auto epmd
jose@uss-defiant:~$ epmd &
[1] 33646
jose@uss-defiant:~$ epmd -names
epmd: up and running on port 4369 with data:
name client_java_node at port 37539
name server_node at port 45073
jose@uss-defiant:~$
```

Make sure epmd application is running.

- ps ax | grep epmd
- epmd &

```
import com.ericsson.otp.erlang.*;
```

```
...
```

```
while (true) {
```

```
OtpErlangObject newMessage = mailBox.receive();
```

```
if (newMessage instanceof OtpErlangTuple) {
```

```
OtpErlangTuple erlangTuple = (OtpErlangTuple) newMessage;
```

```
OtpErlangPid senderPID = (OtpErlangPid) erlangTuple.elementAt(0);
```

```
OtpErlangString clientName = (OtpErlangString) erlangTuple.elementAt(1);
```

```
String messageStr = "Hello " + clientName.stringValue();
```

```
OtpErlangString greetings = new OtpErlangString(messageStr);
```

```
mailBox.send(senderPID, greetings);
```

```
}
```

```
}

We need to write code that is able to process incoming messages. We know their structure.
```

Explicit cast operation

Everytime a message is fetch, it is required to determine its type and perform some validations.

We make use of some classes to retrieve information sent by another process.

In this case I have the PID of the sender, I can make use of it to reply back.

Per vedere le sdk installate possiamo utilizzare `sdk list java`, per installare una nuova versione possiamo utilizzare il comando ...

Laboratorio 8 - Jakarta EE, Servlet and JSP

Nelle pagine web HTML c'è il problema di dover ripetere alcune sezioni in modo statico, con JSP è possibile inserire queste sezioni in modo dinamico con un loop.

Q&A (by Francesco Taverna)

CAPITOLO 2 - CONCORRENZA (Benefici della parallelizzazione)

- Concorrenza, legge di Amdahl, grafico al variare di n. Quali sono i casi in cui all'aumentare di n la curva diminuisce.

CAPITOLO 4 - ERLANG

(Concorrenza e distribuzione)

- What is OTP. What is a gen_server? What would you use it for? What are callbacks?

CAPITOLO 5 - JAVA

(Java threads e Sincronizzatori/JVM Runtime Data Area)

- com'è organizzata la memoria della JVM? cosa accade quando un thread chiama una funzione? quali sono le problematiche associate.

Risposta: you need to reserve a chunk of memory for the variable

- ma in quale memoria e quanto spazio necessario?

Risposta: Verrà chiesto una stack memory per ogni thread

- Come puoi sapere che un thread ha raggiunto un certo punto della sua esecuzione?

Risposta: you need a special register: the program counter, ne avremo uno per thread.

- cos'è la memoria heap, a cosa serve? quali sono le policy?

Risposta: java è un ling di programma orientato agli oggetti, sono proprio gli oggetti ad essere istanziati nella memoria heap

- quando viene istanziato un oggetto, come puoi sapere quanto spazio questo occuperà? where is kept the code associated with the method?

Risposta: Nella "method area" che contiene i metodi delle classi e non solo (non ho ben capito se contiene anche i campi delle classi)

(Java threads e Sincronizzatori/Thread Pooling ed Executor framework)

- Dove posso essere costruiti questi "prodotti" già pronti? Come funziona con le varie classi i vari parametri ecc. + about the executor for thread pooling and the Callable interface.

Risposta: Abbiamo bisogno di una istanza di executor service, richiamiamo un metodo che crei questa istanza (factory method), restituiscono un'istanza a partire da un metodo.

- What are Java Futures, what are they for, what's the advantage in a concurrent setup

CAPITOLO 6 - VERSO IL TEMPO LOGICO (Clock per il tempo logico)

- Concept of logical clock, lamport timestamps, vector timestamps, differences and properties.
- Vector timestamps. What are their uses, how do you compare two vector timestamps. + dimostrazione
- Lamport timestamp. cosa sono, perchè sono utili, come vengono usati. Definisci cos'è un "evento" in questo contesto. (con tanto di disegno e descrizione di ogni cosa sul disegno: cosa indicano i punti e cosa indicano le frecce)

Risposta: un evento è un astrazione che usiamo per modellare ciò che realmente accade. Il ts viene mappato sul messaggio, perchè il destinatario deve venire a conoscenza del ts del sender. il receiver prenderà come timestamp corrente il max tra il suo ed il ts appena arrivato.

- Uso pratico del Lamport_ts ?

CAPITOLO 8 - COMMUNICATION SUPPORT FOR DISTRIBUTED SYSTEM

(RMI/Class Loading in Java)

- class loading in JVM. che differenze sono state aggiunte per le applicazione distribuite?
- What is RPC and what is RMI? When are they used? Explain the differences, describe the architecture of a system that uses RPC, how are parameters passed. Remote references in RMI

Risposta: Quando una classe si incontra per la prima volta deve essere caricata in memoria. Devo stabilire quale class loader se ne deve occupare, si parte sempre da quello più in basso, se poi questo non può viene delegato a quelli più in alto.

- Class Loading in Java e come cambia quando abbiamo oggetti remoti.
- Se un programma Java dobbiamo caricare una classe, come avviene il caricamento e quando viene fuori questo problemi?

Risposta: Avviene tramite il ClassLoader che delegherà la responsabilità ad un altro class loader in maniera gerarchica.

- Perchè sono 3? Non basterebbe averne 1?

Risposta: Il comportamento è diverso, perchè devono trovare il file .class da utilizzare

- Perchè è un problema trovare i file .class?
- Ho due classi, la classe paperino (scritta da me) e la classe String (del sys), c'è qualcosa di diverso?
- Dove cerca il sistema per il file .class?

Risposta: Nel classpath

- Mentre la classe string dove sta?

Risposta: Nel jar di java

- Quando ho un programma distribuito come posso comportarmi?

Risposta: Posso scaricare il .class da uno specifico nodo

- I basic ClassLoader possono farlo?

Risposta: No ho bisogno di un ClassLoader specifico

(Garbage Collection)

- Garbage collection come viene fatto in locale. Come viene implementato in Java.
- Garbage collection da locale a distribuita. Come funzionano i collector
- Tri-color marking algorithm
- markAndSweep vs ReferenceCounting -> come adattarli a sistemi distribuiti -> lease

(Web Services)

- What is the difference between big web services and rest web services.

CAPITOLO 11 - ENTERPRISE APPLICATIONS (Enterprise Java Beans)

- EJB e come vengono gestiti dall'Application Server, quali sono i tipi e come viene gestito il lifecycle
- About servlets, jsp
- Java servlet
- JSP. What are they used for, processing of a JSP, relation of a JSP with a servlet

CAPITOLO 12 - FUNDAMENTAL DISTRIBUTED ALGORITHMS

(Classi di algoritmi)

- Wave algorithms

(Mutual Exclusion: The Distributed Way)

- Algoritmi distribuiti: Ricart-Agrawala

(Leader Election)

- Leader Election in un contesto in cui ci possono essere dei fault.
- The bully's algorithm

CAPITOLO 13 - SYNCHRONIZATION OF PHYSICAL CLOCKS

(Sincronizzazione interna: l'algoritmo di Christian)

- Synchronization of physical clocks, general principles
- Se io sono su una rete locale e devo sincronizzare i clock dei singoli nodi, come posso fare? Illustra i vari protocolli.

Risposta: Non è possibile avere un global clock

- La richiesta specifica è che siano tra di loro sincronizzati (secondo me si potrebbe avere un server nella rete locale)

Risposta: Ci sono delle tecniche per il management della latency (Christian Algorithm) che utilizza il RTT. Supponiamo che l'overhead per l'invio e la ricezione siano gli stessi

- In pratica, quale è il massimo dell'errore che posso permettere nel sincronizzare i clock?

Risposta: L'errore massimo è nel caso in cui un invio è immediato e tutto il RTT è dovuto all'invio/ricezione del secondo pacchetto, quindi il massimo errore è $RTT/2$

(Sincronizzazione interna: l'algoritmo di Berkeley)

- Come sincronizzare il clock di un sistema distribuito? INTERNAMENTE
- Algoritmo di Berkeley, quale problema risolve? Descriverlo approfonditamente

Risposta:

1. Deve settare un valore unico di clock
2. Abbiamo un nodo master (time daemon) che ripetutamente richiede il valore del clock a tutti i nodi nella network
3. I nodi rispondono con il proprio clock
4. Il master calcola una media scartando anche degli outlier
5. Una volta calcolata la media invia l'offset a ogni nodo rispetto alla media calcolata
6. Ora tutti i nodi sono sincronizzati

Ora ci sono dei problemi, io ottengo una risposta che corrisponde al valore del clock nel momento in cui il messaggio viene ricevuto, questo non considera l'overhead. Quindi il master non ottiene per forza un valore accurato.

- Perché il server centrale non invia esattamente il valore ma invia l'offset?

Giova dice che va bene lo stesso, secondo me no. Giovanni afferma che non deve essere invalidata la monotonia del clock (problema relativo all'update). Facciamo un passo indietro, il master fa questa lettura inviando e ricevendo messaggi.

(NTP)

- Network Time Protocol

ESERCIZI ERLANG

1. implementa una funzione erlang che riceva una lista, decida se questa è palindroma oppure no

hint: c'è modo di passare una lista ordinata al contrario

```
%%lista palindroma
-module(exercise6).
-export([is_palindrome/1]).

is_palindrome(List) ->
    List == lists:reverse(List).

%%senza funzione libreria
is_palindrome(List) ->
    List == reverse(List).

reverse2([]) -> [];
reverse2(L) -> reverse2(L, []).

reverse2([], R) -> R;
reverse2([H | T], R) -> reverse2(T, [H|R]).
```

2. Implementare una funzione di riduzione.

```
%%implementare una funzione di riduzione
-module(exercise7).
-export([reduce/3]).

% Caso base: se la lista è vuota, restituiamo l'accumulatore
reduce(_Fun, Acc, []) ->
    Acc;
% Caso ricorsivo: applichiamo la funzione e passiamo al resto della lista
reduce(Fun, Acc, [Head | Tail]) ->
    reduce(Fun, Fun(Head, Acc), Tail).

% Somma di una lista di numeri
Sum = my_reduce:reduce(fun(X, Acc) -> X + Acc end, 0, [1, 2, 3, 4, 5]).
% Risultato: 15

% Concatenazione di una lista di stringhe
Concat = my_reduce:reduce(fun(X, Acc) -> X ++ Acc end, "", ["Hello", " ", "World"]).
% Risultato: "Hello World"
```

3. How would you implement a quicksort in Erlang

```

%%ordinare una lista (quicksort)
-module(exercise1).
-export([quicksort/1]).

quicksort([]) -> [];
quicksort([Pivot | Rest]) ->
    Smaller = [X || X <- Rest, X =< Pivot],
    Greater = [X || X <- Rest, X > Pivot],
    quicksort(Smaller) ++ [Pivot] ++ quicksort(Greater).

```

4. scrivere una funzione invert che restituisce la lista invertita

```

-export([reverse/1]).

-export([reverse2/1]).

%%caso lista vuota
reverse([]) -> [];

reverse([H | T]) -> reverse(T) ++ [H].

%[1 2 3]

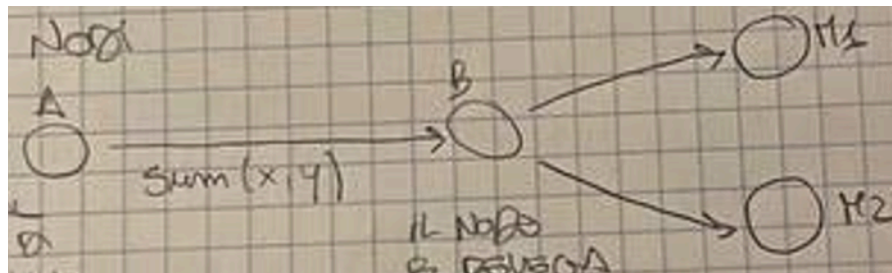
% reverse([3 | ]) -> reverse() ++ [3] ----> [3]
% reverse([2 | 3]) -> reverse(3) ++ [2] ---> [3 2]
% reverse([1 | 2 3]) -> reverse(2 3) ++ [1] ---> [3 2 1]

%%caso con accumulatore (tail recursion)
reverse2([]) -> [];
reverse2(L) -> reverse2(L, []).

reverse2([], R) -> R;
reverse2([H | T], R) -> reverse2(T, [H|R]).

```

5. Due nodi:



A chiede al nodo B di fare una somma.

Il nodo B delega l'operazione al nodo M1 se x è pari mentre a M2 altrimenti.

Scrivere la struttura (codice) del nodo B: fare attenzione a chi deve rispondere al nodo A (B attenda la risposta del nodo M1 (M2) tramite la receive, quindi risponde al nodo A.

Risposta: La condizione si può fare con $x \text{ rem } 2 == 0$ (tramite delle guardie sul pattern matching del messaggio).

```

nodeB() ->
  receive
    {X, Y, PidA} when X rem 2 == 0 ->
      PidM1 ! {X, Y, self(), PidA};

    {X, Y, PidA} when X rem 2 /= 0 ->
      PidM2 ! {X, Y, self(), PidA},

    {sum, Sum, PidA} -> PidA ! {sum, Sum}
  end.

nodeA(PidB, X, Y) ->
  pidB ! {X, Y, self()},
  receive
    {sum, Sum} -> io:format("The sum is "++Sum);
  end.
  
```

```

nodeM1() ->

    receive
    | {X, Y, PidB, PidA} ->
    |   Sum = X + Y,
    |   PidB ! {sum, Sum, PidA}
    end.

nodeM2() ->

    receive
    | {X, Y, PidB, PidA} ->
    |   Sum = X + Y,
    |   PidB ! {sum, Sum, PidA}
    end.

```

6. fare un semaforo

```

%%semaforo
-module(exercise5).
-export([start/1, acquire/1, release/1]).

%% Avvia il semaforo con un numero massimo di risorse disponibili
start(Max) ->
    spawn(fun() -> loop(Max) end).

%% Processo che gestisce lo stato del semaforo
loop(Count) ->
    receive
    | {acquire, Pid} when Count > 0 ->
    |   Pid ! ok,
    |   loop(Count - 1);
    | {acquire, Pid} ->
    |   receive
    |   | {release} -> loop(Count + 1)
    |   after 0 ->
    |   | Pid ! wait,
    |   | loop(Count);
    |   end;
    | {release} ->
    |   loop(Count + 1)
    end.

```

```

%% Funzione per acquisire il semaforo
acquire(Sem) ->
    Sem ! {acquire, self()},
    receive
    | ok -> ok;
    | wait -> acquire(Sem) % Riprova finché non riesce
    end.

%% Funzione per rilasciare il semaforo
release(Sem) ->
    Sem ! {release}.

```

7. sortare una lista

Vedi 3.

8. moltiplicare tutti i numeri di una lista

```

%% moltiplicare tutti gli elementi di una lista
-module(exercise2).

-export([rmul/1]).

rmul([]) -> 1;

%%in questo caso è ricorsiva senza utilizzare tail recursion
rmul([H|T]) -> H * rmul(T).

%%utilizzo di tail recursion utilizzando un accumulatore
rmul(L) -> rmul(L, 1).
rmul([H|T], Acc) -> rmul(T, Acc * H);
rmul([], Acc) -> Acc.

```

9. estrarre solo numeri pari o dispari da una lista

```

-export([filter_even/1]).
-export([filter_even/2]).

-export([filter_odd/1]).
-export([filter_odd/2]).

%%solo pari
filter_even([]) -> [];
filter_even([H | T]) when H rem 2 == 0 -> [H | filter_even(T)];
filter_even([_ | T]) -> filter_even(T).

%%con list comprehension
filter_even(List) ->
    [X || X <- List, X rem 2 == 0].

%%solo dispari
filter_odd([]) -> [];
filter_odd([H | T]) when H rem 2 != 0 -> [H | filter_odd(T)];
filter_odd([_ | T]) -> filter_odd(T).

%%con list comprehension
filter_odd(List) ->
    [X || X <- List, X rem 2 != 0].

```

10. date due liste A e B, ritornare solo gli elementi di A che non compaiono in B

```

-module(exercise4).

-export([difference/2]).
%%soluzione con list:member
difference([], _) -> [];
difference([H | T], B) when lists:member(H, B) -> difference(T, B);
difference([H | T], B) -> [H | difference(T, B)].

%%soluzione senza list member
-module(list_utils).
-export([difference/2]).

difference([], _) -> [];
difference([H | T], B) when is_in_list(H, B) -> difference(T, B);
difference([H | T], B) -> [H | difference(T, B)].

is_in_list(_, []) -> false;
is_in_list(X, [X | _]) -> true;
is_in_list(X, [_ | T]) -> is_in_list(X, T).

```